



Frank Witte

Testmanagement und Softwaretest

Theoretische Grundlagen
und praktische Umsetzung

2. Auflage

EBOOK INSIDE



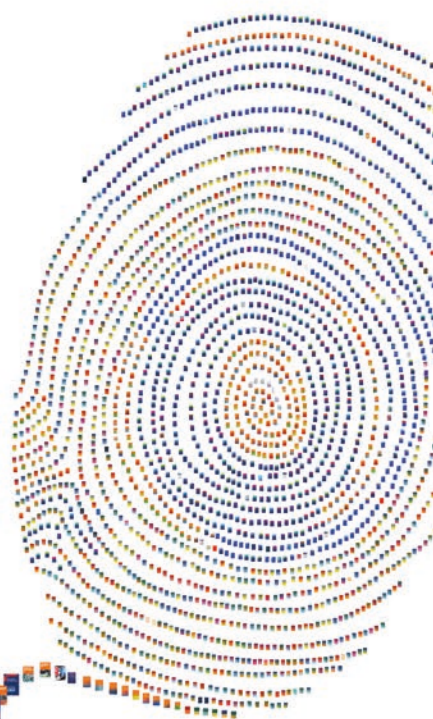
Springer Vieweg

Testmanagement und Softwaretest

Lizenz zum Wissen.

Sichern Sie sich umfassendes Technikwissen mit Sofortzugriff auf tausende Fachbücher und Fachzeitschriften aus den Bereichen: Automobiltechnik, Maschinenbau, Energie + Umwelt, E-Technik, Informatik + IT und Bauwesen.

Exklusiv für Leser von Springer-Fachbüchern: Testen Sie Springer für Professionals 30 Tage unverbindlich. Nutzen Sie dazu im Bestellverlauf Ihren persönlichen Aktionscode **C0005406** auf www.springerprofessional.de/buchaktion/



Jetzt
30 Tage
testen!

Springer für Professionals.

Digitale Fachbibliothek. Themen-Scout. Knowledge-Manager.

-  Zugriff auf tausende von Fachbüchern und Fachzeitschriften
-  Selektion, Komprimierung und Verknüpfung relevanter Themen durch Fachredaktionen
-  Tools zur persönlichen Wissensorganisation und Vernetzung

www.entschieden-intelligenter.de

Springer für Professionals

 Springer

Frank Witte

Testmanagement und Softwaretest

Theoretische Grundlagen und praktische
Umsetzung

2., erweiterte Auflage



Springer Vieweg

Frank Witte
Landshut, Deutschland

ISBN 978-3-658-25086-7 ISBN 978-3-658-25087-4 (eBook)
<https://doi.org/10.1007/978-3-658-25087-4>

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <http://dnb.d-nb.de> abrufbar.

Springer Vieweg

© Springer Fachmedien Wiesbaden GmbH, ein Teil von Springer Nature 2016, 2019

Das Werk einschließlich aller seiner Teile ist urheberrechtlich geschützt. Jede Verwertung, die nicht ausdrücklich vom Urheberrechtsgesetz zugelassen ist, bedarf der vorherigen Zustimmung des Verlags. Das gilt insbesondere für Vervielfältigungen, Bearbeitungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.

Die Wiedergabe von allgemein beschreibenden Bezeichnungen, Marken, Unternehmensnamen etc. in diesem Werk bedeutet nicht, dass diese frei durch jedermann benutzt werden dürfen. Die Berechtigung zur Benutzung unterliegt, auch ohne gesonderten Hinweis hierzu, den Regeln des Markenrechts. Die Rechte des jeweiligen Zeicheninhabers sind zu beachten.

Der Verlag, die Autoren und die Herausgeber gehen davon aus, dass die Angaben und Informationen in diesem Werk zum Zeitpunkt der Veröffentlichung vollständig und korrekt sind. Weder der Verlag, noch die Autoren oder die Herausgeber übernehmen, ausdrücklich oder implizit, Gewähr für den Inhalt des Werkes, etwaige Fehler oder Äußerungen. Der Verlag bleibt im Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutionsadressen neutral.

Springer Vieweg ist ein Imprint der eingetragenen Gesellschaft Springer Fachmedien Wiesbaden GmbH und ist ein Teil von Springer Nature.

Die Anschrift der Gesellschaft ist: Abraham-Lincoln-Str. 46, 65189 Wiesbaden, Germany

Vorwort

In diesem Buch gebe ich Ihnen Hinweise und konkrete Tipps, wie man beim Testmanagement und bei Softwaretests am besten vorgeht und werde dafür theoretische Grundlagen zusammen mit Anregungen zur Umsetzung in der betrieblichen Praxis verbinden. Ich verfüge über langjährige Erfahrung in diesem Bereich und habe in vielen Projekten in unterschiedlichen Branchen immer wieder gesehen, wie der Test der Software erfolgreich werden kann, wo es Fallstricke gibt und was getan werden muss, um sie zu vermeiden.

Da Softwaretest und Testmanagement einem ständigen Wandel unterworfen sind, neue Trends aufkommen und sich verstärkt haben, habe ich mich dazu entschlossen, eine 2., überarbeitete Auflage meines Anfang 2016 erstmals erschienenen Buches zu veröffentlichen.

Mit dieser zweiten Auflage gibt es auch eine „eBook-inside-Option“, siehe auch letzte Seite.

Zum Testen ist es immer erforderlich, das Testobjekt genau einzugrenzen – welche Anwendung, welches Betriebssystem, welche Testumgebung ... und die Eingrenzung des Untersuchungsgegenstands steht auch am Anfang dieses Buches.

Der Softwaretest findet oft in großen Projekten statt. Dabei gibt es jede Menge Störungen von außen: Profilierung einzelner Manager oder Mitarbeiter, Kampf zwischen Abteilungen, Managemententscheidungen, die einen gesamten Bereich betreffen und deren Sinn man sich beim besten Willen nicht erschließen kann. Wenn es wichtiger ist, die Rendite des Unternehmens im Quartal um einen Zehntelprozentpunkt zu steigern und dadurch Projekte scheitern, wenn man meint, ein funktionierendes System durch nicht durchdachte Entscheidungen (etwa vorschnelle Offshore-Verlagerung, angebliche Synergieeffekte, bevorstehender Verkauf oder Merger des Unternehmens) außer Kraft setzen zu müssen und dabei ganze Teams restlos und nachhaltig demotiviert. Hin und wieder bekommt man solche Knüppel zwischen die Beine geworfen, so dass man sich nicht wundern muss, dass es dann manchmal keine nennenswerten Fortschritte gibt bzw. dass man sich im Gegenteil manchmal wundern muss, dass überhaupt noch ein Entwicklungsprojekt Fortschritte macht.

Dazu gehören auch Karrieredenken und Optimierung eigener Egoismen, sture Abgrenzungen, persönliche Eitelkeiten, Krisen und Dilettantismus, mangelnde Flexibilität, Bürokratismus, psychische Probleme wie Burnout, Boreout und innere Kündigung von

Mitarbeitern, Machtansprüche und persönliche Eitelkeiten. Solche Rahmenbedingungen können jedes Projekt zum Scheitern bringen.

Manchmal werden laufende Projekte auch unerwartet gestoppt. Man wundert sich dann zwar warum man sparen will, koste es was es wolle, aber ändern kann man es in der Regel nicht. Oft erschließen sich die wahren Hintergründe dem Projektmitarbeiter dabei nicht.

Auch politische Termine, starre Budgetrestriktionen und mangelnde Kostentransparenz, kein Bewusstsein über die Folgen oder den Wunsch, die Wahrheit einfach nicht sehen zu wollen, belasten Innovationen weit mehr als es in der allgemeinen Wahrnehmung und selbst weit mehr als es in der gefilterten Wahrnehmung des Topmanagements gesehen wird.

Ein prominentes Beispiel für politische Termine ist die Einführung des LKW-Mautsystems Toll Collect:

Toll Collect wurde im März 2002 als Joint Venture der Deutschen Telekom, Daimler Chrysler und der französischen Cofiroute (Compagnie Financière et Industrielle des Autoroutes) gegründet. Cofiroute war einbezogen worden, weil von den Bewerbern Erfahrung mit vergleichbaren Projekten verlangt worden war und Telekom bzw. Daimler Chrysler dies nicht nachweisen konnten. Die Gesellschafteranteile von Toll Collect verteilten sich auf die Deutsche Telekom (45 %), Daimler Chrysler (45 %) und Cofiroute (10 %). Das Unternehmen beschäftigt nach eigenen Angaben rund 750 Mitarbeiter.

Mit dem System von Toll Collect sollte die Erfassung der Straßenbenutzungsgebühren für LKWs von einer pauschalen Gebühr auf eine wegstreckenerfasste Autobahngebühr umgestellt werden. Nach einigen politischen Auseinandersetzungen hatte Toll Collect im Juli 2002 den Zuschlag erhalten, im September 2002 wurde der Vertrag mit dem Bundesverkehrsministerium unterzeichnet. Für den Betrieb des Mautsystems sollte Toll Collect 12 Jahre lang ca. 650 Mio. Euro pro Jahr aus den Mauteinnahmen erhalten.

Der Staat verlangte schließlich von Toll Collect, der Tochterfirma des Konsortiums, 3,3 Milliarden Euro Schadenersatz plus 1,7 Milliarden Konventionalstrafe, weil das Mautsystem, das im Sommer 2003 eigentlich hätte in Betrieb gehen sollen, tatsächlich erst eineinhalb Jahre später funktionierte. 2005 erhob die damalige rot-grüne Regierung deswegen Schiedsklage. Toll Collect seinerseits klagte zurück, weil der Bund wegen der Start-Verzögerungen Geld einbehält.

Bei der Planung von Toll Collect war man davon ausgegangen, dass bis Juni 2003 das System entwickelt wird und zum 01.09.2003 eingeführt wird, inklusive der Integrations- und Testphase sowie der Urlaubszeit.

Der alte Toll Collect Maut-Vertrag mit allen Anlagen und Nebenvereinbarungen hatten einen Umfang von 17.000 Seiten. Haben die Abgeordneten dieses Werk wirklich gelesen?

Des Weiteren meldete die EU Wettbewerbsbehörde wegen eines möglichen Monopols von Daimler Chrysler für die OBU Bedenken an. Die OBU ist die On Board Unit, also das Gerät was die Wegstrecke per Satellitenortungssystem GPS erfasst, mit der Anzahl der Fahrzeugachsen, der Schadstoffklasse, dem Fahrzeugkennzeichen und den gebührenpflichtigen Autobahnabschnitten zu einem Paket zusammenfasst und diese Daten mit einem Mobilfunksender per SMS an den Zentralrechner von Toll Collect

schickt. Die EU Wettbewerbsbehörde verfügte dabei einige Auflagen für die behördliche Zulassung zur Bedingung, z. B. dass die OBU auch mit den Geräten anderer Hersteller kompatibel sein musste. Das führte zu einem erheblichen Terminverzug bei der Einführung des Systems, die für den 31.08.2003 angesetzt war. Dadurch wurde dem Staat ein erheblicher Einnahmeverlust zugefügt, weil das pauschale System zu diesem Zeitpunkt abgeschaltet wurde.

Es gab vor allem im Bereich der OBUs massive technische Probleme:

- OBUs reagierten nicht auf Eingaben.
- OBUs ließen sich nicht ausschalten.
- OBUs schalteten sich grundlos aus.
- OBUs zeigten unterschiedlich hohe Mautbeträge auf identischen Strecken an.
- OBUs wiesen mautpflichtige Autobahnstrecken als mautfrei aus.
- OBUs wiesen Strecken außerhalb des Autobahnnetzes als mautpflichtig aus.
- Siemens-Geräte passten nicht in den genormten Einbauschacht.

Diese Fehler wurden teilweise von der fehlerhaften Software verursacht, die die Geräte auch automatisch per GSM Mobilfunk aktualisieren sollte. Da die Fehler so gravierend waren, musste der Einführungstermin mehrfach verschoben werden bis schließlich der 01.01.2005 als Einführungstermin festgeschrieben wurde. Über die entstandenen Kosten, vor allem durch den Mautausfall, wurde lange kontrovers diskutiert: Die Bundesregierung hatte mit Mehreinnahmen von 2,6 Mrd. Euro im Vergleich zur alten Maut gerechnet und diesen Betrag bereits fest im Bundeshaushalt eingeplant.

Im März 2005 lief das System endlich mit einer Zuverlässigkeit von 99 %, aber die OBUs waren in ihrer Funktionalität eingeschränkt, da einige Fehler nicht so schnell beseitigt werden konnten. Vor allen Dingen die automatischen Updates der Software über GSM waren deaktiviert und konnten erst in einer nächsten Stufe zum 01.01.2006 eingeführt werden. Erst mit dieser Änderung konnten Streckenneubauten, Änderungen der Kilometerpauschale und Umwidmung von Bundesstraßen wegen der „Mautflucht“ vom System verarbeitet werden.

Dadurch erzielten das Projektmanagement und die verspätete Einführung einen Effekt auf die Umwelt, der weit über die Bedeutung des Mautsystems hinausging.

Zahlreiche Spediteure und Fuhrunternehmen kamen mit dem neuen System nicht klar und beklagten eine holprige Einführung. Das System verursachte auch erheblichen Mehraufwand bei LKW-Fahrern und in Logistikunternehmen. Das als Vorzeigeprojekt geplante System von Toll Collect hatte sich am Ende zu einem Lehrstück in Sachen Projektfehler entwickelt.

Es gibt also zahlreiche Kriterien, die Projekte verzögern, verteuern oder ruinieren, die außerhalb fachlicher Thematik liegen. Die meisten Projekte scheitern nicht wegen ingenieurtechnischer oder mathematischer Herausforderungen, nicht aufgrund komplizierter Technologie, sondern aufgrund mangelnder Kommunikation, trivialer Planungsfehler und relativ einfacher Probleme im betrieblichen Ablauf.

Es gibt genügend Experten und Bücher, die sich mit persönlichen Befindlichkeiten, nötigen Soft Skills und gruppendynamischen Prozessen intensiv befassen, so dass ich hierauf nicht eingehen werde.

Dieses Buch soll sich auf fachliche Themen beschränken, die mit dem Softwaretest direkt in Verbindung stehen.

Ein wesentlicher Grund, warum IT-Projekte in die Schieflage kommen, sind unklare Prozessvorgaben, schlechte Kommunikation und eine mangelhafte Organisation. Zumindest in diesem Bereich kann man mit professionellem Testmanagement einiges gegensteuern.

Die Herausforderung der geeigneten Kommunikation ist durch die neuen Medien, ein geändertes Kommunikationsverhalten und ein Eintreten einer jüngeren Generation in den Arbeitsmarkt auch im Testmanagement in den letzten Jahren noch schwieriger geworden.

Ich konzentriere mich in der Betrachtung weitestgehend auf den Systemtest. Andere Testschritte und Projektphasen sind zwar zur Erreichung des Projektziels nicht weniger wichtig, sie sollen auch hier teilweise gestreift werden, wo es auf deren Vollständigkeit und Erfüllung ganz besonders ankommt, aber der Systemtest soll besonders beleuchtet werden.

Meine berufliche Erfahrung als Testkoordinator, Testmanager und Tester und die Projektarbeit als Freiberufler in den letzten 25 Jahren hat dieses Buch ebenfalls beeinflusst. Ähnlichkeiten mit lebenden oder toten Personen sind also rein zufällig – aber so manche Anekdote kann vielleicht plastisch das Spannungsfeld darstellen, in dem sich der Testverantwortliche befindet.

Wenn übrigens im Folgenden von Testmanagern, Testkoordinatoren, Testern ... die Rede ist, meine ich immer auch Testmanagerinnen, Testkoordinatorinnen, Testerinnen und verwende nur aus Gründen der Lesbarkeit die männliche Form.

In diesem Buch soll das Thema Softwaretest demnach so betrachtet werden, als ob die Rahmenbedingungen stimmen, also man die Anwendung programmieren und fertig stellen möchte, dass das Entwicklungsprojekt überhaupt politisch gewollt ist, dass es keine gegenläufigen Tendenzen auf Managementebene gibt, man keine Querschläger von innen oder außen bekommt, es keine persönlichen Differenzen gibt und man sich nur der Sache widmen kann. Schon das ist alles bei Weitem nicht selbstverständlich, aber man muss bestimmte Fakten voraussetzen, um überhaupt in der Lage zu sein, eine Anwendung erfolgreich testen zu können.

Selbst mit durchweg positiven Rahmenbedingungen wird ein Testprojekt schon kompliziert genug. Softwaretest bezieht sich in der Regel auf komplexe, innovative Technologien, umfangreiche Systeme und verschachtelte Prozesse, die daher auch komplexe Testverfahren benötigen. Mit der zunehmenden Digitalisierung vieler Anwendungen und der verstärkten Durchdringung der Informationstechnologie in vielen Lebensbereichen muss der Softwaretest dem steigenden Qualitätsanspruch der Systeme gerecht werden und benötigt dafür ein ausgereiftes Management.

Inhaltsverzeichnis

- 1 Der Testprozess** 1
 - 1.1 Die Säulen im Testprozess 2
 - 1.2 Phasen im Testprozess 3
 - 1.3 Historie des Themas Softwaretest 4
 - 1.4 Standardisierung von Testprozessen 5
 - 1.5 Die Rolle des Testers im Testprozess 5
 - 1.6 Rahmenbedingungen beim Softwaretest 7
 - 1.7 Verfahren zur Prozessoptimierung 8
 - Literatur 9
- 2 Grundsätze für Softwaretester** 11
 - 2.1 Testen zeigt die Anwesenheit von Fehlern 11
 - 2.2 Vollständige Tests sind unmöglich 12
 - 2.3 Mit dem Testen frühzeitig beginnen 12
 - 2.4 Beim Testen flexibel sein 13
 - 2.5 Tests müssen geplant werden 13
 - 2.6 Testfälle regelmäßig prüfen und reviewen 13
 - 2.7 Das Testumfeld berücksichtigen 14
 - 2.8 Unterschiedliche Personen als Tester und Programmierer 14
 - 2.9 Dokumentation und Nachvollziehbarkeit der Tests 14
 - 2.10 Operation gelungen, Patient tot? 15
 - 2.11 Ermittlung der Fehlerrate 15
 - 2.12 Testnutzen 17
 - 2.13 Testphilosophien 18
 - Literatur 19
- 3 Testnormen** 21
 - 3.1 IEEE 829 Standard for Software Test Documentation 22
 - 3.2 ISO-Standard 9126..... 22
 - 3.3 ISO-Standard IEC-29119..... 25
 - Literatur 26

4	Rollen und Verantwortlichkeiten im Testmanagement	27
4.1	Testmanager	27
4.2	Testkoordinatoren	28
4.3	Testdesigner	29
4.4	Testautomatisierer	30
4.5	Testsystemadministrator	30
4.6	Tester (Testdurchführer)	31
4.7	Qualifikation für Testverantwortliche	31
	Literatur	32
5	Grundlagen des Testmanagements	33
5.1	Integration der Testprozesse in den Life Cycle der Softwareentwicklung	34
5.2	Integrierte Toolunterstützung im gesamten Testprozess	34
5.3	Einheitliche Testumgebungen und Testdaten	35
5.4	Softwaretest als Management-Aufgabe	35
5.5	Industrialisierung und Standardisierung der Testprozesse	35
	Literatur	36
6	Marktsituation beim Softwaretest	37
6.1	Ist-Situation	37
6.2	Tester und Entwickler sind bei vielen IT- Trends nicht im Driver-Seat	40
6.3	Tester und Entwickler sind zögerlich bei innovativen Techniken	42
6.4	Viel Potenzial zur Effizienzsteigerung des Testens bleibt ungenutzt	43
6.5	Shift-Left ist in phasenorientierten Prozessmodellen nicht etabliert	44
6.6	Testen läuft Gefahr, zum Selbstzweck zu werden	44
6.7	Tester und Entwickler kritisch gegenüber Standards	45
6.8	Künftiger Forschungsbedarf	46
	Literatur	48
7	Testvorbereitung	49
7.1	Teststrategie	49
7.2	Kennzahlen zur Wahl der geeigneten Teststrategie	52
7.3	Häufigkeit der Änderungen und Anzahl der Releasezyklen	54
7.4	Änderungsumfang pro Release	54
7.5	Testziele	55
7.6	Anforderungen an das Konfigurationsmanagement	57
7.7	Testendekriterien	58
7.8	Testorganisation	59
	Literatur	60
8	Requirements Engineering	61
8.1	Grundsätze des Anforderungsmanagements	61
8.2	Kriterien für die Formulierung von Requirements	63
8.3	Requirements Engineering in agilen Projekten	65

8.4	Beispiele zur Formulierung von Requirements	67
8.5	Nachträgliche Erstellung von Requirements	68
8.6	Probleme beim Requirements Engineering	69
8.7	Review der Requirements	71
8.8	Nebenabsprachen und Schattenprozesse	72
8.9	Verzögerungen beim Requirements Management	72
	Literatur	73
9	Teststufen	75
9.1	Modultest	75
9.2	Integrationstest	76
9.3	Systemtest	78
9.4	Abnahmetest	81
9.5	Mischformen und Abgrenzung der einzelnen Teststufen	82
9.6	Teststufen im V-Modell	82
9.7	Blackbox- und Whitebox-Verfahren	84
9.8	Mehrere Teststufen statt „Big Bang“	87
	Literatur	88
10	Testabdeckung und Überdeckungsmaße	89
	Literatur	92
11	Fehlermanagement	93
11.1	Definition von Fehlern	94
11.2	Fehlerarten	94
11.3	Kosten pro Fehler – die Barry-Boehm-Kurve	95
11.4	Dynamik der Fehlerkosten	96
11.5	Konsequenzen für die Fehlerbehandlung in den Teststufen	101
11.6	Fehlerverfolgung	102
11.7	Aufbau einer Fehlermeldung	103
11.8	Tools für die Fehlerverwaltung	103
11.9	Auswirkungen von Fehlern	109
11.10	Schätzungen für die Fehlerdichte	110
11.11	Programmierfehler – Beispiele	112
	Literatur	118
12	Testplanung	119
12.1	Erarbeitung der Teststrategie und der Testmaßnahmen	119
12.2	Gemeinsames Verständnis über Testvorgehen und Testziele	120
12.3	Tests nach Auslieferung des Produkts	121
12.4	Testdaten	121
12.5	Das Testprojekt im Entwicklungszyklus	123
12.6	Zeitliche Planung der Testaktivitäten	124

13	Testumgebung	127
	Literatur	130
14	Testkonzeption	131
14.1	Grundlagen	131
14.2	Aufbau des Testkonzepts	134
14.3	Testkonzept nach IEEE 829	136
14.4	Testarten, Testinhalte und Testphasen	139
14.5	Testkonzept und betriebliche Realität	140
	Literatur	142
15	Kennzahlen zur Bewertung von Tests	143
15.1	Testprozessreife	144
15.2	Ermittlung der Testproduktivität	145
15.3	Kennzahlen für die Testeffektivität	148
15.4	Die COCOMO-II-Gleichung	148
15.5	Berechnung der Testdauer und des Testaufwands	149
15.6	Planungsrisiken bei der Aufwandsermittlung	151
15.7	Risikozuschläge in Testprojekten	152
15.8	Vom Entwicklungsaufwand zum Testaufwand	153
15.9	Vom Gesamtaufwand zur Detailbetrachtung	154
15.10	Berücksichtigung vorhandener Ressourcen	154
15.11	Genügend Fehlerpuffer miteinbeziehen	155
15.12	Erfahrungen aus früheren Projekten nutzen	155
	Literatur	156
16	Testvoraussetzungen	157
17	Beschreibung der Testfälle	159
17.1	Struktur der Testbeschreibung	159
17.2	Testspezifikation und Testimplementierung	160
17.3	Beschreibung der Geschäftsprozesse	161
17.4	Aufbau des einzelnen Testfalls	162
17.5	Strukturierung der Testfälle	163
17.6	Anzahl Testfälle	164
17.7	Abhängigkeiten zwischen Testfällen	164
17.8	Priorisierung von Testfällen	166
17.9	Funktionale und nichtfunktionale Testfälle	166
18	Testmethoden	173
18.1	Äquivalenzklassenbildung	173
18.2	Unterschiedliche Testtiefen	177
18.3	Überdeckungsmaße	178

18.4	Test Maturity Model	184
18.5	Test Process Improvement (TPI)	185
	Literatur	185
19	Testdurchführung	187
19.1	Änderungen der Testfallbeschreibung während der Testdurchführung	188
19.2	Strukturiertes Testen und exploratives Testen	188
19.3	Intuitive Testfallermittlung	190
19.4	Durchführung explorativer Tests	191
	Literatur	191
20	Testbericht	193
20.1	Kriterien für Testberichte	193
20.2	Beschreibung der Rahmenbedingungen	194
20.3	Bericht der Testergebnisse	195
	Literatur	197
21	Produktiveinführung von Systemen	199
21.1	Pilotbetriebe	200
21.2	Produktivnahme mit Übergangsphase	201
21.3	Hilfsmodelle	202
	Literatur	203
22	Reviews im Testprozess	205
22.1	Allgemeines über Reviews	205
22.2	Vorteile von Reviews	205
22.3	Reviewprozess	207
23	Werkzeuge für die Unterstützung des Testmanagements	209
23.1	Evaluation von Testtools	209
23.2	Suche geeigneter Werkzeuge	210
23.3	Anforderungen an geeignete Test-Tools	212
	Literatur	213
24	Die optimale Testabdeckung	215
25	Testmetriken	217
25.1	Arten von Testmetriken	218
25.2	Bewertung der Komplexität von Testfällen	219
25.3	Problematik von Metriken	223
25.4	Verwässerung von Metriken	224
25.5	Metriken, bei denen man (fast) nur verlieren kann	225
25.6	Aussagen unterschiedlicher „Key Performance Indicators“	227
25.7	Beispiel für eine Fehlermetrik	230
	Literatur	230

26	Testautomatisierung	231
26.1	Chancen und Risiken der Testautomatisierung	232
26.2	Vorgehen bei der Testautomatisierung	233
26.3	Planung der Testautomatisierung	234
26.4	Voraussetzungen für automatische Testdurchführung	235
26.5	Codenahe und GUI-Testautomatisierung	235
26.6	Untersuchungen vor der Automatisierung	236
26.7	Ablauf der Testautomatisierung	236
26.8	Testergebnisse bei automatisierten Tests	237
26.9	Qualifikation der Softwaretester bei automatisierten Tests	238
26.10	Automatische Testskripts und Änderungen in der Software	239
26.11	Automatische Generierung von Testdaten	240
26.12	Fehlerfreie Ergebnisse ergeben vollständige Reproduzierbarkeit	240
26.13	Fazit	241
	Literatur	241
27	Offshoring von Tests	243
	Literatur	247
28	Test von Internet-Anwendungen	249
28.1	Struktur bei Web-Tests	249
28.2	Fragen an die Web-Applikation	250
28.3	Phasen beim Web-Test	252
28.4	Testabdeckung beim Test von Internetanwendungen	252
28.5	Test unterschiedlicher Sprachen	254
28.6	Look and Feel	255
28.7	Test nach Zielgruppen	255
28.8	A/B-Testing und Multivariate Testing	256
	Literatur	262
29	Testen in der Cloud	263
	Literatur	264
30	Mobile Testing	265
30.1	Besonderheiten beim Mobile Testing	265
30.2	Auswirkungen auf das Testmanagement	266
	Literatur	266
31	Agiles Testen	267
31.1	Besonderheiten beim Agile Testing	268
31.2	Testmanagement in agilen Testprojekten	270
31.3	Kombination traditioneller Testmethoden und agilen Methoden	271
	Literatur	271

32	Geschäftsbasiertes Testmanagement	273
	Literatur	276
33	TPI Next	277
33.1	Kernbereiche im TPI Next Modell	277
33.2	Der Änderungsprozess beim geschäftsbasierten TPI	278
	Literatur	282
34	Testumgebungsmanagement	283
	Literatur	285
35	Testaufwandsschätzung	287
35.1	Grundlagen von Schätzverfahren	287
35.2	Modelle zur Testaufwandsermittlung	288
	Literatur	293
	Weiterführende Literatur	295
	Stichwortverzeichnis	297

Der Softwaretest besteht aus mehreren Phasen. Ein definierter effizienter Testprozess verbessert die Abläufe und erhöht die Softwarequalität.

Der idealtypische Testprozess kann wie in Abb. 1.1 dargestellt werden.

Das Testmanagement erstreckt sich von der Testplanung über die Erstellung der Testspezifikation und Durchführung der Tests bis hin zur Protokollierung der Testergebnisse und der Auswertung der Tests.

Das Testmanagement organisiert, koordiniert und überwacht die entsprechenden Aktivitäten durch:

- Incident Management: organisatorischer und technischer Prozess der Reaktion auf erkannte oder vermutete Störungen in IT-Bereichen sowie hierzu vorbereitende Maßnahmen und Abläufe.
- Problem Management: ermitteln, bewerten und Korrektur aufgetretener Fehler.
- Change Management: Bearbeitung zusätzlicher oder geänderter Anforderungen während des Projektablaufs.
- Release Management: Planung und Konfiguration der einzelnen Software Releases.

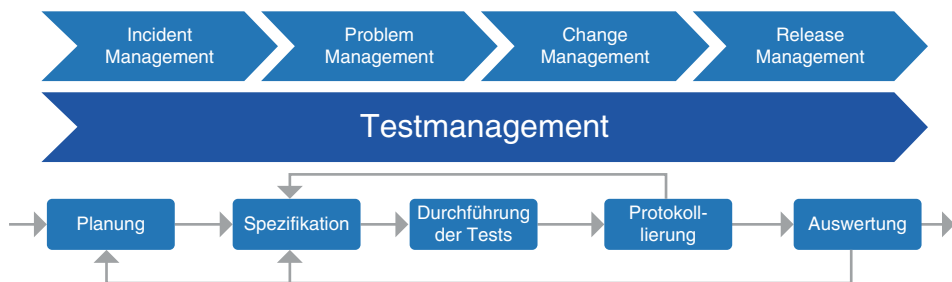


Abb. 1.1 Der Testprozess

Ein definierter Testprozess dient dazu, die einzelnen Testaktivitäten voneinander abzugrenzen, Ablaufpläne zu verfeinern und inhaltliche und zeitliche Abhängigkeiten der einzelnen Testaktivitäten aufzuzeigen.

Der weltweit verbreitete ISTQB-Testprozess unterscheidet folgende Aktivitäten:

- Planung und Steuerung,
- Analyse und Design,
- Realisierung und Durchführung,
- Auswertung und Bericht,
- Abschluss.

Die einzelnen Aktivitäten des Testprozesses sind nicht strikt nacheinander abzuarbeiten, eine zeitliche Überlappung wird sich in der Praxis weder vermeiden lassen noch ist sie unbedingt erforderlich.

1.1 Die Säulen im Testprozess

Ein erfolgreicher Testprozess steht auf mehreren Säulen, die in diesem Buch näher beschrieben werden (Abb. 1.2).

Das Testmanagement muss bereits von Anfang an in die Softwareentwicklung mit eingebunden werden. Die Teststrategie definiert die grundlegenden Festlegungen, ein methodisches Vorgehen bei der Organisation der Testprozesse legt die Basis für einen erfolgreichen Ablauf. Eine Testplanung und Konzeption des Testvorgehens definiert wer, was, wann, wie testet und welche Umgebungsparameter erforderlich sind. Dazu benötigt man fachliches Know-how über die Anwendung, die betrieblichen Abläufe und standardisierte Vorgehensweisen. Professionelle Testtools unterstützen die administrative Abwicklung.



Abb. 1.2 Säulen im Testprozess

1.2 Phasen im Testprozess

Der grundsätzliche Testprozess kann aus dem V-Modell mit folgenden Stufen abgeleitet werden:

- Testplanung,
- Testdesign,
- Testspezifikation,
- Testdurchführung,
- Testprotokollierung,
- Testauswertung.

In der Phase der Testplanung werden das Testkonzept und der Testplan erstellt. Dabei werden das Testobjekt, die Testumgebung, die Konfiguration des Testsystems und die Testressourcen beschrieben. Dabei wird festgelegt, welcher Testumfang mit welchen Tools getestet werden soll.

Beim Testdesign wird die Testplanung verfeinert. Die Testanforderungen, nötigen Testszenarien und Testdekriterien werden in diesem Schritt dokumentiert. Je nach Umfang des Projekts kann diese Phase aber auch mit der Testplanung zusammenfallen [[KNAU2008](#)].

In der Phase der Testspezifikation werden die Testfälle im Detail beschrieben. Die Testvoraussetzungen, die erforderlichen Eingaben und erwarteten Ausgaben werden dokumentiert und Abhängigkeiten zwischen den einzelnen Testfällen ermittelt. Dabei werden sowohl funktionale als auch nichtfunktionale Anforderungen berücksichtigt.

Die Testdurchführung geschieht manuell oder automatisiert, in der Praxis sind meist Mischformen anzutreffen. Die Testdurchführung ist fast immer iterativ, weil die Software phasenweise entwickelt wird, die entwickelten Features demnach im Laufe der Zeit erweitert werden und weil behobene Fehler immer wieder mit Regressionstests nachgetestet werden müssen. Die Entwicklung schreitet in der Regel auch in Phasen voran, so dass der Testumfang sich während der Projektdauer um neue Testaktivitäten erweitert.

Bei der Testprotokollierung werden die Testergebnisse aufgezeichnet, manuell mit Checklisten oder Testberichten oder automatisiert mittels Logfiles. Diese Ergebnisse werden im Rahmen der Testauswertung analysiert und aufbereitet. Dabei stellt man evtl. fest, dass man zu früheren Phasen im Testprozess zurückkehren muss, etwa wenn die Testspezifikation fehlerhaft oder lückenhaft war.

Dieser Testprozess soll aber nur einen grober Rahmen zur Orientierung geben, und keine starre Vorgabe sein. In agilen Testansätzen geht man verstärkt dazu über, den Testprozess möglichst offen zu halten und den Umständen der jeweiligen Aufgabenstellung anzupassen.

Modellhafte Ansätze sind immer abzuwandeln. Jede Aufgabenstellung, jede Software, jedes Testprojekt und jedes Team steht vor individuellen Herausforderungen, wie es die Phasen effizient umsetzt.

1.3 Historie des Themas Softwaretest

Softwaretests waren bis ca. 1975 nahezu unbekannt. Ende der 70er-Jahre wurde das erste kommerzielle Testlabor eröffnet, da erste Unternehmen die Notwendigkeit eines Softwaretests erkannten, aber dafür keine Ressourcen zur Verfügung hatten, und die Testtheorie erst ansatzweise begonnen hatte. Als sich das Thema Softwaretest in den 80er- und 90er-Jahren des letzten Jahrhunderts langsam etablierte, kam es des Öfteren vor, dass die Software fast fertiggestellt war und erst dann getestet wurde. Eigene Experten oder eine definierte Testabteilung waren bei vielen Unternehmen noch nicht vorhanden. Ein Berufsbild für den Softwaretester war damals erst im Entstehen, eine Standardisierung von Testprozessen nur ansatzweise vorhanden.

Nach mehreren negativen Erfahrungen und einigem Lehrgeld ist in den Unternehmen auf breiter Front das Bewusstsein gewachsen, wie wichtig ein entwicklungsbegleitender Softwaretest ist, um den Ansprüchen an qualitativ hochwertige Lieferungen zu genügen – sei es, dass die Software an Kunden geliefert wird, oder dass es sich um interne Anwender handelt (z. B. ein Warenwirtschaftssystem das in der eigenen kaufmännischen Abteilung zur Fakturierung eingesetzt werden soll).

Bereits 1979 formulierte G. Myers Kriterien für den Softwaretest, die noch heute ihre Gültigkeit besitzen:

1. Ein notwendiger Bestandteil eines Testfalls ist die Definition der erwarteten Werte oder des Resultats.
2. Ein Programmierer versucht nicht, sein eigenes Programm zu testen.
3. Die Resultate eines jeden Tests sollen gründlich überprüft werden.
4. Testfälle müssen für ungültige und unerwartete ebenso wie für gültige und erwartete Eingabedaten definiert werden.
5. Ein Programm darauf hin zu untersuchen, ob es nicht tut, was es tun sollte, ist nur die eine Hälfte der Schlacht. Die andere Hälfte besteht darin zu untersuchen, ob das Programm etwas tut, was es nicht tun soll.
6. Wegwerftestfälle sollten vermieden werden, es sei denn, das Programm ist wirklich ein Wegwerfprogramm.
7. Es sollten keine Testverfahren unter der stillschweigenden Annahme geplant werden, dass keine Fehler gefunden werden.
8. Testen ist eine extrem kreative und intellektuell herausfordernde Aufgabe.
9. Testen ist der Prozess, ein Programm in der Absicht auszuführen, Fehler zu finden.
10. Ein guter Testfall ist dadurch gekennzeichnet, dass er mit hoher Wahrscheinlichkeit einen bisher unbekannten Fehler anzuzeigen imstande ist.
11. Ein erfolgreicher Testfall ist dadurch gekennzeichnet, dass er einen bisher unbekannten Fehler entdeckt.

1.4 Standardisierung von Testprozessen

Eine der Hauptursachen für die Fehleinschätzung von Aufwänden und Problembereichen ist, dass die Softwareindustrie wesentlich jünger ist als andere Industrien. Kaum mehr als 40 Jahre sind vergangen, seit die ersten Softwareunternehmen entstanden. Das sollte man sich immer wieder vor Augen führen, wenn man über ihre künftige Entwicklung spricht. Das heißt: Die Softwareindustrie kommt erst allmählich in die „reife“ Entwicklungsphase, in der die klassischen Industriezweige – wie zum Beispiel die Elektro-, die Automobilindustrie oder der Maschinenbau – sich schon lange befinden. Wenn man die heutige Reife der Softwareindustrie mit der klassischen Industrien vergleicht, befindet sich die Softwareindustrie in etwa im Jahre 1930. [OUTP2014]

Die Automobilindustrie treibt die Standardisierung in der Produktion seit Henry Ford voran. Die Pharmaindustrie setzt schon lange auf Cluster und Kooperationen mit Mitbewerbern in Entwicklung und Marktbearbeitung. Für die Elektro- oder Textilbranche ist die Auslagerung der Produktion und Entwicklung in Offshore-Länder eine Existenzbedingung. All diese Trends haben inzwischen auch die Softwareindustrie erreicht.

Deswegen müssen Software-Hersteller ihre Geschäftsmodelle in immer kürzeren Zeitabständen überdenken. Sie müssen regelmäßig ihre Geschäftsprozesse hinterfragen und untersuchen, wie sie höhere Qualität und geringere Kosten miteinander vereinbaren können. Unternehmen müssen über ihre Fertigungstiefe nachdenken und Tätigkeiten auslagern oder automatisieren. Außerdem müssen das Management professionalisiert und die internen Prozesse neu überdacht werden.

Software hat inzwischen alle unsere Lebensbereiche durchdrungen: sei es im Auto oder im Haushalt durch embedded Systeme, in der Fabrik, im Büro, oder in einem Online Shop. Allein im Google Play Store werden heute bereits über 4 Millionen Apps zum Download angeboten, und die Zahl steigt ständig weiter. Da jede Software vor Auslieferung an den Endnutzer bzw. Auftraggeber getestet werden muss, ist es wichtig zu prüfen, wie diese Herausforderungen unter Berücksichtigung der Kostenrahmen und der Qualitätsziele umgesetzt werden können.

1.5 Die Rolle des Testers im Testprozess

Parallel zur Professionalisierung der Softwareindustrie hat sich das Rollenbild des Testers in den vergangenen Jahren massiv verändert. Noch vor etwa zwanzig Jahren war es in den meisten Unternehmen nicht einmal existent. Die in den primär sequentiellen Vorgehensmodellen definierte Phase „Test“ wurde oft als Knautschzone, als Puffer für die in der Regel zu enge Projektplanung angesehen. Der Aufwand für den Softwaretest und somit auch für die Tester war schon im Vorfeld als Einsparungspotenzial überzogener

Projektbudgets definiert. Man wollte nicht viel in eigene Tester investieren, sondern die Entwickler selber testen lassen („der Auftraggeber sieht sie sich ja selbst noch mal an“) und Tester galten oft als minderbegabte Entwickler. Die Sicht auf die Bedeutung des Softwaretests war in etwa so, als ob man vor der Einweihung einer neuen Brücke gerade einmal einen einzigen LKW über die Brücke fahren lässt. Hält die Brücke, wird sie freigegeben, stürzt sie am nächsten Tag unter der Last der Autos ein, war der Lastkraftwagenfahrer schuld.

Dass ein Entwickler die selbst erstellte Software testet, widerspricht völlig der Philosophie, ein Produkt herzustellen. „Testing is a destructive process, even a sadistic process“ (Myers). Derjenige, der ein Produkt definiert, entwickelt und implementiert, ist am wenigsten dazu geeignet, die Ergebnisse seiner Tätigkeit destruktiv zu betrachten. Er wird schon unbewusst das Interesse daran haben, nur das Positive an seinem Erzeugnis zu sehen.

Daher soll nie der Entwickler das Endprodukt testen. Diese Erkenntnis hat sich zum Glück (nach jahrelanger Überzeugungsarbeit und vielen Fehlversuchen) inzwischen in den Unternehmen durchgesetzt. Man bezweifelt heute nicht mehr, dass es für den Systemtest eine eigene Instanz geben muss.

Mit jedem neuen Schaden wuchs die Erkenntnis, den Softwaretest ernster zu nehmen, besser zu planen und professioneller umzusetzen. Der Softwaretest wurde im Lauf der Jahre zu einer eigenen Disziplin und zu einer unabhängigen Instanz und einem selbstbewussten Rollenverständnis. Durch standardisierte Ausbildungen und Zertifikate, z. B. vom ISTQB, hat sich das Berufsbild des Softwaretesters in den letzten Jahren klarer geformt. Testen wird nicht mehr als eine Begleiterscheinung der Programmierung, sondern als eigenständige Disziplin betrachtet. Das Bewusstsein, dass der Softwaretest kein Studentenjob, keine Nebenbeschäftigung für nicht ausgelastete Sachbearbeiter oder gescheiterte Programmierer sein darf, ist inzwischen auf breiter Front vorhanden. Mit dieser Professionalisierung kamen auch Testtechniken, Testmethoden und Testwerkzeuge auf, und die Frage, ob und wozu überhaupt getestet werden soll, wird nicht mehr gestellt.

Die fortschreitende Digitalisierung der Umwelt macht einen zunehmenden Einsatz von Softwareprodukten erforderlich, die ständig weiterentwickelt und an neue Strukturen angepasst werden müssen. Daher bietet sich für den Softwaretester ein immer breiteres Betätigungsfeld. Vor allem größere Unternehmen der IT-Branche mit eigener Software-Entwicklungsabteilung verfügen oftmals über ein eigenes Team von Softwaretestern, welches die Produkte ausgiebig testet, ehe sie letztendlich auf den Markt kommen. Aber auch für Existenzgründer ist die Tätigkeit als Softwaretester eine lohnenswerte Aufgabe.

Umso wichtiger wird aber die Frage, wie man Testprozesse optimieren und effizient umsetzen kann, damit der Softwaretest mit der zunehmenden Professionalisierung und zunehmenden Reife der Softwareindustrie angemessen Schritt halten kann. Da das Thema Softwaretest mit der steigenden Bedeutung auch zunehmend Kosten verursacht, muss man schon im kaufmännischen Sinne bestrebt sein, Testprozesse zu verbessern, um die Softwarequalität zu erhöhen ohne die Kostenrahmen der Projekte zu sprengen.

1.6 Rahmenbedingungen beim Softwaretest

Die Professionalisierung der Software-Entwicklung führte dazu, dass auch dem Softwaretest mehr Bedeutung zugewendet werden muss:

- **komplexere Produkte:** Dieser Trend wird bei Autos oder elektronischen Produkten besonders transparent: Das Endprodukt beinhaltet mehr Features, unterschiedliche Komponenten, die in Wechselwirkungen zueinander stehen. Eine Änderung an irgendeiner Stelle im System führt zu einem Fehlverhalten an einer ganz anderen Stelle, teilweise völlig unerwartet und nicht vorhersehbar. Das Navigationsgerät im Auto muss unabhängig vom MP3-Player und der Klimaanlage funktionieren.
- **schnellere Produktzyklen:** Der Erwartungsdruck, neue Produkte auf den Markt zu bringen, die Dynamik im Markt ist gestiegen. Durch das Internet werden Produkte vergleichbarer, die Informationsfülle ist gewachsen, die Datenfülle angestiegen. Dadurch werden Innovationen und Neuentwicklungen beschleunigt, das menschliche Wissen wächst in immer kürzeren Intervallen. Das Internet erlaubt mehr Vergleichbarkeit, die Konkurrenz ist nur noch einen Mausklick vom eigenen Produkt entfernt. Die Reifephase ist verkürzt, schon aus Gründen kürzerer Konjunkturzyklen und der Erwartung schnellerer Return-on-Invests. Gerade für Hochlohnländer, die nicht über den Preis konkurrieren können, ist dabei höhere Komplexität und bessere Qualität zu einem wesentlichen Unterscheidungsmerkmal geworden. Die schnelle Reaktion auf geänderte gesetzliche Richtlinien, Marktgegebenheiten und Kundenanforderungen bedeutet, dass Unternehmen bessere Software in kürzeren Abständen ausliefern müssen. Erschwerend kommt hinzu, dass sich Änderungen vielfach noch während des Entwicklungsprozesses ergeben und entsprechende erste Anpassungen bereits vor Fertigstellung der Software durchzuführen sind.
- **stärkerer Wettbewerb:** Die Globalisierung macht Produkte und Dienstleistungen leichter vergleichbar und austauschbarer. Durch eine international wachsende Zahl von Unternehmen und Arbeitskräften nimmt der Innovationszyklus zu. Eine größere Freiheit von Kapitalströmen, mehr Exporte bis zu einer stärkeren Mobilität von Arbeitskräften bewirken eine stärkere Internationalisierung und eine Erhöhung des Wettbewerbs.
- **verteilte Entwicklung:** Komplexe Produkte bestehen aus zahlreichen Komponenten, die aufeinander abgestimmt sein müssen. Die Fertigungstiefe hat in allen Industrien abgenommen, die Spezialisierung steigt, man muss Komponenten von anderen Entwicklungsgruppen, anderen Firmen oder anderen Ländern im eigenen Produkt mit integrieren. Die Entwicklungstiefe hat ebenfalls abgenommen, der Lieferant ist nicht mehr nur eine verlängerte Werkbank, sondern spezialisiert sich auf die Optimierung seiner Komponenten. In der Automobilindustrie wird das besonders deutlich. Durch die gestiegenen Anforderungen an die Software-Integration steigen auch die Anforderungen an den Softwaretest [ICBR2017].

- neue Methoden im Projektmanagement: Der verstärkte Einsatz agiler, iterativ-inkrementeller Vorgehensweisen erfordern neue Ansätze im Softwaretest. Dadurch werden das frühzeitige Einbinden wichtiger Stakeholder, kurze Planungs- und Arbeitsphasen sowie ein kontinuierliches Neupriorisieren der zu erledigenden Aufgaben wichtiger [PROA2018].

1.7 Verfahren zur Prozessoptimierung

Um Softwareentwicklungsprozesse zu optimieren und vergleichbar beurteilen zu können, gibt es inzwischen zahlreiche Verfahren (CMMI, SPICE usw.). Diese Verfahren können Hinweise geben, müssen aber konsequent umgesetzt werden, um die entsprechenden Wirkungsgrade zu entfalten.

Oft werden Normen nur übergestülpt und stehen als theoretischer Block neben den eigentlichen Geschäftsprozessen. Jeder noch so gute Ansatz muss aber adaptiert, abgeändert und verfeinert werden, sonst bleibt das neue Verfahren eine Worthülse und lässt frustrierte Entwickler und Tester zurück. Dazu ist es wichtig, den Nutzen und die wesentlichen Inhalte eines Verfahrens zu verstehen.

Eine Beurteilung der Softwareproduktreife an sich findet aber meist gar nicht statt. Selbst eine optimale Gestaltung der Entwicklungsprozesse reicht nicht aus, um auch eine hohe Produktqualität zu garantieren. Vor allem die Bedeutung der Testphase wird dabei leicht unterschätzt.

Es leuchtet eher ein, mit zusätzlichen Entwicklern weitere Features zu programmieren, weil es dabei um Herstellung geht, weil die Produktivität im Vordergrund der Betrachtung steht, weil klar wird, dass man schneller fertig wird oder ein umfangreicheres Produkt mit zusätzlichen Extras bieten kann. Man sieht dann vor seinem inneren Auge ein umfassenderes, komplizierteres und innovativeres Produkt. Wenn man dieselbe Kapazität, um die man die Entwicklung aufstocken könnte, aber in den Test steckt, sieht man auf den ersten Blick weder neue Features noch zusätzliche Innovation. Im Gegenteil, je mehr man testet, desto mehr steigen die Kosten und desto mehr verschiebt sich der Fertigstellungstermin nach hinten. Qualität versteckt sich manchmal hinter der glänzenden Oberfläche, und nicht immer hat der Softwaretest eine kräftige Lobby im Unternehmen. Das liegt teilweise auch an der Qualität des Testmanagements.

Softwaretestern fehlt es häufig an Eigenmarketing. Ein erfolgreicher Softwaretester benötigt vor allem ein gutes Gedächtnis, Verständnis für Zahlen, Detailgenauigkeit, Akribie und ein logisches Denkvermögen. SAP plant, dass bis zum Jahr 2020 ca. 1 % seiner Belegschaft Autisten sein sollen, die vor allem als Softwaretester eingesetzt werden sollen [DAPZ2017]. Ein Schweizer Dienstleister für Softwaretest, Asperger Informatik, stellt bewusst Mitarbeiter mit dem Asperger-Syndrom, einer leichteren Form von Autismus, ein [ASPI2017]. Diese Mitarbeiter verfügen über herausragende Qualitäten für Konzeption, Durchführung und Analyse von Softwaretests aber nicht unbedingt über Soft Skills und haben manchmal Defizite in der zwischenmenschlichen Kommunikation. Häufig geht es bei Testberichten um komplexe Zusammenhänge, endlose Zahlenreihen und Detailprobleme, die nur schwer in eindrucksvollen bunten Bildern zu präsentieren sind.

Wenn die Testabteilung nur als notwendiges Übel angesehen wird, sie nur über das Image der unbequemen Erbsenzähler verfügt und nur als Sammelbecken gescheiterter Programmierer gilt, die für die kreativen Aufgaben unfähig sind und nur vorgegebene Checklisten ankreuzen, dann wird sie im Unternehmen nicht das Ansehen bekommen das ihr zusteht und das sie dringend benötigt. Es ist unbedingt erforderlich, dass der Entwicklungsleiter und der Leiter der Testabteilung auf Augenhöhe miteinander verhandeln. Bei Projektmeetings muss der Leiter der Testabteilung mit am Tisch sitzen und eigene Akzente setzen. Der entsprechenden Stellung des Testers muss von der Betriebsorganisation Rechnung getragen werden.

Dem Leiter der Testabteilung kommt sogar in besonderer Weise die Aufgabe zu, seinen Posten auch im Networking auszufüllen, gerade wenn seine Abteilung unter den bezeichneten Imageproblemen leiden sollte.

Literatur

- [ASPI 2017] <http://www.asperger-informatik.ch/index.php>. Zugriffen am 18.05.2019
- [DAPZ2017] <https://www.deutsche-apotheker-zeitung.de/news/artikel/2013/05/23/autisten-als-it-spezialisten-gesucht>. Zugriffen am 18.05.2019
- [ICBR2017] http://www.softec.wiwi.uni-due.de/uploads/tx_itochair3/publications/ICBReport20_05.pdf
- [OUTP2014] <http://www.output.at/erp-systeme/software-am-wendepunkt>
- [KNAU2008] <http://www.knaupes.net/theorie-der-softwaretests/>
- [PROA2018] <https://proagile.de/unterschied-klassisch-agil/>



Es gibt einige generelle Leitlinien für Softwaretester, eine Testphilosophie die allen Softwaretest-Aktivitäten zugrunde liegt und als gemeinsame Richtschnur in jedem Testprojekt, ob groß oder klein, vorhanden sein sollte. Diese Grundsätze verhelfen zu einem gemeinsamen Verständnis und einer richtigen Einordnung der Testaktivitäten.

2.1 Testen zeigt die Anwesenheit von Fehlern

Testaktivitäten schlagen fehl und erzeugen Fehler. Das bedeutet im Umkehrschluss: Je höher die Testabdeckung ist, desto geringer ist das Risiko, dass noch unentdeckte Fehler im Testobjekt vorhanden sind. Testen zeigt aber nicht, dass die Software fehlerfrei ist, selbst wenn keine Fehler nachgewiesen werden. Testen ist kein mathematischer Beweis.

Hin und wieder wird der Tester angegriffen, wenn ein Fehler nicht während des Tests gefunden wurde, sondern erst nach dem Rollout. In diesem Fall ist es wichtig, das Nichtentdecken des Fehlers zu analysieren, aber auch darauf hinzuweisen, dass die Testabdeckung bewusst begrenzt war. Vielleicht wurde im Moment der Testplanung ein Geschäftsvorfall für nicht relevant genug erachtet, um im Test nachgewiesen zu werden, und/oder das vorhandene Budget reichte nicht aus, die Testabdeckung entsprechend zu erweitern. Beim Softwaretest ist es wichtig, aus den begrenzten Mitteln so viel Qualität wie möglich zu erreichen, aber keine Testabdeckung von 100 %.

2.2 Vollständige Tests sind unmöglich

Softwaretests beziehen sich immer auf die Betrachtung von Stichproben, es werden nicht alle möglichen Eingabeparameter und deren Kombinationen mit allen möglichen Vorbedingungen getestet, sondern diejenigen, die am ehesten praxisrelevant sind. Der Testaufwand richtet sich dabei nach Priorität und Risiko. Der Testaufwand ist dabei immer mit dem Testnutzen abzugleichen. Wo es sinnvoll erscheint, sollten Tests automatisiert werden um die Testabdeckung zu steigern. Tester werden oft dafür angegriffen, dass sie etwas nicht bemerkt haben, was dann beim Kunden oder beim Endanwender festgestellt wurde. Der Tester muss in diesem Fall begründen können, warum dieser Test nicht durchgeführt wurde und warum er nicht vorgesehen war. Der Tester sollte das aber nicht als persönliche Schuldzuweisung verstehen. Teilweise wird manchmal ein verheerender Umkehrschluss gezogen: Wozu leisten wir uns im Unternehmen eine teure Testabteilung, wenn sie die Fehler trotzdem nicht entdeckt? Ohne Tester hätten wir dasselbe (desolate) Ergebnis gehabt, aber nicht so viel Geld dafür bezahlen müssen. In diesem Fall empfiehlt sich immer die Argumentation, dass ohne Softwaretest die Kosten noch wesentlich höher und das Desaster noch wesentlich größer gewesen wäre, weil dann auch andere Fehler nicht aufgefallen wären.

Ein Test dient zur Feststellung, ob ein Programm wie in der Spezifikation festgehalten funktioniert und nichts anderes tut. Ein Test zeigt aber nicht, ob die Spezifikation wirklich sinnvoll ist. Man kann also erfolgreich getestet haben und trotzdem ein Produkt entwickelt haben, das an den Vorstellungen und Wünschen der Anwender vorbeigeht.

2.3 Mit dem Testen frühzeitig beginnen

Mit dem Test sollte so früh wie möglich im Software- bzw. Systemlebenszyklus begonnen werden. Selbst wenn noch keine testfähige Software zur Verfügung steht, kann man bereits die Requirements analysieren, das Testkonzept schreiben, Testprozeduren dokumentieren und die nötigen Umweltvoraussetzungen für den Test schaffen damit man gut vorbereitet ist, wenn die erste lauffähige Version zur Verfügung steht. Systematisches Testmanagement begleitet die Testaktivitäten von Anfang an und in jeder Entwicklungsstufe. Dadurch lassen sich Fehler bereits in den Entwicklungsstufen aufdecken, in denen sie entstanden sind und werden nicht erst in späteren Phasen gefunden. Wenn Fehler frühzeitig im Lebenszyklus des Projekts gefunden wurden, verkürzt das Korrekturzyklen und vermindert Integrationsaufwand. Grundsätzlich ist es vorteilhaft, wenn so viele Vorbereitungsaktivitäten wie möglich schon vor dem Test getroffen wurden. Andererseits sieht man manche Features erst wenn man die Software tatsächlich einem Test unterzogen hat, auch hier gibt es also eine Grenze. Oft wird aber gerade in der Phase, in der man noch keine testfähige Anwendung zur Verfügung hat, wertvolle Zeit versäumt und in einer späteren Phase kurz vor Lieferung des Endprodukts wird es dann umso hektischer im Team.

2.4 Beim Testen flexibel sein

Wenn in einem bestimmten Softwaremodul ein Fehler gefunden wurde, sind erfahrungsgemäß weitere Fehler nicht weit. Fehler treten in der Regel nicht gleichmäßig verteilt über alle Komponenten auf, viel wahrscheinlicher ist die Häufung von Fehlern in einzelnen Komponenten. Für das Testen heißt das, dass flexibel auf solche erkannten Häufungen eingegangen werden muss. So wichtig eine exakte Testplanung ist, so wichtig ist es darauf zu achten, kreativ mit neuen Herausforderungen umzugehen. Das zeigt sich vor allem in explorativen Tests, also Tests die durch Ausprobieren von Fehlerfällen oder neuen, nicht vorgesehenen Kombinationen, durchgeführt werden. Tester müssen eine gewisse Phantasie entwickeln können und sich auch in den Nutzer hineinversetzen, der das System nicht streng nach der vorgegebenen Anweisung benutzt. Der Tester soll gerade auch die Funktionen genau untersuchen, die nicht exakt spezifiziert sind.

2.5 Tests müssen geplant werden

Um zu verlässlichen Aussagen zu Kosten und Terminen zu kommen, sind die Testfälle aufgrund der Komplexität der Anwendung zu bewerten und daraus Tests abzuleiten. Dabei muss man die Anzahl der voraussichtlich benötigten Testzyklen, den Umfang der Testbeschreibung und die Dauer für die Durchführung der Tests berücksichtigen und mit den vorhandenen Ressourcen abstimmen. Gerade dann wenn es mehrere voneinander abhängige Projekte im Unternehmen gibt, ist es wichtig, eine verlässliche Testplanung zu haben und sie jederzeit bei neuen Erkenntnis zu korrigieren und zu aktualisieren.

2.6 Testfälle regelmäßig prüfen und reviewen

Neue Anforderungen, geänderte Umgebungsvariablen und erweiterte Szenarien erfordern neue Testfälle. Beim Testen darf man also nicht nur buchstabengetreu der vorhandenen Testspezifikation folgen, sondern sie kritisch betrachten, sie ergänzen und aktualisieren wo es vonnöten ist. Unerfahrene Tester und eine fehlende Testmethodik führen zu redundanten und fehlenden Testfällen. Mehrfache Tests (evtl. in verschiedenen Testphasen) für dieselbe Funktion führen zu Kostensteigerung und Terminverzug. Fehlende Testfälle führen zu mangelnder Testabdeckung und damit zu einem erhöhten Risiko fehlerhafter Software und geringerer Qualität des Endprodukts. Gerade erfahrene Tester müssen dazu angehalten werden, ihre Testfälle exakt zu dokumentieren, damit das Know-how und die Kreativität mit denen mittels explorativer Testfälle zusätzliche Konstellationen oder Seiteneffekte nachgewiesen wurde auch für künftige Versionen und Projekte produktiv genutzt werden kann. Ein strukturiertes Testmanagement und eine methodische Planung von Testaktivitäten sind entscheidend für den Testerfolg [COMP2012]. Aus diesem Grund ist

es auch so wichtig, die einzelnen Testphasen voneinander abzugrenzen und sich im Vorfeld genau zu überlegen, welche Funktionalität an welcher Stelle mit welcher Testumgebung sinnvollerweise zu testen ist. Dabei ist es auch entscheidend, immer wieder die Testfälle kritischen Reviews zu unterziehen. Reviews mögen anfangs als Kostentreiber erscheinen, aber sie sind wichtig für den Erfolg des Endprodukts und dienen dadurch in Summe sogar zur Einsparung von Kosten.

2.7 Das Testumfeld berücksichtigen

Der Test ist an Einsatzgebiet und Umgebung des zu testenden Systems anzupassen. Die Testabdeckung, die Testendekriterien und der Testumfang sind bei jedem System neu zu bewerten und neu zu definieren. Die Konfiguration des Testsystems setzt entscheidende Rahmenbedingungen für die Durchführung der Testaktivitäten. Die spezifische Systemarchitektur, der Einsatzzweck und Anzahl der Anwender beeinflussen die Testaktivitäten. Sicherheitskritische Systeme oder Anwendungen in der Medizintechnik verlangen detailliertere und umfassendere Prüfungen als eine graphische Oberfläche einer App oder einer Spielesoftware. Auch die zu tolerierenden Fehler richten sich nach Art der getesteten Applikation.

2.8 Unterschiedliche Personen als Tester und Programmierer

Das Testen von Software ist eine eher destruktive Aufgabe. Man muss an das Produkt anders herangehen als der Entwickler. Dazu kommt, dass man das eigene Produkt naturgemäß milder betrachtet als ein neutraler Beobachter. Ein Programmierer, der sein eigenes Programm testet, findet prinzipiell weniger Fehler, da er versucht zu beweisen, dass sein Programm keine Fehler enthält. Fehler, die durch falsch verstandene Anforderungen entstehen, können nur durch außen stehende Personen aufgedeckt werden. Gerade weil zum Testen auch teilweise andere theoretische Grundlagen beherrscht werden müssen als für die Entwicklung von Software, sollten Tester und Programmierer aus unterschiedlichen Mitarbeitern bestehen.

2.9 Dokumentation und Nachvollziehbarkeit der Tests

Eingangsdaten, erwartete Ergebnisse und die erhaltenen Daten müssen dokumentiert werden. Das dient der Reproduzierbarkeit der Testaktivitäten: Bei gleichen Eingangsdaten muss der Test auf die gleichen Ergebnisse kommen.

Wenn dem nicht so ist, deutet das darauf hin, dass es noch nicht erhobene Parameter und Nebenbedingungen gibt, die für den Softwaretest ebenfalls zu berücksichtigen sind.

2.10 Operation gelungen, Patient tot?

Selbst wenn man strikt nach den Systemanforderungen testet und keine Fehler findet, heißt es noch lange nicht, dass das System den Nutzererwartungen entspricht. Wenn die Anforderungen an der Realität vorbeigehen oder zu umständlich sind, wird das System keine Nutzerakzeptanz finden, obwohl man doch eigentlich alles richtig gemacht hat. Daher empfiehlt es sich, den Endnutzer frühzeitig einzubinden, wo es möglich ist Prototypen zu entwickeln und das System im Pilotbetrieb zu validieren.

Gerade wenn man beim Test strikt nur von Requirements ausgeht, ohne nach links und rechts zu sehen und das Umfeld in der Praxis hinreichend berücksichtigt, läuft man Gefahr, dass man zwar formal der Software einem umfassenden Test unterzogen hat, aber dann am Ende ein Produkt ausliefert, das an den wahren Wünschen des Anwenders vorbeigeht.

2.11 Ermittlung der Fehlerrate

Erfahrungsgemäß treten Fehler oft gehäuft an bestimmten Abschnitten auf. Daher sollten Fehler vor allem in den Code-Segmenten gesucht werden, in denen schon viele Fehler aufgetreten sind. Beim Testen ist es hilfreich, einen Anhaltspunkt zu haben, wie viele Fehler eine bestimmte Software-Komponente wahrscheinlich beinhaltet. Um solche Schätzzahlen angeben zu können, gibt es mehrere Gleichungen als Ergebnis empirischer Untersuchungen.

Um diese Formeln zu bestimmen, wurde eine Vielzahl unterschiedlicher abgeschlossener Software-Projekte ausgewertet und die dabei erhobenen Daten mit einer mathematischen Relation angenähert. Werden diese Formeln auf ein neues Projekt angewendet, erhält man einen ungefähren Schätzwert zur Fehlererwartung.

Nach Basili kann man die Zahl der zu erwartenden Fehler wie folgt ermitteln:

$$F = 4,04 + 0,0014 \text{ LOC}^{4/3}.$$

Dabei ist LOC die Programmlänge in Codezeilen (*engl. lines of code*) und F die zu erwartende Fehlerzahl. Der geplottete Graph dazu ist in Abb. 2.1 zu sehen.

Um den Wert für LOC zu ermitteln, zählt man jede Zeile im Code des Testobjekts außer Leerzeilen und Kommentaren. Nach obiger Formel erhält man also für ein Programm mit 1000 „effektiven“ Programmzeilen eine erwartete Fehlerzahl von **17,7 Fehlern**.

Eine andere Formel zur Abschätzung der zu erwartenden Fehlerzahl lautet:

$$F_r = C_1 + C_2 (\text{SCHG} / \text{KLOC}) - C_3 \text{ISKL} - C_4 (\text{DOCC} / \text{KLOC}).$$

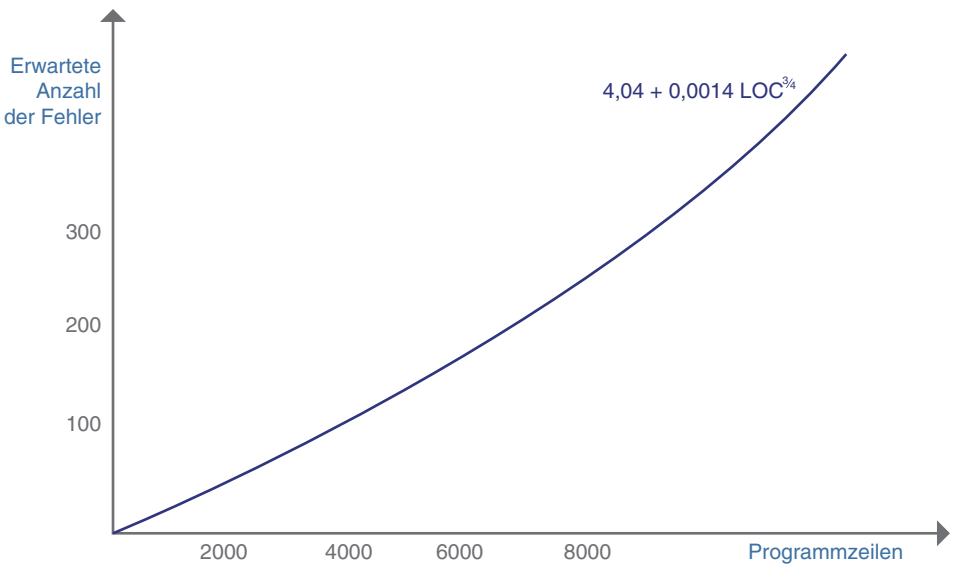


Abb. 2.1 Graph zum Erwartungswert der Fehlerzahl

Die Konstanten und Variablen in dieser Gleichung sind wie folgt definiert:

F_r	Fehlerrate pro 1000 Programmzeilen (LOC)
C_1	Konstante 67,98
C_2	Konstante 0,46
C_3	Konstante 9,69
C_4	Konstante 0,08
SCHG	Anzahl der Änderungen am Lastenheft während der Entwicklung
KLOC	Codeumfang in 1000 Programmzeilen (LOC)
ISKL	durchschnittliche Erfahrung des Entwicklungsteams mit einer Programmiersprache in Jahren
DOCC	Anzahl der geänderten oder neu erstellten Seiten in Designdokumenten [THAL2000]

Beispiel

In einem Software-Projekt wurde eine Komponente von 2000 Programmzeilen von einem Entwicklungsteam geschrieben, das seit durchschnittlich drei Jahren mit Java programmiert. Während der Entwicklung musste viermal das Lastenheft geändert werden. Die vorliegende Komponente liegt in der fünften Version vor, d. h. das ursprüngliche Designdokument wurde viermal geändert.

Die Fehlerrate berechnet sich dabei wie folgt:

SCHG	= 4
KLOC	= 2
ISKL	= 3
DOCC	= 4

Die erwartete Fehlerzahl der Software-Komponente (1000 Programmzeilen) ist also nach dieser Formel **39,99 (also 40) Fehler**.

Die tatsächlichen Verhältnisse in einem konkreten Projekt können möglicherweise immer noch stark von diesen Zahlen abweichen. Man erhält aber ein Gespür dafür, ob man sich mit den erzielten Ergebnissen auf dem richtigen Weg befindet und bekommt die Sicherheit, ob man wahrscheinlich schon hinreichend getestet hat oder besser noch ausgiebiger testen sollte.

Ein anderer Ansatz ist, die Anzahl der Entwicklerstunden zusätzlich als Basis zur Ermittlung der zu erwartenden Fehler zu verwenden. Die Ermittlung über die Lines of Code hat manchmal das Problem, dass in 3 Zeilen Programmcode mitunter mehr Logik vorhanden ist als in 100 Zeilen.

2.12 Testnutzen

Testen dient vor allen Dingen zwei Zwecken:

- a) zur Qualitätssicherung und Unterstützung der Entwickler:

Tests sollen während Realisierung oder Umsetzung die Anzahl der Fehler möglichst gering halten und dafür sorgen, dass die Entwickler bereits während des laufenden Projekts aus erkannten Fehlern lernen können und entsprechende Verbesserungsmaßnahmen einleiten. Im Entwicklungszyklus einer IT-Lösung muss die Qualitätssicherung von Anfang an in den Projektablauf eingebunden sein und darf sich nicht nur auf eine Testphase zum Projektabschluss beschränken.

- b) zur Abnahme der entwickelten IT-Lösung:

Mit der Abnahme bestätigt der Auftraggeber, dass seine Anforderungen erfüllt sind, die in Projektauftrag bzw. Lastenheft oder Kundenspezifikation dokumentiert sind. Die Abnahme ist Voraussetzung für den Projektabschluss [SCHL2009].

Dabei besteht für das Testmanagement immer die Anforderung, zwei widerstrebende Ziele zu erfüllen:

- Wunsch nach standardisierten Testprozessen,
- individuelle Anforderungen eines bestimmten Projekts.

Deswegen müssen systematisches Vorgehen, Einhaltung von Qualitätsgrundsätzen und konsequente Klärung und Umsetzung testrelevanter Themen in gleicher Weise beachtet werden.

Die Managementebene misst den Erfolg einer Aufgabe anhand des dafür notwendigen Aufwands. Der Mehrwert einer Maßnahme muss demnach größer sein als dessen Kosten. Diese Aussage lässt sich auch auf den Nutzen von Testverfahren in Form des Return-on-Invest (ROI) abbilden.

$$\text{ROI} = (\text{Testnutzen} - \text{Testkosten}) / \text{Testkosten}$$

In die Berechnung des Testnutzens fließen dabei folgende Größen ein:

- vermiedene Fehler (Eintrittswahrscheinlichkeit · Kosten des Fehlers).

Erfahrungsgemäß beträgt der prozentuale Anteil der vermiedenen Fehler in unterschiedlichen Projekten zwischen 18 und 40 %, die Varianz hat also eine sehr große Spanne.

2.13 Testphilosophien

Um die Rolle des Testers zu bewerten, gibt es unterschiedliche Meinungen, die sich in sog. „Testschulen“ wiederfinden.

Die **Analytical School** versteht den Softwaretest als eine technische Disziplin, die wissenschaftliche Methoden anwendet, um die korrekte Funktionsweise von Software aufzuzeigen.

Beispiele für Methoden und Werkzeuge dieser Schule sind z. B. die Grenzwertanalyse und die Messung der Testüberdeckung.

Die **Factory School oder Standard School** möchte mit einem effizienten Softwaretest Kalkulationssicherheit und Wiederholbarkeit erreichen. Die Bestimmung der Produktqualität steht im Mittelpunkt und ist für den Projektfortschritt entscheidend. Durch eine weitgehende Standardisierung der Vorgehensweise können auch weniger qualifizierte Mitarbeiter für den Softwaretest eingesetzt werden.

In der **Quality School** steht die Einhaltung definierter Quality Gates im Mittelpunkt, die anhand der erfolgreichen Durchführung der Testprozesse bewertet werden. Eine hohe Produktqualität ist das primäre Ziel.

Die **Context-Driven School** ist die erste der beiden Schulen, die sich explizit auf agile Entwicklung bezieht. Hier entscheiden Tester in Zusammenarbeit mit dem Kunden, welche Eigenschaften der Software geprüft werden. Ziel ist auch, die „wahren“ Anforderungen an die Software zu ermitteln. Dadurch orientiert sich die Testabdeckung stark an den wesentlichen Kriterien der Software. Zu dieser Schule zählt auch das explorative Testen [PIIN2011].

Die **Agile School** versteht Testen als den Nachweis, ob die in User Stories definierten Anforderungen an die Software tatsächlich umgesetzt wurden. Der Test ist daher immer aus der Rolle des Kunden zu definieren. Eine Automatisierbarkeit und Wiederholbarkeit der Testfälle ist für diesen Ansatz erforderlich. Das „Test Driven Development“ wird der Agile School zugerechnet. Beim Test Driven Development wird der Test definiert, bevor der Programmcode geschrieben wird. Wenn der Code den Test bestanden hat, wird der entsprechende Test angepasst. Das führt zu besseren Unittests und einer besseren Testbarkeit der Anwendung.

Diese einzelnen Richtungen existieren in der Praxis nebeneinander und nicht in Reinkultur. Bei unterschiedlichen Befragungen unter Testexperten erzielten Quality, Analytical und Context-Driven School vergleichbare Zustimmungsraten.

Die Analytical School bezieht sich eher auf die Basisprozesse und sollte daher nicht aus der Rolle des Testmanagements betrachtet werden.

Factory School und Quality School setzen eine geordnete Planung und definierte Planungsschritte voraus. Daher werden diese Schulen auch als planungsbasiert bezeichnet. Context-Driven und Agile School beziehen sich auf agile Organisationen und werden als erfahrungsbasierte Testverfahren bezeichnet.

Literatur

- [COMP2012] <http://www.computerwoche.de/a/die-zehn-groessten-fehler-im-software-test-management,1877489>
- [PIIN2011] http://pi.informatik.uni-siegen.de/stt/30_3/01_Fachgruppenberichte/TAV/08_TAV30_Testmanagement-Dschungel_abstract_HH_04.pdf
- [SCHL2009] <http://www.bernhardschloss.de/images/Testmanagment%20in%20IT-Projekten.pdf>
- [THAL2000] Thaller GE (2000) Software-Metriken. Technik, Berlin

Normen bringen einen hohen betriebs- und volkswirtschaftlichen Nutzen, da sie gemeinsame Standards schaffen und damit eine Voraussetzung bilden für den erfolgreichen Wettbewerb unterschiedlicher Unternehmen. Normen leisten einen wichtigen Beitrag zur Regulierung, indem sie den Staat von technischen Detailregelungen entlasten. Auf der Grundlage der Normen der IEEE und ISO arbeitet das ISTQB an der Standardisierung von Systemtests. Ziel ist die Standardisierung und Vergleichbarkeit von Testprozessen. Der erste Standard für Software-Systemtests war der „IEEE-Standard 829 – Test Documentation“. Dieser Standard wurde 1983 eingeführt und in den Jahren 1987 und 2008 überarbeitet. Die Norm mit dem Titel „Standard for Software and System Test Documentation“ beschreibt den Systemtestprozess und der Testdokumentation. Als Systeme werden dabei nicht nur die Software, sondern auch darunterliegende Hardware-Plattformen verstanden.

Ein voll ausgebauter Testprozess besteht im Sinne des ANSI/IEEE Standard 829 aus folgenden Teilprozessen:

- Testplanung,
- Testfallspezifikation,
- Testentwurf (Design der Testabläufe),
- Testdatenerstellung (Ist und Soll),
- Testausführung,
- Testauswertung,
- Testabnahme.

Allerdings ist es nur selten möglich und sinnvoll, den gleichen Testprozess für mehrere Projektarten zu verwenden. Dafür sind die einzelnen Aufgabenstellungen zu unterschiedlich. Jedes Projekt erfordert eine eigene Ausprägung des Prozesses. Das Tailoring der Testprozessmodelle gehört zu den wesentlichen Aufgaben eines Testmanagers.

3.1 IEEE 829 Standard for Software Test Documentation

Im IEEE 829 Standard for Software Test Documentation werden acht Dokumente beschrieben, die in drei Kategorien unterteilt sind:

- **Übersicht**
 1. **Testkonzept** (test plan): Das Testkonzept bestimmt Abgrenzung, Vorgehensweise, Mittel und Ablaufplan der Testaktivitäten. Es bestimmt die Elemente und Produktfunktionen, die getestet werden sollen, die Testaufgaben, die durchgeführt werden müssen, das verantwortliche Personal für jede Aufgabe und das Risiko, das mit dem Konzept verbunden ist.
- **Test-Spezifikation** (test specification)
 1. Die **Testentwurfsspezifikation** (test design specification) verfeinert die Beschreibung der Vorgehensweise für das Testen der Software. Sie identifiziert die Produktfunktionen, die von den Tests abgedeckt werden müssen. Sie beschreibt weiterhin die Testfälle und Testabläufe, die benötigt werden, um Tests zu bestehen und spezifiziert die Bestehens- oder Verfehlenskriterien der einzelnen Produktfunktionen.
 2. Die **Testfallspezifikation** (test case specification) dokumentiert die zu benutzenden Eingabewerte und erwarteten Ausgabewerte. Testfälle sind vom Testdesign getrennt. Dies erlaubt die Verwendung der Testfälle in mehreren Designs und die Wiederverwendung in anderen Situationen.
 3. Die **Testablaufspezifikation** (test procedure specification) ist die Beschreibung aller Schritte zur Durchführung der spezifizierten Testfälle und Implementierung des zugehörigen Testdesigns.
- **Testbericht** (test reporting)
 1. Der **Testobjektübergabebericht** (test item transmittal report) beschreibt die Übergabe der Testfälle für den Fall, dass getrennte Entwicklungs- und Testteams eingebunden sind oder für den Fall, dass ein offizieller Zeitpunkt für den Beginn einer Testausführung erwünscht ist.
 2. Das **Testprotokoll** (test log) dient zur Aufzeichnung der Ereignisse während einer Testausführung.
 3. Der **Testabweichungsbericht** (test incident report) dokumentiert alle Ereignisse, die während einer Testausführung auftreten und weitere Nachprüfungen erfordern.
 4. Der **Testabschlussbericht** (test summary report) fasst die Testaktivitäten zusammen, die mit einer oder mehreren Testentwurfsspezifikationen zusammenhängen.

3.2 ISO-Standard 9126

Der ISO-Standard 9126 behandelt Softwarequalität im Allgemeinen und schreibt vor, was für die Messung von Qualität erforderlich ist. Zuverlässigkeit, Performance und Sicherheit sind Qualitätsziele, die durch Testprozesse bestätigt werden.

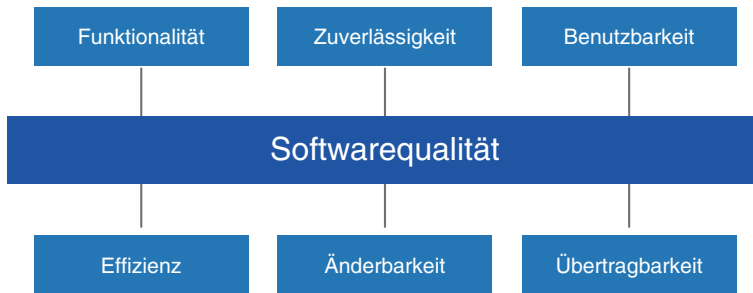


Abb. 3.1 Qualitätsmerkmale von Softwaresystemen

Abb. 3.1 zeigt die aufgeführten Qualitätsmerkmale, Teilmerkmale werden im Anhang der Norm nur als Vorschläge aufgeführt.

Zu den einzelnen Qualitätsmerkmalen gehören folgende Kriterien:

- **Funktionalität:** Vorhandensein der Funktionen mit festgelegten Eigenschaften. Diese Funktionen erfüllen die definierten Requirements.
 - Angemessenheit: Eignung von Funktionen für spezifizierte Aufgaben, zum Beispiel aufgabenorientierte Zusammensetzung von Funktionen aus Teilfunktionen.
 - Korrektheit: Lieferung der richtigen oder vereinbarten Ergebnisse oder Wirkungen, zum Beispiel die benötigte Genauigkeit von berechneten Werten.
 - Interoperabilität: Fähigkeit, mit vorgegebenen Systemen zusammenzuwirken.
 - Sicherheit: Fähigkeit, unberechtigten Zugriff, sowohl versehentlich als auch vorsätzlich, auf Programme und Daten zu verhindern.
 - Ordnungsmäßigkeit: Merkmale der Software, die bewirken, dass die Software anwendungsspezifische Normen oder Vereinbarungen oder gesetzliche Bestimmungen und ähnliche Vorschriften erfüllt.
 - Konformität: Fähigkeit des Softwareprodukts, Standards, Konventionen oder gesetzliche Bestimmungen und ähnliche Vorschriften bezogen auf die Funktionalität einzuhalten.
- **Zuverlässigkeit:** Fähigkeit der Software, ihr Leistungsniveau unter festgelegten Bedingungen über einen festgelegten Zeitraum zu bewahren.
 - Reife: geringe Versagenshäufigkeit durch Fehlerzustände.
 - Fehlertoleranz: Fähigkeit, ein spezifiziertes Leistungsniveau bei Software-Fehlern oder Nicht-Einhaltung ihrer spezifizierten Schnittstelle zu bewahren.
 - Wiederherstellbarkeit: Fähigkeit, bei einem Versagen das Leistungsniveau wiederherzustellen und die direkt betroffenen Daten wiederzugewinnen. Zu berücksichtigen sind die dafür benötigte Zeit und der benötigte Aufwand.
 - Konformität: Grad, in dem die Software Normen oder Vereinbarungen zur Zuverlässigkeit erfüllt.

- Benutzbarkeit: Aufwand, der zur Benutzung der Software erforderlich ist (Einarbeitungs- und Schulungsaufwand) und individuelle Beurteilung der Benutzung durch eine festgelegte oder vorausgesetzte Benutzergruppe.
 - Verständlichkeit: Aufwand für den Benutzer, das Konzept und die Anwendung zu verstehen.
 - Erlernbarkeit: Aufwand für den Benutzer, die Anwendung zu erlernen (zum Beispiel Bedienung, Ein-, Ausgabe).
 - Bedienbarkeit (Usability): Aufwand für den Benutzer, die Anwendung zu bedienen.
 - Attraktivität: Anziehungskraft der Anwendung gegenüber dem Benutzer.
 - Konformität: Grad, in dem die Software Normen oder Vereinbarungen zur Benutzbarkeit erfüllt.
- Effizienz: Indexwerte zum Verhältnis zwischen Leistungsniveau der Software und eingesetzten Betriebsmitteln und zum Verhältnis zwischen dem Leistungsniveau der Software und dem Umfang der eingesetzten Betriebsmittel unter festgelegten Bedingungen.
 - Zeitverhalten: Antwort- und Verarbeitungszeiten sowie Durchsatz bei der Funktionsausführung.
 - Verbrauchsverhalten: Anzahl und Dauer der benötigten Betriebsmittel bei der Erfüllung der Funktionen. Ressourcenverbrauch, wie CPU-Zeit, Festplattenzugriffe usw.
 - Konformität: Grad, in dem die Software Normen oder Vereinbarungen zur Effizienz erfüllt.
- Wartbarkeit/Änderbarkeit: Messung des Aufwands zur Durchführung vorgegebener Änderungen an der Software. Bei Änderungen handelt es sich in diesem Zusammenhang um Korrekturen, Verbesserungen oder Anpassungen an Änderungen der Umgebung, der Anforderungen oder der funktionalen Spezifikationen.
 - Analysierbarkeit: Aufwand, um Mängel oder Ursachen von Versagen zu diagnostizieren oder um änderungsbedürftige Teile zu bestimmen.
 - Modifizierbarkeit: Aufwand zur Ausführung von Verbesserungen, zur Fehlerbeseitigung oder Anpassung an Umgebungsänderungen.
 - Stabilität: Wahrscheinlichkeit des Auftretens unerwarteter Wirkungen von Änderungen.
 - Testbarkeit: Aufwand, der zum Test der neuen oder geänderten Software notwendig ist.
 - Konformität: Übereinstimmungsgrad mit Normen, gesetzlichen Vorgaben oder Vereinbarungen zur Änderbarkeit.
- Übertragbarkeit (Portierbarkeit): Eignung der Software, von der Umgebung in eine andere übertragen werden zu können. Als Umgebung wird in diesem Zusammenhang die organisatorische Umgebung, Hardware- oder Software-Konfiguration bezeichnet.
 - Anpassbarkeit: Fähigkeit der Software, diese an verschiedene Umgebungen anzupassen.

- Installierbarkeit: Aufwand, der zum Installieren der Software in einer festgelegten Umgebung notwendig ist.
- Koexistenz: Fähigkeit der Software neben einer anderen mit ähnlichen oder gleichen Funktionen zu arbeiten.
- Austauschbarkeit: Möglichkeit, diese Software anstelle einer spezifizierten anderen in der Umgebung jener Software zu verwenden, sowie der dafür notwendige Aufwand.
- Konformität: Erfüllungsgrad für Normen oder Vereinbarungen zur Übertragbarkeit.

3.3 ISO-Standard IEC-29119

Die neue ISO-Norm IEC-29119 definiert Testbegriffe, Testprozesse, Testdokumentation und einzelne Testtechniken. Von organisatorischen Teststrategien über projektspezifische Testprozesse und Testpläne mit dem Teststufen, Testfallspezifikationen, Testentwurf, Testdurchführung und Testberichtswesen soll diese Norm jeglichen Software-Entwicklungs- und Wartungsprozess unterstützen.

Die Norm besteht aus folgenden Teilen:

- ISO/IEC 29119-1: Concepts & Definitions,
- ISO/IEC 29119-2: Test Processes,
- ISO/IEC 29119-3: Test Documentation,
- ISO/IEC 29119-4: Test Techniques,
- ISO/IEC 29119-5: Keyword Driven Testing [[SOFT2013](#)].

Zusätzlich besteht auf Basis des Testprozesses, der in ISO/IEC/IEEE 29119-2 beschrieben ist, die Norm

- ISO/IEC 33063: Process Assessment Model.

Der Testprozess ist in der ISO-IEC 29119-2 in folgende Superprozesse aufgeteilt:

- organisatorische Testprozesse,
- Testmanagementprozesse,
- statische Testprozesse,
- dynamische Testprozesse.

Jeder Superprozess enthält wiederum Unterprozesse, z. B. statische Analyse, Inspektionen und Reviews unter dem statischen Testprozess und Testentwurf, Testaufbau, Testausführung, Testauswertung und Problemberichtswesen unter dem dynamischen Test.

Diese Norm ersetzt die bestehenden Normen:

- IEEE 829 Test Documentation,
- IEEE 1008 Unit Testing,
- BS 7925-1 Vocabulary of Terms in Software Testing,
- BS 7925-2 Software Component Testing Standard.

Für den Softwaretest in besonders sicherheitskritischen Systemen sind weitere Normen zu beachten. So wird beispielsweise im Umfeld medizinischer Software durch die allgegenwärtigen grundlegenden Anforderungen der MDD 93/42 EWG und durch die damit verknüpften einschlägigen Normen ein strenger Rahmen gesetzt. Dieser erstreckt sich von der Validierung der Testprozesse über den Testentwurf, die Dokumentation der Tests, Verknüpfungen mit Anforderungs-, Fehler-, und Risikomanagement bis in den Abschluss der Testaktivitäten hinein. Welche Normen im Softwaretest berücksichtigt werden müssen und wie der Softwaretest geplant und durchgeführt werden muss, hängt zudem davon ab, ob eine Testabteilung nur verifizierende oder verifizierende und zusätzlich validierende Aufgaben hat. Unter Verifikation versteht man einen Prozess, der für ein Programm oder ein System sicherstellt, dass es zu einer Spezifikation „konform“ ist („Ist das System richtig gebaut?“). Im Gegensatz dazu steht die Validierung, d. h. die dokumentierte Plausibilisierung, dass ein System die Anforderungen in der Praxis erfüllt („Funktioniert das System richtig?“). Eine weitere Einflussgröße bildet das gelebte (und dokumentierte) Software-Entwicklungsmodell.

Literatur

[SOFT2013] <http://www.softwaretestingstandard.org/>



Rollen und Verantwortlichkeiten im Testmanagement

4

In einem Testprojekt sind für bestimmte Aktivitäten definierte Rollen festgelegt. Von Projekt zu Projekt sind diese Rollen abhängig von den verfügbaren Mitarbeitern jeweils neu festzulegen. Es sind dabei auch Mischformen möglich oder die Vereinigung mehrerer Rollen in einer Person, aber genauso gut die Aufteilung der Verantwortlichkeiten einer Rolle auf mehrere Mitarbeiter.

4.1 Testmanager

Der Testmanager ist in der Regel der Teilprojektleiter für das Teilprojekt „Test“. Daher deckt er neben den Projektmanagementaspekten der Teilprojektleitung, die spezifischen Themenfelder des Tests mit ab und koordiniert die entsprechenden Aufgaben:

- Planung und Steuerung von Softwaretests,
- Ermittlung von Testaufwand, Sicherstellen der Termine unter Beachtung von Kosten- und Budgetrestriktionen,
- Erstellung der Testplanung und des Testkonzepts,
- Koordination der Testaktivitäten und Teststeuerung: Beschreibung der Testspezifikationen, Durchführung der Testfälle, Sicherstellung der Testressourcen,
- Erstellung von Metriken und Reporting der Zwischen- und Endergebnisse,
- Ermittlung von Testrisiken und Eskalation an das Management.

Ob man einen Testmanager benötigt oder ob man die Aufgaben zwischen Projektleiter und Testkoordinator bzw. Tester sinnvoll aufteilen kann, hängt von der Projektgröße und der Komplexität des Softwareprojekts ab.

Ein Testmanager muss über Führungskompetenz und Erfahrungen im Projektmanagement verfügen. Neben fachlichen Erfahrungen im Softwaretest muss er strukturiert arbeiten, darf nicht den Überblick verlieren, benötigt eine eigenverantwortliche und systematische Arbeitsweise mit hohem Qualitätsanspruch und gute Kommunikationsfähigkeiten.

4.2 Testkoordinatoren

Für definierte Teilaufgaben oder die Leitung fachlicher oder technischer Test-Teams werden teilweise Testkoordinatoren definiert. Sie unterstützen den Testmanager und haben ähnliche Aufgaben wie der Testmanager, nur eben für einen Teilbereich.

Fachliche oder technische Testteams werden teilweise von einem Testkoordinator und nicht vom Testmanager geleitet. Auch das Abweichungsmanagement oder die Definition der Testobjekte kann an den Testkoordinator übertragen werden. Alternativ können Teile dieser Zusatzthemen auch bei einem „Testumgebungskoordinator“ angesiedelt sein, der in erster Linie für die Konfiguration der Testinfrastruktur und die Verwaltung der Testdaten verantwortlich ist.

Sofern es im Unternehmen mehrere parallel arbeitende Testprojekte gibt, kann es sinnvoll sein, die Rolle eines Testumgebungskoordinators als Querschnittsfunktion im Unternehmen zu bündeln. Für den Test bestimmter Geschäftsprozesse oder für den Test einer Systemintegration sind oftmals viele parallele Tests erforderlich, die auf eine bestimmte Testkonfiguration zugreifen. Hier besteht die Herausforderung, jedem Teilprojekt ein Zeitfenster zuzuweisen, und definierte Rahmenbedingungen bzw. Voraussetzungen für alle Beteiligten sicherzustellen. Zu beachten ist, dass durch die Tests von Projekt A nicht die Daten so verändert werden, dass Projekt B die erforderliche Testkonfiguration und die definierte Testumgebung nicht mehr vorfindet.

Geschäfts- und Supportprozesse müssen bereits bei der Testphase in die Testaktivitäten mit einbezogen werden. Ansonsten scheitert dann häufig die Einführung des Softwareprodukts in der Praxis.

Das Berufsbild des IT-Testkoordinators bzw. der IT-Testkoordinatorin (laut „Bundesagentur für Arbeit“) besteht aus folgenden Aufgaben:

- durch angemessene und aussagekräftige Tests den Software- und Hardware-Entwicklungsprozess begleiten und unterstützen,
- begleitende Tests für den Software- und Hardware-Entwicklungsprozess konzipieren,
- Testziele, geplante Teststufen und einen groben Ablaufplan in enger Zusammenarbeit mit Kunden und Spezialisten aus den Bereichen Systemanalyse, Systementwicklung und Produktion festlegen,
- Umfang der Tests und Testendekriterien, Vorgehensweise für den Test, Testobjekte (die eigentlichen Testgegenstände) festlegen; zu testende Qualitätseigenschaften identifizieren; Ressourcenplanung, konkrete Terminplanung zur Durchführung der Testaktivitäten durchführen,

Tab. 4.1 Ressourcen im Testprozess

	März	April	Mai	Juni
Benötigte Ressourcen	4	4	6	4
Verfügbare Ressourcen	4	5	5	4

- automatisierte Testsuites erstellen,
- Testumgebung planen und bereitstellen,
- manuelle und automatisierte Tests auf den Ebenen Unit-, Funktions-, System- und Akzeptanztest unter Verwendung der Testfälle und Testszenarien durchführen,
- Testumgebung aufbauen, Testtreiber und -skripte implementieren,
- Verhalten des Testgegenstands beobachten und protokollieren,
- vermutete Fehler im Testprotokoll festhalten,
- Fehlerlisten erstellen, Fehler nach dem Grad der Auswirkung auf die Funktionsfähigkeit des Testgegenstandes kategorisieren,
- Testbericht einschließlich Testprotokoll erstellen.

Einige dieser Punkte sind in den einzelnen Kapiteln zum Softwaretest beschrieben, und sie fallen an, egal ob man allein für den Softwaretest verantwortlich ist oder nicht.

Als Testkoordinator geht es zusätzlich um die fachliche Führung eines Teams und die Abstimmung der einzelnen Aufgaben und Arbeitspakete. Die Koordination der Testaktivitäten muss sich dabei auch am Projektfortschritt anderer Abteilungen (z. B. Entwicklung, Systemarchitektur) orientieren und personelle Ressourcen mit ihren Stärken/Schwächen (z. B. gewissenhafter Testdurchführung, in der Programmierung von Skripten oder in gewandtem Reporting) optimal einsetzen. Danach muss man den Testaufwand mit diesen Ressourcen abgleichen.

Dabei hat es sich als positiv erwiesen, immer zunächst die Aktivitäten festzulegen ohne Betrachtung der nötigen Ressourcen, und dann entsprechend aufzuteilen. Aus der zeitlichen Verteilung der Kapazitäten ergeben sich die ersten Engpassbetrachtungen.

In Tab. 4.1 erkennt man, dass man mit den Ressourcen im April so weit wie möglich Vorarbeiten tätigen muss, um den Zusatzaufwand im Mai so bald wie möglich vorzuziehen.

Die Betrachtung wird komplexer wenn nun bestimmte Ressourcen für bestimmte Arbeiten spezialisiert sind. Eine weitere Komplexität entsteht, wenn unterschiedliche Projekte zum selben Zeitpunkt bestimmte Fähigkeiten beanspruchen. Sonst kann es vorkommen, dass zu einem bestimmten Zeitpunkt Mitarbeiter mit bestimmten Rollen Überstunden leisten und Wochenendeinsätze fahren müssen, während zu anderen Phasen die vorhandenen Mitarbeiter nicht ausgelastet sind.

4.3 Testdesigner

Testdesigner definieren Testfälle und legen die Reihenfolge der Testdurchführung fest, indem sie die Priorisierung der einzelnen Tests vornehmen. Ein Testdesigner muss über Erfahrungen im Softwaretest und Grundsätze des Software-Engineerings verfügen. Der Testdesigner

beherrscht die eingesetzten Spezifikationsmethoden, kann die Testtools konfigurieren und besitzt das nötige fachliche Hintergrundwissen, um die Anforderungen zu verstehen und inhaltlich zu durchdringen, die er in sinnvolle und exakt definierte Testfälle übersetzen muss. Der Testdesigner muss sich auch über die für das Projekt geforderte Testabdeckung Gedanken machen.

Folgende Aufgabengebiete gehören typischerweise dazu:

- Analyse und Prüfung komplexer Softwaresysteme (Anforderungen, Spezifikationen, Systemmodelle etc.),
- Festlegung der Testbarkeit von Requirements,
- Entwicklung von Testfällen in einem definierten Tool,
- Planung und Steuerung der Testaktivitäten,
- Definition von zu nutzenden Testdaten,
- Überprüfung aller Testdokumentationsunterlagen,
- Optimierung von Arbeitsabläufen und Geschäftsprozessen,
- Aufwandsermittlung für Testkonzeptionen und Testdurchführung,
- Priorisierung von Testfällen.

4.4 Testautomatisierer

Testautomatisierer sind für die Erstellung der Skripte verantwortlich. Erfahrungen in der Software-Entwicklung und der Testdurchführung sind notwendige Voraussetzungen, eigene Erfahrungen als Entwickler sind dabei sinnvoll. Für den Erfolg der Testautomatisierung ist auch eine umfassende Kenntnis der Unternehmensprozesse erforderlich, um die automatisierten Verfahren optimal in den Entwicklungs- und Testprozess zu integrieren. Ansonsten gehen die Testskripte an den zentralen Anforderungen vorbei.

Bei der Testautomatisierung geht es sowohl um reine Programmiertätigkeiten als auch um das Design und die Beschreibung der automatisierten Skripts. Es kann sinnvoll sein, diese Rollen innerhalb der Testautomatisierung zu trennen, je nach Größe und Ausbildungsstand der Mitarbeiter.

4.5 Testsystemadministrator

Der Testsystemadministrator ist verantwortlich für die Konfiguration des Testsystems, die Software, Hardware und die Netzwerkumgebung. Er verwaltet die eingesetzten Tools und Datenbanken.

Dafür sind fundierte Kenntnisse über die aktuelle Hardware, im Bereich Datenbanken und der Installation von Servern und der Softwareverteilung erforderlich. Erfahrungen im Konfigurationsmanagement erleichtern den Einstieg.

4.6 Tester (Testdurchführer)

Tester wirken bei der Dokumentation der Testfälle mit und führen die fachlichen und/oder technischen Tests durch. In der Sonderrolle „Testautomatisierer“ erstellen sie Skripte zur automatisierten Erstellung von Testfällen.

Tester benötigen ein umfassendes Systemverständnis über die Anforderungen, den Entwurf und die Implementierung der Software.

In letzter Zeit sind für den Testdurchführer mehrere Trends zu beobachten:

- Er wird durch automatisierte Skripts ersetzt.
- Seine Rolle wird mehr und mehr in Billiglohnländer ausgelagert.
- Entwicklungen wie Crowdfunding, bei dem viele Tester über das Web zu einem geringen Stundensatz als Nebenbeschäftigung einfache Tests durchführen, nehmen zu.

Andererseits werden für Testszenarien, die komplexe Geschäftsvorfälle abbilden, nach wie vor kreative, erfahrene Tester benötigt, die den Testumfang durch exploratives Testen erweitern.

Oft sind die einzelnen Rollen nicht klar voneinander abgegrenzt, ein Tester kann also grundsätzlich durchaus teilweise Aufgaben eines Testkoordinators übernehmen. Wenn es aber mehrere Beteiligte in einem Testprojekt gibt, empfiehlt sich eine klare Trennung der einzelnen Verantwortungsbereiche.

4.7 Qualifikation für Testverantwortliche

Allgemein sollten Tester über folgende Eigenschaften verfügen:

- qualifizierte Ausbildung/Studium,
- analytisches Denken,
- systematische und strukturierte Arbeitsweise,
- gute Auffassungsgabe,
- hohes Qualitätsbewusstsein,
- Kreativität,
- Organisationstalent,
- Team- und Kommunikationsfähigkeit,
- Aufgeschlossenheit,
- Interesse an innovativen Technologien,
- dynamische Arbeitsweise.

Zertifizierungen (z. B. vom ISTQB – Test Analyst oder Test Manager) sind geeignet, um die Qualifikation von Testern zu prüfen und verhelfen zu gemeinsamen Standards

[ELEK2011]. Die Notwendigkeit einheitlicher Ausbildungsstandards wurde in den letzten Jahren immer stärker erkannt. Noch im Jahre 2007 hatten weltweit nur 10.000 Tester ein entsprechendes Zertifikat, im Jahr 2016 waren es bereits 52.000. Der Beruf des Testers erfährt dadurch in den letzten Jahren eine immer stärkere Standardisierung. Auch in der Ausbildung und in Universitäten werden heute mehr und mehr theoretische Lehrinhalte zum Softwaretest unterrichtet. Das zeigt, dass sich das Thema Softwaretest inzwischen etabliert hat.

Literatur

[ELEK2011] <http://www.elektroniknet.de/embedded/entwicklungstools/artikel/82651/0/>



Für die Aktivitäten zum Softwaretest müssen im Unternehmen konkrete Abläufe und Arbeiten organisiert werden. Dafür ist ein definiertes Testmanagement verantwortlich. Ob das Testmanagement von einem Testkoordinator oder einem eigenen Testmanager durchgeführt wird, ist letztlich nachrangig, entscheidend ist, dass überhaupt ein Testmanagement vorhanden ist. Diese Erkenntnis hat sich in den letzten Jahren mehr und mehr in den Unternehmen durchgesetzt, ganz selbstverständlich ist es aber noch nicht.

Mehr als jeder zweite Softwarefehler, der nach einer Änderung entdeckt wird, entsteht durch Mängel im Testmanagement. Das ergab eine Studie von „Steria Mummert Consulting“. Daran erkennt man das gewaltige Potential, das in einer Verbesserung des Testmanagements liegt.

Oftmals mangelt es an einem systematischer Vorgehenskatalog für das Testen von Software. Der Testfallkatalog ist nicht oder nur teilweise vorhanden. Immer noch entscheiden viele Verantwortliche erst am Projektende, welche Funktionen der Software überprüft werden sollen und führen lediglich unsystematische Stichprobentests durch. Häufig fallen Tests auch dem Termindruck oder dem Rotstift zum Opfer. IT-Firmen folgen dem Wunsch ihrer Kunden und unterbieten sich gegenseitig mit immer kürzeren Release-Zyklen. Gleichzeitig stellen enger getaktete Umsetzungsfristen bei Gesetzesanpassungen die IT-Abteilungen auf die Probe.

Laut „Steria Mummert Consulting“ wird damit genau an der falschen Stelle gespart und geschlachtet: Würden Fehler erst kurz vor Aufnahme des Regelbetriebs auftauchen, dauere die Korrektur meist deutlich länger und benötige mehr Mitarbeiter im Vergleich zu frühzeitig entdeckten Fehlern. Der Mehraufwand für eine nachträgliche Fehlerbeseitigung beträgt nach Schätzung der Analysten im Schnitt etwa das Fünffache [HEIS2011].

Für das effektive und effiziente Testing wurden „fünf goldenen Regeln“ für Softwaretester definiert, die in den folgenden Kapiteln vorgestellt werden.

5.1 Integration der Testprozesse in den Life Cycle der Softwareentwicklung

Der Test Life Cycle bewegt sich dabei parallel zum Entwicklungsprozess der Software. Der Softwaretest geht dabei von den Anforderungen aus, definiert die Testvorgehensweise und die daraus resultierenden fachlichen und technischen Testfälle. Die Testfälle werden in der Regel parallel zu Software-Design und Entwicklung der Applikation erstellt. Der Test soll dabei nicht nur die fehlerfreie Lauffähigkeit der Software sicherstellen. Zum Testen gehört auch die Prüfung, dass alle geplanten Anforderungen fachlich und technisch vollständig sowie korrekt umgesetzt werden. Dieses Ziel kann unter Effizienzgesichtspunkten nur erreicht werden, wenn Testbelange schon in einer frühen Phase des Entwicklungsprozesses hinreichend berücksichtigt werden. Mit dem Review der Anforderungen beginnt die Testphase.

5.2 Integrierte Toolunterstützung im gesamten Testprozess

Der Test Lifecycle umfasst die Verwaltung der Anforderungen, das Management der Testfälle, die Testplanung, die Testdurchführung, das Testreporting und das Fehlermanagement. Integrierte Testtools unterstützen dabei, Anforderungen zu definieren und zu verwalten. Aus den Anforderungen werden Testfälle abgeleitet und zugeordnet. Auf der Basis der Testfälle erfolgt die Testplanung. Die Ergebnisse werden in einem oder wiederholten Testläufen verifiziert. Die Ergebnisse jedes Testlaufs werden in der Testmanagement-Software hinterlegt. Die Verknüpfung von Fehlermanagement, Testdurchführung, Testfällen und den Anforderungen stellt sicher, dass Auswirkungen von Fehlern und gegenseitige Abhängigkeiten auf die betroffenen Anforderungen jederzeit nachvollziehbar sind [BORL2014]. Dies ist insbesondere für den Umgang mit bekannten Problemen wichtig, so genannten „Known Issues“. Im Umkehrschluss lassen sich bei Änderungen in den Anforderungen die betroffenen Testfälle schnell und ohne großen Aufwand identifizieren, anpassen und zu neuen Testläufen zusammenstellen.

Ein wesentlicher Baustein für effizientes Testen ist die sinnvolle Testautomation. Die Automatisierung von Testfällen betrifft die Erstellung der Vorbedingungen, die automatische Durchführung der einzelnen Testschritte, den Abgleich mit dem erwarteten Ergebnis und die Wiederherstellung des definierten Anfangszustandes bis hin zum Reporting. Um die automatisierte Testdurchführung direkt aus der Testmanagement-Software zu starten und die Ergebnisse dort zentral zu verwalten, sollten die Automatisierungs-Werkzeuge in die Testmanagement-Software integriert werden. In der Praxis sind häufig nach wie vor – vor allem bei größeren, komplexen Projekten – unterschiedliche Tools anzutreffen und redundante Erfassung von Daten oder Medienbrüche vorhanden. Solche ungünstige Rahmenbedingungen erschweren Testaktivitäten, verzögern die Abläufe und erhöhen die Fehlerquote. Sollte das der Fall sein, muss der Testmanager von Anfang an darauf hinweisen und das auch bei seiner Planung entsprechend berücksichtigen.

5.3 Einheitliche Testumgebungen und Testdaten

In einer Entwicklungsumgebung werden Testdaten und Testsysteme in der Regel durch den Entwickler nur rudimentär zur Verfügung gestellt. Als Folge entstehen in der späteren Integration hohe Reibungsverluste. Häufig reagieren die umliegenden Systeme anders als erwartet, oder die Testdaten werden in ungewohnter Konstellation angetroffen. Erfolgreiches Testen benötigt somit durchgängige, konsistente Testdaten und geordnete Testumgebungen in definierten Zuständen. Release Management und Konfigurationsmanagement müssen eng mit dem Testmanagement verbunden sein. Die Testumgebungen sollten den Release- und Datenständen in der Produktionsumgebung so früh wie möglich entsprechen und sie möglichst realitätsgenau abbilden. Insbesondere die Testdatenbewirtschaftung rückt verstärkt in den Fokus, da Massendatenverarbeitung (z. B. für CRM- oder Business Intelligence-Systeme) realistische Abbilder der Produktionswelt benötigen [[HEIS2011](#)].

5.4 Softwaretest als Management-Aufgabe

Qualitätssicherung ist der letzte Schritt vor dem Rollout im Entwicklungszyklus. Nur durch ein konsequentes Testmanagement kann sichergestellt werden, dass mögliche Terminverschiebungen in Vorphasen nicht zulasten der Qualitätssicherung gehen, weil keine Zeit mehr zum Testen übrig bleibt. Zudem wird Software immer komplexer und die abgebildeten Prozesse immer kritischer. Dadurch steigt die Bedeutung der Qualitätssicherung weiter an. Diese Relevanz muss sich in den Organisationsstrukturen der Projekte und der Linienorganisation widerspiegeln. Es sollte in jeder Projektorganisation schon mit Beginn der Planungsphase ein Testmanager vorhanden sein, der auf derselben Ebene agiert wie der Entwicklungsleiter. Nur mit einem entsprechenden Einfluss und einer entsprechenden Position werden die erforderlichen Testaktivitäten im Projektmanagement angemessen berücksichtigt [[HEIS2011](#)].

5.5 Industrialisierung und Standardisierung der Testprozesse

Allgemeine Vorgehensweisen, generische Templates, definierte exakt abgegrenzte Teststufen und Glossare sollten einmalig für alle Projekte in einer Organisation Verwendung finden. Dadurch müssen nicht die Rahmenbedingungen bei jedem Projekt neu bestimmt werden, sondern man kann sich schneller auf die inhaltliche Struktur der Testfälle konzentrieren. Auch Testdaten, die bestimmte Kriterien erfüllen, können in mehreren Projekten Anwendung finden. Industrialisierte Testprozesse verkürzen Durchlaufzeiten und realisieren Optimierungspotenziale. Die stärkere Industrialisierung und Standardisierung von Test-

prozessen bedingt eine stärkere Automatisierung von Testaktivitäten. Durch neue Trends wie z.B. der agilen Entwicklung wird der gesamte Software-Entwicklungsprozess stärker industrialisiert. Der Softwaretest stellt dabei einen integralen Bestandteil dar.

Literatur

- [HEIS2011] <http://www.heise.de/developer/artikel/Auswertung-der-Umfrage-Software-test-in-der-Praxis-1369290.html>
- [BORL2014] <http://www.pressebox.de/inaktiv/micro-focus-ismaning/Borland-Hohe-Software-qualitaet-erfordert-zentrales-Testmanagement/boxid/691727>

6.1 Ist-Situation

Von den Hochschulen Bremen und Bremerhaven, der Technischen Hochschule Köln, der ANECON Software Design und Beratung GmbH, dem German Testing Board (GTB) und dem Swiss Testing Board (STB) wurde im Jahre 2012 eine anonyme Online-Umfrage zum Thema „Softwaretest in der Praxis“ im deutschsprachigen Raum durchgeführt. Der Befragung lagen drei rollenspezifische Fragebögen zugrunde. Ihr Ziel war einerseits die Ermittlung des aktuellen Stands der Qualitäts- und Testaktivitäten in der Praxis, andererseits sollte der Handlungsbedarf für Forschung, Ausbildung und Beratung im Testen analysiert werden. Diese Umfrage wurde im Zeitraum von September 2015 bis Januar 2016 erneut durchgeführt.

Ein Großteil der Fragen von 2011 wurde für die aktuelle Umfrage übernommen, so dass eine sehr gute Abschätzung der Entwicklung in den letzten vier Jahren gegeben werden kann: Was hat sich verändert? Wie sieht der Testalltag heute in den Unternehmen aus? Was hat sich im Testalltag etabliert? Ergänzt wurde die aktuelle Umfrage um einen Fragebogen für Forscherinnen und Forscher im Bereich Qualitätssicherung. Gefragt wurde, wie die Forschungsaktivitäten im Bereich Qualitätssicherung und Test gesehen werden, wo Forschungsbedarf besteht und wie zufrieden die Forschenden mit ihrer Forschungssituation sind [SWTU2016].

Diese Umfrage zeigte die in Abb. 6.1 gezeigten Ergebnisse.

In der Polarisierung „agiler Prozess“ versus „phasenorientierter Prozess“ haben sich beide Vorgehensweisen inzwischen nahezu angenähert, in der Praxis gibt es folgende Verteilung: 43 % nennen den eingesetzten Prozess agil, 47 % nennen ihn phasenorientiert, 10 % haben kein Prozessmodell. Interessant ist, dass nur 4 % vom Management angeben, dass kein explizites Vorgehensmodell eingesetzt wird, während 12 % des operativen Bereichs dies

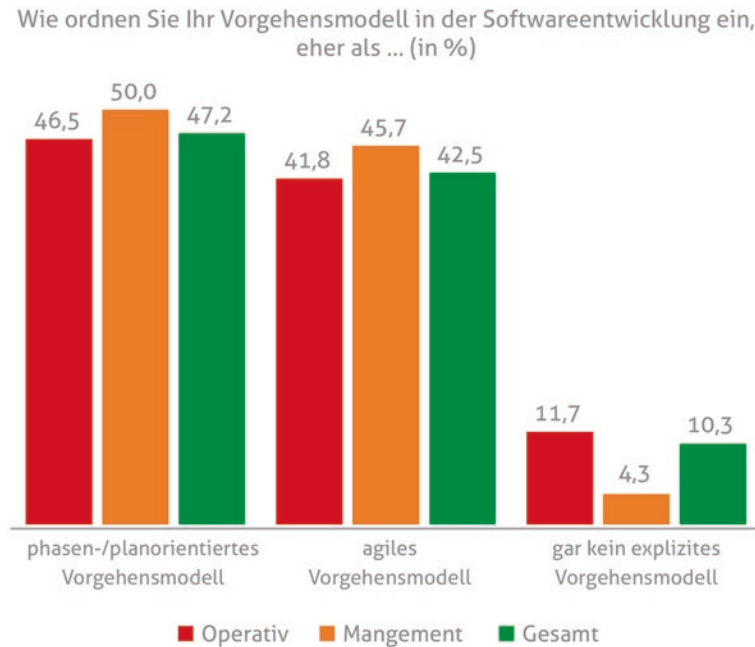


Abb. 6.1 Vorgehensmodell in der Softwareentwicklung

so sehen. Die Mehrheit aller Projekte schätzen heute testgetriebene Softwareentwicklung (67 %), User Stories (59 %), Stand-up-Meetings (53 %), Product-Backlogs (46 %), Continuous Integration (40 %) und zentrale Storyboards (36 %) als bedeutend für die Qualitätssicherung ein.

Seit 2011 kann deutliche Verschiebung beobachtet werden (s. Abb. 6.2). Jeweils um sieben Prozentpunkte haben phasen-/planorientierte Vorgehensmodelle sowie die Nutzung von keinem Vorgehensmodell abgenommen, damit ergibt sich eine Zunahme von 14 Prozentpunkten für das agile Vorgehen.

Werden einzelne agile Praktiken in Bezug auf die Bedeutung für die Qualitätssicherung allgemein betrachtet, ergibt sich eine noch deutlichere Verschiebung: Die Mehrheit aller operativen Mitarbeiter schätzen testgetriebene Softwareentwicklung (67 %), gemeinsame Aufwandsschätzungen (59 %), User Stories (59 %), sowie Stand-up-Meeting (53 %), Retrospektive (53 %) und Refaktorisierung (51 %) als bedeutend bzw. sehr bedeutend für die Qualitätssicherung ein (s. Abb. 6.3). 40 % nutzen meistens oder immer einen Continuous Integration Server für den Integrationstest. Weitere 14 % setzen diese Technik wenigstens teilweise ein. Diese Praktiken, die mit der Diskussion zu agilen Vorgehensmodellen eng verbunden sind, finden sich nicht nur beim agilen, sondern ebenso beim phasen-/planorientierten Vorgehen.

Wie ordnen Sie Ihr Vorgehensmodell in der Softwareentwicklung ein,
eher als ... (in %)

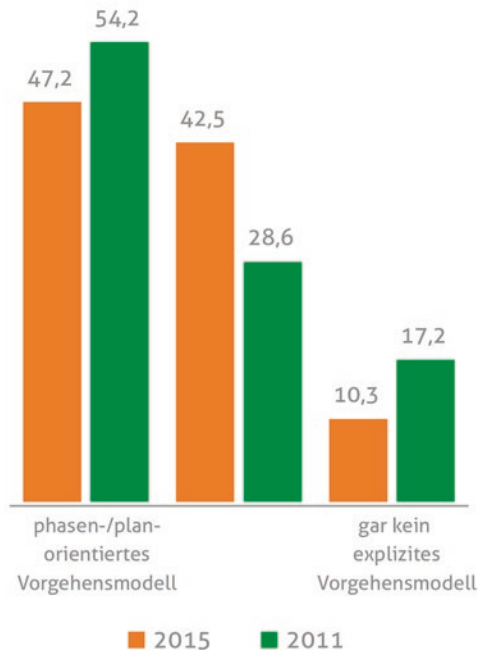


Abb. 6.2 Trend zum Vergleich der Vorgehensmodelle

Continuous integration beschreibt den Prozess des fortlaufenden Zusammenfügens von Komponenten zu einer Anwendung (zum Beispiel mit Hilfe von Jenkins). Die Continuous Integration hat enorme Vorteile:

- Integrations-Probleme werden laufend entdeckt und behoben (gefixt) – nicht erst kurz vor einem Meilenstein
- Frühe Warnungen bei nicht zusammenpassenden Bestandteilen.
- Sofortige Unittests entdecken Fehler zeitnah. Die Testabdeckung wird durch laufende automatisierte Tests erhöht.
- Ständige Verfügbarkeit eines lauffähigen Standes für Demo-, Test- oder Vertriebszwecke.
- Die sofortige Reaktion des Systems auf das Einchecken eines fehlerhaften oder unvollständigen Codes „erzieht“ die Entwickler im positiven Sinne zu einem verantwortlichen Umgang und kürzeren Checkin-Intervallen. Der Merge-Aufwand wird immer größer, je länger man mit der Integration wartet.

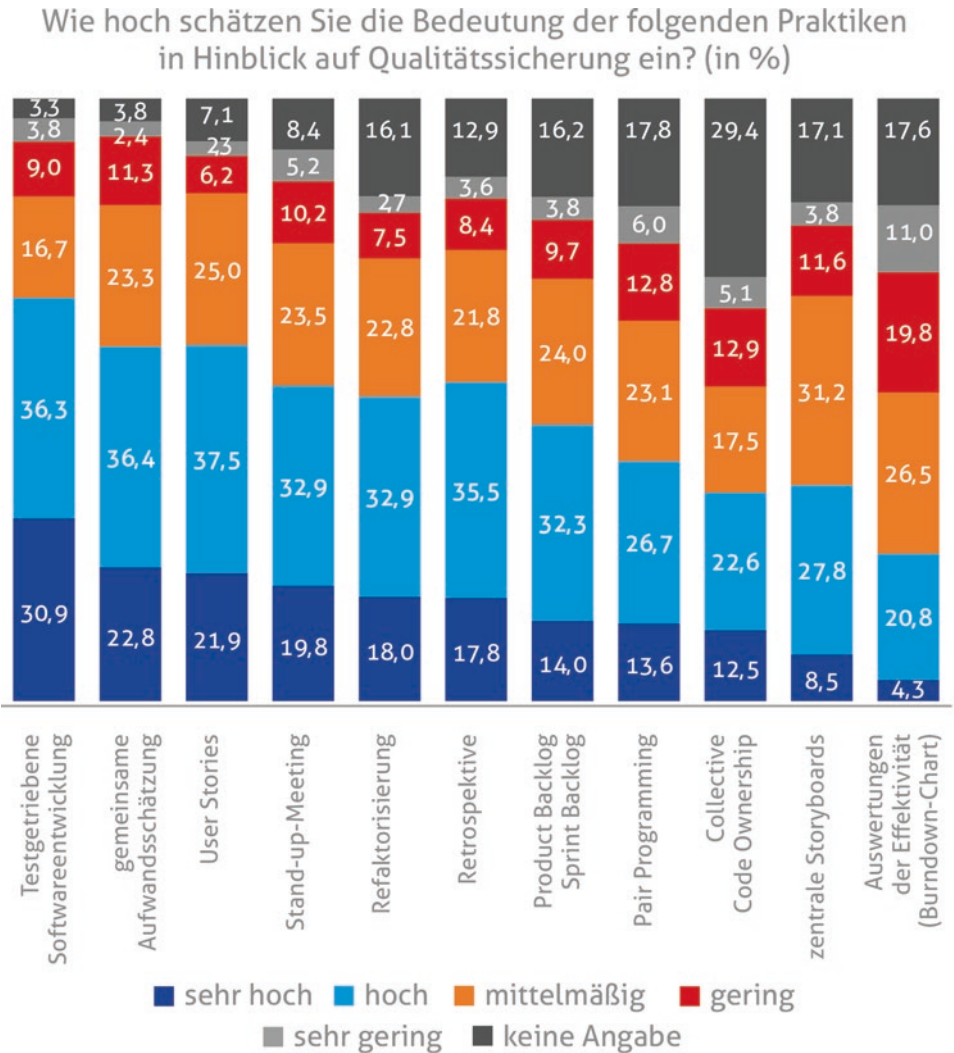


Abb. 6.3 Bedeutung agiler Praktiken im operativen Umfeld

6.2 Tester und Entwickler sind bei vielen IT- Trends nicht im Driver-Seat

Neue IT-Trends wie Industrie 4.0/Internet of Things (22 %), Big Data (23 %) oder auch Mobile Systeme (45 %) sind aus Sicht des Managements zukunftsrelevant und werden in den Unternehmen bereits angemessen berücksichtigt (s. Abb. 6.4). Im Operativen wird die Vorbereitung auf die Trends wesentlich kritischer gesehen. Dem kritischeren Blick wird auch von der Forschungsseite zugestimmt: 74 % sehen hier noch einen deutlichen Forschungsbedarf für IoT, 60 % für Big Data und 50 % für mobile Systeme.

Wie gut ist Ihr Unternehmen für die zukünftigen Herausforderungen in den folgenden IT-Anwendungsbereichen vorbereitet? (in %)

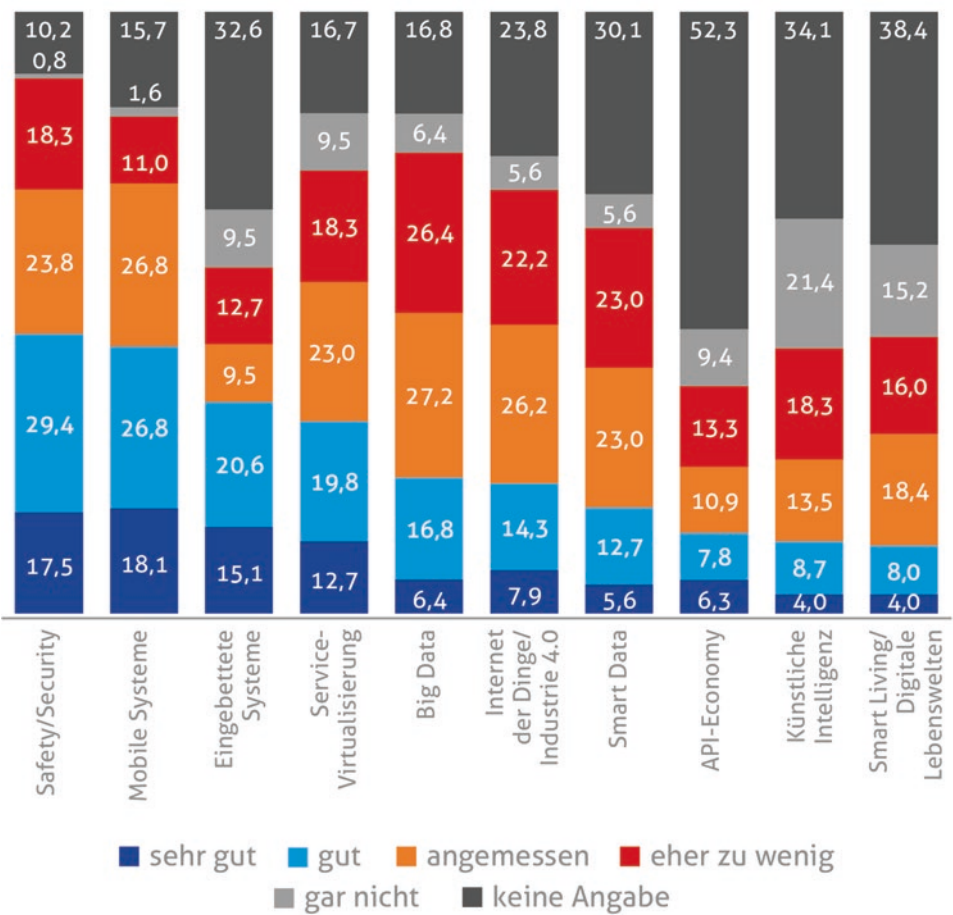


Abb. 6.4 Neue IT-Trends in verschiedenen Anwendungsbereichen

Die Tester und Entwickler sehen überwiegend das Projektmanagement in der Verantwortung oder in der Lage, auf Unzulänglichkeiten hinzuweisen: Aus ihrer Sicht entscheidet in 77 % der Projekte der Projektmanager und in 58 % der Testmanager über die grundlegenden Maßnahmen der Qualitätssicherung. Weniger als ein Drittel der Tester und Entwickler sehen ihre eigene Gruppe in der Verantwortung. Dieses Bild wird von den Managern bestätigt: Sie glauben, dass 73 % der Projektmanager und Testmanager hier entsprechend agieren. Eine inhaltlich organisatorische Verankerung, über die IT-Trends aktiv bearbeitet werden, ist über folgende Organisationen und Institutionen gegeben: 88 % kennen ISTQB, 65 % ASQF, 56 % ist iSQI und 52 % ist BITKOM bekannt.

Die wesentlichen Erkenntnisse zur Digitalisierung und den daraus entstehenden Herausforderungen ist in Schlagworten angekommen und dem Management also durchaus

bewusst. Die Umsetzung in den einzelnen Projekten ist aber nach wie vor schleppend. Das liegt meiner Ansicht nach daran, dass zu wenig Ressourcen für echte Prozessoptimierungen abgestellt werden und neue Trends zu wenig im täglichen Projektgeschäft verankert werden: man arbeitet meist wie früher, obwohl sich die Zeit gewandelt hat. Das einzelne Projekt bekommt nicht die Mittel, um Veränderungen im Ablauf durchzuführen und handelt sich mit suboptimalen Methoden weiter.

6.3 Tester und Entwickler sind zögerlich bei innovativen Techniken

Die heute eingesetzten Testtechniken haben sich seit Jahren kaum verändert: So wird die Testfallermittlung immer noch regelmäßig in Freitextform durchgeführt (52 %), wobei 42 % wenigstens formularbasiert vorgehen. Auch Testdaten werden in 44 % der Fälle als Freitext dokumentiert, in 28 % der Fälle wenigstens formularbasiert. Jegliche Form formaler Methoden wie grafische Modellierung (5 %), formale Modelle (7 %) oder formale Sprachen (21 %) werden nur bedingt eingesetzt. Dieser Status ist seit 2011 nahezu konstant. Auch die Forscher sehen hier keine Notwendigkeit einer weiteren Forschungsintensivierung. Es sind demnach ausreichend innovative Methoden vorhanden, die jedoch nur schwer ihren Weg in die Praxis finden. Auch andere neue Techniken wie Crowdfunding (1 %) oder Service-Virtualisierung (11 %) werden nur sehr zögerlich eingesetzt.

Crowdfunding wurde mit einigem Elan vor einigen Jahren begonnen, hat sich aber inzwischen als nur begrenzt sinnvoll einsetzbar erwiesen. Die Idee beim Crowdfunding war, den Test zu beschleunigen um bedarfsweise zahlreiche Tester flexibel und explorativ testen zu lassen. Beim Crowdfunding wird das Testen von Softwareanwendungen an die Internetgemeinde ausgelagert. Der entscheidende Vorteil liegt in der enormen Vielfalt an Endgeräten und Betriebssystemen der Tester, die ein Unternehmen in dieser Form intern kaum abbilden kann. Anhand einer Vielzahl verschiedener Kriterien wie Alter, Beruf, Hobbies oder Einkommensklasse können die Probanden so ausgewählt werden, dass sie die Merkmale der potenziellen Zielgruppe einer Anwendung entsprechen. Die Tester können mit einem frischen Blick und ohne Betriebsblindheit die Funktionalität und Benutzerfreundlichkeit von Apps, Webseiten und Co. unter die Lupe nehmen.

Es hat sich aber herausgestellt, dass es dadurch für den Testmanager enorm schwierig wird, den Überblick zu bewahren. Die Qualität der Fehlerbeschreibungen ist oft nicht ausreichend und manche Fehler werden mit unterschiedlichen Beschreibungen mehrfach erfasst oder lassen sich auf dieselbe Ursache zurückführen. Die Bewertung des Fehlverhaltens und die Priorisierung der Fehler verbleiben in jedem Fall innerhalb des Unternehmens bzw. Softwareprojekts. Es ist auch fraglich, ob man kurzfristig genügend Tester in einer vorher definierten Zielgruppe findet. Für viele Anwendungen sind zudem das fachliche Know-how und das Hintergrundwissen über Prozesse erforderlich, die nur ein professioneller Tester mitbringt.

6.4 Viel Potenzial zur Effizienzsteigerung des Testens bleibt ungenutzt

Viele Möglichkeiten, das Testen effizienter zu gestalten, scheinen nicht genutzt zu werden: Die Qualitätssicherungsabteilung ist in über 63 % der Fälle weiterhin Inhouse und direkt vor Ort angesiedelt. Externe Dienstleister mit ihrem spezialisierten Know-How werden immerhin in 40 % der Fälle häufig hinzugezogen.

Auch die Testautomatisierung als klassische Möglichkeit zur Steigerung der Effizienz wird nur teilweise umgesetzt (s. Abb. 6.5): Nur 48 % geben z. B. an, im Integrationstest wenigstens 50 % automatisiert zu haben. Im Systemtest sinkt dieser Wert auf 39 %, im Systemintegrationstest sogar auf 23 %. Bei der Steuerung des gesamten Testprozesses, innerhalb derer Potenziale offenkundig werden können, ergeben sich folgende Werte: So wird in 78 % der Projekte die Testabdeckung innerhalb der Anforderungen gemessen, eine

Wie hoch ist der Prozentsatz der Automatisierung im ... (in %)

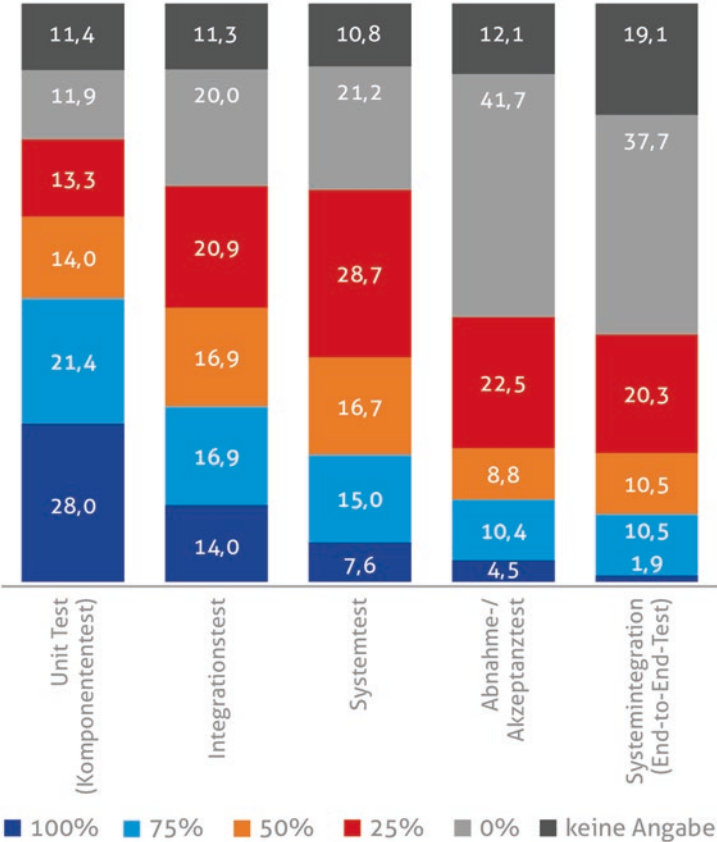


Abb. 6.5 Automatisierungsgrad pro Teststufe

Codeabdeckung wird bei 40 % der Befragten ermittelt. Auch bezüglich der Testfälle selbst wird in 60 % der Projekte gemessen, wie viele davon jeweils durchgeführt werden. Die Forschung sieht hier aktuell nur noch wenig Forschungsbedarf (22 %).

Die Erkenntnisse sind also letztlich bekannt, es scheitert aber häufig an der konsequenten Umsetzung.

6.5 Shift-Left ist in phasenorientierten Prozessmodellen nicht etabliert

In Projekten mit einem phasenorientierten Prozessmodell hat sich die Erkenntnis, dass das Testen umso effizienter und effektiver ist, je früher es im Softwareentwicklungsprozess angewendet wird (Shift-Left), operativ nach wie vor nicht durchgesetzt: So wird z. B. in der Phase „Vorstudie“ in 43 % der Fälle selten oder nie irgendeine Qualitätssicherungsmaßnahme eingesetzt. Auch das Fachkonzept wird in 23 % der Fälle und der Systementwurf in 18 % ohne jegliche QS-Maßnahmen umgesetzt. Erst in der Implementierung (96 %) und der Integration (96 %) werden solche Maßnahmen durchgeführt. Seit 2011 sind die Reviews für die frühen Artefakte der Softwareentwicklung, wie Anforderungen und Architektur, sogar um mehr als 10 Prozentpunkte zurückgegangen.

Hier liegt ein großes Optimierungspotenzial brach. Es würde aus meiner Sicht helfen, den Nutzen stärker in EUR zu quantifizieren um das Bewusstsein schon bei der Projektplanung und Quotierung des Budgets zu stärken. Oft liegt es aber auch daran, dass zu wenige erfahrene Ressourcen im Testmanagement verfügbar sind, um diese Ansätze zielgerichtet umzusetzen.

6.6 Testen läuft Gefahr, zum Selbstzweck zu werden

Softwareprodukte (s. Abb. 6.6) werden aus dem Blickwinkel der Tester und Entwickler mit keinen (30 %) oder nur noch wenigen schwerwiegenden Fehlern (56 %) ausgeliefert. Aus ihrer Sicht sind nur knapp 5 % der Kunden mit der Qualität eher unzufrieden. Dem entgegen schätzen allerdings 52 % der Tester und Entwickler die Testintensität als zu gering ein, d. h. die Tester und Entwickler scheinen das Good-Enough der Kunden und die damit einhergehende Zufriedenheit nur bedingt berücksichtigen zu können. Die Forscher bekräftigen diese Vermutung, da sie einen erhöhten Forschungsbedarf beim Nachweis der Wirksamkeit und Effizienz der QS-Maßnahmen mit 68 % und die Untersuchung der Wechselwirkung zwischen den QS-Maßnahmen mit knapp 70 % sehen.

Dieser in jüngerer Zeit verstärkte zu beobachtende Trend liegt aber auch daran, dass viele Anwendungen inzwischen einen höheren Reifegrad erreicht haben als das in früheren Jahren der Fall war. Problematisch wird es, wenn dadurch Testaktivitäten vernachlässigt werden, weil sich dadurch ein guter Qualitätsstandard schnell wieder verschlechtern kann.

Treten nach Auslieferung Ihrer Produkte schwerwiegende Fehler auf? (in %)

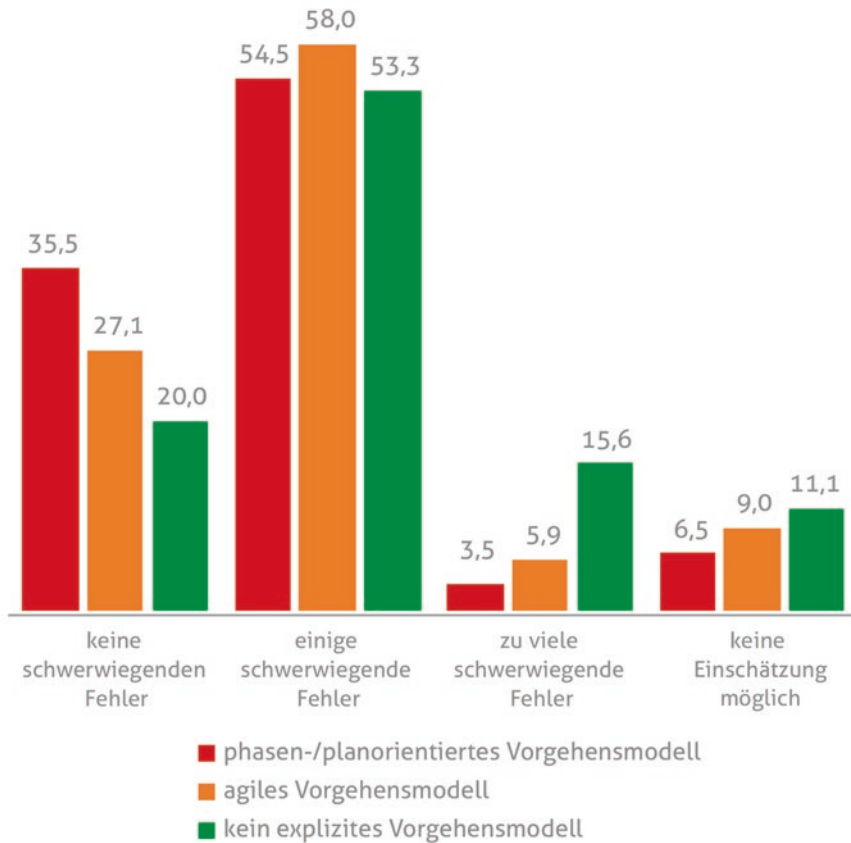


Abb. 6.6 Verteilung nach Vorgehensmodelle zu jeder Antwortalternative auf die Frage der Einschätzung zu Fehlern nach Auslieferung

6.7 Tester und Entwickler kritisch gegenüber Standards

Obwohl die Testdisziplin mittlerweile über 30 Jahre alt ist, werden Standards weiterhin zögerlich eingesetzt: So dominiert bei den verwendeten Testprozessen trotz der Verfügbarkeit eines ISTQB-Testprozesses (34 %) und einer ISO 29119 (11 %) noch der individuell zugeschnittene Testprozess (60 %). Im Vergleich zu 2011 ist allerdings ein Anstieg der Umsetzung von Standards zu erkennen. Während systematische Audits des Prozesses wie TPI/TPI-Next (12 %) und TMMi (3 %) im gleichen Zeitraum an Bedeutung weiter verloren haben (77 % führen überhaupt keine Audits durch). Auch die Forscher sehen für den Bereich der Prozessbewertung keine Zukunft: 34 % sind für Einstellung der Forschung [GTBS2016].

Ob das allerdings der richtige Weg ist, ist zumindest fraglich: Vielleicht müsste man eher prüfen, warum denn die Standards nur teilweise umgesetzt werden bzw. wie man Standards so definieren kann, dass sie stärker in die betriebliche Anwendung kommen. Evtl. kann man auch Standards und individuelle unternehmensspezifische Besonderheiten besser kombinieren.

6.8 Künftiger Forschungsbedarf

Forschungsbedarf wird vor allem dort gesehen, wenn es um die Wirksamkeit und die Wechselwirkungen einzelner Qualitätsmaßnahmen geht (s. Abb. 6.7).

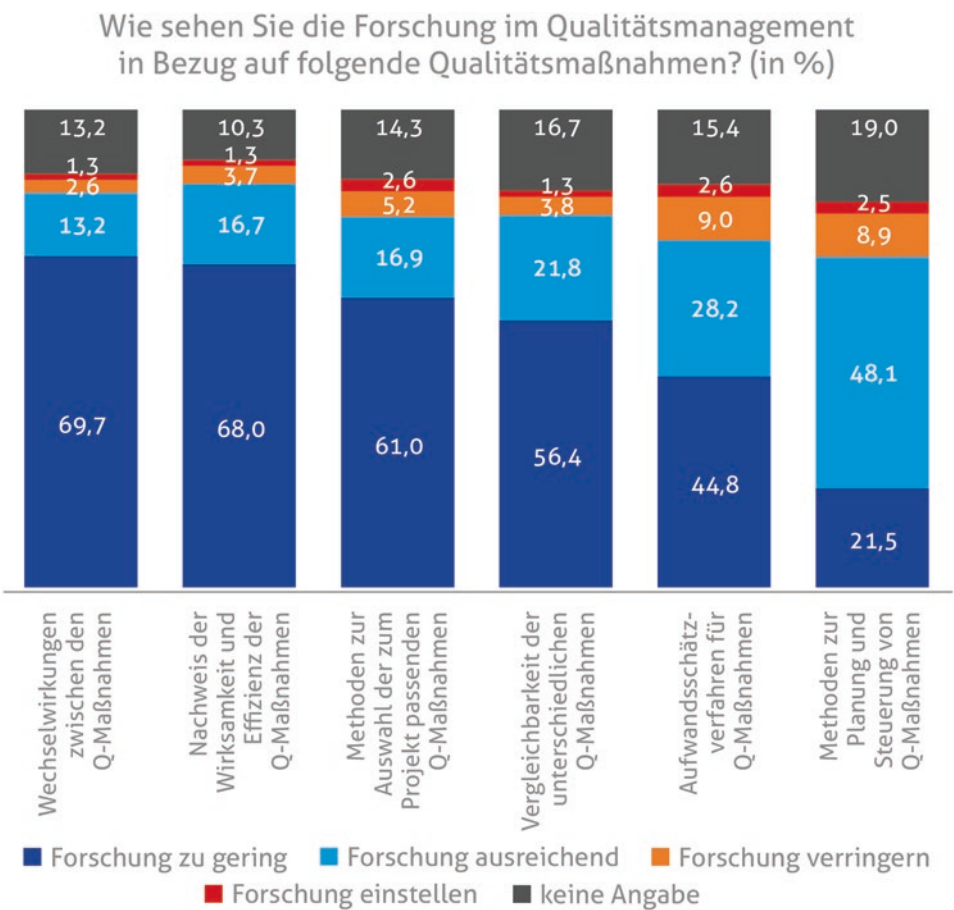


Abb. 6.7 Forschung zu den Qualitätsmaßnahmen

Fast 70 % sehen die Forschung zu den Wechselwirkungen der einzelnen Qualitätsmaßnahmen als zu gering an. Auch beim Nachweis der Wirksamkeit fehlt es noch an fundierten Forschungsergebnissen (68 %). Ebenso kann noch nicht davon ausgegangen werden, dass klar ist, welche Qualitätsmaßnahmen die passenden für das jeweilige Projekt sind. 61 % der Forscher sehen die Forschung in diesem Punkt als zu gering an. Dies liegt auch daran, dass bei der Vergleichbarkeit der Maßnahmen Forschungsdefizite bestehen (56 %). Insgesamt betrachtet sehen die Forscher einen erheblichen Forschungsbedarf bei Wirksamkeit, Wechselwirkung und Auswahl der jeweiligen Qualitätsmaßnahmen. Dies bestätigt auch die zu beobachtende Unsicherheit der operativen Mitarbeiter in der Einschätzung der Testintensität. Bei allgemein anwendbaren Techniken und Verfahren zur Qualitätssicherung ergibt sich folgendes Bild: Zwischen 50 % und 60 % sehen bei folgenden Techniken und Verfahren die Forschung als ausreichend an:

- Model Checking (60 %)
- Statische Analyse (manuell, Reviews) (58 %)
- Statische Analyse (werkzeuggestützt) (58 %)
- Simulation (53 %)
- Codebasierter Test (50 %)

Ein erhöhter Forschungsbedarf wird bei formaler Verifikation (zur Laufzeit, Run-Time Verification) mit 36 % und bei Testspezifikationssprachen/Techniken (TTCN-3, TestML, BDT, ...) mit 34 % gesehen. „Spitzenreiter“ bei „Forschung einstellen“ ist codebasierter Test mit 7 %.

Bei den agilen Praktiken zeigt sich ein sehr ähnliches Bild des Defizits an ausreichender Forschung (s. Abb. 6.8). Zwischen 50 % und 60 % sehen erhöhten Forschungsbedarf bei folgenden Themen:

- Integration des Testens im agilen Vorgehen
- Agiles Testen (methodische Ansätze)
- Continuous Integration/Deployment

Bei **BDD (Behavior Driven Development)** und **ATDD (Acceptance Test Driven Development)** sehen 49 % bzw. 48 % die Forschung als zu gering und jeweils 25 % als ausreichend an. Jeweils 4 % möchten die Forschung zu diesen beiden Praktiken verringern. 45 % sehen die Forschung bei TDD (Test Driven Development) als ausreichend an – bei dieser Frage mit Abstand der größte Wert für ausreichende Forschung. 36 % möchten im Gegensatz dazu TDD-Forschungsaktivitäten erhöhen. Beim explorativen Testen sind 29 % für eine Erhöhung, 28 % sehen die Forschung als ausreichend an und 13 % möchten sie verringern oder sogar einstellen. Allerdings gaben von den Forschern auch 29 % keine Einschätzung zum explorativen Testen ab.

Wie sehen Sie die Forschungsaktivitäten in Bezug auf eher agile Praktiken bei allgemein anwendbaren Techniken und Verfahren? (in %)

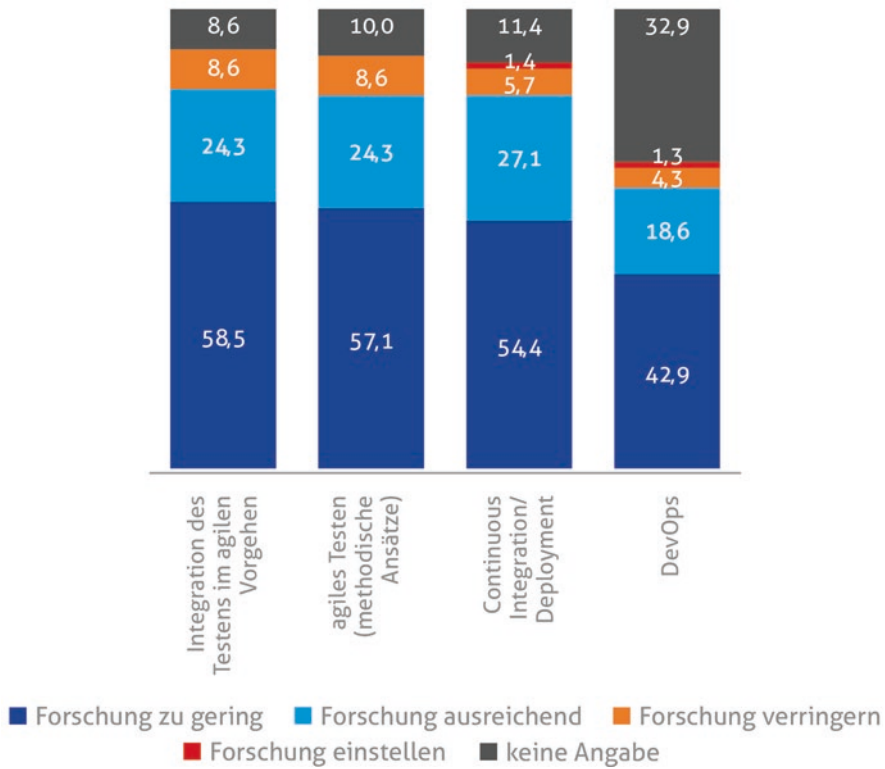


Abb. 6.8 Forschung zu agilen Projekten

Literatur

- [GTBS2016] http://www.softwaretest-umfrage.de/2016/GTB_Survey_Management_Summary_KVAS_Neu.pdf
- [SWTU2016] http://www.softwaretest-umfrage.de/2016/Softwaretest_Broschuere_2016.pdf

Bevor man mit der Testdurchführung starten kann, sind strategische Überlegungen, Planungsaktivitäten und konkrete Vorbereitungsmaßnahmen erforderlich. Einige grundlegende Fakten sind unternehmensweit zu klären. Sie gelten in der Regel komplett oder zumindest teilweise für alle beteiligten Projekte des Unternehmens. Auch wenn diese Aspekte nicht in jedem Projekt neu bewertet werden müssen, ist es wichtig, dass in jedem einzelnen Projekt das grundlegende Verständnis dafür vorhanden ist.

7.1 Teststrategie

Zu Beginn der Testaktivitäten muss eine Strategie definiert sein. Dabei muss man zuerst überlegen, was der Test zeigen soll und welche Ziele man mit den Testaktivitäten verfolgt. Welche Anforderungen werden an die Anwendung gestellt, wie sicherheitsrelevant ist das System, wie langlebig, wie hoch ist das Budget?

Der Test soll die Qualität sicherstellen. Einerseits verlängern Softwaretestphasen bei bestehender Funktionalität das Projekt und machen es teurer, andererseits sollte man sich dabei immer einen Satz von Henry Ford vor Augen führen: „Qualität ist wenn der Kunde zurück kommt und nicht das Auto“. Es gibt eine allgemein anerkannte Erkenntnis, dass es etwa sieben-mal mehr kostet, einen ehemaligen Kunden zurückzugewinnen als einen bestehenden Kunden zu halten. Manchmal ist es besser zu Beginn weniger Funktionen auszuliefern und dadurch eine hohe Qualität sicherzustellen als Hunderte von Features die kaum benutzt werden. Das ist aber naturgemäß immer ein Drahtseilakt.

Als Softwaretester hat man in der Regel keinen Einfluss darauf, den Umfang oder die Features zu bestimmen. Man kann aber durchaus im Vorfeld darauf hinweisen, was es bedeutet, dass für die gewünschte Qualität Maßnahmen wie

- Aufstockung des Testpersonals,
- zusätzliche Kosten,
- Lieferung definierter Funktionen zu einem späteren Zeitpunkt,

aus Sicht des Testers aus bestimmten Gründen erforderlich wären.

Es ist sogar notwendig, vor übertriebenen Hoffnungen von Anfang an zu warnen und manche Euphorie von Marketing und Entwicklung zu bremsen. Ein Softwaretester muss Risiken realistisch einschätzen können, von einem Softwaretester wird zu Recht erwartet, dass er den gesamten Prozess im Blick hat und je frühzeitiger der Tester im Projekt warnt umso eher kann man gegensteuern und Risiken mindern.

Vor dem Test muss man immer einige grundlegende Überlegungen zur Teststrategie anstellen:

- Auf welcher Hardware muss die Software getestet werden, wie ist die Testkonfiguration?
- Welche Arten von Hardware sind in welcher Teststufe vorhanden?
- Ist die Hardware noch aktuell wenn das System in den Rollout und den Feldeinsatz geht?
- Wo soll der Test stattfinden? Gibt es einen eigenen Raum für den Test? Ist es am Arbeitsplatz des Testers möglich oder muss man immer hin- und herwandern? Wenn der Test z. B. in einem Auto stattfindet oder im Freien: Lassen sich die Fehler vor Ort erfassen und der Testfortschritt auch unterwegs tracken?
- Welche Angaben braucht der Entwickler zur Fehlerbehebung, in der Konsequenz: Welche Daten müssen im Einzelnen aufgezeichnet werden? Ist gewährleistet, dass die Daten nur einmal am Entstehungsort erfasst werden? Allein durch getrennte Arbeitsplätze von Büro und Testlabor kann es zu erheblichen Verzögerungen im Projektablauf kommen, wenn man beim Test nicht in Echtzeit auf bestimmte Daten zugreifen kann. Eine Studie hat in diesem Zusammenhang um 30 Jahren ermittelt, dass Daten in Unternehmen zu 79 % mehr als einmal erfasst werden. Interessanterweise hat sich dieser Prozentwert in den letzten 25 Jahren kaum verändert, obwohl doch die Probleme auf der Hand liegen: Redundante Datenerfassung führt immer zu Qualitäts- und Zeitverlust sowie zusätzlichen Kosten. Eine Flut an Excel-Tabellen mit unterschiedlichen Bearbeitungsständen ist gerade in den letzten Jahren über viele Abteilungen hereingebrochen. Prozessoptimierung vor diesem Hintergrund betrachtet zahlt sich immer aus.

Bei der Betrachtung und Bewertung von Risiken muss immer das Umfeld betrachtet werden, in dem die Software eingesetzt wird. Das betrifft sowohl die User die mit der Software arbeiten sollen als auch das technische Umfeld in der die Software eingesetzt wird. Welche Abhängigkeiten bestehen zwischen den einzelnen Komponenten? Man muss sich

also immer fragen, ob der Test überhaupt schon sinnvoll gestartet werden kann, wenn bestimmte Bedingungen noch gar nicht vorhanden sind.

Ich hatte einmal das Videokodiersystem einer Poststraße zu testen. Der Test war gar nicht sinnvoll möglich, bevor die Sortiermaschine als vorgelagerte Komponente richtig gearbeitet hat. Nachdem die Sortiermaschine nach vielen Monaten Verzug arbeitete, konnten erst die Fehler im Nachfolgesystem aufgedeckt werden. Der Integrationstest war zu diesem Zeitpunkt aber bereits um mehrere Monate verzögert. Als die Fehler dann auftraten war das leider kurz vor der drohenden Penalty-Forderung ...

Wenn der Systemtest von bestimmten Voraussetzungen in der Systemumgebung abhängig ist und diese noch nicht fertig sind, kann es notwendig sein, zu Simulationen zu greifen. Bei Tests für Software im Kraftfahrzeug wird z. B. häufig auf den „HIL-Test“ („Hardware in the loop“) zurückgegriffen. Der HIL-Test ist ein Verfahren, bei dem ein eingebettetes System (z. B. reales elektronisches Steuergerät oder reale mechatronische Komponente) über seine Ein- und Ausgänge an ein angepasstes Gegenstück (einen HIL-Simulator) angeschlossen wird, der als Nachbildung der realen Umgebung des Systems dient.

„Hardware in the Loop“ dient als Methode zum Testen und Absichern von eingebetteten Systemen, zur Unterstützung während der Entwicklung sowie zur vorzeitigen Inbetriebnahme von Maschinen und Anlagen. Das zu steuernde System (z. B. Auto) wird über Modelle simuliert, um die korrekte Funktion der zu entwickelnden Steuereinheit (z. B. eine Motorsteuerung) zu verifizieren.

Die HIL-Simulation muss in der Regel in Echtzeit ablaufen und wird in der Entwicklung benutzt, um Entwicklungsaufwand zu minimieren, Programmierzeiten zu verkürzen und Kosten zu sparen. Insbesondere lassen sich wiederkehrende Abläufe simulieren. Dies hat den Vorteil, dass eine neue Entwicklungsversion unter den gleichen Kriterien getestet werden kann wie die Vorgängerversion. Somit kann detailliert nachgewiesen werden, ob ein Fehler beseitigt wurde oder nicht. Es kann dadurch auch das Systemverhalten in Grenzsituationen erprobt werden, ohne Auto und Fahrer dabei zu gefährden.

Die HIL-Simulation bedeutet stets eine Vereinfachung der realen Konfiguration und kann daher den Test am Zielsystem nie ganz ersetzen. Bei zu großen Unterschieden zwischen HIL-Simulation und der Realität ist die Komplexität der gewählten Simulationsmodelle zu stark reduziert. Inzwischen wird die HIL-Simulation nicht nur dann angewendet, wenn die Testumgebung noch nicht vollständig vorhanden ist, sondern als zusätzliche Teststufe bei der Integration der Software. Wenn die Software schon auf dem HIL-Rechner Fehlverhalten zeigt, muss man sie erst gar nicht in der endgültigen Systemumgebung (z. B. im Fahrzeug) testen.

Je größer die Abhängigkeit von anderen Komponenten oder Systemen ist und je komplexer das Gesamtsystem, desto stärker muss das eigene System mit Unterstützung von Simulationen getestet werden.

7.2 Kennzahlen zur Wahl der geeigneten Teststrategie

Je größer und komplexer der Systemtest ist, desto mehr Tester werden benötigt. In einem Projekt Wiener Banken kamen 96 Tester auf 67 Entwickler, in einem Projekt der US-Regierung gab es über 160 Tester für 100 Entwickler. Man kann davon ausgehen, dass der Test einer service-orientierten Architektur doppelt so viel kostet wie die Entwicklung der einzelnen Web-Services. Vor diesem Hintergrund ist das Management daran interessiert, Zahlen über den Return On Investment (ROI) zu bekommen.

Dafür gibt es im Wesentlichen zwei Kennzahlen:

$$\text{Fehlerentdeckungsrate} = \frac{\text{gewichtete Fehler (Tester)}}{\text{gewichtete Fehler (gesamt)}}$$

Die Fehlerfindungsrate ist der von den eigenen Testern gemeldete Anteil aller gemeldeten Fehler. Je höher der von den Testern selber gemeldete Fehleranteil ist, umso höher ist die Fehlerentdeckungsrate. Dies ist eine klare Aussage über die Effektivität des Testens.

$$\text{Vertrauensgrad} = (\text{gemeldete Fehler (gewichtet)} / \text{ausgeführte Testfälle}) \cdot \text{Testüberdeckung}$$

Der Vertrauensgrad ist das Verhältnis der gefundenen Fehler zur Anzahl der getesteten Testfälle, multipliziert mit der Testüberdeckungsrate. Je weniger Fehler pro Testfall aufgedeckt werden und je höher die Testüberdeckung ist, desto größer ist der Vertrauensgrad. Wenn eine ausreichend hohe Testüberdeckung erreicht ist und nur noch wenige niedrig gewichtete Fehler bei vielen Testfällen auftauchen, ist das ein Zeichen für die Systemstabilität und das Vertrauen in den Testbetrieb steigt. Wenn aber entweder viele Fehler gefunden werden oder die Testabdeckung gering ist, steigt das Misstrauen.

Der Entwurf und die Auswahl der am besten geeigneten Teststrategie richten sich danach, ob ein System neu eingeführt wird oder eine bestehende Anwendung abgelöst wird. Bei einem neuen System sind grundlegende Überlegungen über Testkonzept, Testumfang und Testplanung zu definieren. Beim Wartungstest liegt der Schwerpunkt auf den gezielten Tests der Erweiterungen und Anpassungen und auf der Prüfung der bestehenden Funktionalität (Regressionstest). Alle Features, die im früheren Release schon verfügbar waren, müssen weiterhin fehlerfrei funktionieren.

Für neue Funktionalitäten, die explizit zu testen sind, ist auf Basis der zusätzlichen Anforderungen, ggf. auch aufgrund der Releasenotes die bestehende Testspezifikation zu erweitern. Diese neuen Features müssen in geeigneten Teststufen getestet und in das Gesamtsystem integriert werden. Lediglich der Umfang der neu zu testenden Funktionen ist dabei geringer als bei einer kompletten Ablösung des Altsystems. Geänderte Funktionalitäten müssen mit geänderten bestehenden Testfällen und angepassten Testdaten und geänderten erwarteten Ergebnissen zu testen.

Für beseitigte Fehler vorangegangener Testläufe oder Fehlerbehebungen bereits ausgelieferter Versionen werden bestehende Testfälle in einem Fehlernachtest (Retest) ausgeführt. Für Funktionalitäten, die in einer neuen Version nicht mehr aktiv sind, muss nachgewiesen werden, dass ihre Deaktivierung erfolgreich war. Für diesen Nachweis müssen die alten Testfälle letztmalig ausgeführt werden. Im Regressionstest wird zusätzlich geprüft, ob nicht geänderte Funktionalitäten nach wie vor die erwarteten Sollergebnisse liefern. Damit wird nachgewiesen, dass die Änderung an einer bestimmten Stelle im Code nicht unerwünschte Seiteneffekte auf andere Softwareteile hatte. Auch bei Wartungstests werden nicht funktionale Testfälle (z. B. Performancetests, Lasttests) durchgeführt.

Außerdem sind folgende Kriterien für die Wahl der richtigen Teststrategie wichtig:

- Die **Änderungsquantität** sagt aus, wie viele Codezeilen, Komponenten, Funktionen, Schnittstellen, Anwendungen oder Systeme geändert werden müssen.
- Die **Änderungsqualität** beschreibt wie komplex die Umsetzung der Anforderung ist (z. B. muss nur ein Hilfetext erweitert werden oder sind umfangreiche Algorithmen und mathematische Funktionen zur Berechnung anzupassen).
- Die **Änderungskritikalität** bezeichnet das Risiko, das mit der Änderung verbunden ist (z. B. Anwendungen mit hohen Nutzerzahlen oder bilanzrechtlichen Auswirkungen auf der einen Seite, selten genutzte Auswertung für einen kleinen Fachbereich auf der anderen Seite) [BOCH2010].

Abb. 7.1 zeigt ein Beispiel für unterschiedliche Testarten bei unterschiedlichen Veränderungen in der Anwendung.

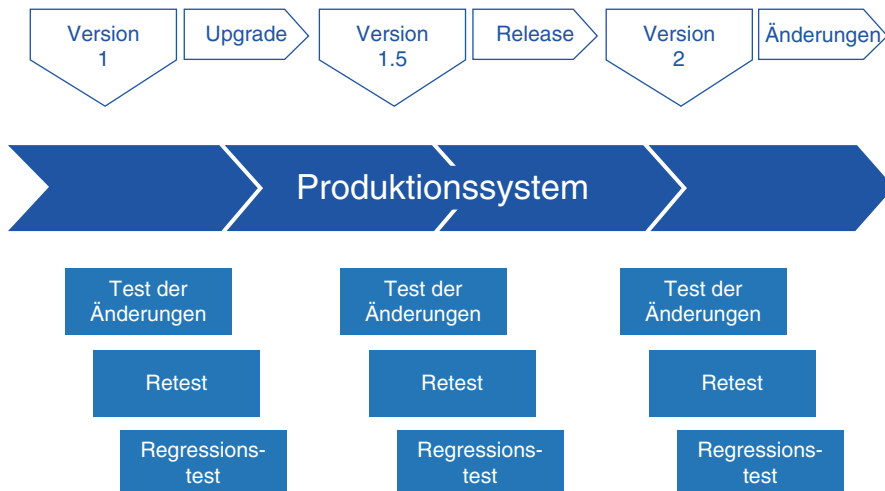


Abb. 7.1 Unterschiedliche Testarten bei unterschiedlichen Veränderungen in der Anwendung

7.3 Häufigkeit der Änderungen und Anzahl der Releasezyklen

Bei kundenbezogener Entwicklung können die Änderungshäufigkeiten sehr hoch sein, beispielsweise ein vereinbarter Releasezyklus mit Lieferung an den Kunden in 2- oder 4-wöchigen Abständen. Dann wird zum Beispiel jeden Tag die Software neu integriert (ein „Preliminary Build“ erstellt) und immer wieder eine neue Software gebildet.

Meist weiß man zwar im Vorfeld durchaus, dass eine gewisse definierte Zeitphase für den Test zur Verfügung stehen sollte. In der Praxis passiert es dann aber doch immer wieder, dass z. B. eine Funktion unbedingt zu Release 4 am 15.3. fertig sein muss aber die Funktion erst am 14.3. integriert werden konnte, weil sie vorher noch nicht ansatzweise funktionsfähig war.

Daher empfiehlt sich in diesem Fall folgendes Vorgehen: In regelmäßigen Abständen, z. B. alle 4 Wochen, eine Version bilden, die intensiv getestet wird. Das sollte dann auch die Version sein, die ausgeliefert wird, oder in Ausnahmefällen zumindest noch die, die ausgeliefert wurde (damit das Unternehmen eine Chance hat die Fehler noch vor dem Kunden zu finden, selbst wenn das Release schon ausgeliefert ist). Die anderen Versionen, die täglich oder stündlich von der Entwicklung geliefert werden, sollten in mehr oder weniger detaillierten Tests parallel dazu verifiziert werden. Dabei sollte man wieder die Priorisierung der Testfälle als Anhaltspunkt für den Testumfang pro Release nutzen. Dazu empfiehlt es sich einen „Quick-Test“/Schnelltest zu definieren, der auf einem dazu extra vorhandenen Rechner bzw. einer besonders definierten Testkonfiguration getestet wird. Es ist zu bedenken, dass allein der Software-Update für eine neue Version eine gewisse Zeit verschlingt.

Bei einer solchen zyklischen entwicklungsbegleitenden Software-Abgabe hatte ich schon einmal erlebt, dass die Software für die Kundenauslieferung am Tag vor dem vereinbarten Meilenstein in vier Versionen an die Testabteilung geliefert wurde. In der Konsequenz war man dann fast den gesamten Tag mit Software-Updates beschäftigt und konnte die neuen Lieferungen nur sehr oberflächlich testen. An diesem Tag wurden Versionen A, B, C und D geliefert, die Version C wurde an den Kunden ausgeliefert und das war dann die Version, die von den meisten Testern definitiv am kürzesten (effektiv etwa 10 Minuten lang) getestet worden war ...

Fazit: Es macht manchmal wenig Sinn, jede Version komplett zu testen und dabei immer wieder von Null anzufangen. Man muss dann zu viele Testzyklen vorzeitig abbrechen. Es ist aber sinnvoll gewisse „Quick-Tests“ für die Zwischenversionen zu definieren und die Version, die voraussichtlich auch ausgeliefert wird, dann mit einer sinnvollen höheren Testabdeckung auf Herz und Nieren zu prüfen.

7.4 Änderungsumfang pro Release

Die Funktionen und Teilsysteme sind oft voneinander abhängig und beeinflussen sich gegenseitig. Daher gilt es, nicht das gesamte Softwarepaket auf einmal zu entwickeln und zu testen, sondern sukzessive zu integrieren und immer wieder zu testen.

Je größer die Änderungen von einem Release zum nächsten sind, desto umfangreicher ist das Risiko für Seiteneffekte. Wenn z. B. von einer zur nächsten Softwareversion an drei Stellen Steuerungsparameter verändert wurden, und man nach dem erneuten Test eine Verschlechterung der Systemperformance feststellt, dann weiß man nicht, woran es gelegen hat. Dafür müsste man die drei Parameter isoliert betrachten und pro Änderung eine Version erstellen.

Da das aber in der Praxis viel zu viele Releases und einen exponentiell wachsenden Testumfang zur Folge hätte, gilt es auch hier einen Kompromiss zu finden. Wenn die Änderung der drei Parameter insgesamt eine bessere Performance verspricht und das Risiko des oben beschriebenen Seiteneffektes gering ist, so macht es durchaus Sinn die einmalig verbesserte Version zu testen.

Hier gilt es eine mathematische Risikobewertung vorzunehmen (Eintrittswahrscheinlichkeit und Testdauer in Beziehung setzen).

$$\begin{aligned} \text{Wahrscheinliche Testdauer} &= \text{Kürzere Testdauer} \times \text{prozentuale Wahrscheinlichkeit} \\ &+ \text{längere Testdauer} \times \text{prozentuale Wahrscheinlichkeit} \end{aligned}$$

Auf das obige Beispiel bezogen:

$$0,9 \cdot 1T + 0,1 \cdot 3T = 1,2 \text{ T.}$$

Bedeutet: Die Testdauer liegt zu einer Wahrscheinlichkeit von 90 % bei 1 Tag, zu einem Restrisiko von 10 % bei 3 Tagen, rechnerisch also bei 1,2 Tagen. Es ist also besser nur 1 Release zu testen anstatt 3, in der Planung ist aber dann ein Faktor von 20 % (über das ganze Projekt hindurch) auf den Wert zu addieren. Bei weiteren Parametern und Einflussgrößen muss man eine komplexere Planung vornehmen.

7.5 Testziele

Ziel eines erfolgreich durchgeführten Tests ist es nicht, dass die zu prüfende Einheit fehlerfrei ist. In einem Testbericht darf daher nie „Die Software hat keine Fehler“ formuliert werden, sondern immer nur „Mit den ausgewählten Testfällen konnte kein Fehler gefunden werden“.

Testen ist kein Prozess, der nachweist, dass das Programm tut was es soll. Und der Zweck des Softwaretests ist es nicht, zu zeigen, dass ein Programm die geforderten Funktionen korrekt ausführt. Nur eine formale Verifikation im Sinne eines mathematischen Beweises würde die Fehlerfreiheit eines Programmes nachweisen.

Bei der Definition der Testziele ist es wichtig, sich diese grundsätzlichen Gedanken zum Softwaretest immer wieder zu verdeutlichen. Diese Form des zielgerichteten Denkens kennt jeder, der schon mal ein Programm geschrieben hat. Es werden Fehler im Programmcode gemacht, die auch bei einer Durchsicht nicht gefunden werden. Ein hinzugezogener, neutraler Beobachter sieht die Fehler aber fast auf den ersten Blick.

Ein ähnliches Phänomen lässt sich beim Schreiben eines Textes zu beobachten. Tippfehler, unklare Formulierungen oder fehlerhafte Satzstellungen werden vom Autor beim Korrekturlesen leicht übersehen, da er vor seinem „inneren Auge“ schon die richtige Form sieht. Der sogenannte „Querleser“ kennt diese innere Form des Autors nicht und kann so die Fehler in der Syntax, die Tipp- und Grammatikfehler korrigieren. Spätestens jetzt sieht auch der Schreiber seine Fehler und fragt sich, wie er das übersehen konnte. Dies kann auch beim Testen von Software passieren, indem die eigene Erwartungshaltung durch das Unterbewusstsein eine Fehlerfreiheit vorgaukelt, die gar nicht vorhanden ist.

Auch hinter einer Aussage wie „Der Test ist erfolgreich durchgeführt worden“ verbirgt sich eine Menge an Psychologie. Die meisten Projektmanager definieren einen Test dann als erfolgreich, wenn keine Fehler gefunden wurden. Das widerspricht aber zum einen der Definition des Testerfolgs in der Theorie, zum anderen ist diese Haltung für den Tester keine allzu hohe Motivation, wenn seine Arbeit erfolgreich war und dies als nicht erfolgreich aufgezeichnet wird. Denn: Ein erfolgreicher Test in diesem Sinne würde ja bedeuten, der Tester habe seine Arbeit umsonst gemacht. Deswegen muss in das definierte Testziel immer der Qualitätsanspruch mit einbezogen werden.

Folgende Eigenschaften soll ein systematischer Softwaretest erfüllen:

- Reproduzierbarkeit: Die Ergebnisse entstehen systematisch und nachvollziehbar und sind nicht Zufällen oder unbekannten Randbedingungen unterworfen.
- Planbarkeit: Es ist möglich, den Testaufwand und den Testnutzen (gefundene Fehler) vorherzusagen.
- Wirtschaftlichkeit: Das Verhältnis von Testaufwand und Testnutzen wird stetig optimiert durch eine realitätsnahe Testumgebung, detaillierte Dokumentation, Einsatz geeigneter Test-Tools und Optimierung der Testprozesse.
- Risiko- und Haftungsreduktion: Der systematische Test gibt Sicherheit, dass kritische Fehler nicht auftreten. Vertragsstrafen, Rückrufe und schwerer Imageschaden wird vermieden.

Das bedeutet in der Praxis folgende anzustrebende Kriterien für den Softwaretest:

- Es existiert eine Testplanung.
- Die einzelnen Testschritte bzw. Testprozeduren sind in einer Testspezifikation eindeutig und umfassend beschrieben.
- Das Programm wird gemäß Testspezifikation ausgeführt.
- Ein Vergleich der tatsächlichen Ergebnisse mit den erwarteten Ergebnissen wird vorgenommen und festgestellte Abweichungen werden genau untersucht.
- Tests, die nicht bestanden wurden, werden nach Behebung der Softwarefehler wiederholt.
- Testergebnisse werden dokumentiert, ggf. mit Screenshots und Logfiles.
- Fehlersuche und Fehlerbehebung erfolgen separat.
- Definierte Testziele beschreiben die Testerfüllung.
- Die Testspezifikation wird laufend erweitert, verfeinert und aktualisiert.

7.6 Anforderungen an das Konfigurationsmanagement

Das Softwaresystem besteht aus einer Vielzahl unterschiedlicher Komponenten und Module, die miteinander kompatibel sein müssen, damit das System als Ganzes funktioniert. Von jedem dieser Bausteine entstehen im Laufe der Entwicklung neue, erweiterte, korrigierte oder verbesserte Versionen oder Varianten. Da an diesem Prozess mehrere Entwickler und Tester parallel beteiligt sind, ist es umso wichtiger, jederzeit den Überblick zu behalten, welche Bausteine aktuell sind und welche Komponenten mit welchen anderen Komponenten zusammengehören. Dafür ist ein ausgereiftes Konfigurationsmanagement unabdingbar.

Wenn in einem Projekt das Konfigurationsmanagement nicht ausreichend umgesetzt wird, entstehen dadurch folgende typische Probleme:

- Entwickler überschreiben gegenseitig ihre am Quellcode oder anderen Dokumenten vorgenommenen Änderungen, da der gleichzeitige Zugriff auf gemeinsame Dateien nicht verhindert wird.
- Es ist nicht transparent, welche Versionen des Source Codes einer bestimmten Komponente im Entwicklungsteam existieren und welche davon gerade aktuell sind.
- Es ist unklar, welche Versionen verschiedener Einzelmodule zusammenpassen und zu einem größeren Teilsystem integriert werden können. Das verhindert Integrationsarbeiten und verzögert nachfolgende Teststufen.
- Compiler und andere Entwicklungswerkzeuge werden in unterschiedlichen Versionen eingesetzt.
- Wenn Änderungen am Quellcode einer Komponente gegenüber der Vorversion unbekannt sind, werden Fehleranalyse, Fehlerkorrektur und Regressionstest erheblich erschwert.
- Es ist nicht mehr ersichtlich, welche Testfälle zu welchem Versionsstand eines Testobjekts gehören und welche Testergebnisse bei einem Testlauf an einem Testobjekt einer bestimmten Version aufgetreten sind. Dadurch ist eine Testauswertung nicht möglich und ein Reporting mit Interpretation der Testergebnisse nicht ausführbar.

Wenn man nicht mehr nachvollziehen kann, welche Änderung von wem, wann und weshalb durchgeführt wurde, und nicht mehr klar ist, wie der Stand vor und nach der Änderung war, entsteht im betrieblichen Ablauf schnell ein chaotischer Zustand.

Folgende Anforderungen bestehen aus Sicht des Softwaretests an das Konfigurationsmanagement:

- **Versionsverwaltung:** Katalogisieren, Speichern und Abruf unterschiedlicher Versionen eines Objekts und Verwaltung von Kommentaren, aus denen die jeweilige Änderungshistorie eindeutig hervorgeht.
- **Konfigurationsverwaltung:** Bestimmung und Administration aller Dateien (Objekte) in der jeweils passenden Version, die zusammen ein Teilsystem bilden.
- **Statusverfolgung von Fehlern und Änderungen:** Aufzeichnung von Problemberichten und Änderungsanforderungen und die Möglichkeit, deren Umsetzung an den Konfigurationsobjekten einwandfrei nachzuvollziehen.

7.7 Testendekriterien

Bereits im Vorfeld der Testdurchführung muss definiert sein, wann man mit dem Test zu Ende sein will. Zu jeder Studie, jeder Untersuchung und jeder wissenschaftlichen Ausarbeitung gehört am Anfang die genaue Definition und Abgrenzung des Untersuchungsgegenstands. Das trifft auch für den Test in Software-Projekten zu.

Das Testende kann z. B. erreicht sein, wenn

- alle Testfälle laut Spezifikation durchlaufen wurden,
- alle Testfälle einer bestimmten Priorität oder einer gewissen Risikogruppe getestet wurden,
- über einen festgelegten Zeitraum von Softwaretests kein neuer Fehler gefunden wurde,
- strukturelle Kriterien erfüllt sind (z. B. ein definierter Anweisungsüberdeckungsgrad nachgewiesen wurde),
- der vereinbarte und budgetierte Testaufwand geleistet ist [[ISTE2009](#)].

Es ist Aufgabe des Testmanagers, ein sinnvolles Testende zu definieren, das sich an Funktionsumfang, Terminen und Kosten genauso wie am Qualitätsanspruch orientieren muss. Das ist auch wichtig, um das Projektziel abzugrenzen und die Erwartungen nichts ins Unendliche hochzuschrauben.

Eine allgemeine Definition für Testendekriterien besteht aus folgenden Punkten:

- Eine definierte Testabdeckung ist erreicht.
- Eine definierte Restfehlerzahl wurde unterschritten.
- Pro (Test-)Zeiteinheit wird nur noch eine bestimmte Fehlerzahl gefunden.
- Der Ausliefertermin für das Softwareprodukt ist erreicht.
- Es sind Grenzen für den Abbruch bzw. die Weiterführung eines Tests definiert: Wenn ein Entwickler mehr als n Fehler (z. B. $n = 30$) zu korrigieren hätte, soll dies zu einem Testabbruch führen müssen; Fortsetzung erst, wenn der Entwickler weniger als m Fehler (z. B. $m = 3$) zu bearbeiten hat.
- Die Zahl der in einer Zeiteinheit (z. B. 1 Woche) gefundenen Fehler ist geringer als die in dieser Zeiteinheit abgeschlossenen Fehlerbearbeitungen [[HAHS2014](#)].

Einzelnen Testmethoden können spezielle Testendekriterien zugeordnet werden.

Zum Beispiel im Modell der „Category Partition Method“, das selbst relativ allgemein gehalten ist, werden folgende Kriterien für Testendekriterien beschrieben:

- Jede Kombination von Wahlmöglichkeiten muss einmal getestet werden.
- Jede Exception (Programmausnahmesituation, Fehler) muss einmal getestet werden.
- Die Testsuite sollte jeden Pfad oder Zweig mindestens einmal durchlaufen, dies wird „Method Scope Branch Coverage“ genannt.

Tab. 7.1 Planungsbasierte und entscheidungsbasierte Testkriterien

Kriterium	Planungsbasiert	Entscheidungsbasiert
Testzeitpunkt	Nach der Entwicklung	Während der Entwicklung
Anwendung von Testmethoden	Vorgabe durch das Testmanagement	Best Practices durch das Testteam
Standards, Normen	Ja	Nein
Orientierung	Testprozess	Produkt/Nutzer
Definition von Testfällen	Vollständig vor Testbeginn	Gestaltbar bis Testausführung
Software-Änderungen	Sonderfall	Kurzfristig = Normalfall
Testorganisation	Abgrenzung Entwicklung versus Testen	Entwicklung integriert Testen
Zu testende „Losgröße“	Gesamtes Produkt	Inkrement bis zum Gesamtprodukt
Rolle des Testmanagers	Zuteilung konkreter Arbeitspakete	Management des selbstorganisierten Ablaufs
Stellenwert der Qualität	Qualität wird erst im Test bewertet	Qualität ist Teil der Software-Entwicklung

Testkriterien lassen sich wie in Tab. 7.1 dargestellt nach planungsbasierten und entscheidungsbasierten Testkriterien klassifizieren.

7.8 Testorganisation

Die Testorganisation beschreibt Rollen und Termine im Testprozess. Grundlegende Festlegungen für das Testmanagement werden mit der Testorganisation beschrieben.

Zusammengefasst werden die Fragen wann, was, wie, wer, womit und wo testet mit Testorganisation definiert (Abb. 7.2).

Eine gute Testorganisation ist eine notwendige Voraussetzung zur Beschleunigung von Abläufen und zur effizienten Gestaltung von Testaktivitäten.

Um Testfälle wirkungsvoll automatisieren zu können, müssen die Testfälle reproduzierbar und standardisiert sein. Die Testabläufe müssen klar und umfassend dokumentiert sein, um externe Dienstleister und Zulieferer optimal einzubinden. Die verwendeten Begrifflichkeiten müssen als Glossar in der Teststrategie eindeutig definiert sein.

Inzwischen gibt es verschiedene Bewertungsmethoden, die Unternehmen einen Rahmen liefern, um den Status-quo des Testmanagements zu ermitteln und es schrittweise zu optimieren. Die Methoden, beispielsweise TMMi, TPI, MMAST, TAP, TMap verstehen sich dabei als Leitfaden. Testmanager bekommen einen Überblick darüber, wie ausgreift ihr Testmanagement derzeit ist, ob geplante Investitionen in bestimmte Verbesserungsmaßnahmen aktuell überhaupt sinnvoll sind oder ob in der momentanen Situation nicht andere Schritte Vorrang haben.

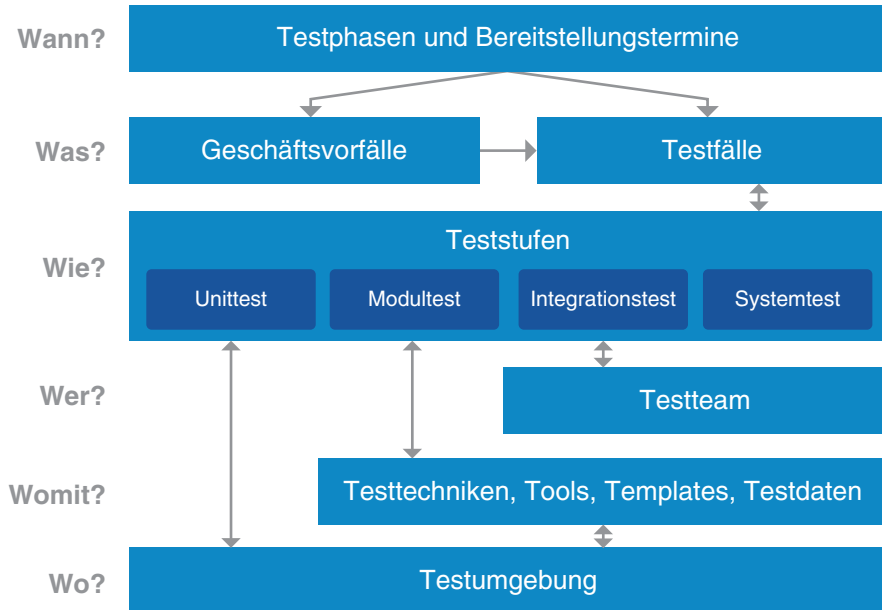


Abb. 7.2 Testorganisation

Literatur

- [ISTE2009] http://www.iste.uni-stuttgart.de/fileadmin/user_upload/iste/se/people/schmidberger/downloads/Grundlagen_Testen-110_k.pdf
- [BOCH2010] Bochynek N, Delgado JD (2010) Testen in der Finanzwelt. Diaz & Hilterscheid, Berlin. isbn:978-3-00-028082-5
- [HAHS2014] <http://michael.hahsler.net/stud/done/nest/Diplomarbeit.pdf>

Das Requirements Engineering dient zum systematischen Management und Verfahren der Anforderungsanalyse und Anforderungsverfolgung und stellt die Grundlage erfolgreicher Softwareprojekte dar. Effiziente und fehlerarme Systeme benötigen klar beschriebene, nicht widersprüchliche und vollständige Anforderungen. Requirements Engineering umfasst die Ermittlung, Analyse, Spezifikation und Validierung der Eigenschaften und Rahmenbedingungen eines Softwaresystems, die über seinen gesamten Lebenszyklus gewünscht werden bzw. relevant sind.

8.1 Grundsätze des Anforderungsmanagements

Die Aktivitäten für den Softwaretest müssen mit der Requirementphase beginnen und die Entwicklung von Anfang an begleiten. Selbst wenn noch keine Zeile codiert wird, kann man sich mit dem Testkonzept, der Strukturierung der Testfälle, der Ermittlung sinnvoller Testdaten beschäftigen. Dieser Aufwand ist erheblich.

Anforderungen (Requirements) legen die qualitativen und quantitativen Eigenschaften eines Produkts fest. Diese Anforderungen beschreiben Eigenschaften oder Bedingungen (üblicherweise vom Auftraggeber festgelegt), die erforderlich sind, um ein Problem zu lösen oder ein Ziel zu erreichen. Anforderungen bezeichnen auch Eigenschaften oder Bedingungen, die ein System oder eine Systemkomponente erfüllen muss um einen Vertrag, eine Norm, eine Spezifikation oder andere formal festgelegte Dokumente oder dokumentierte Repräsentationen der beschriebenen Eigenschaften und Bedingungen zu erfüllen.

Die Anforderungsbeschreibung legt dabei fest, was das Produkt können soll, wie es aussieht und mit welchen Mitteln es realisiert werden soll, nicht aber wie es realisiert wird. Es wird zwischen funktionalen Anforderungen („Was soll das zu entwickelnde Softwaresystem tun?“) und nichtfunktionalen Anforderungen („Welche Eigenschaften soll das zu

entwickelnde Softwaresystem zusätzlich zur Funktionalität aufweisen?“) unterschieden. Auch Vorschriften und Standards (z. B. VDE, DIN, TÜV) zählen zu den nichtfunktionalen Anforderungen.

Funktionale Anforderungen enthalten

- Funktionalitäten:
 - Algorithmen
 - Abläufe
- Daten:
 - Eingaben
 - Ausgaben
 - Datenmodelle
- Schnittstellen:
 - Benutzerschnittstellen
 - Schnittstellen zu anderen Systemen

Nichtfunktionale Anforderungen beinhalten

- Leistungsanforderungen:
 - z. B. Anzahl Benutzer, Datenumfang, Antwortzeiten, Prozesshäufigkeit
- Qualitätsanforderungen:
 - z. B. Zuverlässigkeit, Sicherheit, Robustheit, Portierbarkeit
- Realisierungsanforderungen:
 - Entwicklung, Entwicklungsmethode
 - Aufwand
 - Richtlinien
 - Abnahme
 - Wartung
 - Dokumentation

Wenn man die Testfälle bereits in der Phase des Requirements Engineering formuliert, kommt es früher zur Aufdeckung unklarer Anforderungen. Das hat Rückwirkungen auf einen schnelleren Entwicklungsprozess und führt zu weniger Fehlern bei der Testdurchführung.

Zahlreiche Studien haben ergeben, dass immer noch zwischen 30 und 40 % der gesamten Entwicklungszeit darauf verwendet werden, Fehler aufgrund unspezifischer Anforderungen wieder auszubügeln.

Beim Requirements Engineering sind Reviews besonders wichtig. Sonst wird zwar getestet und auch ein Testumfang nachgewiesen, es wird sogar offiziell nachgewiesen, dass jedes zu testende Requirement von einem Testfall abgedeckt ist – aber der Test ist sinnlos, weil er das Requirement nicht vollständig nachweist und die Anforderung nicht exakt abbildet.

Im Anforderungs- oder Requirement-Management werden die Eigenschaften oder die Leistung der Software definiert und verwaltet. Wenig oder falsch verstandene Anforderungen führen zu ineffizienten Entwicklungsprozessen, Nacharbeiten und im schlechtesten Fall zum Scheitern von Projekten. Sehr wichtig ist beim Requirements Engineering die Abstimmung zwischen Softwareentwicklern und den Fachabteilungen über den gesamten Entwicklungsprozess hinweg beim Sammeln, Dokumentieren, Priorisieren und Validieren von Anforderungen. Indem frühzeitig alle „Betroffene“ in die Entwicklung eingebunden sind, wird die Kluft zwischen Business und IT überbrückt, was letztlich auch der Softwarequalität nützt.

Erfahrungsgemäß verleiten so manche Metriken dazu, sich zu früh zufrieden zu geben und genau die entscheidende Voraussetzung des Requirements Engineering zu übersehen. Das Management und der Endkunde sind zunächst zufrieden, wenn die Testabdeckung auf dem Papier nachgewiesen ist. Rein formal ist dann getestet worden, aber Qualität ist doch nicht erreicht. Das ist eine der Hauptursachen für Fehler, die im Live-Betrieb erst auftreten und dann besonders teuer werden, ein Punkt, der deshalb besonderes Augenmerk erfordert.

8.2 Kriterien für die Formulierung von Requirements

Professionelles Requirements Engineering ist die Voraussetzung für gute Testprozesse. Folgende Grundregeln sind beim Formulieren von Requirements zu beachten. Je besser sie eingehalten werden, desto einfacher lassen sich daraus das Testkonzept und die Testprozeduren ableiten:

- Jedes Requirement muss mindestens einem Anforderer zugeordnet werden. Wenn es niemanden gibt, warum muss es dann überhaupt erfüllt werden?
- Jedes Requirement muss eindeutig und klar formuliert sein, damit eine auftragsbezogene Umsetzung und eine Kalkulation des Aufwands möglich sind.
- Die Anforderungen sind klar strukturiert und in Kapitel und Unterkapitel aufgeteilt.
- Jedes Requirement muss getestet werden können (Fit Criterion), so dass für den Abnahmetest ein fest definiertes Kriterium gilt. Das bedeutet, dass die Wertebereiche der zu testenden Parameter klar beschrieben sein müssen.
- Jedes Requirement muss von allen Stakeholdern (Auftraggeber, Entwickler, Tester) in gleicher Weise verstanden werden und keinen Interpretationsspielraum zulassen.
- Jedes Requirement muss eindeutig (mit einer ID) identifiziert sein. Diese ID darf bei einer Löschung oder Änderung der Anforderung nicht gelöscht werden, sondern sie muss als gelöscht Requirement erkennbar bleiben.
- Jedes Requirement muss in kurzen Sätzen verständlich beschrieben sein und darf keine verschachtelten Formulierungen oder Nebensätze mit komplizierten Konstruktionen enthalten.

- Jedes Requirement muss positiv formuliert sein, da eine negative Formulierung schwer testbar ist.
- Jedes Requirement muss klar definiert sein, Formulierungen wie „sollte“ oder „könnte“ sind zu vermeiden. Es müssen eindeutige Kriterien bestimmt sein, für die das Requirement gilt.
- Die einzelnen Anforderungen sind vollständig, widerspruchsfrei und verständlich formuliert (sofern es möglich sind Bilder statt Worte und natürliche Sprache statt Formeln zu verwenden).
- Die Requirements müssen begründet sein.
- Die Requirements müssen eine nachvollziehbare Historie besitzen. Es muss erkennbar sein, wann ein Requirement hinzugekommen ist, geändert wurde, was sich zu welchem Zeitpunkt geändert hat und ob es noch gültig ist.
- Jedes Requirement hat einen Status (z. B. Draft, Approved, Rejected, etc.).
- Das Softwarewerkzeug zur Verwaltung der Requirements verfügt über eine permanente Schnittstelle zu anderen Systemen zur Softwarepflege (z. B. Mercury Quality, Borland Silk).
- Jede Anforderung weist eine Quelle auf.
- Jede Anforderung ist versioniert. Auch nachträglich lassen sich neu hinzugekommene, geänderte oder entfernte Anforderungen leicht ermitteln.
- Das verwendete Tool bietet eine automatisierte Konsistenz und Vollständigkeitskontrolle an.
- Aus den Anforderungen können einfach und ohne großen Aufwand Links auf externe Webseiten, Prototypen und Detailbeschreibungen hinterlegt und wieder aufgefunden werden.
- Mehrere Benutzer können gleichzeitig mit dem Werkzeug zur Verwaltung der Anforderungen arbeiten.

Im Rahmen des Requirements Engineering ist zu prüfen, ob es Widersprüche zwischen einzelnen Requirements gibt. Daher ist auf eine definierte Struktur der Anforderungen zu achten. In einem meiner früheren Projekte waren die meisten Requirements am Ende in einem Kap. „Sonstige fachliche Anforderungen“ formuliert und nicht dem einzelnen Objekt zugeordnet worden. Entsprechend unstrukturiert sah im Endergebnis die Software aus. Die Requirements müssen vollständig sein und dürfen keine impliziten Anforderungen, die ein Stakeholder im Kopf hat, oder Rahmenbedingungen enthalten, die allgemein vorausgesetzt werden. Das ist vor allem dann problematisch, wenn die Beteiligten schon lange im selben Unternehmen und derselben Systemumgebung arbeiten und viele Fakten unbewusst als selbstverständlich voraussetzen.

Die Requirements müssen dafür auch in einem definierten und versionierten Dokument zusammengefasst werden. Wenn während der Entwicklungsphase in Meetings oder bei Systemtests bemerkt wird, dass bestimmte Aspekte nicht hinreichend beschrieben sind, ist

es wichtig, das an einer zentrale Stelle in der Anforderungsverwaltung auch weiterzufolgen und nicht einen Schattenprozess zu beginnen.

Wenn Requirements in einem Tool festgelegt werden, aber dann doch in einem Meeting oder einer E-Mail zwischen Kunde und Entwickler Abweichungen festgelegt und Nebenabsprachen getroffen werden, ist die Testbarkeit der Anwendung nicht mehr gewährleistet. Speziell darauf wird in vielen Unternehmen noch zu wenig geachtet. Oft bestehen schon langjährige Beziehungen zwischen Kunde und Entwicklungschef, und man scheut manchmal den Aufwand die gesamte Prozesskette (über Systemrequirements bis Softwaretest) wieder aufzurollen. Meist rächt sich das aber in unvollständigen Tests oder nicht klar definierten, zu erwarteten Ergebnissen. Das Handling von Change Requests im Projekt gestaltet sich teilweise schwierig und komplex. Auch dafür ist ein Prozess zu definieren. Dabei geht es unter anderem um die Administration der Change Requests, die Abstimmung der Anforderungen mit dem Auftraggeber, die Kalkulation der zusätzlichen Aufwände und der Überprüfung der Auswirkungen und den Nachweis im Test [EC4U2014].

Leider wurde die Notwendigkeit dieser Basisanforderungen in der Praxis noch immer nicht konsequent genug umgesetzt. Sofern der Testmanager darauf hinwirken kann, auch zu einer späteren Projektphase hier noch Optimierungen zu erzielen, sollte es getan werden, denn mit einem unklaren Anforderungsmanagement sind Fehler in späteren Phasen vorprogrammiert.

8.3 Requirements Engineering in agilen Projekten

In agilen Projekten gibt es spezielle Herausforderungen an das Requirements Engineering, die mit folgenden Grundsätzen beschrieben werden:

- Spezifikation der Details so spät wie möglich,
- keine Umsetzungssicht,
- der Detaillierungsgrad wird durch das Risiko und den zeitlichen Abstand zur Umsetzung gesteuert,
- Effizienz im Requirements Management,
- Änderungen akzeptieren und konsequent umsetzen.

Der spätestmögliche sinnvolle Zeitpunkt zur Erstellung der Spezifikation ist der, wenn die Spezifikation wirklich die Umsetzung beschreibt, also ein Zeitpunkt ab dem sich die Voraussetzungen nicht mehr ändern, der aber noch vor der Programmierung der Funktionen ist, da sonst kein Informationsvorsprung durch die Realisierung mehr erreicht wird. Dazu kann die Testspezifikation als Detailspezifikation genutzt werden, in der dann erst kurz vor Beginn der einzelnen Sprints Testfälle beschrieben werden, anhand derer die Entwicklung

testgetrieben voranschreitet. Das ist in der Praxis aber nur dort möglich, wo eine enge Zusammenarbeit von Anwendungsentwicklung und Softwaretest gegeben ist.

Dabei ist aber zu beachten, dass vertraglich kritische Themen, die zur Umsetzung der Anforderungen erforderlich sind, noch vor Vertragsabschluss geklärt sein müssen. Bei größeren Projekten oder bei der Zusammenarbeit mehrere Teams werden Schnittstellenspezifikationen zur reibungslosen Projektabwicklung teilweise schon zu einem früheren Zeitpunkt benötigt.

Da sich Requirements und Testspezifikation zu einer Gesamtspezifikation ergänzen, ist es sinnvoll, die Details nicht in der Anforderungsspezifikation, sondern in der Testspezifikation festzuhalten. Bei agilen Projekten sind modellbasierte Testkonzepte Anforderungsspezifikationen aus einer anderen Sicht. Das bedeutet, dass in der Testspezifikation grundlegende Requirements-Spezifikationstechniken angewendet werden können. Durch diese Methode können Testfälle mit hoher Wahrscheinlichkeit konsistent gehalten werden, da Abweichungen zum Code bei der Testdurchführung sofort auffallen und dann geändert werden müssen. In einem weiteren Schritt kann man daraus auch die Benutzerdokumentation teilweise automatisieren. Man muss im gesamten Projekt also nicht die Requirementsspezifikation, die Testspezifikation und das Benutzerhandbuch zu 100 % erstellen, sondern kann das Testdokument als Grundlage verwenden, um andere Dokumente modular aufzubauen, was Aufwand bei der Dokumenterstellung einspart.

In der Praxis passiert es mitunter, dass die Entwicklung die Systemarchitektur und teilweise sogar die Requirements-Phase überholt, dass also das Produkt schon vorhanden ist obwohl es noch gar nicht ausreichend definiert ist.

Hier empfehle ich ein pragmatisches Vorgehen. Mit den in den Lehrbüchern vorgeschlagenen Modellen kommt man dabei nur bedingt weiter. Man muss in solchen Fällen (die in der Praxis eher die Regel als die Ausnahme sind) zusammen mit der Testdurchführung die Requirements und die Systemarchitektur nachziehen und sehr eng mit den Abteilungen zusammenarbeiten, die Lücken sukzessive füllen und immer wieder Rückkopplungsprozesse beachten. Dazu ist gerade bei Problemen dieser Art eine transparente vertrauensvolle Kommunikation zwischen allen Beteiligten erforderlich. Dann ist zwar vielleicht kein ideales Softwareprojekt mehr möglich, aber man kann eine halbwegs stabile Basis schaffen. Dieses Vorgehen bedeutet aber in jedem Fall Zusatzaufwand, der auch transparent dargestellt werden sollte.

Das Requirement-Management muss sehr dynamisch zwischen Kunden bzw. beauftragender Abteilung und Requirements Engineer ablaufen. Gerade hier vergeht oft sehr viel Zeit, um Anforderungen formell nachzuziehen und auch zeitnah in die Dokumentationen aufzunehmen. Das wird dann manchmal nach hinten geschoben, d. h. man hat am Ende zwar ein fertiges Softwareprodukt, aber kein Lastenheft und kein Pflichtenheft, das den erreichten Zustand exakt beschreibt. Bei Erweiterungen oder Änderungen der Software rätselt man dann erneut, warum eine Funktion in einer bestimmten Weise implementiert ist oder ob sie überhaupt in dieser Form gewünscht war. Besser wäre es natürlich, diese Lücken nach der Auslieferung genau zu überprüfen und dann ggf. später nachzuziehen, wenn diese Aktivitäten im Projekt hinten runtergefallen sind, weil niemand mehr

dazu Zeit hatte. Leider ist aber gerade das dann meistens – aufgrund mangelnden Budgets – nicht der Fall. Also setzt man dann beim nächsten Entwicklungsprojekt wieder auf einer lückenhaften Basis auf.

8.4 Beispiele zur Formulierung von Requirements

Am Beispiel eines Fahrzeugs sollen im Folgenden schlechte Requirements exemplarisch aufgezeigt werden:

- Das Fahrzeug muss einen niedrigen Verbrauch haben.
- Das Fahrzeug muss mindestens fünf Sitze haben.
- Das Fahrzeug soll einen hohen Sicherheitsstandard haben.
- Das Fahrzeug muss eine ausreichend große Ladefläche haben.

Das sind Requirements, die Interpretationen offen lassen, die weder ein Entwickler noch ein Tester genau nachprüfen kann.

Besser wäre es die Anforderungen wie folgt zu formulieren:

- Der durchschnittliche Kraftstoffverbrauch bei 100 km Landstraße ohne Beladung darf 7 Liter pro 100 Kilometer nicht überschreiten.
- Bei einer freien Ladefläche von mindestens 2,20 m · 1,50 m müssen 6 Personen transportiert werden können.
- Die standardisierten Crash-Tests müssen den Nachweis der höchsten Sicherheitsstufe = 5 Sterne erfüllen.
- Der Laderaum muss mehr als 1,85 Meter und weniger als 2 Meter hoch sein.

Am Beispiel einer Buchhaltungs-Software wäre ein schlecht formuliertes Requirement „Das System muss alle Geschäftsvorfälle richtig verbuchen“. In diesem Fall muss auf die exakten Buchungsregeln referenziert werden.

Man sagt in diesem Zusammenhang, auch Requirements sollten „SMART“ sein:

- *S*pecific – Anforderungen müssen exakt definiert sein.
- *M*asurable – Requirements müssen messbar und testbar sein.
- *A*ssignable – Es ist zu definieren wer die Anforderung umsetzt.
- *R*ealistic – Die Erwartungen müssen realistisch sein und die vorhandenen Umgebungsvariablen und Ressourcen berücksichtigen.
- *T*ime-related – Die Zeit zur Zielerreichung muss spezifiziert sein.

Abb. 8.1 zeigt das Requirements Engineering im Entwicklungsprozess.

Der Auftraggeber sollte ein hohes Eigeninteresse daran haben, Requirements gut zu formulieren, um ein Produkt zu erhalten, welches seine Vorstellungen und Anforderungen auch erfüllt.

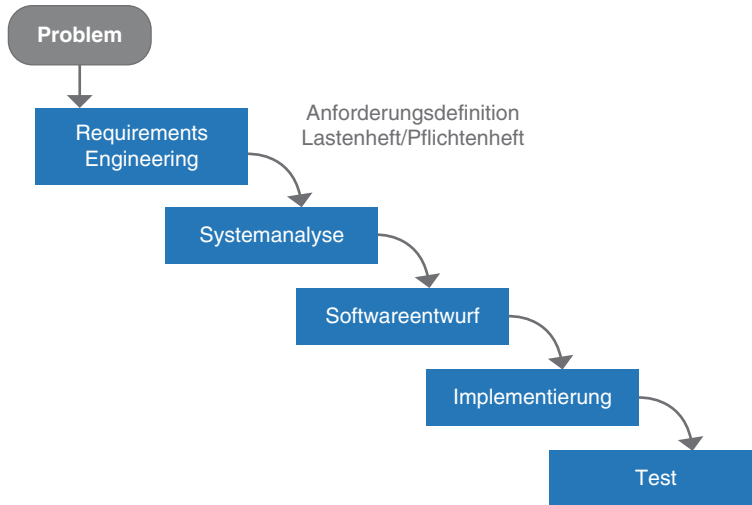


Abb. 8.1 Requirements Engineering im Entwicklungsprozess

8.5 Nachträgliche Erstellung von Requirements

Es kommt leider immer noch vor, dass ein Test starten soll aber gar keine Anforderungen oder nur sehr lückenhaft definierte Requirements vorliegen. Teilweise sind Trivialitäten zwar beschrieben, aber die Besonderheiten fehlen oder es gibt Interpretationsspielräume bei der Auslegung der Anforderungen. Die erste Frage, die sich in so einer Situation stellt, wäre: Wie programmieren die Entwickler überhaupt, wenn sie nicht wissen, was vom Endprodukt gefordert ist?

Der Tester muss in diesem Fall darauf drängen, dass Requirements erstellt werden müssen. Der Tester sollte auch allen Beteiligten klar aufzeigen, dass der Test immer nur so gut oder so schlecht sein kann wie die Basis. Lückenhafte, interpretierbare, widersprüchliche Anforderungen sind dabei sehr problematisch. Es kann auch sein, dass der Tester eine fertige Anwendung vorfindet, die schon getestet worden ist, die aber nun um weitere Features erweitert worden ist oder portiert wurde und daher neu getestet werden muss. Es gibt aber angeblich keinerlei Anforderungen, oder niemand findet sie, oder sie wurden vor Jahren aufgenommen und nie mehr aktualisiert.

Streng genommen, müsste der Tester den Test ganz ablehnen: Woher soll man wissen, was das System können soll, wenn der Tester es nur erraten kann? Aber andererseits muss man von einem Tester auch erwarten können, dass er sich in den Endnutzer hineinversetzt und selber das System zu verstehen versucht.

Dem Tester bleibt also gar nichts anderes übrig, als Anforderungen nachträglich zu erstellen, dabei bestimmte Annahmen zu treffen und sie sich dann vom Auftraggeber bestätigen zu lassen. Wenn dazu keine Zeit mehr ist, dann muss man das transparent darstellen: Es wird eine bestehende Applikation getestet und der Tester definiert teilweise eigenständig

das erwartete Ergebnis. Dass das ein großes Risiko sein kann, weil das evtl. an den Bedürfnissen des Auftraggebers vorbeigeht, sollte dann immerhin bis ins Management hinauf bekannt sein.

Zum Glück wird dieser Fall inzwischen immer seltener, da das Bewusstsein für konsequentes Requirements Engineering in den Unternehmen permanent wächst. Es ist eine der Situationen, die in keinem Lehrbuch und keinem modellhaften Ansatz vorkommen, aber mit denen erfahrene Softwaretester doch hin und wieder werden umgehen müssen.

8.6 Probleme beim Requirements Engineering

Die Requirements müssen klar, exakt und vollständig definiert sein. Beim Anforderungs-Review muss also der Tester überprüfen, mit welchen Mitteln er das Requirement abtesten kann und wie er sicherstellt, dass das Requirement auch wirklich erfüllt ist.

In den meisten Projekten ist es ja nicht so, dass gar keine schriftlich niedergelegten Anforderungen existieren – oft sind diese lückenhaft, widersprüchlich, schwer interpretierbar oder unverständlich.

Wenn der Tester diese Fehler erkennt, ist es nicht ausreichend, einfach nur die lückenhaften Requirements zu prüfen und sich darauf zurückzuziehen, dass eben nicht mehr beschrieben war und genau das was lückenhaft und unscharf beschrieben war ja 1:1 getestet wurde.

In diesem Fall muss der Ersteller der Testspezifikation bereits Rückfragen stellen:

- Was bedeutet dieses Requirement genau?
- Was soll bei bestimmten Nebenbedingungen geschehen?
- Bestimmte Randbedingungen fehlen in der Beschreibung der Requirements, wie soll der Tester damit umgehen?
- Anforderungen widersprechen sich – was soll der Tester dann prüfen und welches Ergebnis wird erwartet?
- Dieselbe Anforderung ist mehrfach beschrieben. Auch das ist ein Problem, denn wenn eine doppelt beschriebene Anforderung dann ergänzt, erweitert oder geändert wird und es passiert nur an einer Stelle, dann betrifft das auch den Test des Gesamtsystems. Daten sollten grundsätzlich immer nur einmalig am Entstehungsort erfasst werden. Aber noch immer wird die Mehrzahl der Daten mehrmals im Prozess erfasst, werden Daten übertragen, stehen in unterschiedlichen Systemen und Tabellen. Das alles senkt die Produktivität weit mehr als es allgemein erkannt wird.

Beim Testreporting muss immer auch die Qualität der Requirements mit einbezogen werden. Wenn die Basis des Systems schon lückenhaft ist, wie soll dann ein erfolgreicher Test überhaupt durchgeführt werden können? Der Ersteller des Testkonzepts muss also regelmäßig beim Auftraggeber zurückfragen, wenn er etwas nicht verstanden hat. Das zeigt, dass man sich mit der Materie intensiv beschäftigt hat und als Softwaretester über den

Tellerrand blickt. Der Kontakt zwischen Tester, Requirements Engineer und Auftraggeber ist aber häufig kaum vorhanden, man hat die Requirements vom Kunden irgendwann erhalten und Rückfragen gehen über eine zwischengeschaltete Instanz. An diesem Punkt sieht man eindrucksvoll, wie sehr eine nicht vorhandene oder nur rudimentäre Kommunikation ein Projekt erschweren kann.

Dabei muss man sich immer gut in den Endkunden hineinversetzen. Vor allem die nicht idealen Umweltbedingungen sind zu berücksichtigen – ein System verhält sich draußen in der Kälte, in der schmutzigen Produktionshalle oder im Bergbaustollen anders als im sauberen klimatisierten Büro unter Laborbedingungen. Ich hatte einmal ein Warenwirtschaftssystem zu testen, das in einer Hofeinfahrt einer Kohlehandlung von einem Lagerarbeiter bedient wurde, der im Winter eine Lampe über der Tastatur hängen hatte damit ihm die Finger beim Tippen nicht einfroren. Solche Bedingungen waren beim Test in der Testumgebung im Büro nie simuliert worden.

Man sollte immer davon ausgehen, dass der Enduser kein Informatiker ist, sondern in erster Linie Anwender. Man sollte beim Systemdesign und beim Test immer an Personen denken, die mit der Anwendung wahrscheinlich Probleme haben. Vor allem Fehlermeldungen müssen so aussagefähig sein, dass der User weiß wie er damit umzugehen hat und was er als nächstes zu tun hat bzw. wie er den Fehler selber beheben kann. Die Userfreundlichkeit ist ein wesentlicher Aspekt beim Test, und die Testabdeckung richtet sich auch daran, welche Art von Enduser der Nutzerkreis sein wird – ein System das nur von wenigen Ingenieuren benutzt wird, die langjährige Experten in ihrem Fachgebiet sind, ist anders zu testen als eine App für ein Smart Phone.

Oft sind gerade triviale Anforderungen sehr deutlich formuliert, während komplexere Sachverhalte in den Requirements nur lückenhaft oder beispielhaft beschrieben sind. Manchmal enthält ein definiertes und getracktes Requirement gleich mehrere Anforderungen – bekommt lediglich eine Nummer, besteht aber streng genommen aus mehreren Requirements. Dann bedeutet das, dass für solch ein Pseudo-Sammel-Requirement auch mehrere Testfälle benötigt werden.

Manchmal sind Requirements auch doppelt, hin und wieder widersprechen sie sich. Daher ist es so wichtig, dass der Tester von Anfang an beim Requirements-Prozess dabei ist: Der Tester muss verstehen, was die Anforderung machen soll, damit er im Nachhinein effizient arbeiten kann, sinnvolle Testfälle definieren und planmäßig testen.

Teilweise gibt es auch auf Kundenseite unterschiedliche Anforderungen. Das bedeutet, dass der Auftraggeber das Requirement dann abschwächt oder unklar formuliert. Wenn das beim Test auffällt, ist darauf hinzuwirken, dass diese Formulierungen detailliert werden. Eine geänderte Systemumgebung oder eine unklare Beschreibung der Systemumgebung können ebenfalls zu Problemen führen. Die Konfiguration und die Umgebungsvariablen müssen ebenso messbar und nachvollziehbar dokumentiert sein wie die Anforderungen an die Software an sich.

Eine besondere Problematik entsteht, wenn der Endnutzer ein externer Kunde des Anbieters ist und die Kommunikationswege sehr formalisiert ablaufen. Dann werden manchmal die Ausnahmen in Meetings vom Vertrieb und Einkauf abgesprochen, die sich dann

nicht in den formulierten Requirements wiederfinden, weil man zeitbedingt nicht einen extra Change Request erstellen und bewerten will. Der Tester bekommt dann teilweise gar nichts davon mit, was ein Software-Projektleiter mit dem Endkunden alles vereinbart hat. Dann wird es vielleicht gar nicht getestet bzw. nur einmalig getestet und ist beim Regressionstest der nächsten Software vergessen.

8.7 Review der Requirements

Wenn die Anforderung nicht exakt umrissen ist, dann kann der Test keinen exakten Nachweis führen. Fehler im Anforderungsmanagement tragen sich in die weiteren Teststufen fort. Der Tester muss sich daher fragen, ob er die Requirements wirklich vollständig verstanden hat. Beim Review der Requirements muss der Tester vor allem darauf achten, wie er daraus Testfälle generieren kann und ob nicht wesentliche Anforderungen vergessen wurden.

Auch wenn die Requirementphase eventuell lückenhaft war, besteht bei der Erstellung des Testkonzepts die Möglichkeit, Rückfragen zu stellen und die Anforderungen genauer zu formulieren. Wenn man erst mal bei der Testdurchführung ist und feststellt, dass der Test die geforderte Systemeigenschaft nicht genau oder nur teilweise gewährleisten kann, ist es einerseits schon viel teurer geworden um entsprechend gegenzusteuern, andererseits aber immer noch nicht zu spät.

Der Tester sollte sich daher nicht scheuen, bei unklaren Requirements Rückfragen zu stellen und diese bei den Reviews abzuweisen bzw. erläuternde Bemerkungen zu verlangen.

Beim Requirements Engineering muss auf eine genaue Zuordnung der Anforderung zum Testobjekt geachtet werden. Häufig werden Releases undefiniert oder neu aufgeteilt. In diesem Fall muss die Zuordnung zum entsprechenden Release aktualisiert werden und bei der Planung der Testaktivitäten berücksichtigt werden, so dass der richtige Umfang an Anforderungen im entsprechenden Release getestet wird. Wenn hier Fehler unterlaufen, werden Anforderungen zu früh getestet, die noch gar nicht implementiert sind. Das führt zu nicht unerheblichen Kostensteigerungen und Terminverzug. Oder aber es bleiben Funktionen ungetestet, weil die Releaseplanung sie in einem anderen Release verortet hat.

Leider wird vor allem das Requirements Engineering grandios unterschätzt. Der Tester muss eine „kundenzentrierte Betrachtung der Software“ vornehmen – im Systemtest ist der Tester der Kunde und der Tester muss sich überlegen, was er erwarten würde, wenn er die Software für viel Geld gekauft hätte. Aber selbst bei unvollständigen oder mangelhaften Requirements kann man immer noch das Pferd von hinten aufzäumen: Was bewirkt die Anwendung, was ist der Zweck der Software, welche Zielgruppe beschreibt den User? Wenn man sich in die Zielgruppe hineinversetzt kann man von der Anwendung, von den Funktionen her die man an der Oberfläche sieht, sinnvolle Testfälle entwickeln. Je besser sich der Tester in den Endanwender hineinversetzt und sich überlegt, welche Funktion er inne hat (also ist es der Autofahrer, der Mitarbeiter in der Buchhaltungsabteilung, der Verkäufer, der Paketbote ...?), umso besser wird der Systemtest sich auf das Wesentliche konzentrieren.

Wenn in der Konzeptphase aufgrund löchriger Requirements Testfälle erstellt wurden, ist es umso wichtiger, sie dem Auftraggeber bzw. dem Projektmanagement vorzulegen und bei den Testaktivitäten auch darauf zu verweisen wo eigenständig bestimmte Annahmen getroffen wurden. Man sollte in jedem Fall darauf drängen, für die Begleitung des Requirements Engineering auch für die Testabteilung genügend Zeit bereitgestellt zu bekommen. Das wird sich später im Testablauf in jedem Fall auszahlen.

8.8 Nebenabsprachen und Schattenprozesse

Häufig kommt es während der Entwicklung zu Rückfragen an den Auftraggeber, weil die Requirements das Produkt nicht vollständig beschreiben, sondern einen Interpretationsspielraum offen lassen.

In diesem Fall müssen der Prozess zur Klärung der Rückfragen und die Kommunikationswege klar definiert sein. In vielen Projekten existieren neben dem offiziellen Anforderungskatalog E-Mails, interne Vermerke und Meetings, bei denen nicht alle Beteiligten einbezogen wurden. Man sollte unbedingt darauf achten, wo und wie solche Festlegungen dokumentiert werden. Wenn Festlegungen, die für den Systemtest wichtig sind, als Kommentar im Quellcode stehen, werden sie beim Test nicht berücksichtigt. Generell sind weitere parallele Anforderungsdokumente und ein „Schattenprozess“ unbedingt zu vermeiden, sonst können sich in vielen Bereichen von Softwaredesign über Softwaretest bis zur Wartung erhebliche Probleme ergeben.

Es muss auch darauf geachtet werden, dass Festlegungen schnell getroffen werden können und dass das Upgrade der Anforderungsdokumente zeitnah erfolgen kann. Erfahrungsgemäß dauert das offizielle Verfahren manchmal so lange, dass man gar nicht mehr fertig würde, wenn man geänderte Anforderungen oder Detaillierungen der Spezifikation nicht sofort berücksichtigen würde. Die Konsequenz ist, dass dann darauf manchmal ganz verzichtet wird, d. h. man hat zwar ein Anforderungsdokument aber es spiegelt nur teilweise die Anforderungen wieder, der Rest der Anforderungen befindet sich als unterschiedliche Informationen im Unternehmen und in den Köpfen verteilt. Wenn man in solchen Projekten dann versucht, Teile der Aufgaben auszulagern, an Externe oder offshore zu vergeben, kann es dann sehr problematische Auswirkungen geben.

8.9 Verzögerungen beim Requirements Management

Manchmal sind aufgrund der komplexen Systemumgebung die Anforderungen noch gar nicht fertig ausformuliert, die Entwicklung aber bereits fortgeschritten. In diesem Fall überholt die Entwicklung das Requirements Management. Besonders wenn Altsysteme durch eine neue Applikation abgelöst werden, kann das der Fall sein. Man implementiert ein Produkt und passt es an die Anforderung aus einem neuen System an, ohne vorher die Requirements exakt berücksichtigt zu haben.

Gerade wenn ein System erweitert oder durch eine neue Technologie abgelöst wird, die bereits in einem anderen Projekt eingesetzt wird, besteht die Gefahr, dass die Entwicklung schneller ist als das Requirements Management oder die Systemarchitektur.

In diesem Fall empfiehlt sich ein pragmatisches Vorgehen, das aber möglichst nahe am Modell bleibt und immer wieder dazu zurückkehrt. Wenn die Entwicklung schneller ist, dann ist es wichtig, Requirements nachträglich abzuleiten und als interne Systemrequirements zu definieren. Es reicht nicht, wenn die Anwendung nur codiert ist und die zu Grunde liegenden Produktanforderungen „in der Luft hängen“.

Wenn man schon den idealtypischen Zustand nicht erreicht, so ist es umso wichtiger, dass alle Wünsche, Anregungen und Änderungen die während des Entwicklungs- oder Testprozesses in das Produkt eingeflossen sind, zumindest nachträglich ihren Niederschlag in den Requirements finden.

Wie immer geht es bei der Anwendung von Modellen aus Lehrbüchern nicht darum, sie zu 100 % umzusetzen, sondern dem Ideal so nahe wie möglich zu kommen und sich dafür pragmatisch dem Optimum zu nähern.

Literatur

[EC4U2014] <http://www.ec4u.de/wp-content/uploads/2012/01/Leitfaden-Systematisiertes-Anforderungsmanagement.pdf>

Der Test eines komplexen Softwaresystems wird in mehrere Phasen (Teststufen) unterteilt. Die Einordnung der Teststufen (auch Testzyklen genannt) folgt häufig dem Entwicklungsstand des Systems. Es geht dabei darum, dass der Test einer bestimmten Funktionalität so früh wie möglich erfolgen soll. Da aber nicht jeder Test in jeder Teststufe schon möglich ist, muss man diese Teststufen eindeutig abgrenzen und prüfen, welcher Test in welchem Testzyklus sinnvoll ist.

9.1 Modultest

Ein **Modultest** (auch als **Unittest** bezeichnet) wird in der Softwareentwicklung durchgeführt, um funktionale Einzelteile (Module) von Computerprogrammen zu testen, d. h. sie auf korrekte Funktionalität zu prüfen.

Da Algorithmen auf Modulebene meist nur eine begrenzte Komplexität aufweisen und über klar definierte Schnittstellen aktiviert werden, können sie mit relativ wenigen Testfällen ziemlich umfassend getestet werden. Dies gilt als Voraussetzung für die anschließende Teststufe des Integrationstests, um dort die Testfälle auf das integrierte Zusammenwirken größerer Funktionsteile oder der gesamten Anwendung ausrichten zu können. Die modulspezifischen Detailkonstellationen lassen sich damit auf Stichproben beschränken, was die Anzahl der erforderlichen Testfälle drastisch reduziert. In der Regel wird der Modultest vom Entwickler durchgeführt.

Vorteilhaft am Modultest ist, dass er früh im Entwicklungsprozess durchgeführt und gut mit Werkzeugen unterstützt werden kann. Modultests lassen sich leicht in den Entwicklungsprozess integrieren. Der Modultest reicht aber nicht mehr aus, wenn mehrere Module oder Komponenten und deren Interaktion getestet werden soll.

Beim Modultest existieren evtl. die aufgerufenen bzw. die aufrufenden Programmteile noch nicht. Daher werden Testtreiber eingesetzt, die den Teil des Systems simulieren, der die Testkomponente benutzt. Komponenten, welche die zu testende Komponente aufrufen, werden Teststümpfe genannt. Da manchmal der Aufwand, Teststümpfe zu erstellen, fast so groß wäre wie die Komponente selber zu erstellen, wird das System meist „Bottom-up“ erstellt, also im Modultest bewusst nur mit einer geringeren Testabdeckung durchgeführt. Für Unittests werden Testtreiber (driver) benötigt, welche die Komponente mit Eingaben versorgen bzw. Ausgaben entgegennehmen. Als Platzhalter (stubs) werden diejenigen Komponenten bezeichnet, die von dieser Komponente aufgerufen werden.

Teilweise wird der Begriff Komponententest auch für den Modultest verwendet. Das ist aber nicht ganz exakt, denn der Komponententest prüft bereits das Zusammenspiel einzelner Klassen. Der Komponententest ist also bereits eine weitere Teststufe und wird manchmal vor dem Integrationstest ebenfalls durchgeführt.

Auf der Modul- und Komponentenebene steht der Quellcode zur Verfügung. Diese Testebene wird auch als Entwicklertest bezeichnet, da diese Teststufe in der Regel von den Entwicklern der Applikation durchgeführt wird. Testfallentwurfsverfahren auf Quellcode-Ebene werden auch „White-Box-Tests“ genannt.

9.2 Integrationstest

Der Begriff **Integrationstest** bezeichnet in der Softwareentwicklung eine aufeinander abgestimmte Reihe von einzelnen Tests, die dazu dienen, verschiedene voneinander abhängige Komponenten eines komplexen Systems im Zusammenspiel miteinander zu prüfen. Die erstmals im gemeinsamen Kontext zu testenden Komponenten haben jeweils einen Modultest und den Komponententest erfolgreich bestanden und sind für sich isoliert fehlerfrei funktionsfähig.

Beim Integrationstest muss ein Verständnis über die Abhängigkeiten und Integrationschritte vorhanden sein. Die Software-Architektur muss klar sein, die Schnittstellen spezifiziert und der Datenaustausch zwischen den einzelnen Komponenten getestet sein. Für den Integrationstest erstellt man nun Testszenarien, die das Zusammenwirken der beiden betroffenen Komponenten zeigen. Die Anzahl der Integrationsstufen richtet sich nach der Komplexität der Software.

Der Integrationstest deckt z. B. inkompatible Schnittstellenformate oder unterschiedliche Interpretationen übergebener Daten auf. Er weist auch auf Durchsatz- oder Lastprobleme hin, wenn Daten zwar richtig übergeben werden, aber zum falschen oder verspäteten Zeitpunkt oder in zu schnellen Zeitintervallen.

Für den Integrationstest unterscheidet man verschiedene Verfahren:

- Urknalltest: Alle Komponenten werden einzeln entwickelt und dann in einem Schritt integriert. Das erschwert allerdings die Fehlersuche.
- Die Bottom-Up-Integration erfolgt inkrementell in umgekehrter Richtung zur Benutzt-Beziehung. Dazu sind Testtreiber, aber keine Teststümpfe erforderlich. Nachteilig ist,

dass Fehler in der obersten Schicht sehr spät entdeckt werden, wobei die oberste Schicht aber die Logik der Anwendung enthält. Dennoch ist diese Integrationsart in der Praxis am häufigsten anzutreffen.

- Bei der Top-Down-Integration erfolgt die Integration in Richtung der „Benutz-Beziehung“, also von der Ebene, die das Modul aufruft. Man entdeckt Fehler in der obersten Schicht, also logische Fehler, sehr früh, benötigt aber Teststümpfe.
- Bei der Sandwich-Strategie kombiniert man den Top-Down- mit dem Bottom-Up-Ansatz: Die Zielschicht wird als zu testende Schicht zunächst identifiziert. Dann werden individuelle Komponententests durchgeführt. Im nächsten Schritt werden kombinierte Schichttests vorgenommen: Test der obersten Schicht mit der Zielschicht, Test der Zielschicht mit der unteren Schicht, schließlich alle Schichten zusammen [KOSCH2006].

Abb. 9.1 zeigt die Testarten und Testinhalte im V-Modell.

Wenn man ein sehr komplexes und umfangreiches Softwaresystem hat, muss man mehrere Integrationsstufen bewerkstelligen. Man integriert also sukzessive mehrere Komponenten, Dabei ist zu beachten, die Testaktivitäten immer in der Stufe durchzuführen, wo sie das erste Mal anfallen. An dieser Stelle müssen sie auch getestet werden, weil sich sonst der Fehler „fortpflanzt“. Anders herum macht es aber auch keinen Sinn, denselben Funktionstest an mehreren Stellen durchzuführen.

Für die Software-Integration sind die einzelnen Komponenten genau zu beschreiben: Welche Version von Subsystem A mit welcher Version von Subsystem B bilden das Gesamtsystem?

Im V-Modell werden unterschiedliche Testarten nach ihren Inhalten unterschieden

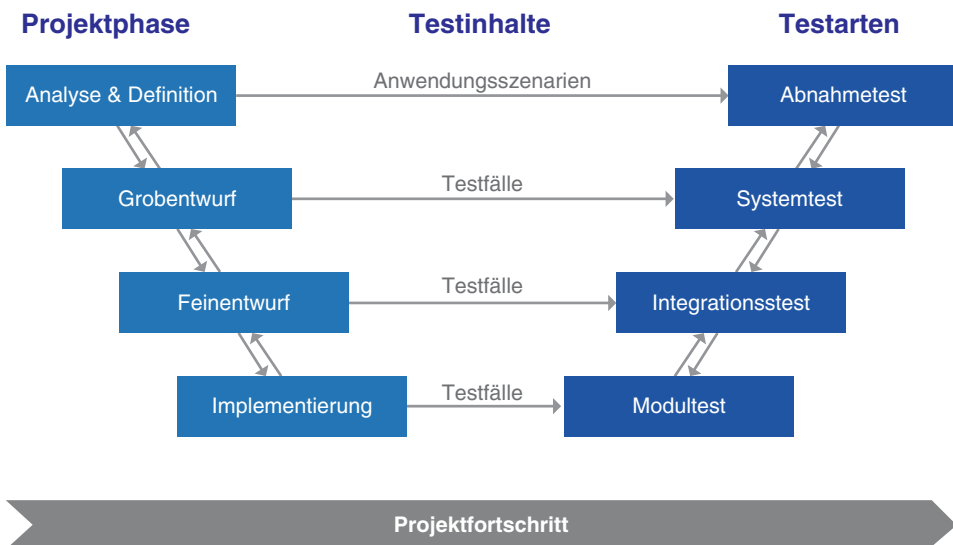
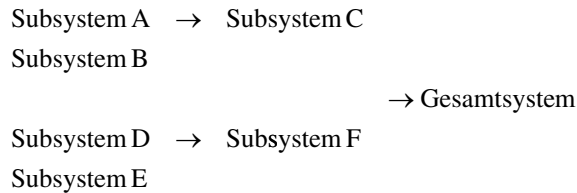


Abb. 9.1 Testarten und Testinhalte im V-Modell

Manchmal gibt es folgendes Vorgehen:



Angenommen, ein Feature in Subsystem A stand noch nicht bereit als der Test für Subsystem C durchgeführt wurde. Der Gesamtsystemtest wird durchgeführt, und danach eine Änderung im Subsystem A nachträglich geliefert oder ein Fehler behoben.

In diesem Fall muss erst der Test von Subsystem C und dann der Test für das Gesamtsystem wiederholt werden. Ansonsten ist nicht gewährleistet, dass das Gesamtsystem noch funktioniert.

Nur das Gesamtsystem zu testen, wäre in diesem Falle nicht ausreichend, weil man dann den Fehler nicht exakt lokalisieren kann und ein späterer Upgrade nur von Subsystem C nicht mehr definiert möglich wäre. Die Wiederholbarkeit der Integrationstests der einzelnen Subsysteme muss unbedingt gegeben sein.

An dieser Stelle hört man oft den Einwand „Aber es war doch nur eine ganz kleine Änderung, in einem einzigen Modul, warum den ganzen Ablauf noch einmal aufrollen?“ Gerade wenn es mehrere Integrationsstufen nacheinander sind, mag das auf den ersten Blick verlockend erscheinen, und manchmal sind die Termine so knapp, dass man gar keine andere Wahl hat (etwa wenn ein Programmfehler der sich bereits in der Produktion auswirkt kurzfristig behoben werden muss). Dann ist es aber immer noch besser die Integrationstests aller Integrationsstufen nach dem dringend erforderlichen Rollout durchzuführen als auf den Integrationstest ganz zu verzichten. Unterlassene Integrationstests richten sonst in späteren Phasen wesentlich höheren Schaden an.

Der Aufwand für den Integrationstest wird häufig unterschätzt. Bei einer inkrementellen Entwicklung liefern die einzelnen Komponenten ihre Software meist bis zum Start des Systemtests und werden sukzessive integriert. Bei einer komplexen Software-Architektur sind daher Abhängigkeiten im Vorfeld zu prüfen und beim Integrationstest sukzessive zu integrieren.

Dazu muss aber eine professionelle Integrationsstrategie vorhanden sein: Im obigen Beispiel kann man davon ausgehen, dass Subsystem B im Integrationstest von Subsystem C getestet wird und keine Auswirkungen auf Subsystem F hat. In der Praxis ist aber gerade das in vielen Fällen nicht der Fall, weil die einzelnen Subsysteme sich untereinander beeinflussen.

9.3 Systemtest

Der **Systemtest** ist die Teststufe, bei der das gesamte System gegen die gesamten Anforderungen (funktionale und nicht funktionale Anforderungen) getestet wird. Gewöhnlich findet der Test auf einer Testumgebung statt und wird mit Testdaten durchgeführt.

Die Testumgebung soll die Produktivumgebung des Kunden simulieren, d. h. ihr möglichst ähnlich sein. In der Regel wird der Systemtest durch die realisierende Organisation durchgeführt.

Für den Systemtest muss eine Testumgebung erzeugt werden, die weitestgehend mit der Produktivumgebung übereinstimmen soll. In jedem Fall sind aber Tests direkt in der Produktivumgebung zu vermeiden!

Zum einen besteht die Gefahr, dass das Produktivsystem durch den Test beschädigt wird und es zu Verzögerungen im betrieblichen Ablauf kommt. Das ist bei Systemen in der Fertigung ein nicht unerhebliches Risiko. Das kann dann zu Produktionsausfällen oder Vermögensschäden führen. Tests in der Produktivumgebung sind meistens nicht reproduzierbar, weil sich die Datenbasis und damit die Ausgangsbedingungen in der Produktivumgebung permanent verändern und eine Produktivumgebung nicht einfach auf den Ausgangszustand vor der Testdurchführung zurückgesetzt werden kann.

Beim Pilotbetrieb auf einer Lieferstelle (also in der Produktivumgebung) hatte ich einmal erlebt, dass ein Beleg mit einem maximalen Betrag von 9,9 Mio. Euro erzeugt wurde, um das Layout des Belegdrucks in der Lieferstelle vor Ort zu prüfen. Man wollte dabei sehen, ob sehr große Beträge im Summenfeld noch ausreichend Platz finden und sich einen visuellen Eindruck vom Beleg verschaffen. Man nahm den Testkunden „Osterhase“ mit Zuordnung zu einer Pseudo-Verkäufern timer. Nach der Kontrolle wurde der Beleg direkt wieder storniert. In der Umsatzstatistik des nächsten Monats stach die Lieferstelle unter allen anderen positiv hervor. Bei der Detailanalyse der exorbitant hohen Daten erst wurde festgestellt, dass der Umsatz des Testkunden „Osterhase“ in die Vertriebsstatistik eingeflossen war. Der Pseudo-Verkäufer bekam dadurch eine hohe Sonderprovision gutgeschrieben. Die direkte Auswirkung der testweisen Fakturierung auf das Buchhaltungssystem war durch die Stornierung zwar aufgehoben worden, aber nicht die Effekte auf die Vertriebsstatistik. Man hatte dadurch zwar den Test positiv bestanden, aber nicht ausreichend die Auswirkungen auf andere Systeme im Hintergrund bedacht.

Im V-Modell wird im Systemtest gegen den funktionalen Testsystementwurf getestet. Platzhalter und Testtreiber kommen nicht mehr zur Anwendung. Beim Systemtest werden funktionale und nicht funktionale Qualitätsmerkmale der Software getestet. Die ISO 9126 unterscheidet hierbei nach Funktionalität, Benutzbarkeit, Effizienz, Änderbarkeit und Übertragbarkeit.

Folgende Aspekte werden beim Systemtest für die funktionalen Anforderungen untersucht:

- Angemessenheit: Eignen sich die verwendeten Funktionen für die vorgesehenen Anforderungen?
- Interoperabilität : Arbeitet das System in der Systemumgebung fehlerfrei?
- Ordnungsmäßigkeit: Werden Normen und Vorschriften (z. B. gesetzliche Anforderungen) eingehalten?
- Richtigkeit: Erzielen die Funktionen die erwarteten Ergebnisse?
- Sicherheit: Sind die Daten und Anwendungen vor unberechtigten Zugriffen oder Kopien geschützt?

Für funktionale Anforderungen gibt es folgende Testansätze:

- **Geschäftsprozessbasierter Test:** Einzelne Geschäftsvorfälle oder die Rangfolge der Prozesse bilden die Basis zur Ableitung und Priorisierung der Testfälle.
- **Anwendungsfallbasierter Test:** Die häufigsten Anwendungen bzw. ihre Reihenfolge dienen als Testbasis.
- **Anforderungsbasierter Test:** Die Testfälle werden aus den Anforderungen abgeleitet.

Das Testen nichtfunktionaler Anforderungen ist oft weniger präzise beschrieben und lässt sich meist schwerer quantifizieren. Sofern aber Anforderungen interpretierbar sind, sind auch die Testergebnisse unklar. Dadurch werden die Testergebnisse nichtfunktionaler Tests eher subjektiv bewertet.

Doch auch hier gibt es abstrakte Ansätze, an denen man sich orientieren kann, z. B. bei der Bewertung von Benutzeroberflächen oder der graphischen Gestaltung von Websites (Logo immer an derselben Stelle, einheitliche Navigationsleisten, klare Regeln zur Verlinkung). Bei Performancetests sollte eine akzeptable Antwortzeit genau definiert sein um sich nicht auf intuitiven Aussagen verlassen zu müssen.

Folgende Tests werden als nichtfunktionale Tests bezeichnet:

- **Lasttest, Performancetest, Massentest, Stresstest:** Bei diesem Test wird das System gezielt an seine Belastungsgrenzen gebracht. Dazu gehört der Test des Antwortzeitverhaltens bei bestimmten Funktionen (z. B. zahlreiche parallele Datenbankaufrufe), die Verarbeitung großer Datenmengen und Dateien, wie sich das System bei dauerhafter Überlastung und bei Wiedereintritt in den Normalbetrieb verhält. Hierbei ist besonders darauf zu achten, dass es keine Abstürze von Funktionen gibt und die Zugriffszeiten akzeptabel bleiben. Es ist entscheidend, realistische Zugriffe zu simulieren. Nach der Einführung eines neuen Webportals der Bundesanstalt für Arbeit stürzte nach wenigen Stunden der Server ab, da man im Simulationsprogramm zwar mit 100 parallelen Usern getestet hatte, aber zeitweise mehr als 10.000 User parallel auf das System zugriffen. Bei Lasttests muss man stets von der maximalen Systemlast ausgehen und den Extremwert noch um ca. 20 % erhöhen und darf keine Durchschnittswerte ansetzen.
- **Datensicherheit:** Schutz gegen unberechtigte Zugriffe auf das System und die Daten.
- **Stabilität:** Systemverhalten bei Dauerbetrieb über einen längeren Zeitraum (z. B. Simulation von permanenten Testabfragen über ein ganzes Wochenende) und das Wiederanlaufverhalten nach Systemausfällen.
- **Robustheit:** Systemverhalten bei Fehlbedienungen, Falscheingaben und Wiederanlaufzeiten.
- **Kompatibilität:** Prüfung, ob das System zu Standards, Normen und Vorschriften kompatibel ist, die Schnittstellen zu anderen Programmen korrekt sind und ob andere Anwendungen durch den Systemeinsatz nicht negativ beeinflusst werden.

- **Benutzbarkeit:** Test der Oberflächen, der Übersichtlichkeit und Verständlichkeit der Anwendungen, der Klarheit der Dokumentation, Verifikation aussagekräftiger kontext-sensitive Hilfetexte bei allen Feldern und der intuitiven Verständlichkeit der Struktur der Applikation (bzw. der Website).
- **Übertragbarkeit:** Portierbarkeit und Installierbarkeit der Software auf andere Systeme, vorhandene Hardwarerestriktionen zum Einsatz des Systems.
- **Änderbarkeit:** Ordnungsgemäße Dokumentation der Anwendung, Einhaltung von Coding-Standards und Grundlage einer modularen Systemarchitektur.

9.4 Abnahmetest

Ein **Abnahmetest**, **Verfahrenstest**, **Akzeptanztest** oder auch **User Acceptance Test (UAT)** ist ein Test der gelieferten Software durch den Kunden/Endanwender bzw. Auftraggeber. Oft sind Akzeptanztests Voraussetzung für die Rechnungsstellung. Dieser Test kann unter Umständen (z. B. bei neuen Anwendungen) bereits auf der Produktionsumgebung mit Kopien aus Echtdaten durchgeführt werden.

Das Ziel des Abnahmetests ist nicht mehr, Fehler aufzudecken. Zum Abnahmetest sollten die Fehler aus vorhergehenden Teststufen bekannt und bereits behoben sein. Der Abnahmetest wird vielmehr dazu verwendet, damit der Kunde Vertrauen in das System gewinnt. Daher ist auch nur eine kleine, eng umrissene Testabdeckung erforderlich. Die Testfälle bestimmt der Kunde selbst. Vor allem häufige und wichtige Anwendungsfälle und typische Geschäftsprozesse, für die der Einsatz der Software erfolgen soll, werden dazu herangezogen.

Die Zielgruppen der Abnahmetests leiten sich aus dem Grad der Individualisierung der Software ab. Individualsoftware wird durch den Kunden getestet. Es müssen alle Anforderungen der einzelnen Anwendergruppen erfüllt werden, je nach Nutzung des Systems. Massenprodukte, Standardsoftware oder Web-Oberflächen (z. B. Online-Shoppingsysteme) werden mit einer repräsentativen Auswahl von Anwendern getestet. Es ist oft schwierig, eine allgemeine Akzeptanz zu erzielen. Daher sollte man typische Anwender mittels Profilen beschreiben (z. B. Alter, Beruf, Online-Affinität, technische Vorbildung) und für jeden dieser Gruppen geeignete Repräsentanten auswählen, die den Abnahmetest nach eindeutig definierten Testfällen durchführen.

Die vertragliche und regulatorische Abnahme der Software durch den Kunden ist oft ein wichtiger Vertragsbestandteil und Meilenstein im Projekt. Durch die Abnahme der Software werden Zahlungsfristen, Wartungsvereinbarungen oder Garantiefristen ausgelöst. Daher ist es für den Projekterfolg entscheidend, nachprüfbare Abnahmekriterien schon bei Vertragsabschluss aufzustellen, damit die Vertragsparteien abgesichert sind und die Erwartungen von vornherein klar definiert sind. Gesetzliche Bestimmungen oder andere Auflagen (z. B. ASIL, CENELEC, BaFin) müssen bei der Abnahme umgesetzt sein, auch ohne explizit im Vertrag erwähnt worden zu sein.

Der Abnahmetest wird teilweise in der Testumgebung des Kunden durchgeführt. In diesem Fall ist die Systemumgebung im Vorfeld exakt zu definieren. Teilweise kann es nämlich noch zu unliebsamen Überraschungen kommen, wenn man feststellt, dass die eigene Systemumgebung bestimmte Konstellationen gar nicht finden konnte, die in der Testumgebung des Kunden bei der Abnahme dann als Fehler aufgedeckt werden.

Sofern Testfälle für den Abnahmetest vom Kunden bereits im Vorfeld offengelegt werden, ist unbedingt zu empfehlen, sie vor der Abnahme schon einmal probeweise durchzuführen. Dann bemerkt man rasch, an welchen Stellen besonderer Handlungsbedarf besteht.

9.5 Mischformen und Abgrenzung der einzelnen Teststufen

Die vorgenannten Teststufen sind in der Praxis oft nicht scharf voneinander abgegrenzt, sondern können, abhängig von der Projektsituation, fließend oder über zusätzliche Zwischenstufen verlaufen. So könnte zum Beispiel die Abnahme des Systems auf der Grundlage von Testergebnissen (Reviews, Testprotokolle) von Systemtests erfolgen.

Besonders für System- und Abnahmetests wird das Blackbox-Verfahren angewendet, d. h. der Test orientiert sich nicht am Code der Software, sondern nur am Verhalten der Software bei spezifizierten Handlungen (Eingaben des Benutzers, Grenzwerte bei der Datenerfassung, etc.).

9.6 Teststufen im V-Modell

Das **V-Modell** ist ein Vorgehensmodell in der Softwareentwicklung, bei dem der Softwareentwicklungsprozess in Phasen organisiert ist. IT-Systeme des Bundes schreiben sie seit 2005 verbindlich für die Planung und Durchführung von IT-Projekten vor. Neben diesen Entwicklungsphasen definiert das V-Modell (siehe Abb. 9.2) auch das Vorgehen für den Softwaretest phasenweise.

Jeder Entwicklungsstufe stehen also genau spezifizierte Testmaßnahmen gegenüber. Im V-Modell erfolgt die Validierung nach der Verifizierung. In vielen Projekten wird lediglich durch eine Testphase verifiziert, d. h. geprüft, ob das Software-Produkt der Spezifikation (z. B. dem Pflichtenheft) entspricht. Oft gibt es aber Missverständnisse beim Abgleich des Lastenheftes (fachliche Anforderungen) mit dem Pflichtenheft (technische Realisierung). Deshalb sollten auch die fachlichen Anforderungen validiert werden, d. h. es ist zu prüfen, ob sie gültig sind, wie sie genau zu verstehen und zu interpretieren sind. Wird z. B. die Unterbrechbarkeit von Dialogabläufen gefordert, kann dies aus fachlicher Sicht anders gemeint sein, als es technisch zu realisieren ist. Tab. 9.1 zeigt die Unterschiede zwischen Validierung und Verifikation.

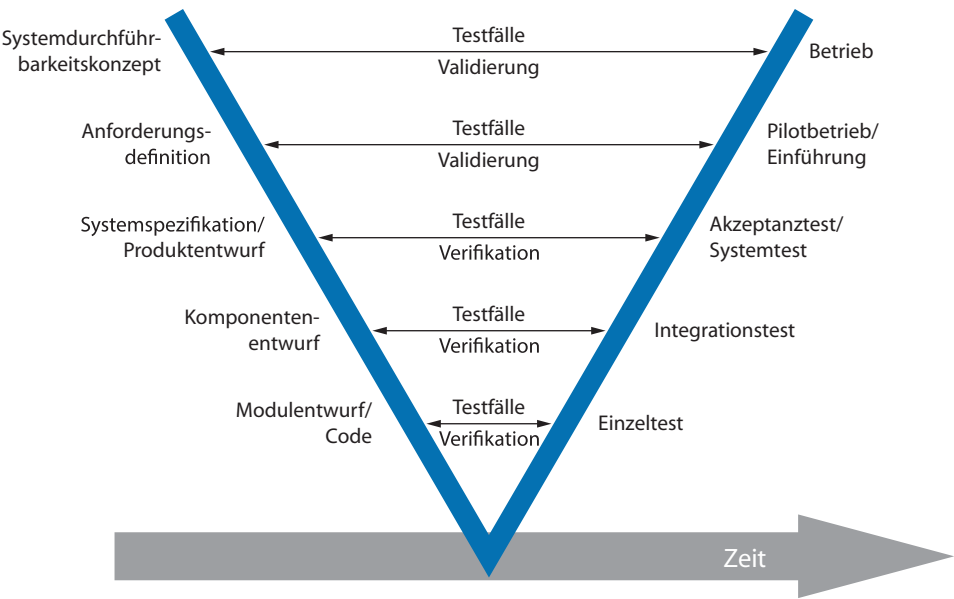


Abb. 9.2 Validierung und Verifikation in den einzelnen Teststufen

Tab. 9.1 Validierung und Verifikation

Validierung	Eignung bzw. der Wert des Produktes bezogen auf seinen Einsatzzweck	Wird ein passendes Produkt entwickelt? „Am I building the right product?“
Verifikation	Überprüfung der Übereinstimmung zwischen einem Produkt und seiner Spezifikation	Wird das Produkt richtig entsprechend den Vorgaben entwickelt? „Am I building the product right?“

- Allerdings hat das V-Modell in der Praxis auch einige Nachteile:
- Die Anforderungen an ein neues System sind zu Beginn nie vollständig bekannt. Daher sind die Requirements häufig lückenhaft und müssen zu einem späteren Zeitpunkt verfeinert und ausführlicher spezifiziert werden.
 - Während der Entwicklung kann sich eine produktive Zusammenarbeit zwischen Benutzern und Entwicklern ergeben, aus der neue Realisierungsmöglichkeiten resultieren. Es gibt Rückfragen, neue Ideen und Vorschläge zur Implementierung, die in die Entwicklung eines besseren Endprodukts direkt einfließen können. Das ist beim V-Modell nicht vorgesehen, das Modell ist relativ starr.
 - Es gibt für eine Anforderung verschiedene Realisierungsmöglichkeiten.
 - Die Realisierung bestimmter Anforderungen lässt sich nicht theoretisch garantieren, das bedeutet, dass ihre Realisierung vorgezogen werden muss.

Inzwischen gibt es neben dem V-Modell andere Ansätze, wie z. B. das agile Testen.

9.7 Blackbox- und Whitebox-Verfahren

Die Prüfmethode für den Softwaretest werden in zwei große Blöcke unterteilt (siehe Abb. 9.3).

Statische Prüfmethode

Statische Prüfmethode dienen zur Verifikation der Software. Verifikation bedeutet in diesem Zusammenhang die Überprüfung der Software auf Übereinstimmung mit der Spezifikation. Diese Methoden lassen sich nicht oder nur sehr schwer automatisieren. Zu diesen Verfahren zählt die Überprüfung der Software auf bestimmte Qualitätsmerkmale, wie die Aussagekraft von Methodenbezeichnungen und Kommentaren. Die Analyse des Quellcodes findet ohne Ausführung der Software statt („Schreibtischtest“).

In sequentiellen Prozessmodellen findet die Testphase erst am Ende der Entwicklung statt. Aus diesem Grund wurden statische Prüfmethode eingeführt, um schon früh kritische Fehler in der Software zu finden. Beispiele für statische Prüfmethode sind die Fagan Inspektion und der Walkthrough, bei denen einige Testfälle erstellt werden und der Quellcode manuell nach Fehlern durchsucht wird. Dabei werden überwiegend Entwurfsfehler gefunden.

Dynamische Prüfmethode

Im Vergleich zu den statischen Prüfmethode werden dynamische Prüfmethode zur Validierung der Software eingesetzt. Bei diesen Verfahren wird die Eignung der Software

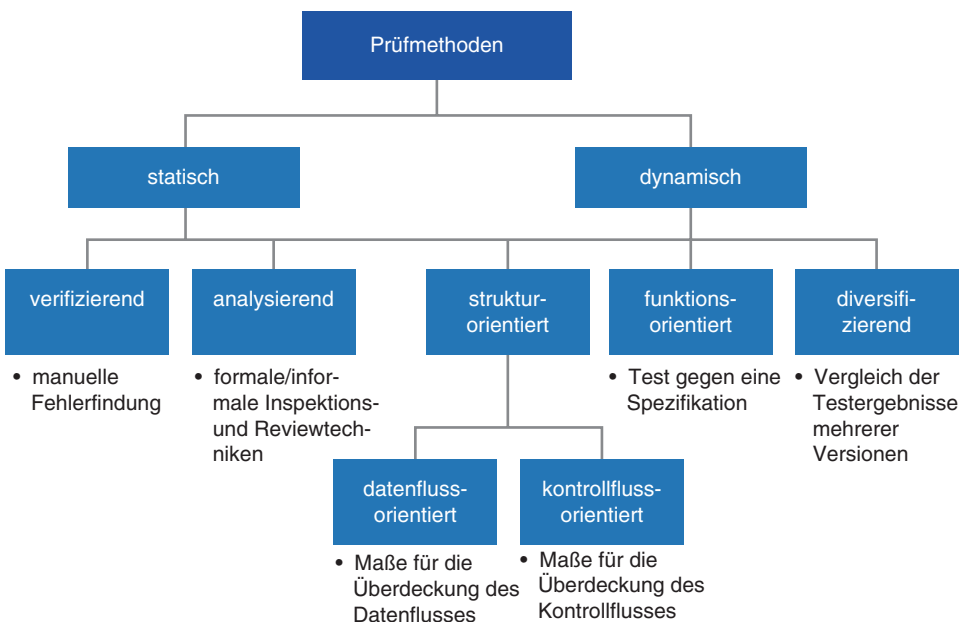


Abb. 9.3 Klassifikation von Testverfahren nach Prüfmethode

bezogen auf den Einsatzzweck untersucht. Mit Test werden hierbei diejenigen dynamischen Prüfmethoden bezeichnet, bei denen die Software zur Ausführung kommt.

Die dynamischen Verfahren besitzen eine weitere Unterteilung bzgl. der Prüftechnik:

- Strukturorientiertes Testen bestimmt die Testfälle auf Basis des Quellcodes. Dabei wird zwischen kontrollflussorientierten Tests, die die Ablauflogik testen, und datenflussorientierten Tests, die die Datenzugriffe kontrollieren, unterschieden.
- Funktionsorientiertes Testen bestimmt die Testfälle auf Basis der Spezifikation. Es wird geprüft, ob die Software die Spezifikation erfüllt. Beim Modultest wird die Software gegen die Modulspezifikation getestet, beim Schnittstellentest gegen die Schnittstellenspezifikation und beim Abnahmetest gegen die Anforderungen des Kunden.
- Diversifizierendes Testen hingegen prüft verschiedene Versionen der Software. Dabei werden die Testergebnisse der verschiedenen Versionen miteinander verglichen. Ein Test gilt als erfolgreich, wenn die Ergebnisse des Tests bei verschiedenen Versionen identisch sind [KNAU2008].

Eine aussagekräftigere Unterteilung der dynamischen Prüfmethoden lässt sich bzgl. des Informationsstandes machen. Dabei wird unterschieden, ob der Tester Kenntnis über den zugrunde liegenden Quellcode besitzt (White-Box-Test) oder nicht (Black-Box-Test).

Beim Blackbox-Verfahren ist der innere Aufbau des Testobjekts nicht bekannt bzw. wird nicht zum Test herangezogen. Testfälle werden aus der Spezifikation abgeleitet oder liegen bereits als Teil der Testspezifikation vor. Die Blackbox-Testverfahren gehen davon aus, dass die Implementierung einer Funktionalität unbedeutend ist. Stattdessen wird beim Test die in der Spezifikation verlangte Funktionalität nachgewiesen. Es handelt sich also um das Testen auf höherer Ebene. Die Blackbox-Tests konzentrieren sich dabei auf die sichtbare Funktionalität des Programms und nicht auf dessen interne Strukturen. Der Black-Box-Test ist benutzerorientiert, da er feststellt, ob das Programm überhaupt funktioniert, nicht wie es programmiert wurde. Der Tester versucht Bedingungen zu finden, bei denen sich das Programm nicht gemäß der zugrunde liegenden Spezifikation verhält. Die Testdaten werden aus der Spezifikation abgeleitet. Das zu testende System wird dabei als Ganzes betrachtet, nur sein Außenverhalten wird bei der Interpretation und Analyse der Testergebnisse herangezogen.

Black-Box-Tests verhindern, dass Entwickler selber Tests „um ihre eigenen Fehler herum“ entwickeln und somit Lücken in der Implementierung übersehen. Ein Entwickler, der Kenntnisse über die innere Funktionsweise eines Systems besitzt, könnte unabsichtlich durch gewisse zusätzliche Annahmen, die außerhalb der Spezifikation liegen, wesentliche Aspekte in den Tests vergessen oder anders als die Spezifikation auslegen. Ein Black-Box-Test eignet sich außerdem als zusätzliche Stütze zur Überprüfung der Spezifikation auf Vollständigkeit, da eine unvollständige Spezifikation häufig Fragen bei der Entwicklung der Tests aufwirft.

Der White-Box-Test kann darüber hinaus zusätzliche, nicht geforderte oder nicht spezifizierte Funktionalität entdecken. Beim White-Box-Test steht dem Tester der Quellcode

zur Verfügung. Es wird die interne Struktur des Programms unter Berücksichtigung der Spezifikation für einen Soll-Ist-Vergleich getestet. Bei den White-Box-Tests wird davon ausgegangen, dass einige Strukturelemente (Anweisungen, Ausdrücke) fehlerbehaftet sind, wodurch eine Überprüfung aller möglichen Pfade nötig wird. Jedoch kann nicht getestet werden, ob auch alle möglichen Pfade implementiert wurden, die in der Spezifikation aufgeführt sind. Eine Automatisierung der Testdurchführung gestaltet sich hierbei meist einfacher als bei Black-Box-Tests. Das strukturbasierte Testen fällt unter die Kategorie der White-Box-Tests.

Im Allgemeinen wird der White-Box-Test eher auf den Modultest, der Black-Box-Test eher auf Integrations- und Systemtest angewendet. Das zeigt die Testpyramide in Abb. 9.4.

Vorteile von Black-Box-Tests gegenüber White-Box-Tests:

- bessere Verifikation des Gesamtsystems,
- Test semantischer Eigenschaften bei geeigneter Spezifikation,
- Portabilität von systematisch erstellten Testsequenzen auf plattformunabhängige Implementierungen.

Nachteile von Black-Box-Tests gegenüber White-Box-Tests:

- größerer organisatorischer und administrativer Aufwand,
- zusätzlich eingefügte Funktionen bei der Implementierung werden nur zufällig getestet,
- Testsequenzen einer unzureichenden Spezifikation sind unbrauchbar.

In der testgetriebenen Softwareentwicklung gibt es als Mischform beider Ansätze den „Grey-Box-Test“. Der Grey-Box-Test wird vom Tester wie ein White-Box-Test geschrieben, jedoch bereits vor der eigentlichen Implementierung des Quellcodes, was ihn mit einem Black-Box-Test verbindet, da noch kaum Informationen über den vorliegenden Quellcode existieren.

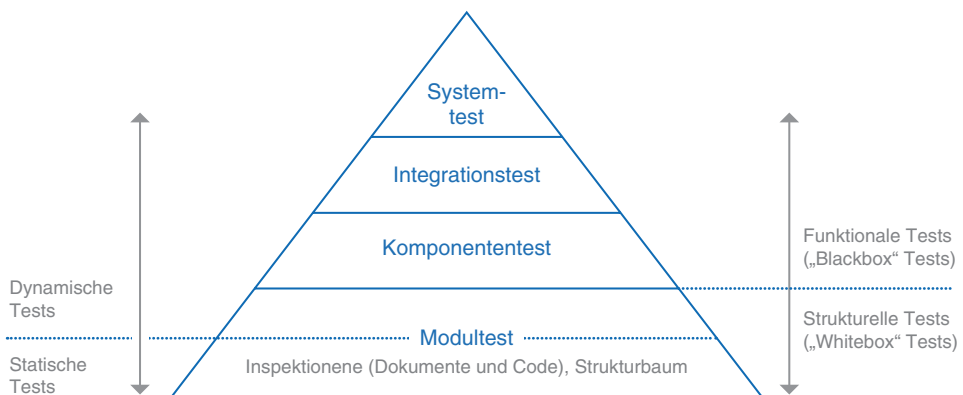


Abb. 9.4 Testpyramide

9.8 Mehrere Teststufen statt „Big Bang“

Die im V-Modell und weiterführenden Modellen, auch agilen Projekten definierten unterschiedlichen Teststufen aber sind in jedem Fall ein Vorteil gegenüber einem einzigen Tests des Gesamtsystems direkt vor dem GoLive. Problematisch ist, wenn der Systemtest zu früh erfolgt, wenn vorher das einzelne Modul nie getestet wurde oder die Modularität der Software nur begrenzt vorhanden ist. Auch wenn der Systemintegrationstest mit einer geringen Testabdeckung durchgeführt wurde, kann es im Systemtest problematisch werden.

Auch wenn die Systemintegration so funktioniert, dass der „Big Bang“ erfolgen kann und nicht sukzessive sinnvolle Teilfunktionen miteinander integriert werden, kann es zu Problemen und Verzögerungen kommen, weil die Fehlerbehebung und Aktualisierung der fehlerhaften Komponenten dann evtl. weitere Änderungen im Systemdesign und der Softwarearchitektur nach sich zieht [BOCH2010]. In der Praxis wird aber gerade auf diese beiden Stufen bei vielen Unternehmen zu wenig Wert gelegt. Man will ja schnell ein funktionierendes Produkt sehen und ausliefern können. Manchmal ergaben sich bei der Entwicklung bereits Verzögerungen und der Einführungsstermin drängt.

Je nach Komplexität der Software muss man die Teststufen anders definieren. Wenn z. B. eine Headunit im Auto, die aus Klimaanlage, Radio, Navigationssystem, Telefon und DVD-Player besteht, getestet werden soll, dann wird ein Integrationstest für jedes der Teilsysteme und der Systemtest für das Gesamtsystem durchgeführt. Man kann dann aber auch „Subsystemtest“ zum Test des Navigationssystems sagen, weil schon in diesem eigenen Teilbereich so viele Funktionen und Teilfunktionen vorhanden sind, dass ein vorhergehender Integrationstest etwa nur für die Komponente „Karte“ oder nur für die Funktion „Routenplanung“ allein sinnvoll ist. Bei Tests vor allem im Automotive-Bereich wird oft der HiL-Test dazwischen geschaltet, der Test „Hardware in the loop“ auf einem Rechner, der die Funktionen im Auto simulieren soll. Der Systemtest ist meist die letzte Chance um vor der Auslieferung noch festzustellen wie die Software arbeitet.

Nach der letzten Phase des Systemintegrationstests ist das Produkt komplett integriert. Das bedeutet, dass es immer noch zu Fehlverhalten kommen kann, die vorher als einzelne Module nur simuliert wurden, nicht bemerkt werden konnten. Durch die Integration immer neuer Komponenten kann es auch durchaus vorkommen, dass Software, die vorher funktioniert hat nun plötzlich nicht mehr funktioniert.

Es empfiehlt sich, alle Tests so früh wie möglich im Testprozess durchzuführen. Das, was in einer bestimmten Testphase schon sinnvoll getestet werden kann, sollte auch in der frühestmöglichen Phase getestet werden und nicht erst am Ende der Prozesskette, weil dieses Vorgehen die Kosten pro Fehler reduziert.

Der Tester sollte immer darauf drängen, dass die Teststufen eindeutig beschrieben sind und alle am Projekt Beteiligten unter diesen Definitionen dasselbe verstehen. Ein eigenes Glossar im Unternehmen bzw. im Projekt empfiehlt sich in jedem Fall. Unterschiedliches Verständnis über bestimmte Begriffe ist eines der Schlüsselprobleme für misslungene Kommunikation im Projekt. Oft scheitert man deswegen an der erfolgreichen Umsetzung, weil mit bestimmten Begriffen verschiedene Vorstellungen assoziiert werden. Das sollte man sich in diesem Zusammenhang immer wieder vor Augen führen.

Dabei sollte man sich möglichst an Standarddefinitionen der Begriffe halten, möglichst so wie sie z. B. in Wikipedia oder in standardisierten Regelwerken aus Normen zu finden sind. Wenn man im Unternehmen oder im Projekt aus gutem Grund von einer Standarddefinition abweicht, dann muss die Begründung und die erstellte Erläuterung exakt dargestellt werden. Gerade in größeren Unternehmen werden Begriffe in unterschiedlichen Projekten aus historischen Gründen manchmal mit unterschiedlicher Bedeutung verwendet. Im Rahmen einer CMMI-Zertifizierung fragte ein Auditor bei einem großen deutschen Elektrokonzern in einer Abteilung fünf Mitarbeiter aus Software-Entwicklung, Software-Architektur und Testabteilung, was sie unter einem Integrationstest verstehen und wie sie ihn von den anderen Teststufen abgrenzen. Er bekam dabei von den einzelnen Mitarbeitern fünf völlig unterschiedliche und in sich widersprüchliche Antworten. Wenn aber das Verständnis über die Abgrenzung der Teststufen nicht vorhanden ist, läuft man Gefahr, dass manche Funktionen doppelt und andere gar nicht getestet werden. Dadurch werden viele Fehler, die viel früher hätten gefunden werden müssen, erst in einer späten Teststufe aufgedeckt. Wenn man hier erfolgreich gegensteuert, kann zum einen der gesamte Testprozess beschleunigt werden, aber auch die Kosten für den Softwaretest in Summe können nachhaltig gesenkt werden.

Literatur

- | | |
|-------------|---|
| [BOCH2010] | Bochynek N, Delgado JD (2010) Testen in der Finanzwelt. Diaz & Hilterscheid, Berlin. isbn:978-3-00-028082-5 |
| [KNAU2008] | http://www.knaupes.net/theorie-der-softwaretests/ |
| [KOSCH2006] | http://www.informatik.uni-bremen.de/st/Lehre/swp0607/testen-1x2.pdf |

Ein Test mit allen möglichen Eingabedaten und deren Kombination wäre grundsätzlich ein möglicher Test. Das ist aber wegen der großen Zahl von möglichen Eingabewerten und Kombinationen unrealistisch, wäre viel zu langwierig und zu teuer. Man muss also eine sinnvolle Untermenge an Testfällen definieren. Zur Auswahl der wichtigsten aus den möglichen Testfällen gibt es mehrere sinnvolle Verfahren.

Im Unittest testet der Entwickler selbst, oder zwei Entwickler testen ihren Code gegenseitig. Hier liegt schon das erste Problem: Ein Entwickler wird seinen Code in aller Regel nur insoweit testen, dass er nicht abstürzt und z. T. nur entsprechend der persönlichen Einschätzung zur Funktionsfähigkeit. Für sämtliche anderen Aspekte (und ganz besonders auch dahingehend, ob die Entwickler wirklich die richtige Umsetzung des Kundenproblems vorgenommen haben), sollte ein vom Entwicklungsteam unabhängiges Testteam eingesetzt werden.

Für den Unittest werden folgende Kennzahlen verwendet:

Line Coverage

Line Coverage misst, wie viele Statements vorgenommen wurden (ein Statement ist normalerweise eine Zeile Code inkl. Kommentare, Bedingungen usw.).

Function Coverage

Function Coverage wird mit den meisten Standard-Debuggern getestet. Sie sagt aus, ob eine Funktion aufgerufen wurde oder nicht. Sie sagt aber nichts darüber aus, wie die Funktion aufgerufen wurde, ob sie richtig arbeitet und die richtigen Werte zurückliefert. Sie sagt auch nichts darüber aus, wie oft diese Funktion aufgerufen wurde.

Die Function Coverage wird mit dem Anweisungsüberdeckungstest nachgewiesen. Diese Testform verlangt, dass alle Anweisungen des zu testenden Programms mindestens einmal ausgeführt werden. Es werden alle Knoten des Kontrollflussgraphen durch einen

Test abgedeckt. Dieser Test besitzt mit 18 % die geringste Fehleridentifizierungsquote. Dies liegt an der fehlenden Berücksichtigung von Kontrollstrukturen und den Datenabhängigkeiten zwischen verschiedenen Programmteilen. Als eigenständiges Testverfahren ist der Anweisungsüberdeckungstest nicht ausreichend geeignet, findet aber Anwendung im Zweigüberdeckungstest.

Branch Coverage

Branch Coverage misst, ob der WAHR- und FALSCH-Zweig für jede Bedingung durchlaufen wurde (IF/WHILE/FOR).

Beispiel

```
public int getNameLength(Boolean is CoolUser) {  
    User user = null;  
    If (isCoolUser) {  
        User = new John();  
    }  
    return user.getName().length();  
}
```

Wenn man nun die Funktion isCoolUser auf „wahr“ setzt, bekommt man 100 % Statement Coverage. Aber man wird einen Null-Pointer bekommen wenn man sie mit „falsch“ aufruft. Man sieht also dass mit einer 100 %-igen Line Coverage nur 50 % Branch Coverage erreicht hat, und dass sowohl der Test als auch der Programmcode unvollständig ist.

Decision Coverage

Die Decision Coverage prüft, ob jeder Eingangs- und Ausgangspunkt innerhalb des Programms mindestens einmal durchlaufen wurde. Dadurch wird getestet, ob alle möglichen Ausgangsvariablen, die das Programm durchlaufen muss mindestens einmal angesteuert wurden.

Bei mehreren Parametern, die kombiniert werden müssen, sind die Pfade mehrmals zu durchlaufen:

```
if (a or b) and c then
```

Die Entscheidungskriterien sind in diesem Fall wie folgt erfüllt:

- a = true, b = true, c = true
- a = false, b = false, c = false

Die Decision Coverage ist aber nicht ausreichend nachgewiesen, denn der Wert von b im zweiten oder der Wert von c in der dritten Bedingung würden das Ergebnis nicht beeinflussen. Also muss man diese Anweisung mit folgenden Parametern testen, um die Decision Coverage zu prüfen:

- `a=false, b=false, c=true`
- `a=true, b=false, c=true`
- `a=false, b=true, c=true`
- `a=false, b=true, c=false`

Um alle möglichen Kombinationen aller Variablen zu testen, ist ein Test auf Multiple Condition Coverage erforderlich.

Im obigen Beispiel wären in diesem Fall folgende acht Bedingungen zu prüfen:

- `a=false, b=false, c=false`
- `a=false, b=false, c=true`
- `a=false, b=true, c=false`
- `a=false, b=true, c=true`
- `a=true, b=false, c=false`
- `a=true, b=false, c=true`
- `a=true, b=true, c=false`
- `a=true, b=true, c=true`

Die Decision Coverage wird mit dem Zweigüberdeckungstest nachgewiesen.

Bei diesem Test werden sämtliche Zweige des Kontrollflussgraphen mindestens einmal durchlaufen. Dabei besteht der Nachteil, dass Kombinationen von Zweigen nicht geprüft werden und mehrfache Schleifendurchläufe nicht ausreichend geprüft werden. Zusätzlich kann es Schwierigkeiten bereiten, Tests mit den passenden Daten zu erstellen, um alle Zweige zu erreichen. Die Fehleridentifizierungsquote für diese Testform liegt bei 34 %.

Condition Coverage

Die Condition Coverage prüft, ob jeder boolesche Parameter auf einen wahren und einen falschen Wert getestet wurde. Dabei ist darauf zu achten, dass die Entscheidungskriterien eindeutig sind. Ein boolescher Parameter muss immer entweder wahr oder falsch sein. Das bedeutet im Umkehrschluss, dass sich nur eindeutige Alternativen in Booleschen Parametern abbilden lassen.

Der einfache Bedingungsüberdeckungstest weist die Condition Coverage nach. Dabei werden die Entscheidungsstrukturen des Programms geprüft. Es wird verlangt, dass Testfälle existieren, in denen einmal eine Teilentscheidung den Wert „true“ und im anderen Fall den Wert „false“ annimmt. Nicht berücksichtigt werden geschachtelte Bedingungen; ist bei einer „oder“-Verknüpfungen der erste Vergleich wahr und bei einer „und“-Verknüpfung der erste Wert falsch, findet keine Überprüfung des zweiten Vergleichs statt. Somit können Fehler im zweiten Ausdruck nicht gefunden werden.

Daher wurde der Mehrfach-Bedingungsüberdeckungstest entwickelt. Diese Testform fordert, dass auch alle zusammengesetzten Entscheidungen gegen wahr und falsch geprüft werden. Der Nachteil besteht in der hohen Anzahl an Testfällen: Bei n Teilentscheidungen entstehen 2^n -Testfälle. Ohne Testautomatisierung lässt sich Condition Coverage daher kaum sinnvoll und kostengünstig testen.

Path Coverage

Path Coverage testet, ob jeder mögliche Weg durch einen bestimmten Programmteil einmal ausgeführt wurde. Sie wird mit dem Pfadüberdeckungstest gemessen. Sämtliche Pfade des Programms müssen mindestens einmal durchlaufen werden. Problematisch wird dies bei Schleifen mit unbekannter Wiederholungszahl (while-Schleife), da die Anzahl der Pfade ins unendliche ansteigen kann. Aufgrund dessen spielt der Pfadüberdeckungstest in der Praxis keine bedeutende Rolle.

Zustandsbasiertes Testen

Das zustandsbasierte Testen wird für alle Zustände einer Komponente verwendet:

- Komponente wird in den zu testenden Zustand gebracht
- Test für alle möglichen Stimuli:
 - korrekte Eingaben,
 - fehlerhafte Eingaben,
 - Aktionen, die Zustandsübergänge bewirken [[SNEE2012](#)].

Literatur

[SNEE2012] Sneed HM, Baumgartner M, Seidl R (2012) Der Systemtest, Von den Anforderungen zum Qualitätsnachweis. Hanser, München

Fehler gehören zum Menschsein. Fehler sind Teil der menschlichen Existenz und ein entscheidender Grund für seine enorme Entwicklung. Die Geschichte der Menschheit ist geprägt von Neugier, Wagemut und Irrtümern. Menschen machen kleine und große Fehler, bzw. Fehler mit kleinen und großen Auswirkungen. Wenn Christoph Columbus nicht den Fehler begangen hätte zu glauben, er müsse von Spanien nach Richtung Westen segeln um Indien zu erreichen, wäre Amerika nicht entdeckt worden. Von J. W. von Goethe stammt das Zitat „Wenn Du nicht irrst, kommst Du nicht zu Verstand“.

Beim Test von Software werden Fehler bemerkt. Diese Fehler müssen erfasst, beschrieben, teilweise mit unterschiedlichen Projektmitarbeitern besprochen, behoben und nachgetestet werden. Für diese Tätigkeiten muss ein Prozess definiert sein. Dabei geht es darum, Fehler vollständig zu beheben und die Kosten und die Bearbeitungszeit für die Fehlerbehebung so kurz wie möglich zu gestalten. Zu einem konstruktiven Umgang mit Fehlern sollte man Fehler nicht als Versagen oder Scheitern, sondern als notwendige Investition auf dem Weg zu besseren Lösungen betrachten.

Fehler konstruktiv zu erkennen kann dazu verhelfen, sie bereits im Vorfeld zu vermeiden und sie im Testprozess in einer möglichst frühen Phase zu eliminieren. Wer Fehler systematisch erfasst und analysiert und mit seinem Testteam konstruktiv nach Lösungen sucht, schafft Vertrauen und eine gute Arbeitsatmosphäre. Wenn Fehler langfristig verringert und vermieden werden sollen, müssen wir unsere Denk- und Handlungsweise ändern und unser traditionell negatives und destruktives Fehlerdenken in ein positives Lösungsdenken und gesundes Verhalten umwandeln.

11.1 Definition von Fehlern

Ein Fehler ist:

- jede Abweichung der tatsächlichen Ausprägung eines Qualitätsmerkmals von der vorgesehenen Soll-Ausprägung,
- jede Inkonsistenz zwischen Spezifikation und Implementierung,
- jedes strukturelle Merkmal des Programmtextes, das ein fehlerhaftes Verhalten des Programms verursacht [[UNIS2014](#)].

Ziel des Testens ist, durch gezielte Programmausführung Fehler zu erkennen. Als erfolgreicher (positiver) Test wird dabei ein Test bezeichnet, der einen Fehler aufdeckt, während ein erfolgloser (negativer) Test ein Test ist, der keinen Fehler aufdeckt.

Softwaretests sind dabei nur eines von mehreren Möglichkeiten, Fehler zu vermeiden und die Qualität des Endprodukts zu steigern:

- Fehlervermeidung: Entwicklungsmethoden, Verifikation, statische Analyse,
- Fehlerentdeckung: Test, Asserts, „Quality-Feedback-Agent“ z. B. Flugschreiber,
- Fehlertoleranz: Behandlung von Fehlern zur Laufzeit des Programms, um das Programm trotzdem fortzusetzen [[KOSCH2006](#)].

11.2 Fehlerarten

Funktionale Fehler

Funktionale Tests sind kein Ersatz für Unit Tests, sondern überprüfen den funktionalen Ablauf, wie ihn der Nutzer wahrnimmt. Ein funktionaler Test überprüft z. B., ob ein Delete-Button vorhanden ist und ob er wie erwartet funktioniert. Um die Tests überschaubar zu halten, wird meist nur überprüft, ob die entsprechenden Templates vorhanden sind und für Nutzer mit verschiedenen Rollen und Rechten wie erwartet funktioniert [[PLON2014](#)]. Fehler, die dabei auftreten, werden als funktionale Fehler bezeichnet.

Performancefehler

Früher ging man im Allgemeinen davon aus, sich beim Test einer Anwendung zunächst auf eine umfassende Funktionalität konzentrieren und die Performance zurückstellen sollte. Ein paar Sekunden hier und da lassen sich in der Regel immer noch herausholen, wenn man das Programm im Detail prüft.

Inzwischen geht man aber verstärkt dazu über, Performancetests schon in früheren Projektphasen durchzuführen. Wenn die Antwortzeiten oder Zugriffszeiten völlig indiskutabel sind, kann das auch auf eine unglückliche Systemarchitektur hindeuten. Und Fehler an dieser Stelle behebt man besser in einer frühen Projektphase. Wenn man nämlich am Ende

eine funktionsfähige, aber leider viel zu langsame Software hat, kann das Anpassungen in der Dateistruktur oder bei Datenbankzugriffen bedeuten, wodurch man den gesamten Test noch einmal neu aufrollen und die Entwicklung zeitintensiv nachbessern muss.

11.3 Kosten pro Fehler – die Barry-Boehm-Kurve

Jedes Testprojekt steht und fällt mit den Anforderungen an die Software. Prinzipiell wird Software nach den klassischen Vorgehensmodellen in folgenden sechs Phasen im Rahmen eines Projekts entwickelt:

1. Analyse
2. Entwurf (Design)
3. Implementierung
4. Test
5. Inbetriebnahme
6. Wartung

Während im Wasserfallmodell und V-Modell diese Phasen mit kurzen Rückkoppelungen im Idealfall nur einmal durchlaufen werden, erfolgt eine iterative Wiederholung der Phasen in der evolutionären Softwareentwicklung. Entscheidend dabei ist, dass während der Analyse möglichst alle Anforderungen an die zu entwickelnde Software erkannt werden. Später auftretende Änderungen können nach dem Entwurf der Software oft nur schlecht oder unter hohem Kostenaufwand berücksichtigt werden. Aus dieser Darstellung ergibt sich die auf Barry Boehm zurückgehende Kostenkurve.

Diese zeigt, dass später auftretende Anforderungen wesentlich höhere Kosten (exponentieller Anstieg) verursachen, als frühzeitig erkannte Anforderungen. Deshalb wird in der klassischen Softwareentwicklung der Schwerpunkt auf die Analyse gelegt, um möglichst alle Anforderungen zu erkennen. Darin inbegriffen ist die These, dass die Softwarearchitektur nur schwer änderbar ist und somit nur unter hohen Kosten Änderungen der Anforderungen berücksichtigt werden können.

Dieses Vorgehen findet sich in einer Reihe von Ingenieurdisziplinen wieder. Für die Softwareentwicklung wird oftmals die Metapher des Hausbaus benutzt. Bevor mit dem eigentlichen Bau begonnen werden kann, muss zuerst eine Architektur (Entwurf) anhand der Anforderungen der späteren Bewohner erarbeitet werden. Diese Architektur kann dann durch eine Baufirma realisiert werden. Sollten sich während der Bauausführung Änderungen ergeben, können diese nur schwer berücksichtigt werden, da größere Änderungen eine Überarbeitung der Architektur erfordern würden.

Die Baumetapher hat sich als sehr wertvoll für die Softwareentwicklung erwiesen. Kein Architekt bzw. Konstrukteur entwickelt heute jedes Haus oder jede Brücke komplett neu. Vielmehr wird auf einen reichen Erfahrungsschatz zurückgegriffen und bekannte

Elemente werden miteinander kombiniert. In diesem Zusammenhang entstand der Gedanke wieder verwendbarer Softwarekomponenten. So können heute enorme Produktivitätssteigerungen durch den Einsatz von Frameworks wie Microsoft.NET, Suns Java Klassen oder der Qt Bibliothek von Trolltech erreicht werden.

Jede Phase in der klassischen Softwareentwicklung erzeugt definierte Artefakte. Diese Artefakte beschreiben den kompletten Prozess und das Produkt. Anhand dieser Dokumentation kann das Projekt nachvollzogen und kontrolliert werden. Eine Einarbeitung eines neuen Mitarbeiters ist mit dieser Dokumentation leichter möglich. Darüber hinaus dienen die bereits produzierten Artefakte als Arbeitsgrundlage der folgenden Phasen. Im Idealfall benötigt ein Mitarbeiter kein Vorwissen aus den vergangenen Phasen um seine Arbeit fortzusetzen. In der Endkonsequenz werden Mitarbeiter dadurch theoretisch leichter austauschbar. Dadurch wird eher in Werkzeuge und Prozesse als in Mitarbeiter und deren Qualifikation investiert.

Aufgrund der klar definierten Ergebnisse einer Phase kann objektiv entschieden werden, ob eine Phase in der Softwareentwicklung erfolgreich beendet ist. Ein umfangreiches Controlling ist möglich. Es wurden eine Vielzahl von Softwaremetriken zur Beurteilung der Qualität der produzierten Software, aber auch zur Beurteilung der Qualität des Projektes insgesamt, entwickelt. In vielen Unternehmen sind diese Metriken bereits vorhanden. Diese Metriken unterstützen zukünftige Projekte z. B. bei Aufwandsschätzungen. Insgesamt gesehen findet eine ständige Optimierung der Softwareentwicklung statt. Diese Optimierung ist notwendig, um die stetig komplexer werdenden Anforderungen bewältigen zu können.

Weiterhin existiert die Tendenz eine möglichst hohe Automatisierung, z. B. im Rahmen der Model Driven Architecture, während der Softwareentwicklung zu erreichen. Neben einer Kostenreduktion durch Einsparung von Mitarbeitern wird eine Qualitätssteigerung angestrebt. Dazu sind hohe Investitionen in Werkzeuge notwendig, vergleichbar dem Aufbau von vollständig automatisierten Fertigungsstraßen in der Güterindustrie. Auch hier sind die Parallelen zu anderen Ingenieurdisziplinen unverkennbar [EMER2004].

Dabei zeigt die „Barry-Boehm-Kurve“ (siehe Abb. 11.1) die Dynamik der Fehlerkosten eindrucksvoll auf.

11.4 Dynamik der Fehlerkosten

Das Unternehmen Thyssen definiert Fehlerkosten wie in Tab. 11.1 dargestellt.

Man spricht in Bezug auf die Kosten für die Fehlererkennung in den einzelnen Projektphasen auch von der „1:10:100:1000-Regel“. Diese Regel wird auch als „Zehnerregel“ bezeichnet [OLEV2016]. Sie besagt, dass je weiter ein Fehler sich unentdeckt in die späten Phasen des Werdeganges eines Produktes oder Prozesses bewegt – oder gar bis zum Kunden –, umso höher die Kosten zur Behebung dieses Fehlers werden.

Eindrucksvoll untermauert wird die Zehnerregel durch die Ergebnisse einiger Studien aus den 70er-Jahren in Japan, USA und Großbritannien, die sich mit den Ursachen von Produkt- bzw. Qualitätsmängeln beschäftigten. Alle Analysen lieferten nahezu die gleichen

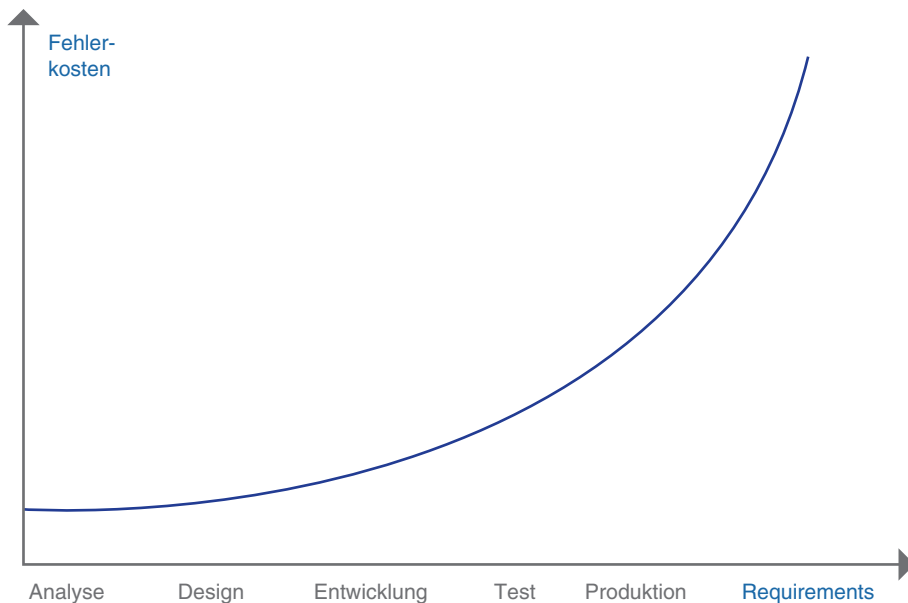


Abb. 11.1 Zeitliche Abhängigkeit der Kosten für Änderungen nach Boehm

Tab. 11.1 Fehlerkosten

Verhütungskosten	Der finanzielle Aufwand für Bemühungen, damit alles von Anfang an richtig gemacht wird.
Prüfkosten	Der finanzielle Aufwand für das Prüfen, ob alles von Anfang an richtig gemacht wurde.
Interne Fehlerkosten	Der Preis, der bezahlt werden muss, wenn herausgefunden wird, dass etwas nicht von Anfang an richtig gemacht wurde.
Externe Fehlerkosten	Der Preis, der bezahlt werden muss, wenn nicht herausgefunden wurde, dass etwas nicht von Anfang an richtig gemacht wurde.

Ergebnisse: Ca. 70 % aller Produktmängel hatten ihre Ursache bereits in der Entwicklung, Konstruktion und Arbeitsvorbereitung. Der Herstellungsprozess selbst hat bezüglich der Endqualität des Produktes offensichtlich eher einen sekundären Einfluss. Eine VD-MA-Studie zum Thema „Qualitätsbezogene Kosten“ Anfang der 90er-Jahre in der Bundesrepublik bestätigt dieses Ergebnis.

In diesem Zusammenhang sind auch die Ansätze von Deming und dessen Aussage „Qualität kann nicht erprüft, sie muss erzeugt werden“ zu sehen. Deming hat hier einen entscheidenden Beitrag für das Qualitätsverständnis geliefert. Entscheidend ist ein umfassendes Qualitätsmanagement, das möglichst frühzeitig im Prozess aufgesetzt wird.

Übertragen auf den Softwaretest hat das für die Dynamik der Fehlerkosten folgende Konsequenzen:

Faktor 1: Bei Entdeckung des Fehlers im Anforderungsmanagement

Wird bereits im Requirements Engineering darauf geachtet, dass Requirements vollständig, nicht widersprüchlich, klar verständlich und testbar sind (siehe auch Abschn. 8.2), spart man in allen nachfolgenden Schritten und hat nur Kosten von einer Arbeitseinheit. Umso erstaunlicher erscheint es da, wie stiefmütterlich in vielen Unternehmen immer noch das Requirements Engineering behandelt wird [REQU2013].

Anforderungen beschreiben, was im Projekt herzustellen und zu liefern ist. Dass es wichtig ist, mit den Anforderungen sorgfältig umzugehen, ist allgemein anerkannt. Dazu haben auch die Publikationen des Chaos-Reports der Standish Group beigetragen, die seit fast drei Jahrzehnten regelmäßig zeigen, dass sich ein Scheitern von Projekten oft auf mangelhaften Umgang mit den Anforderungen zurückführen lässt. Das ist etwa der Fall, wenn Kunden nicht genügend einbezogen werden und Anforderungen unvollständig oder unrealistisch sind, aber auch, wenn sie zu umfangreich sind und Features gefordert werden, die später gar nicht oder nur selten genutzt wurden. Das ist eine häufige Ursache für Mehrkosten und Verzögerungen. In linearen Vorgehensmodellen, insbesondere dem Wasserfallmodell, wird Requirements Engineering typischerweise als eine Phase zum Projektbeginn betrachtet. Doch eine einmalige Festlegung der Anforderungen zu Projektbeginn reicht nicht aus: Neue Ideen entstehen, andere Anforderungen entfallen, Rahmenbedingungen und Details ändern sich. Darum müssen die Anforderungen über die gesamte Projektlaufzeit beziehungsweise den gesamten Lebenszyklus eines Softwareprodukts verwaltet und aktuell gehalten werden (Requirements Management). Damit geht das Requirements Engineering und Requirements Management deutlich über die Vorstellung hinaus, ein Pflichtenheft als Grundlage eines Projekts zu erstellen.

In vielen Fällen werden die Requirements auch als gegeben hingenommen, ohne in einem genauen Review zusammen mit dem Auftraggeber die Fragen zu klären. Das ist zugegeben auch schwierig, weil sich viele Fragen erst während des Entwicklungsprozesses oder auch erst während der Testphase ergeben. Zu diesem Zeitpunkt liegt längst ein Anforderungswerk als Vertragsgrundlage vor, das dann nicht mehr ohne weiteres vom Auftraggeber geändert werden darf, da es ja als Kalkulationsgrundlage dient. Und das Handling über Change Requests ist in vielen Fällen äußerst kompliziert.

Eigentlich sollten Entwickler die Anforderungen daher vor Erstellung des Angebots genau prüfen. Meist liegt die Auftragsanfrage über das neue System erst wochenlang beim Vertrieb bis der betroffene Entwickler es zu sehen bekommt und die Implementierung des umfangreichen Requirementskatalog innerhalb von wenigen Stunden abschätzen soll. Dass man dabei die Anforderungen dann meistens gar nicht so genau gelesen bzw. verstanden hat und dass dabei die Lücken und Widersprüche leicht übersehen werden, versteht sich von selbst.

Oft wird das Requirements Management als Aufgabe betrachtet, die beliebige Rollen ohne besondere Kenntnisse nebenbei mit erledigen können. Es hat teilweise noch größere Imageprobleme als der Softwaretest. In Wirklichkeit ist auch das Requirements Engineering eine schwierige und sehr anspruchsvolle Disziplin im System- und Softwareentwicklungsprozess. Ein Anforderungsingenieur muss die Sprache des Fachbereichs sprechen,

ebenso die der Architekten, Entwickler und Tester. Er muss nicht nur zahlreiche Methoden und Techniken rund um das Requirements Engineering und Management beherrschen, sondern auch eine ausgeprägte Sozialkompetenz mitbringen, um alle Stakeholder in den Prozess einbinden zu können.

Meist will man gerade in der Anfangsphase schnell ein Ergebnis sehen und schnell zumindest ein halbfertiges Produkt präsentieren können, nicht erst lange Anforderungen lesen und prüfen, denn dieser Teil kommt einem naturgemäß etwas zäh und langwierig vor. Dennoch liegt gerade in frühen Phasen der Schlüssel zum Erfolg, da man dadurch viel Aufwand, viele Re-Tests und viele Kosten in späteren Projektphasen vermeiden kann.

Dazu kommt, dass das Anforderungswerk in der Regel die Vertragsgrundlage darstellt und eine Verfeinerung, Detaillierung oder Korrektur dieser Anforderungen die Kalkulation und den vertraglichen Umfang des gesamten Auftrags in Frage stellen können.

Faktor 10: Bei Entdeckung durch den Programmierer selbst

Wird ein Fehler während der Entwicklung in einem Test durch den Programmierer selbst entdeckt, so kann er meist schnell behoben werden. Der zusätzliche Aufwand beschränkt sich auf die Fehlerbehebung selbst. Allerdings muss ein Teil des Programmcodes wieder verworfen werden, die Anforderungen angepasst werden und es gibt Rückkopplungen zur Systemarchitektur und zum Requirements Engineering. Es entstehen dadurch Kosten in Höhe von 10 Arbeitseinheiten. Oft kommt das Softwaredesign nicht rechtzeitig hinterher – die Softwareentwicklung hat schon fertige Programmkomponenten geliefert, die Software-Architektur definiert aber erst nachträglich generische Anforderungen und Datenmodelle zu einem viel zu späten Zeitpunkt. In diesem Fall kommt es zu einem Bruch in der Prozesskette: Der Softwaretest kann dann zwar auf Requirements und Produkt aufsetzen, es fehlt aber ein wesentlicher Schritt im Entwicklungszyklus. Wenn nun nachträglich Änderungen an der Softwarearchitektur durchgeführt werden, muss man sowohl die Entwicklung als auch den Test wieder anpassen. In diesen Fällen fehlt meist die ausreichende Rückkopplung zwischen den Stakeholdern. Gerade dieser Fall kommt meiner Erfahrung nach in der Praxis häufig vor. Oft gibt es ja bereits eine bestehende Softwarelandschaft, und man kann viele der vom Kunden gelieferten Requirements mit seiner Software ja schon abdecken und fängt nicht wie in den modellhaften Überlegungen „auf der grünen Wiese“ an. Hier liegt auch eine wesentliche Ursache für Projektverzögerungen und Budgetüberschreitungen. Erschwerend kommt hinzu, dass es ja „irgendwie immer schon gegangen ist“ und dann auf eine Optimierung der Softwarearchitektur so lange verzichtet wird bis es wirklich nicht mehr geht. Es ist eben eindrucksvoller ein fertiges Softwareprodukt dem Endkunden zu präsentieren als nur ein universales Datenmodell zu generieren.

Faktor 100: Bei Entdeckung durch einen Tester

Wird der Fehler erst durch einen Softwaretester entdeckt, wenn der neue Entwicklungsstand bereits auf einem Testsystem läuft, steigt bereits der Aufwand. Der Tester wird den Fehler dokumentieren, einen entsprechenden Vorgang in der Aufgabenverwaltung/im Bugtracking-System anlegen bzw. wiedereröffnen, den Entwickler darauf aufmerksam

machen, dieser wird den Fehler beheben, der Stand auf dem Testserver wird noch einmal aktualisiert, es erfolgt ein weiterer Test auf dem Testserver, bevor der neue Stand live eingespielt werden kann. Es entstehen also mindestens die Kosten für einen weiteren Testzyklus. Womöglich kann hierdurch auch noch eine Verschiebung des GoLive-Termins entstehen, was nicht nur den Produktmanager besonders ärgern dürfte. Es entstehen bereits Kosten von 100 Arbeitseinheiten.

Die Höhe dieser Kosten variiert dabei nach der Teststufe, wenn der Fehler im Integrationstest entdeckt wird, ist er in jedem Fall günstiger als im Abnahmetest. Das gilt besonders dann, wenn zur Software ein umfassender Validierungs- oder Begutachtungsprozess gehört. Ein Fehler in dieser Phase kann bereits empfindliche Vertragsstrafen auslösen und das Unternehmen finanziell in den Ruin treiben. Dennoch bleibt hier immer noch das Problem innerhalb des Betriebs bzw. der Organisation, es können sich zwar Terminverschiebungen ergeben und Kostenrahmen überzogen werden aber man hat bis hierher noch keine schlechte Qualität an den Kunden und in die Breite geliefert.

Faktor 1000: Bei Entdeckung im Live-Betrieb

Am unangenehmsten und kostspieligsten ist der Fall, wenn der Fehler erst im Live-Betrieb bekannt wird – womöglich noch durch den Hinweis von einem oder mehreren Nutzern. Neben den Kosten durch die aufwändige Fehlerbehebung, wie sie auch bei der Entdeckung auf dem Testsystem anfallen, muss jetzt zusätzlich extra für diesen Fehler der Stand auf dem Live-System aktualisiert werden. Wenn z. B. die Software bereits an mehreren Lieferstellen im Einsatz ist, kommen für die extra durchzuführenden Rollout-Maßnahmen weitere Aufwände hinzu. Zusätzlich gefährdet solch ein Fehler das Image der Software und des gesamten Unternehmens, da die Wirksamkeit der gesamten Qualitätssicherung in Frage gestellt wird. Das sorgt für Verärgerung bei den Nutzern und kann auch Umsätze verhindern, wenn umsatzrelevante Funktionen nicht mehr genutzt werden konnten [RHEI2013]. Dazu können Kosten für Vertragsstrafen, Sachfolgekosten und Vermögensschäden anfallen. Zur Verhinderung oder Eindämmung von Penalty-Zahlungen werden oft in letzter Sekunde enorme Aufwände getätigt, die Kosten für die Überstunden und Wochenend-Einsätze spielen dann nur noch eine verschwindend kleine Rolle. Es entstehen Kosten von 1000 Arbeitseinheiten.

Besonders deutlich wird dies bei Rückrufaktionen von Fahrzeugen. Der amerikanische Hersteller Chrysler musste 1998 30.000 Dodge-Ram-LKWs zurückrufen, weil die Software auf einem der Controller verrücktspielte. Der Fehler resultierte in einer falschen Geschwindigkeitsanzeige und verführte die Fahrer daher zu schnellem Fahren mit hohem Risiko. Auch dieser Fehler wurde behoben, die Händler erhielten Ersatzteile und installierten sie in die Fahrzeuge.

Im Jahr 2002 musste BMW ihre Oberklassemodelle, den 745i und den 745LI zurückrufen. Die digitale Motorsteuerungssoftware funktionierte nicht einwandfrei. Die Motoren hatten durch den Softwarefehler einen unruhigen Lauf und Fehlzündungen erzeugt. 12.000 Fahrzeuge waren betroffen und mussten bei den Händlern neu programmiert werden. In 2003 fand man bei der 7-er Klasse einen weiteren Softwarefehler. Er steckte in einem

Embedded-Chip, auf dem Windows CE lief. Die betroffene Version des Microsoft-Betriebssystems für Handhelds sollte in dem Auto iDrive steuern. Dieses Gerät auf dem Armaturenbrett integrierte etwa 200 sonst getrennt ausgeführte Steuerfunktionen – vom Einstellen der Sitzposition über Navigation, Klimatisierung bis zum autointernen Unterhaltungsprogramm.

Der US-Autobauer Ford musste nahezu 700.000 Fahrzeuge vom Ford Escape und Ford C-Max aus den Modelljahren 2013 und 2014 in die Werkstätten zurückrufen, weil ein Softwareproblem zum verspäteten Auslösen des Airbags führen konnte. In diesem Fall war also die Sicherheit des Fahrers nicht mehr gewährleistet.

11.5 Konsequenzen für die Fehlerbehandlung in den Teststufen

Die Kosten der Fehler steigen also exponentiell, je länger sie unentdeckt bleiben, weil sie von Projektschritt zu Projektschritt vererbt werden und so von Teststufe zu Teststufe immer höheren Schaden anrichten. Soviel Funktionen wie möglich sollten also bereits in der frühesten Teststufe, im Unittest (Modultest) getestet werden, da Fehler, die hier erkannt und behoben werden, sich nicht in höhere Teststufen fortpflanzen.

Die Aufstellung der Kosten pro Fehler gilt in der Detailbetrachtung auch für den Testprozess: ein Fehler, der erst beim Abnahmetest entdeckt wird, ist erheblich teurer als ein Fehler, der während der Entwicklung bzw. im Unittest erkannt wird.

Um zu vermeiden, dass sich Fehler aus einer Entwicklungsstufe in die nächste fortpflanzen und erst beim Kunden bzw. nach Rollout der Anwendung bemerkt werden, sind Tests nach den einzelnen Phasen unabdingbar. In folgenden Stadien der Softwareentwicklung muss deshalb zwingend getestet werden:

- abschließende Prüfung der Anforderungen und exakte Definition der Requirements mit Klärung aller offenen Fragen vor Übergabe an die Programmierer,
- Tests während der Entwicklung durch den Programmierer,
- Tests nach Abschluss der Entwicklung durch einen Softwaretester, ggf. auf Test- und Live-System.

Deshalb wird sich die höhere Qualität durch frühzeitige Softwaretests in vielfacher Hinsicht auszahlen. Bei der Kalkulation des Projekts wird dieser Punkt oft nicht ausreichend gewürdigt. Manchmal hat man aber auch diese Erkenntnis längst gewonnen, weil man es beim aktuellen Projekt auch endlich besser machen möchte als früher, steckt aber noch in Altpjekten fest und kommt nicht hinterher.

Wichtig ist dabei, die einzelnen Funktionen nicht mehrfach in unterschiedlichen Teststufen zu testen, ohne dass es neue Erkenntnisse gibt. Man muss also genau definieren, welche Funktion in welcher Teststufe getestet wird und warum. Hierbei ist kompetentes Testdesign und eine schlüssige Systemarchitektur erforderlich.

11.6 Fehlerverfolgung

Zur Fehlerverfolgung (Fehlertracking) bieten sich ebenfalls professionelle Tools an, die eine zeitgleiche Bearbeitung (möglichst web-gestützt) von unterschiedlichen Clients erlauben. Dabei ist es wichtig, den Status des Fehlers verfolgen zu können. Wann wurde der Fehler eingetragen, gab es Rückfragen, was wurde zur Fehlerbehebung getan, wann wurde die Programmverbesserung implementiert, wann getestet, und wann war der Fehler definitiv behoben?

Die Dauer zur Fehlerbehebung gibt Aufschlüsse über die Auslastung der Entwickler, über die Priorisierung beim Test und über die Komplexität der gefundenen Fehler. Es kann auch sein, dass die reinen Bearbeitungszeiten wesentlich kürzer sind als die Gesamtzeit von Fehlermeldung zur Fehlerbehebung – das deutet darauf hin, dass das Entwicklerteam überlastet ist oder die Abläufe im Unternehmen zu komplex.

Wenn man die „Bearbeitungszeiten“ mit den „Liegezeiten“ vergleicht, gewinnt man Aufschlüsse über fehlende Ressourcen und kann zielgerichtet Aufwände dort erhöhen, wo sie einen Engpassfaktor darstellen und den „Flaschenhals“ ermitteln.

Beispiel

Fehler 1 ist von 2 Abteilungen zu beheben, weil es sich um 2 Änderungen handelt, die in der Software in unterschiedlichen Komponenten entwickelt werden müssen.

Life Cycle Fehler 1:

- 1.4. Fehler erfasst,
- 2.4. Start Analyse Abteilung A,
- 6.4. Ergebnis Analyse Abteilung A (10 Stunden für Abteilung A),
- 6.4. Start Analyse Abteilung B,
- 27.4. Ende Analyse Abteilung B (10 Stunden für Abteilung B),
- 10.5. Start Fehlerbehebung Abteilung A und B,
- 12.5. Ende Fehlerbehebung Abteilung A,
- 19.6. Ende Fehlerbehebung Abteilung B,
- 22.6. Integration der Software,
- 24.6. Nachtest des Fehlers,
- 24.6. Fehler geschlossen.

Hier ist es eindeutig, dass Abteilung B den Engpassfaktor darstellt. Obwohl die reine Fehlerbehebung nur 10 Stunden benötigt, beträgt die Prozesslaufzeit mehrere Wochen.

Es gibt Fehler, die wieder eröffnet werden, weil sie eben doch nicht tatsächlich behoben sind. Das bedeutet, dass ein unterschiedliches Systemverständnis vorhanden ist. Auch der Aufwand pro Fehler sollte untersucht werden – das bedeutet in letzter Konsequenz, dass an dieser Stelle die Anforderungen unklar gewesen sind oder die Fehlerbeschreibung unvollständig war. Ein Fehler mit hohem Fehlerbehebungsaufwand deutet auch darauf hin, dass er erst in einer späteren Teststufe erkannt wurde.

Fehlermeldungen sind hierbei keine, überspitzt formuliert, handbeschriebenen Zettel, welche an den eventuell zwei Büros weiter sitzenden Entwickler gereicht werden, sondern **schematisch, vom Testmanager projekteinheitlich, festgelegte Meldungen**. Jegliche Fehlermeldungen werden in einer vom Projekt verwendeten Fehlerdatenbank festgeschrieben, was neben einer chronologischen Erfassung der einzelnen aufkommenden Fehler eine Übersicht für Schnittstellenrollen, wie z. B. dem Testmanager (Schnittstelle zwischen Projektmanagement und Test(ern)), über aktuell bereits geschlossene, noch offene oder zurückgewiesene Fehler, sowie die verschiedenen Stati bietet.

11.7 Aufbau einer Fehlermeldung

Die schematisch festgelegte Fehlermeldung unterteilt sich in 3 Bereiche:

- **Problembeschreibung:** Schilderung der aufgetretenen Errors im Verlauf des Tests.
- **Identifikation:** Testperson, Software, Testumgebung (Rahmendaten), **Fehlerstatus**.
- **Klassifikation:** Fehlerklasse und Priorisierung.

Von besonderer Relevanz ist neben der Problembeschreibung die Klassifikation bzw. die Einstufung der Meldung, da hier zum einen der erkannte Fehler priorisiert wird zur Darstellung der Wichtigkeit und der damit verbundenen Lösung und zum anderen eine Kritikalität in Form der Fehlerklasse festgehalten wird.

Fehler, die Rückfragen vom Entwickler auslösen, deuten auf ein Defizit bei der Fehlerbeschreibung hin. Diese Fehler sollten ebenfalls gezählt und klassifiziert werden.

Messungen, die der Entwickler benötigt um die Fehlerquelle aufzuspüren (Logfiles, Traces, aber auch Bilder die das Fehlverhalten erklären), sind dem Fehler hinzuzufügen.

Ein einzelner Fehler erzeugt Prozesskosten von ca. 500 bis 2000,- EUR, je nach Phase in der er entdeckt wird und je nach Qualität der Fehlerbeschreibung. Das Fehlermanagement ist ein weiterer Bereich, in dem eine Vereinheitlichung und Standardisierung gewaltige Potenziale freilegen würde. Auch dem Management sollte dabei bewusst sein, wie hoch die Kosten und die Lebensdauer eines Fehlers sind. Es gibt Fehler, die mehrfach behoben, geschlossen, wieder eröffnet werden, teilweise weil die Beschreibung von Anfang an nicht vollständig war oder bei der Behebung des Fehlers nicht nach rechts und links geschaut wurde. Einige Fehler geistern monatelang durchs System und werden immer wieder zwischen Testern und Entwicklern hin- und hergeschoben.

11.8 Tools für die Fehlerverwaltung

Die Fehlerverfolgung sollte in einem professionellen Tool durchgeführt werden. Bevor man sich für ein bestimmtes Fehlerverwaltungstool entscheidet, sollten die Kriterien aufgelistet werden, die es erfüllen soll. Diese ermittelten Kriterien sollten entsprechend

gewichtet werden und dann die einzelnen Anwendungen daraufhin überprüft werden, bevor die Entscheidung für ein bestimmtes Tool getroffen wird.

Dabei sollte man immer auch den Ablauf im Unternehmen bedenken: Fehler wird eröffnet $\Rightarrow$ vom Entwickler behoben $\Rightarrow$ Fehler wird nachgetestet $\Rightarrow$ evtl. wieder geöffnet ... Es empfiehlt sich, im Rahmen der Toolevaluation einige bereits erledigte Fehler mit komplizierten Abläufen mit dem auszuwählenden Tool testweise zu verwalten. Bei größeren Organisationen sollte erst ein Pilotbetrieb in einem kleineren Projekt erfolgen, bevor die Anwendung unternehmensweit eingesetzt wird.

Aus meiner Sicht muss ein geeignetes Tool für die Fehlerverwaltung mindestens folgende Fragen beantworten können:

- Welche Probleme sind aufgetreten?
- Welcher Art sind die Probleme?
- Welche Priorität hat die Behebung des Fehlers?
- Welcher Tester hat das Problem festgestellt?
- Welcher Testfall ist von dem Fehler betroffen?
- Welcher Entwickler ist für das Problem zuständig?
- In welcher Programmversion wurde der Fehler beobachtet?
- Sind evtl. weitere Versionen vom Fehler betroffen?
- Wie schwer wiegt der Fehler?
- Wie ist die betriebliche Auswirkung des Fehlers bzw. die Auswirkung für andere Systeme und Organisationen?
- Wie hoch ist die Priorität für die Fehlerbehebung?
- Hat der Fehler Konsequenzen auf andere Funktionalitäten, die dadurch nicht getestet werden können oder fehlerhaftes Verhalten implizieren?
- Was wurde unternommen, um das Problem zu beheben (z. B. Programmänderung)?
- Ist der Fehler reproduzierbar?
- Gibt es Möglichkeiten, das Problem zu umgehen, wenn ja, wie?
- Ist das Problem wirklich behoben, also nachgetestet?
- Ist ein einmal behobenes Problem wieder aufgetaucht? Wenn ja, wieso?
- In welcher Programmversion wird das Problem voraussichtlich behoben sein? Ist die Behebung dann früh genug oder muss sie vorgezogen werden (Ist etwa mit einem eigenen Release für die Fehlerbehebung, ein „Critical Bugfix“ erforderlich?)?
- Wann wird das Problem voraussichtlich behoben sein?
- Wie hoch ist der geschätzte Aufwand zur Lösung des Problems?
- Wie hoch war der tatsächliche Aufwand um den Fehler zu beheben?
- Wie ist der Status des Fehlers zu welchem Zeitpunkt: Wann wurde der Fehler festgestellt, wann wurde die Software geändert, wann fand der erneute Test statt? Wurde der Fehler erneut eröffnet? Wenn ja, warum – also war etwa die Beschreibung mangelhaft, die Testvoraussetzungen anders, gibt es Integrationsprobleme die immer wiederkehren ...?
- In welchen Formaten erfolgt die Speicherung? Proprietäre Formate erschweren die Verwendung auf unterschiedlichen Plattformen bzw. machen sie unmöglich

- Lassen sich Dokumente an die Fehlerbeschreibung anhängen (Screenshots, Logfiles usw.) damit ein Entwickler die notwendigen Informationen und Traces exakt nachverfolgen kann?
- Sind die Problemdokumente von allen Beteiligten zugreifbar (Dokumentfreigaben, Web-Zugriffe usw.)?
- Lässt sich im System nach Stichworten suchen? Ist die Suche so komfortabel, dass man mit bestimmten Strings auch bei Hunderten oder Tausenden von Fehlern in kurzer Zeit die Beschreibung und den Status eines bestimmten gesuchten Fehlers finden kann?
- Lassen sich Fehler zusammenfassen? Beispiel: Der Tester stellt fest, dass Fehler 149 dasselbe Fehlerbild zeigt wie Fehler 68, Kommentare zur Fehlerursache oder zur Fehlerbehebung oder erklärende Ergänzungen sollten dann nur in einem der Fehler sein, da es sonst zu Redundanzen und ggf. widersprüchlichen Bemerkungen in beiden Fehlerbeschreibungen kommt. Einige Fehlertools erlauben die Duplizierung von Fehlern.
- Ist Mehrbenutzerfähigkeit gewährleistet?

Im Fall eines Softwarefehlers sollte dazu immer dokumentiert werden:

- Konfiguration des Testsystems (einzelne Komponenten). Dafür ist es nötig, dass die Software im Konfigurationsmanagement einwandfrei identifiziert werden kann.
- Einzelne Schritte vor dem Fehler.
- Verhalten des Testsystems vor dem Fehler.
- Beobachtungen während und nach des Fehlers.
- Auswirkungen des Fehlers (war das System noch benutzbar?).
- Schwere des Fehlers.
- Reproduzierbarkeit des Fehlers.
- Zeitpunkt der Fehlererkennung.
- Tester.
- Status des Fehlers (offen, behoben, nachgetestet, erneut geöffnet, geschlossen usw.).
- Anhänge (Logfiles, Screenshots, Fotos usw.).

Wichtig ist auch, dass das Tool unternehmensweit bzw. projektweit (beim Einsatz von Unterlieferanten zu bedenken) eingesetzt werden kann. Besondere Herausforderungen können sich beim Offshoring ergeben, wenn der Offshore-Standort nicht in Echtzeit auf das Fehlererfassungssystem zugreifen kann bzw. die Bandbreiten beim Zugriff auf den Server zu inakzeptablen Antwortzeiten führen.

Es kommt leider immer noch vor, dass Fehler in zwei Tools parallel eingetragen wurden, wobei in der Folge natürlich keine Aktualität gegeben ist, die Informationen auseinanderlaufen und der Erfassungs- und Pflegeaufwand nach wenigen Testzyklen immens wird. Wenn die Fehlererfassung nicht in einem zentralen System projektweit vorgenommen wird, sind Mehraufwände und Qualitätsmängel vorprogrammiert. Daher muss man immer darauf achten, dass jede in den Entwicklungs- bzw. Testprozess involvierte Abteilung an das ausgewählte Tool angeschlossen wird. Auch wenn während der

Projektlaufzeit die Entwicklungs- oder Testorganisation geändert wird, sind diese Abhängigkeiten zu beachten.

In einem meiner früheren Projekte setzte einer der Standorte zunächst ein eigen programmiertes Tool ein, wechselte dann mitten im Projekt auf ein anderes Tool, zog diese Entwicklung aber an anderen Standorten nicht nach, an denen wiederum ein drittes Tool verwendet wurde.

Es war vorauszusehen, dass das natürlich nicht funktionieren konnte – und so kam es denn auch. Man konnte nicht mehr eindeutig miteinander kommunizieren, Fehler, die an zwei Standorten behoben werden mussten, wurden doppelt beschrieben, Fehler konnten auch nach dem Re-Test nicht endgültig geschlossen werden, bei einigen Fehlern war nicht klar, ob es sich um dasselbe Fehlverhalten oder ein anderes Fehlerbild handelte usw. usw.

Auch wenn die Entwicklung an mehreren Standorten oder sogar von unterschiedlichen Unternehmen durchgeführt wird, hat man oft mit dem Problem zu kämpfen, unterschiedliche Systeme zu synchronisieren. Hier liegt ein gewaltiges Optimierungspotenzial brach. Ohne ein definiertes zentrales Projektmanagement mit dem Einsatz einheitlicher Tools bzw. definierter Schnittstellen kann das Doppelarbeit, unklare Stati und lange Abstimmungsmeetings zur Folge haben. Die Lösung liegt in diesem Fall in einer webbasierten Anwendung mit geschützten Feldern und entsprechenden Autorisierungen für beide Unternehmen. In manchen Unternehmen sind die Organisation und die Prozesse mit den zusätzlichen Aufgaben nicht mitgewachsen. Dann wird mit diesen Organisationsmängeln ein Test gestartet, weil man Probleme dieser Art nicht ausreichend beachtet. Ein Testmanager muss hier ein ernstes Veto einlegen, die Probleme solchen Ansinnens aufzeigen und ganz klar das Management davor warnen, dass das Projekt unter diesen Voraussetzungen gefährdet ist, dass Terminprobleme und Kostenerhöhungen zu erwarten sind. Wenn es aus Termingründen dann doch nicht anders möglich ist, weil ja auch die Einrichtung einer entsprechenden Oberfläche Zeit erfordert, muss man zumindest als Testmanager darauf hinweisen.

Umfassende Projekte erreichen teilweise Tausende von Fehlern, die alle zu verwalten sind. Aber die Fehlerzahl allein stellt noch keinen ausreichenden Indikator für die Programmqualität dar.

Testabdeckung $\leftarrow$ Qualität $\rightarrow$ Fehleranzahl

Es empfiehlt sich, aufgetretene Probleme zur Bewertung in Fehlergruppen zusammenzufassen:

- Klasse 1: Systemabsturz, Datenverlust, Produktionsstillstand,
- Klasse 2: Wesentliche Funktion nicht einsetzbar,
- Klasse 3: Funktionale Abweichungen und Einschränkungen von Features,
- Klasse 4: Geringfügige Abweichungen in Sonderfällen,
- Klasse 5: Schönheitsfehler, Rechtschreibfehler oder Layoutfehler.

Bei der Interpretation des Testergebnisses werden die Fehler entsprechend gewichtet. Fehler der Klasse 5 werden mit 1, Fehler der Klasse 4 mit 2, Fehler der Klasse 3 mit 4, Fehler der Klasse 4 mit 8 und Fehler der Klasse 1 mit 16 gewichtet.

Daraus entsteht eine gewichtete Fehleranzahl, die erheblich aussagefähiger ist als die einfache Fehleranzahl. Voraussetzung dieses Verfahrens ist jedoch, dass über die Kriterien für diese Klassifizierung ein gemeinsames Verständnis bei den Projektmitarbeitern und im Management vorhanden ist.

Weitere Kennzahlen die sich aus diesen gewichteten Fehleranzahlen ableiten sind:

$\text{Fehlerfindungsproduktivität} = \text{gewichtete Fehleranzahl} / \text{geleistete Testtage}$

Die Fehlerfindungsproduktivität deutet darauf hin, wie detailliert die Tester testen und/oder welche Reife das Produkt bereits besitzt.

$\text{Effizienz der Testfälle} = \text{gewichtete Fehleranzahl} / \text{ausgeführte Testfälle}$

Diese Größe sagt aus, ob die entscheidenden Testfälle durchgeführt wurden, die dazu geeignet waren, das Fehlverhalten aufzudecken. Sie ist immer mit der geforderten Testabdeckung zu vergleichen. Wenn aber relativ viele Testfälle erst durchgeführt werden müssen, um nach ausgiebigen Tests erst gravierende Fehler zu finden, sollte man über die Testspezifikation noch einmal nachdenken. Genauso verhält es sich, wenn der Kunde bzw. der Anwender Fehlverhalten aufdeckt, welche das System massiv beeinträchtigen, die man mit den eigenen Testfällen nicht bemerkt hat.

$\text{Fehlerdichte} = \text{gewichtete Fehleranzahl} / \text{Systemgröße}$

Die Kennzahl der Fehlerdichte ist erforderlich, um die Anzahl der zu erwartenden Fehler im nächsten Release bzw. nächsten Produkt prognostizieren zu können. Die Systemgröße kann dabei in Zeilen Programmcode, Anweisungen, Function Points, Object Points oder anderen Größenzahlen angeben.

Manchmal ist es auch so, dass ein bereits behobener Fehler durch den Einsatz einer geänderten neuen Komponente wieder hochkommt. Da es wichtig ist, die Historie und den Lebenszyklus eines Fehlers nachzuvollziehen, benötigt man Tools, die den Workflow und die Prozesse im Unternehmen unterstützen, in aller Regel ist z. B. Excel aus meiner Sicht zur Fehlerverfolgung ungeeignet.

Tools sind andererseits nicht die Lösung aller Probleme, wenn der Prozess noch gar nicht definiert ist. Man sollte also zunächst dokumentieren, was man vom Tool genau erwartet, und dann ein geeignetes Tool auswählen.

Der Workflow eines Fehlerverfolgungssystems sieht z. B. wie in Abb. 11.2 dargestellt aus.

Andererseits nützt selbst das beste Tool zur Fehlerverwaltung nur wenig, wenn auf die Grundlagen zur Fehlerbeschreibung nicht geachtet wird. Bei der Fehlerbeschreibung ist immer der Testfall anzugeben, bzw. die genauen Umgebungsbedingungen und Voraussetzungen,

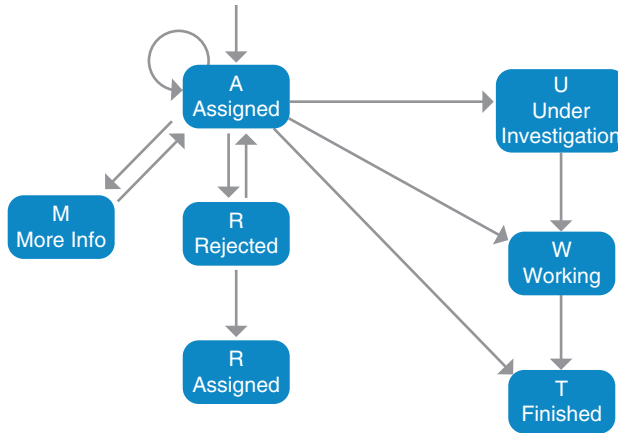


Abb. 11.2 Zustandsübergangsdiagramm für Problem-Objekte

um das entsprechende Fehlverhalten zu reproduzieren. Diese Bedingungen müssen vollständig beschrieben sein. Ansonsten kommt es zu zeitraubenden Rückfragen oder die Fehlerbehebung setzt an der falschen Stelle an. Der Tester sollte erklären können, wie es zu dem Fehlverhalten gekommen ist, d. h. in welchem Anwendungsfall der Fehler aufgetreten ist, welche Eingabeparameter verwendet wurden und in welchem Pfad die Software durchlaufen wurde.

Der Fehler muss auch so exakt und umfassend beschrieben sein, dass der Entwickler, aber auch ein anderer Tester weiß, wie er das Fehlerszenario erzeugen kann. Alle Parameter, die das Fehlverhalten beeinflussen können, sind dabei zu dokumentieren. Das gilt vor allem bei explorativen Tests, in denen beim Test der Applikation nicht auf eine exakte Einhaltung des vorgesehenen Ablaufs geachtet wurde, oder auf Lasttests, die ja gerade die Systemgrenzen verdeutlichen sollen.

Bei der Dokumentation sollten Logfiles oder Screenshots, wenn möglich auch Fotos, unbedingt der Fehlerbeschreibung beigelegt werden – wo immer es sinnvoll erscheint. Ein Bild sagt mehr als 1000 Worte. Auch der exakte Zeitpunkt des Fehlers und die genaue Abfolge der einzelnen Aktionen und Reaktionen sind wichtig für ein komplettes Fehlerbild. Für eine detaillierte Fehlerbeschreibung sollte man sich ausreichend Zeit lassen und genau prüfen, ob man den Fehler genau erfasst hat.

Es empfiehlt sich auch, die betriebliche Auswirkung des Fehlers zu dokumentieren, um die Priorität für die Fehlerbehebung entsprechend bewerten zu können. Dazu gehört auch zu beschreiben, ob es einen Workaround für den Anwender gibt wenn der Fehler auftritt und die Wahrscheinlichkeit des Auftretens zu erfassen.

Eine verständliche Beschreibung ist gerade bei Offshore-Tests unabdingbar. Gerade beim Einsatz einer großen Anzahl von Testern muss daher auf definierte standardisierte Methoden zur Fehlererfassung und Fehlerbeschreibung geachtet werden. Ein hoher Anteil nicht reproduzierbarer Fehler deutet darauf hin, dass die Art der Fehlerbeschreibung nicht optimal ist.

Dazu gehört auch, dass die Entwickler genau festlegen, welche Informationen und Daten benötigt werden, um Fehler zu reproduzieren. Sind z. B. CAN- oder MOST-Traces erforderlich? Kann man den Log-Level für das Reporting so einstellen, dass die Ausgabe „gesprächiger“ wird? Dabei muss man aber auch darauf achten, dass die Performance des Programms sinkt, wenn man zu viele Daten mitloggt, und dass es durchaus auch zu einem Fehlverhalten kommen kann, weil zu viele Ereignisse während der Laufzeit protokolliert werden müssen. In diesem Fall jagt man dann ggf. einem Phantom nach.

Wichtig ist es, diese Festlegungen vor Start der Testdurchführungsphase zu treffen. Ansonsten sind Rückfragen und Verzögerungen bei der Fehlerbehebung vorprogrammiert. Ein Tester sollte Fehler immer sachlich beschreiben. Emotionale Äußerungen wie „funktioniert das schon wieder nicht“ oder „wann wird das endlich behoben“ sind vollkommen kontraproduktiv. Als Tester macht man sich manchmal ohnehin unbeliebt, wenn man das Produkt des Entwicklers kritisiert. Programmierer reagieren dabei manchmal wie sensible Künstler. Ein Tester sollte also nicht noch Salz in die Wunde streuen, sondern bei einer völlig neutralen Beschreibung des Fehlverhaltens bleiben.

11.9 Auswirkungen von Fehlern

Abb. 11.3 veranschaulicht die Auswirkung eines Fehlers.

So kommt es vom Defekt zum Fehlschlagen:

1. Der Programmierer begeht einen Irrtum (mistake)
2. und hinterlässt einen Defekt (defect) im Programmcode.
3. Wird dieser ausgeführt, gibt es eine Infektion im Programmmzustand,
4. der sich als ein Fehlschlagen (failure) nach außen manifestiert.

Es gilt: Fehlschlagen $\Rightarrow$ Infektion $\Rightarrow$ Defekt ($\Rightarrow$ Irrtum) – aber nicht umgekehrt: Nicht jeder Defekt führt zu einem Fehlschlagen [UNIS2014]!

Darin besteht eines der größten Probleme im Softwaretest.

Beispiel – Berechnung des Maximums dreier Zahlen

```
int max3(int x, int y, int z)
{
    int ret;
    if (x > y)
        ret = x;
    else
        ret = max(y, z);
    return ret;
}
```

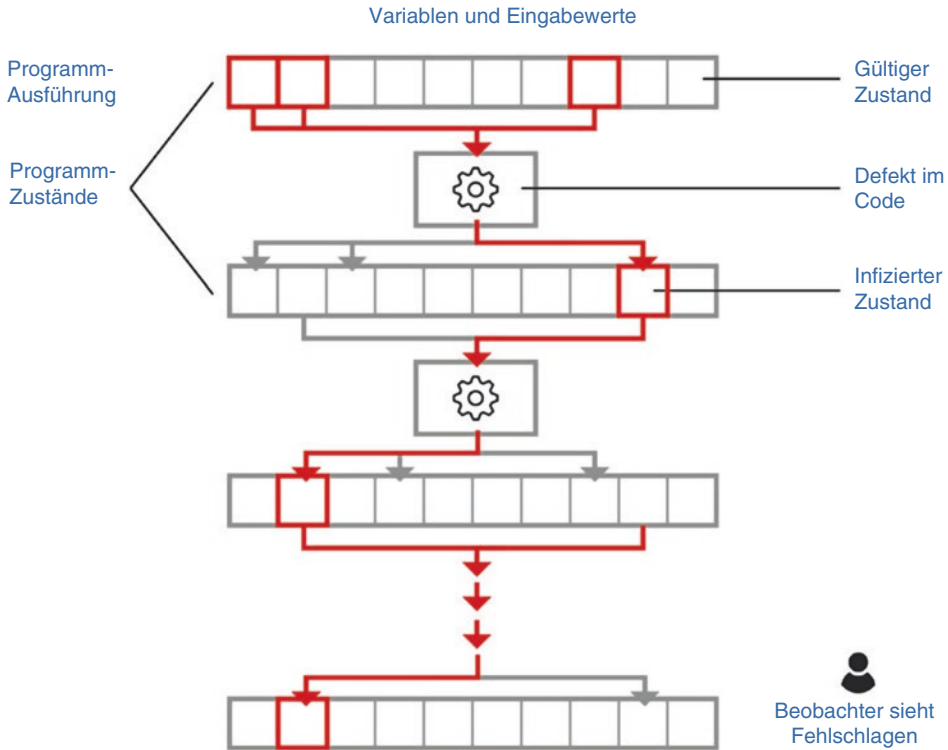


Abb. 11.3 Vom Defekt zum Fehlschlag

Fehlschlagen: $\max3(5, 2, 9)$ liefert 5

Infektion ret hat den Wert 5

Defekt: statt $ret = x$ muss es $ret = \max(x, z)$ heißen (das Maximum von x und z wäre 9 und nicht 5)

Debugging bezeichnet dabei den Rückschluss von Fehlschlägen auf den Defekt.

11.10 Schätzungen für die Fehlerdichte

Grobansatz für die Ermittlung von Fehlern

Zur Kalkulation des Testaufwands gehört eine Schätzung wie viele Fehler in etwa zu erwarten sind. Als Faustregel kann man aus der Anzahl der Programmzeilen den Testaufwand bestimmen. Man setzt pro einer bestimmten Anzahl Programmierzeilen einen Fehler an, bedeutet ein Testszenario ist erforderlich um einen Fehler aufzudecken.

Fehlerfreie Software ist aus betriebswirtschaftlichen und technischen Gründen in der Praxis unmöglich zu erstellen; anzustreben ist daher eine möglichst geringe, zu den Anforderungen der Software passende Fehlerrate. Die angestrebte Fehlerrate ist somit während der Analysephase zu definieren.

Bei Software, deren Ausfall Menschenleben kosten könnte (wie beispielsweise sicherheitskritische Militärsysteme oder medizinische Geräte, z. B. eine Herz-Lungen-Maschine), wird üblicherweise eine Fehlerdichte von $<0,5$ Fehlern pro 1000 Zeilen Code angestrebt.

Anzustreben ist üblicherweise eine Fehlerdichte von <2 (etwa für kaufmännische Anwendungen), normal ist 2–6 und gerade noch akzeptabel (z. B. für eine Informationsseite im Web oder für eine kleine App) ist der Bereich von 6–10.

Über 10 gilt als „Malpractice“ und kann vor einem Gericht zu Kompensationszahlungen führen.

Die Fehlerdichte (Tab. 11.2) kann auch zur Klassifizierung der Produktreife von Software herangezogen werden.

Wissenschaftliche Untersuchungen belegen, dass Software bei Inbetriebnahme noch bis zu 5 Fehler pro 1000 Lines Code hat, also etwa 1 Fehler auf je 3 DIN A4 Seiten manuell erstelltem Source Code. Selbst in den besten aller je untersuchten größeren Softwareprogramme findet man aber immer noch 1 Fehler in je 7500 Zeilen Code. Microsoft selbst schätzte, dass sein Code von Windows 2000 zur Markteinführung ein wirkliches, aber noch nicht genau untersuchtes Problem in je 1000 Lines Code hatte [GREI2015].

Beim Test in der Softwareentwicklung wird eine mehr oder minder große Fehleranzahl als „normal“ unterstellt oder akzeptiert. Hier herrscht auch ein erheblicher Unterschied zur Industrie: Dort werden im Prozessabschnitt Qualitätskontrolle nur noch in Extremsituationen Fehler erwartet.

Nach eigener Schätzung liegen die Kosten, die deutsche Unternehmen darauf verwenden müssen, Schäden zu beheben, die durch nicht lokalisierbare Software-Fehler entstehen (Systemabstürze, Sicherheitsprobleme, aber auch fehlerhafte Anwendungsdaten) bei ca. 2,6 % ihres Umsatzes! Das zeigt den enormen Handlungsbedarf auf, die Software stabiler zu machen, durch größere Investitionen in alle Bereiche der Software-Erstellung, vom Requirementsmanager über Software-Designer und Architekten, Entwickler bis zum Tester, Validierer und Rollout-Verantwortlichen, und wie wichtig es ist genügend hochqualifiziertes Personal dafür einzusetzen.

In einer Umfrage unter 1000 IT-Managern und Fachspezialisten gaben 65 % zu, die genauen Kosten, die ihnen der Softwaretest (in allen Ebenen) verursacht gar nicht zu kennen. Eine weitere Umfrage belegt, dass ca. 30 % aller in Deutschland produzierten Software auch nach ihrer Freigabe an die Nutzer noch schwerwiegende Mängel enthält.

Tab. 11.2 Fehlerdichte und Stabilität von Programmen

Fehlerdichte	Klassifizierung der Programme
$<0,5$	Stabile Programme
0,5 bis 3	Reifende Programme
3 bis 6	Labile Programme
6 bis 10	Fehleranfällige Programme
>10	Unbrauchbare Programme

11.11 Programmierfehler – Beispiele

Im Folgenden werden einige Programmierfehler und deren Auswirkung beschrieben, die zeigen, was für ein Potential in der richtigen Teststrategie liegt und wie teuer Unterlassungen im Testmanagement werden können.

Die Explosion der Ariane 5

Am 4. Juni 1996 startete die ESA eine unbemannte Rakete mit vier Satelliten an Bord von Französisch Guyana aus. Nach 37 Sekunden Flugzeit wich die Rakete stark von ihrem Kurs ab und musste mitsamt ihrer Nutzlast, den vier Cluster-Satelliten, gesprengt werden. Es entstand ein Verlust ca. 500 Millionen US-Dollar für Rakete und Satelliten. Die Entwicklungskosten des Systems betrugen ca. 7 Milliarden US-Dollar.

Ursache für den Absturz: Der Bordcomputer stürzte 36,7 Sek. nach dem Start ab als er versuchte, den Wert der horizontalen Geschwindigkeit von 64 Bit Gleitkommadarstellung in 16 Bit signed Integer umzuwandeln:

$$\pm b_1 b_2 \dots b_{15}.$$

Die entsprechende Zahl war größer als $2^{15} = 32.768$ und erzeugte einen Overflow. Das Lenksystem brach zusammen und gab die Kontrolle an eine zweite, identische Einheit ab. Die Selbstzerstörung wurde ausgelöst, da die Triebwerke abzubrechen drohten.

Folgende Probleme wurden nicht ausreichend berücksichtigt:

1. Die Software stammte von der Ariane 4, aber die Ariane 5 flog ungefähr 5-mal schneller. Man hatte aber auf einen intensiven Test des Navigations- und Hauptrechners verzichtet.
2. Die Software war für den eigentlichen Flug überflüssig und diente nur den Startvorbereitungen. Um einen möglichen Restart im Falle einer kurzen Unterbrechung des Countdowns zu ermöglichen, blieb das Programm 50 Sekunden lang als Sicherheit während des Flugs aktiv, bis die Bodenstation bei einer Startunterbrechung die Kontrolle übernommen hat. Trotz des ganz anderen Systemverhaltens aufgrund der erneuerten Hardware der Ariane 5 wurde dieser Punkt bei der Programmierung nicht neu bewertet.
3. Der Backup-Rechner verwendete exakt das gleiche Programm. Die Systemspezifikation legte fest, dass sich im Fehlerfall der Rechner abschalten und der Ersatzrechner einspringen sollte.
4. Die Umwandlung war nicht abgesichert, da man glaubte, dass die Zahl sowieso nie so groß sein könnte [HUCK1999].

Hier wurde also durch das Fehlverhalten einer Komponente ein Fehler in einer anderen Komponente ausgelöst.

Der Pentium-Prozessor Divisions-Fehler (1994)**Beispiel**

$$x = 4195835.0$$

$$y = 3145727.0$$

$$z = x - (x / y) \cdot y$$

Bei exakter Rechnung und fehlerfreiem Prozessor: 0

Pentium lieferte als Ergebnis: 256

Ursache

Intel verwendet einen speziellen Divisions-Algorithmus, den „Radix-4 SRT Algorithmus“:

1. Sammle die signifikanten Stellen in Divisor und Dividend
2. Lese aus einer Tabelle eine Schätzung für die nächste Stelle des Quotienten.

...

Vorteil des Verfahrens: 2 Bits des Quotienten pro Takt. Die Tabelle sollte 1066 Einträge haben. Durch eine fehlerhafte FOR-Schleife wurden nur 1061 Einträge geladen und in den Chip aufgenommen. Die Auswirkungen des Pentium-Prozessor Divisions-Fehlers im Jahr 1994 waren einige falsche Ergebnisse bei der Gleitpunkt-Division an der vierten Dezimalstelle.

Intel gab zunächst an, der Fehler werde bei einem Normalanwender statistisch nur alle 27.000 Jahre einmal auftreten und sei nur bei der Primzahlerzeugung oder anderen anspruchsvollen Berechnungen relevant. Vor allem die Fachpresse hielt dem andere Schätzungen entgegen. Das deutsche Fachmagazin c't ermittelte in seiner Januar-Ausgabe 1995 eine durchschnittliche Häufigkeit von einem Fehler alle 60 Stunden bei gleitkommaintensiven Anwendungen, räumte aber gleichzeitig ein, „dass die Zahl der wirklich Betroffenen tatsächlich nicht so groß sein dürfte“. IBM stoppte pressewirksam die Auslieferung von Rechnern mit Pentium-CPU und rechnete vor, dass der Fehler statistisch sogar alle sechs Stunden einmal auftreten könne. Die Reaktionen und Behauptungen IBMs waren damals aber nicht unumstritten, war IBM doch mit seiner Power PC-CPU und seinen RS/6000-Workstations einer der schärfsten Konkurrenten von Intel im Highend-Sektor. So wurde der Vorstoß IBMs von vielen eher als marktstrategisches Manöver gewertet. Auch Prof. Nicely vom Lynchburg College, der den Fehler entdeckt hatte, nahm persönlich Intel angesichts derart dramatischer Darstellungen in Schutz. In seinem FAQ vertrat er den Standpunkt, dass ein zwischenzeitlich von Intel in Umlauf gebrachtes White Paper mit einer statistischen Analyse des Fehlers deutlich näher an der Realität liege als die Darstellungen IBMs.

Nachdem Intel den Fehler entdeckt hatte, beseitigte man ihn stillschweigend und begann vermutlich irgendwann im Spätsommer oder Anfang Herbst damit, die Produktion

der verschiedenen Pentium-Varianten nach und nach auf die fehlerbereinigten Versionen umzustellen. Trotzdem lieferte man betroffene CPUs noch bis spät ins Jahr 1994 aus, lange Zeit davon ohne Wissen der Anwender. Kritiker warfen Intel deshalb vor, man hätte den Fehler zunächst vertuschen, dann verharmlosen wollen. Intel behauptete nach Bekanntwerden des Fehlers zunächst, er würde bei den meisten Anwendern nie auftreten. In diesem Zusammenhang soll auch von dem bereits oben erwähnten „statistischen Auftreten alle 27.000 Jahre bei normalen Endusern“ die Rede gewesen sein. Diese Einschätzung löste bei Anwendern und Fachpresse empörte Reaktionen aus. Intel kündigte zunächst an, nur CPUs von Anwendern tauschen zu wollen, die darlegen konnten, dass sie von dem Fehler betroffen seien. Viele Anwender forderten Intel daraufhin auf, alle betroffenen CPUs zu tauschen. Auch die Fachpresse ließ kein gutes Haar an dieser Ankündigung. Nachdem der Druck immer stärker wurde und dem Konzern ein ernstzunehmender Imageschaden drohte, lenkte Intel am 20.12.1994 schließlich ein und kündigte ein umfassendes Austauschprogramm für alle betroffenen CPUs an. Aber Intel zog auch seine Lehren aus dem Vorfall. Der Mitbegründer von Intel, Andy Grove, entschuldigte sich in der Presse für den Ärger, den seine Haltung verursacht hat. Es wurde extra für den Umtausch der fehlerhaften CPUs eine Telefonzentrale eingerichtet. Insgesamt stellte Intel für diesen Vorfall 475 Millionen Dollar zur Verfügung, was über der Hälfte des Gewinns im vierten Quartal des Jahres 1994 entsprach. Am Ende wurden ca. eine Million fehlerhafte Prozessoren umgetauscht. Ein wesentliches Problem war in diesem Fall die Verharmlosung und das Herunterspielen des Fehlers. Wenn man behauptet hätte, man hätte einen Testfall übersehen und würde schnell einen Patch nachliefern, wäre das Problem nicht so beachtet worden, als wenn man in der Debatte die 27.000 Jahre ins Gespräch gebracht hätte. Wenn schon ein derart gravierender Softwarefehler auftritt, sollte man immerhin richtig kommunizieren und nicht versuchen den Fehler herunterzuspielen.

Der Patriot-Raketen-Fehler im Golfkrieg

Während des Golfkriegs verfehlte am 25.02.1991 in Saudi-Arabien eine amerikanische Patriot-Rakete eine irakische Scud-Rakete. Die Scud-Rakete traf eine Kaserne und tötete 28 Soldaten.

Ursache

Der Grund war eine ungenaue Berechnung der Zeit seit Systemstart wegen eines Rundungsfehlers. Die interne Uhr des Prozessors rechnete in Zehntelsekunden. Zur Umrechnung wurde die verstrichene Zeit mit $1/10$ multipliziert, um die verstrichene Zeit in Sekunden zu berechnen.

Dabei ist

$$1/10 = 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots$$

Als Festkommazahl mit 24 Stellen also

0.00011001100110011001100

Rundungsfehler

$$2^{-25} + 2^{-26} + \dots \approx 0.000000095 = 9.5 \cdot 10^{-8}$$

Effekt

Nach 100 Betriebsstunden

$$100 \cdot 60 \cdot 60 \cdot 10 = 3,6 \cdot 10^6$$

ergibt sich als Fehler (Zeitdifferenz)

$$9.5 \cdot 10^{-8} \cdot 3,6 \cdot 10^6 = 0,34 \text{ Sekunden.}$$

Scud-Geschwindigkeit 1676 km/s = 6034 km/h. Die Scud-Rakete ist also in dieser Zeit ca. 0,57 km weitergefliegen und lag damit außerhalb der Reichweite des „Aufspürsystems“ der Patriot [HUCK1999]. Das zeigt eindrücklich, dass bei der Formulierung der Anforderungen an ein Softwaresystem bereits die Parameter zu hinterfragen sind: An welcher Stelle darf gerundet werden? Wie ungenau wird das Ergebnis durch die Rundung? Oft werden einmal formulierte Requirements und Wertebereiche aber nicht weiter hinterfragt.

Das Eisenbahn-Stellwerk Altona

Im Jahr 1995 sollte das alte Stellwerk Altona mit 50 Angestellten durch ein INTEL-486 Echt-Zeit-System ersetzt werden. Damit waren nur noch 10 Personen nötig zum Betrieb. Nach Inbetriebnahme am 13.3.95 stürzte der Rechner ab. Das Schließen des Stellwerks erzeugte ein Verkehrschaos mit Auswirkungen auf den gesamten norddeutschen Bundesbahnverkehr.

Ursache

Bei starkem Zugverkehr versucht das System einen Kellerspeicher (Stack) mit 4 kBytes im RAM-Speicher anzulegen, eigentlich vorhanden waren aber nur 3,5 kByte. 500 Byte mehr waren aus „Sicherheitsgründen“ gewählt worden. Dadurch war keine manuelle Ersatzsteuerung mehr möglich. Die Reparatur durch den Hersteller Siemens nach einigen Tagen erfolgte durch das Hinzufügen von 0,5 MB Speicher. Eine elektronische Winzigkeit hatte einen der größten deutschen Bahnhöfe mit täglich 100.000 Reisenden, 900 durchlaufenden Zügen und 1100 Rangierfahrten lahmgelegt. Unter den Hunderten in einem Spezialchip für das Stellwerk abgespeicherten Programmbefehlen war einer falsch.

„Zwei falsche Ziffern“, sagt Manfred Brandt, Leiter der Sicherheitssysteme Deutsche Bahn bei Siemens, „bewirkten, dass beim Rechner bei mehreren Arbeiten an den 160 Weichen, 250 Signalen und 200 Gleisstromkreisen der Arbeitsspeicher überlief“. Das Herzstück des Rechners, ein gewöhnlicher PC-Prozessor vom Typ Intel 80486, wurde durch die programmierte Software schachmatt gesetzt.

Verlust des Mars Climate Orbiters

Der Start des Mars Climate Orbiters der NASA war für den 11.12.98 vorgesehen. Das Erreichen der Mars-Umlaufbahn und Unterstützung des Mars Polarlander war für den

3.1.99 geplant. Danach sollten Daten von der Oberfläche des Mars gesammelt werden. Die Kartographierung der gesamten Oberfläche sollte mehrere Jahre in Anspruch nehmen, um eventuelle weitere Mars-Landungen vorbereiten zu können. Die angesteuerte Umlaufbahn am 23.9.99 lag 170 km tiefer als geplant.

Ursache

Zwei Gruppen der NASA waren am Projekt beteiligt, eine Gruppe rechnete in Meter, die andere in Inch/Fuß.

Die falsche Steueranweisung führt zu einer falsch berechneten Umlaufbahn und damit zum Verlust des Satelliten. Also ein klassisches Kommunikationsproblem, keine mathematische Herausforderung [[HUCK1999](#)].

Fehler beim Mars-Auto „Pathfinder“

Beim Mars-Roboter „Pathfinder“ 1997 bootete hin und wieder der Bordcomputer und führte manchmal einen System-Neustart durch. Dabei gingen jedes Mal die gesammelten, noch nicht gespeicherten Daten verloren, die Datenerhebung war lückenhaft und zahlreiche wissenschaftliche Untersuchungen konnten aufgrund mangelnder Messergebnisse nicht durchgeführt werden.

Ursache

Der Grund war die Organisation der Datenkommunikation bei mehreren Anforderungen verschiedener Priorität: Der Informations-Bus als Shared-Memory-Bereich zur Datenübertragung zwischen verschiedenen Komponenten mit der Bus-Management-Task (H) zum Datentransport. Die Bus-Management-Task hat die höchste Priorität und findet häufig statt. Die Task (N) wird zum Speichern meteorologischer Daten benötigt, hat eine niedere Priorität, findet selten statt und nur kurz. Die Ausführung von (N) blockierte aber die Ausführung von (H). (M) wiederum ist die Kommunikations-Task (mittlere Priorität, lang). (M) bekommt also Vorzug vor (N), da höhere Priorität.

In folgender Situation kam es zur Umkehr der Prioritäten:

- Task (N) wird ausgeführt.
- Task (H) will arbeiten, wartet aber auf das Ende von (N).
- Task (M) tritt auf, unterbricht (N) wegen der höheren Priorität, (H) ist dadurch aber weiterhin blockiert.
- Eine „watchdog“-Aktivität bemerkt nach einiger Zeit, dass (H) nicht ausgeführt wird und befiehlt den Neustart des gesamten Systems.

Gepäckförderanlage beschädigt Koffer

Bei der für das Jahr 1993 vorgesehenen Neueröffnung des Flughafens in Denver sollte ein voll automatisches Gepäcksystem verwendet werden. Es bestand aus 300 Computern, Laserscanner, Photozellen und einem Ethernet-Netzwerk. Die Eröffnung des Flughafens war

wegen Fehlern im Gepäcksystem um 16 Monate verspätet, es gab viele zerquetschte und verlorene Koffer, das Gepäck musste per Hand sortiert werden. Verlust ca. 3,2 Milliarden US-Dollar [HUCK1999].

Ursache

Über Ethernet wurden zu viele Nachrichten verschickt. Das Netzwerk (LAN) war dadurch überlastet. Die Sortier-Anweisungen kamen nicht rechtzeitig an der Förderanlage an.

Last- und Performancetests mit Simulation des Lastverhaltens sind ein wesentlicher Bestandteil von Systemtestaktivitäten. Dieses Beispiel zeigt eindrücklich ihre Notwendigkeit.

Überweisung an falsche Banken

Das neue US Federal Reserve System der Zentralbank überwies ein neues Computersystem nach seiner Inbetriebnahme 28 Milliarden Dollar an falsche Banken. Zurück kamen nur 24 Milliarden! Ein Teil des Betrags konnte durch Fehler in den Logfiles gar nicht mehr aufgefunden werden, ein weiterer Teil konnte nur durch zeitaufwendige Mahnverfahren wieder hereingeholt werden. Für den Fehlbetrag von 4 Milliarden Dollar hätte man 42.000 Softwaretester mit einem durchschnittlichen Stundenlohn von 50 USD 1 Jahr lang beschäftigen können [HUCK1999].

Raketensonde verloren

Bei der NASA-Venus-Erkundung im Jahre 1962 ging die amerikanische Sonde Mariner-1 unterwegs verloren. Die Ursache war ein Fehler in FORTRAN-Code, ein Komma fehlte an einer Stelle im Programmtext: `DO 3 I = 1.3` statt `DO 3 I = 1,3`. Mit 80 Millionen Dollar ein sehr teurer Kommafehler.

Das zeigt: kritische Programme sollten einem Software-Review unterzogen werden. Vier Augen sehen mehr als zwei. Die Notwendigkeit von Code Reviews ist zwar generell anerkannt, aber oft werden sie aus Termingründen oder mangels fehlender Ressourcen dann doch nicht oder nur teilweise durchgeführt. Gerade wenn Budgets nicht ausreichend kalkuliert sind oder es bereits zu Mehrkosten im Projekt gekommen ist, wird erfahrungsgemäß gern an Code Reviews gespart.

Auszahlung am Geldautomaten

Bei einem neuen Computerprogramm zur Steuerung der Geldautomaten bei einer australischen Bank fehlte eine Abfrage auf die Deckung des Kontos. Ursache war, dass eine Programmzeile versehentlich im Code als Kommentar gekennzeichnet war. Die neue Software zahlte Geld in unbegrenzter Höhe aus – über die Tagesbegrenzung von 200 AUD, auch wenn das Konto nicht gedeckt war. Das sprach sich schnell herum und viele Kunden nutzten die neuen „Kreditmöglichkeiten“. Nach einer Woche musste die Bank jegliche Auszahlung an ihren Automaten einstellen und die Software zurückziehen. Neben den fehlerhaften Auszahlungen war hier vor allem der Imageschaden nachhaltig [BOCH2010].

Falsche Kontonummern

Bei der Hartz-IV-Einführung wurden Überweisungen mit einer führenden Null in der Kontonummer nicht ausgeführt. Es handelte sich um 1,8 Millionen Überweisungen. Die Verbesserung erfolgte teilweise manuell in vielen Kreditinstituten mit zahlreichen Mitarbeitern, die viele extra Überstunden unter anderem am Neujahrswochenende zu leisten hatten. Ursache war, dass man schlichtweg einen Testfall mit einer Kontonummer mit führender Null nicht vorgesehen hatte. Da die Überweisungen ohne führende Null in der Kontonummer ja einwandfrei funktionierten, wurde die zu geringe Testabdeckung zunächst nicht bemerkt.

Literatur

- [GREI2015] <http://greiterweb.de/spw/FehlerReduzierungswege.htm>
- [HUCK1999] <http://www5.in.tum.de/~huckle/bugs.html>
- [OLEV2016] <http://www.olev.de/0/10er-regl.htm>
- [EMER2004] <http://emergenz.hpfsc.de/html/node44.html>
- [RHEI2013] <http://rhein-main-startups.com/tag/110100-regel/>
- [BOCH2010] Bochynek N, Delgado JD (2010) Testen in der Finanzwelt. Diaz & Hilterscheid, Berlin. isbn:978-3-00-028082-5
- [UNIS2014] <https://www.st.cs.uni-saarland.de/edu/einst/12-testen.pdf>
- [REQU2013] <http://www.computerwoche.de/a/requirements-engineering-und-projektmanagement,2538500>
- [PLON2014] <http://www.plone-entwicklerhandbuch.de/plone-entwicklerhandbuch/archetypes-artikeltypen/funktionale-tests>
- [KOSCH2006] <http://www.informatik.uni-bremen.de/st/Lehre/swp0607/testen-1x2.pdf>

In der Testplanung werden Methoden, Strukturen und Werkzeuge für den Test definiert. Es wird definiert, welche Tools für die Testbeschreibung, die Fehlerverwaltung, die Beschreibung der Anforderungen und die Automatisierung verwendet werden. In diesem Zusammenhang muss das grundsätzliche Vorgehen beschrieben werden, wenn eine neue Software von der Entwicklung geliefert wird, also die Prozesse einschließlich der Schnittstellen zu anderen Abteilungen, das Testreporting, die Einbeziehung der Stakeholder und der zeitliche Rahmenplan.

Teilweise (vor allem bei Industriefirmen) müssen parallel mechanische oder elektronische Hardwarekomponenten entwickelt und getestet werden, die teilweise Voraussetzung für bestimmte Testschritte sind. Es ist wichtig, dass diese Testphasen mit den Testphasen des Softwaretests korrespondieren. Verzögerungen bei der Hardwareentwicklung haben schon so manche Anwendung um Monate zurückgeworfen.

12.1 Erarbeitung der Teststrategie und der Testmaßnahmen

Zur Festlegung der Teststrategie gehört die Beschreibung, welche Testdaten verwendet werden, wo sie abgelegt sind und wie sie organisiert werden. Außerdem muss die Testumgebung genau beschrieben sein, sofern sie gemeinsam für alle Testfälle verwendet werden kann.

Dazu gehört die verwendete Hardware mit ihren Leistungsmerkmalen, Eigenschaften des Testlabors, aber auch die Basissoftware, also das Betriebssystem, der Browser und verwendete Tools. Bei größeren Projekten oder großen Organisationen wird empfohlen, Testprozesse, Rahmenvorgaben und allgemein gültige Themen und Inhalte, die in jedem Testkonzept behandelt werden, in übergeordneten Dokumenten (Testpolitik, Testhandbuch, Rahmentestkonzept) festzulegen. Das Ziel ist dabei die Übertragbarkeit auf andere Produkte.

Bei der Testplanung für das einzelne Projekt oder die einzelne Anwendung muss man überprüfen, ob diese Rahmendaten gültig sind und ggf. vorhandene Abweichungen beschreiben.

Teilweise benötigen unterschiedliche Tests unterschiedliche Testkonfigurationen. Auch das ist eindeutig im Rahmen der Teststrategie abzuhandeln. Konfigurationen und spezielle Testvoraussetzungen, die nur für einen einzelnen Testfall gültig sind, müssen immer beim entsprechenden Testfall beschrieben werden, gemeinsame Rahmenbedingungen für das Gesamtprojekt im Testplan.

Bei Regressionstests müssen die Fehlerbehebungen nachgewiesen werden. Dazu ist ebenfalls eine sinnvolle Untermenge an Testfällen zu definieren. Speziell bei Regressionstests muss daher darauf geachtet werden, nicht nur die Funktion, in der der Fehler aufgetreten ist, sondern benachbarte und verbundene Systeme zu testen. Dafür ist ein hoher Wissensstand über die Arbeitsweise des Gesamtsystems erforderlich.

12.2 Gemeinsames Verständnis über Testvorgehen und Testziele

Wenn der Umfang der Requirements und der Aufwand für die Entwicklung bekannt ist und in das gewünschte Endprodukt in etwa absehen kann, muss der zu erwartende Testaufwand bewertet werden. Dazu gilt es nun eine nähere Strukturierung der Aufgaben vorzunehmen (Tab. 12.1).

Eine andere Möglichkeit ist es, die Funktionen der Software (z. B. anhand einer Feature- oder Work Package-Liste) im Einzelnen zu bewerten. In kleineren Einheiten lässt sich der Aufwand genauer planen.

Für die Basis sollen Mitarbeitertage verwendet werden. Für ein größeres Projekt bedeutet eine Stundenbewertung im Vorfeld ein zu großer Aufwand, während eine Bewertung auf Wochenbasis meist zu ungenau ist. Bei der Bewertung auf Wochenbasis entsteht erfahrungsgemäß zu viel Leerlauf weil die Übergabedaten dann nicht exakt genug abgegrenzt sind. Als Ergebnis dieser Berechnung kommt eine Zahl heraus, z. B. 870 Mitarbeitertage.

Tab. 12.1 Testvorgehen und Testziele

	Ermittlung/Einheit	Aufwand pro Einheit	Aufwand in Summe
Testplanung, Testkonzept	Umfang des Projekts		
Erstellung der Testbeschreibungen	Anzahl der Testfälle Komplexität des Testprodukts		
Testdurchführung	Anzahl Releases Dauer pro Release		
Testautomatisierung	Voraussichtliche Anzahl Testfälle Aufwand und Nutzen der Automatisierung		
Testreporting	Anzahl Releases Einmaliges Setup		

Um Urlaub, Krankheit, Fortbildung usw. zu berücksichtigen, rechnet man mit 150 Tagen bzw. 1800 Stunden pro Jahr und Mitarbeiter. Des Weiteren muss berücksichtigt werden, zu wie viel Prozent der Mitarbeiter für das Projekt zur Verfügung steht. Das kann vor allem dann kompliziert werden, wenn sich die Arbeiten in parallelen Projekten in bestimmten Zeiträumen konzentrieren. Dann ist ein Mitarbeiter z. B. im März zu 240 % und im Juni nur zu 30 % ausgelastet, und solche Anzeichen deuten auf einen Terminverzug hin. Die Planung von Testaktivitäten muss daher auf den Tag genau erfolgen. Das bedeutet zwar höheren Planungsaufwand, eine geringere Verdichtungsstufe bedeutet bereits zu viel Unsicherheiten. Die Ergebnisse sollten täglich getrackt werden; mit einer professionellen Toolunterstützung ist das möglich, man kann auch täglich kurze Meetings (die dann aber nicht mehr als 15 Minuten dauern sollten) ansetzen.

Durch die Aufwandsermittlung kann man dann die benötigte Dauer für die Testaktivitäten berechnen. Der Einsatz professioneller Planungstools ist dafür unbedingt zu empfehlen.

12.3 Tests nach Auslieferung des Produkts

Manchmal ist es sinnvoll, den Anteil der Fehler, die nach Auslieferung auftreten können, in einem eigenen Testzyklus zu definieren, dann nämlich, wenn das Projekt in kurzer Zeit aus vertraglichen Gründen fertiggestellt werden musste, kein dringendes Folgeprojekt ansteht bzw. die Zeitplanung für das Folgeprojekt erheblich großzügiger ist. Wenn in einer solchen ungeplanten Testphase die Testabdeckung erhöht werden kann und Fehler noch aufgedeckt werden können, sind die Fehler zwar immer noch teuer, aber evtl. nicht ganz so gravierend. Es verhält sich anders, wenn sie der Kunde erst einige Monate später nach einem erfolgten Rollout entdeckt.

Eine entsprechende Testphase lässt sich aber schwer vor dem Controlling rechtfertigen, und bei einer Planung nach definierten Entwicklungsprozessen ist sie ja auch nicht erforderlich. Dennoch kann es Situationen geben, bei denen dieses Verfahren zur Risikominimierung als sinnvoll geboten erscheint, und dafür in Einzelfällen auch ein Budget bereitgestellt werden sollte, um zum Beispiel nachträgliche Regressanforderungen zu vermeiden oder das Vertrauen in die eigene Software nicht ganz zu verspielen.

Dieses Vorgehen fehlt in standardisierten Prozessen in der Regel. Ist die Software erst einmal vom Endkunden abgenommen, wird kein Geld mehr investiert. Besonders kritisch wird es, wenn Fehler vom eigenen Testteam nach Abnahme der Software durch den Kunden bereits aufgefallen sind, dem Kunden diese Fehler aber verschwiegen werden.

12.4 Testdaten

Die bei Testzyklen verwendeten Testdaten sollten dauerhaft und zentral verwaltet werden. Nur mit denselben definierten Daten ist es möglich einen Testzyklus vergleichbar mit denselben Bedingungen durchzuführen. Dazu gehört auch, neue Daten mit denen ein bestimmter Testfall ergänzt oder ein bestimmter Fehlerfall produziert wurde permanent zu

ergänzen. Wenn man während der Testdurchführung nicht auf den aktuellen vollständigen Datensatz zugreifen kann, sind Verzögerungen und Fehler vorprogrammiert. Ein strukturiertes Testdaten-Management ist daher Voraussetzung für erfolgreiche Softwaretests.

Beim Testdaten-Management sollen eine Reihe von Daten festgelegt werden, die besondere Ausprägungen zeigen.

Beispiel

Zum Test besonderer Adressen für ein Postzustellsystem werden verschiedene Ausprägungen verwendet:

- 12559 Berlin, Straße 653 Haus-Nr. 5
- 32361 Preußisch Oldendorf, Rathausstr. 3
- 33758 Schloss Holte-Stuckenbrock, Gerhart-Hauptmann-Weg 5
- 50858 Köln, Aachener Str. 1177
- 65468 Trebur, Außerhalb 1
- 68159 Mannheim, C3 20
- 86152 Augsburg, Erstes Quersachsengässchen 5
- 86156 Augsburg, Dr.-Dürwanger-Str. 18g
- 86167 Augsburg, Steinerne Furt 58 1/3
- 95361 Ködnitz, Heinersreuth Nr. 5
- 97980 Bad Mergentheim, Andreas-von-Grumbach-Str. 10

Bei diesem Test werden also sehr lange oder zusammengesetzte Ortsnamen, hohe Hausnummern, Hausnummernzusätze und Adressen der Mannheimer Planquadrate in die Testbeschreibung aufgenommen.

Beim Test von Kundenstammdaten sind Kundennamen anzulegen. Um deutlich zu machen, dass es sich um keine echten Kunden machen, empfehlen sich Namen wie „Hans Mustermann“, „Donald Duck“ oder „Königin Beatrix“.

Schon mehrfach habe ich beobachtet, dass manche zwar theoretisch mögliche, aber in der Praxis fast nie auftretende Konstellationen ausgiebig getestet und mit viel Aufwand verifiziert und validiert wurden. Andere Geschäftsvorfälle und Datenausprägungen, die tagtäglich vorkommen, wurden aber in einem viel zu geringen Maße berücksichtigt.

Daher empfiehlt sich für jeden Tester, den Ablauf in der Praxis selbst kennenzulernen. Dann muss man sich auch seltener den Vorwurf der Softwareverifikation im Elfenbeinturm anhören. Es hat sich auch als vorteilhaft erwiesen, einen Praktiker im Projekt bereits in der Planungsphase mit einzubeziehen, einen Mitarbeiter anzuhören der die betriebliche Praxis und den späteren Einsatzbereich vor Ort gut kennt, der den Softwaretest begleitet und der als Pilotanwender wertvolle Hinweise vor einem Rollout geben kann. Dann fühlen sich diejenigen Anwender, die das System die nächsten Jahre benutzen sollen, auch von Anfang an in die Entscheidungsprozesse eingebunden. Einerseits kann man erläutern, warum nicht alles, was in der betrieblichen Organisation gewachsen ist, aufrecht erhalten werden kann, andererseits kann man manchmal mit relativ kleinen Änderungen an der

Anwendung Probleme in der täglichen Praxis leicht lösen. Ein ausgefeiltes Anforderungsmanagement berücksichtigt das zwar schon im Vorfeld, aber es ist immerhin besser, dass umständliche Eingaben oder unklare Abläufe im Test auffallen als erst nach einem umfassenden Rollout.

Vor dem Hintergrund einheitlicher Entwicklungsstandards lässt sich nicht immer die Praxis exakt nachbilden. Generell aber sollte sich die IT an der bestehenden Organisation ausrichten und flexibel bei Organisationsänderungen reagieren können und nicht die IT die innerbetriebliche Organisation festlegen. Manche Testfälle verändern die Testdaten, so dass die Ausgangsbedingungen für weitere Testfälle sich auch ändern können. Das bedeutet eine besondere Herausforderung beim Testdesign. In diesem Fall muss auch gewährleistet sein, dass die Testdaten nach jedem Testzyklus wieder auf einen definierten Ausgangsstand zurückgesetzt werden. Die Veränderung von Testdaten durch die Testdurchführung kann auch bedeuten, dass die Reihenfolge der Abarbeitung bestimmter Testsequenzen vorgegeben ist. Das ist bei der Planung der Testaktivitäten ebenfalls zu berücksichtigen.

12.5 Das Testprojekt im Entwicklungszyklus

Tests sollen entwicklungsbegleitend stattfinden. Im Allgemeinen empfiehlt sich in etwa der in Abb. 12.1 dargestellte Projektverlauf.

Agile Ansätze setzen verstärkt auf den entwicklungsbegleitenden Test, weil ja hier auch inkrementell entwickelt wird.

Aber auch bei klassischen Methoden empfiehlt es sich, den Test frühzeitig mit einzubinden und die Funktionen, die bereits fertiggestellt wurden, schon zu testen. Auch eine phasenweise Integration ist besser als ein „Big Bang“ am Ende der Entwicklung. Hier

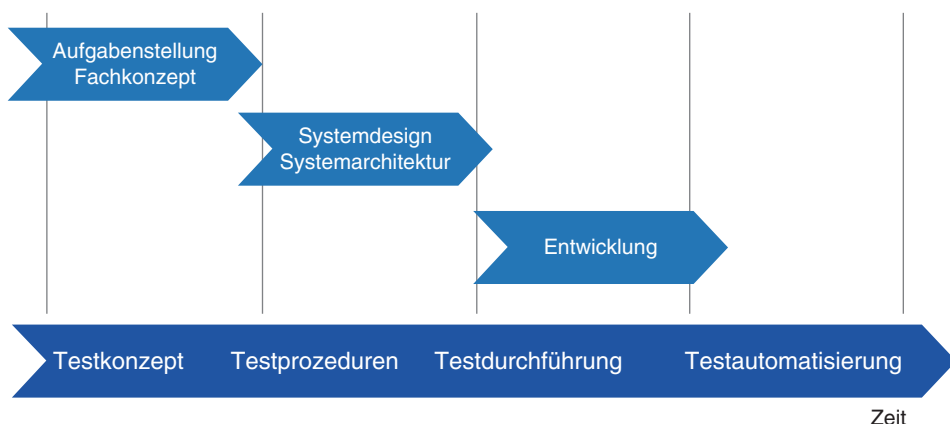


Abb. 12.1 Tests im Entwicklungszyklus

zeigt es sich wieder, dass die Fehlerkosten immer dann am niedrigsten sind, wenn sie frühzeitig erkannt werden.

Dabei muss dem Tester natürlich zu jeder Phase bekannt sein, welchen Umfang er schon testen kann und welche Funktionen noch nicht fertig sind. Die Testabdeckung steigt also auch während der einzelnen Releases eines Projekts sukzessive an.

Manchmal fehlt der Name für die Software, d. h. der Tester weiß dann gar nicht genau was er testet. Hin und wieder hat die Software keinen richtigen Namen, oder einen kryptischen wie „2.3.027536 BugFix XY with AB“, weil das Tool aus dem Konfigurationsprozess selber den Namen generiert hat. Eine eindeutige Bezeichnung des Testobjekts ist aber unbedingt erforderlich, damit der Test auch in einer späteren Phase reproduzierbar ist.

Umso wichtiger ist es, das Testobjekt immer exakt zu benennen. Am Anfang der Testdurchführung sollte immer geprüft werden, welche Software eigentlich getestet wird und ob das richtige Produkt installiert wurde.

Dazu gehört auch, genau nachweisen zu können, welche Komponenten mit welcher Version bei der Integration in die zu testende Software eingeflossen sind. Deswegen ist ein professionelles Konfigurationsmanagement für einen erfolgreichen Softwaretest unabdingbar, und wenn in diesem Bereich Schwächen vorhanden sind, muss der Tester darauf hinweisen, weil sonst seine gesamte Testdurchführung und sein ausgefeiltes Testreporting auf wackligen Beinen stehen.

12.6 Zeitliche Planung der Testaktivitäten

Bei der zeitlichen Planung der Testaktivitäten muss man sich an der Rahmenplanung der Projektplanung, also den Milestones, orientieren. Oft besitzt der Testmanager nicht die Freiheit, Termine selbst vorzugeben, sondern muss sich an den vereinbarten Projektzielen orientieren, da andere Termine (z. B. Rollout, Validierung, Begutachtung) vom Softwaretest abhängen.

Der Vorlauf für den Aufbau und die Konfiguration des Testsystems, für die Erstellung der Testprozeduren, für Einarbeitung und Schulungen des Testteams muss im Rahmen der Testplanung in ausreichendem Maße berücksichtigt werden. Auch Abhängigkeiten von der Entwicklung oder anderen Zulieferern sind dabei zu überdenken. Der Testplaner sollte Vorkehrungen treffen, wie Abhängigkeiten und Risiken möglichst vermieden werden. Es ist nicht sinnvoll, auf 100 % perfekte Ausgangsbedingungen zu warten, denn dann würde der Test nie starten können.

Sinnvoller ist immer ein „entwicklungsbegleitender Test“, also eine sukzessive Entwicklung die von Testaktivitäten lanciert wird. Der agile Testansatz hebt vor allem diesen Aspekt hervor. Das bedeutet aber, dass man gewisse Abstriche hinnehmen muss: Man hat vielleicht noch gar nicht die endgültige Hardware zum Testen, weil auch die Hardware erst noch entwickelt wird, sondern muss auf einem Prototypen testen. Bestimmte Module der Software, die Eingangsdaten für das zu testende Teilsystem zur Verfügung stellen, sind vielleicht noch gar nicht fertig oder existieren nur in einer vorläufigen

Version. Das bedeutet, man muss manchmal Umwege gehen, Testdaten erst einmal selber erstellen, falls erforderlich Mock- und Stub-Objekte einsetzen. Eingaben simulieren und mit dem Mangel leben.

Risiken sollten in einer Matrix mit Eintrittswahrscheinlichkeit und zu erwartenden Kosten bewertet werden (beispielhaft in Tab. 12.2).

Beispiel

Wenn alle drei Risiken in voller Höhe eintreten, wird das Projekt also um 52.000,– Euro teurer als geplant.

An diesem Beispiel sieht man auch, auf welche Risiken man sich besonders konzentrieren sollte: Wenn man kurzfristig mehr Zeit und Entwicklungsaufwand in Komponente B steckt, vermeidet man ein besonders teures Risiko.

Tab. 12.2 Risiken im Testprozess

ID	Risiko	Eintrittswahrscheinlichkeit	Kosten bei Eintrittswahrscheinlichkeit	Kalkulatorische Kosten
R1	Hardware nicht verfügbar	20 %	90 TEUR	18 TEUR
R2	Komponente B nicht rechtzeitig verfügbar	40 %	80 TEUR	32 TEUR
R3	Fluktuation im Testteam	5 %	40 TEUR	2 TEUR

Das Thema „Testumgebung“ wird in IT-Projekten häufig unterschätzt. Zwar heißt es dann in den meisten Testkonzepten allgemein, eine hohe Produktionsnähe solle erreicht werden, was das aber genau heißt, bleibt häufig unklar [BRAND2015].

Zur Testumgebung gehören Hardware, Middleware, Software, aber auch Aspekte wie Organisation, verwendete Testdaten, Datumssimulation, Benutzerkennungen und Zugriffsrechte, Schnittstellen, betriebliche Komponenten und unterschiedliche Konfigurationen.

Laut ISTQB-Glossar gibt es in der Norm IEEE610 eine Referenz: „Testumgebung ist eine Umgebung, die benötigt wird, um Tests auszuführen. Sie umfasst Hardware, Instrumentierung, Simulationen, Softwarewerkzeuge und andere unterstützende Hilfsmittel.“

Um die Testumgebung konkret zu definieren, empfiehlt es sich, die durchzuführenden Testaktivitäten und das jeweilige Testziel für jede einzelne Teststufe zu beschreiben (siehe Tab. 13.1).

Im nächsten Schritt erstellt man eine Sammlung der benötigten Umgebungsmerkmale.

Die Granularität der Merkmale ist im Einzelfall festzulegen. Mittelfristig sollten alle relevanten Anforderungen an die Testumgebung mit dem entsprechenden Schema erfasst werden können.

Beispiel

- **Hardware**
 - Server
 - Client
 - Storage
 - Peripherie
 - Netzwerk

Tab. 13.1 Testaktivitäten und Testziele

Testaktivität	Testziel
Evaluiere	Freies Ausprobieren
Entwickeln und Debuggen	Codeerstellung
Komponententest/Modultest	Frühe Fehlerermittlung hinsichtlich Funktion, Robustheit, Performance
Komponentenintegrationstest	Test der Schnittstellen und komponentenübergreifenden Software
Systemintegrationstest	End-to-End-Test wichtiger Geschäftsprozesse
Fachlicher Systemtest	Test aus Anwendersicht (Funktion, Robustheit, Schnittstellen)
Last- und Performancetest	Test des Gesamtsystems auf Performance und Ressourcenverbrauch durch Simulation von Belastungssituationen
User Acceptance Test	Prüfung der Navigation, Ergonomie, Usability, Barrierefreiheit
Fachlicher Abnahmetest	Vertrauensbildung, Nachweis der Erfüllung der Abnahmekriterien
Technischer Abnahmetest	Test gegen betriebliche Anforderungen
Fehleranalyse und Re-Test	Nachstellen komplexer Produktionsfehler, Korrekturnachweis

- **Software**
 - Betriebssystem-Server
 - Betriebssystem-Client
 - Datenbanken
 - Testobjekt-Konfiguration
- **Middleware**
 - Application Server
 - Load Balancer
- **Daten**
 - Basisdaten (z. B. Postleitzahlenverzeichnis, Tabelle mit Bankleitzahlen)
 - Historiendaten (z. B. Kundenstammdaten, Artikelstammdaten)
 - Menge
 - Aktualität
- **Sicherheit**
 - Authentication Server
 - Verschlüsselung
 - Zertifikate
- **Administration**
 - Wartungszeiten
 - Nutzerkennungen
- **Sonstige**
 - interne Partnersysteme
 - externe Partnersysteme
 - Mandantenfähigkeit
 - Barrierehilfen
 - Sprachdienste
 - Laufzeitbibliotheken
 - Datumssimulation

In einem weiteren Schritt kann man nun die Informationen beider Schritte zu einer Matrix zusammensetzen, und dabei die Produktionsnähe pro Aktivität beschreiben.

Nach Befüllen der Tabelle durch die jeweiligen Stakeholder lassen sich gleichartige Zeilenmerkmale zusammenfassen (Clustering) und so die benötigten Umgebungen und ihre Eigenschaften (Obermengen angekreuzter Umgebungsmerkmale) ableiten.

Als Ergebnis können unterschiedliche Systemumgebungen entstehen wie z. B.

- Entwicklungsumgebung,
- Integrationsumgebung,
- Produktionsumgebung,
- Produktionsspiegelung (Test).

Weiterhin gilt es, das wichtigste Kriterium zur Vervielfältigung von Testumgebungen zu beachten: die Notwendigkeit, parallel verschiedene Versionen einer Umgebung zur Verfügung stellen zu müssen. Während die meisten Testaktivitäten darauf abzielen, die nächste zu veröffentlichende Version eines Programms zu analysieren, entsteht durch die Analyse von Produktionsfehlern oder zusätzliche Migrationsaktivitäten (z. B. Datenbank-Upgrades) der Bedarf nach Umgebungen bereits ausgerollter Versionen.

Man muss in der Testabteilung darauf achten, dass auch nach dem Rollout eines Programms eine vollständige Testumgebung mit Abbild des Produktivsystems zur Verfügung steht, damit Fehler, die in der Produktionsumgebung auftreten, jederzeit nachgetestet werden können. Man benötigt also mindestens ein System, was den Ist-Stand abbildet und ein weiteres für Weiterentwicklungen oder Änderungen der Software. Es muss unbedingt vermieden werden, diese beiden Optionen auf einer Umgebung miteinander mischen zu wollen, weil man dabei leicht Unterschiede in der Basissoftware übersieht und nicht unerhebliche Installationsaufwände erzeugt. Im Falle eines Fehlers in der Produktivumgebung muss der Tester schnell reagieren können, um den Fehler sofort nachzustellen und dem Entwickler direkt gezielte Hinweise zur Fehlerbehebung geben zu können. Auch der Entwickler sollte in solchen Fällen die Möglichkeit haben sich am Testsystem durch zusätzliche Logfiles oder die Möglichkeit des Debuggings weitere Informationen zur gezielten Lokalisierung des Fehlers zu verschaffen. Wenn das Testsystem dann erst wieder auf den Softwarestand gebracht werden muss, das auch in der Produktion eingesetzt wird, vergeht wertvolle Zeit und es besteht eine weitere Fehlerquelle, dass bei der Installation und Konfiguration der Software ein Fehler auftritt und dadurch eine Abweichung von Test- und Produktivsystem entsteht.

Die oben beschriebene Matrix bietet somit die Grundlage für Management, Organisation und Planung von Testumgebungen. Sie beantwortet zum einen die Frage „Was wird wo getestet“, zum anderen definiert sie die Menge aller von einer Umgebung zu erfüllenden Anforderungen. Außerdem beantwortet diese Matrix auch die Beantwortung von Detailfragen: Wird etwa der Einsatz von Automatisierungswerkzeugen (Testrobotern) in Integrationsumgebungen mit der Begründung abgelehnt, diese Umgebung solle schließlich „produktionsnah“ sein, fällt das Argument in sich zusammen, wenn der Einsatz eines Testroboters das konkrete Testziel nicht gefährdet.

Außerdem gestattet sie die zielgenaue Ansiedlung zukünftiger Testaktivitäten auf der dafür geeigneten Umgebung. Durch die Ermittlung von Testziel und benötigter Produktionsnähe kann man leicht anhand des entstandenen Profils die geeignete Umgebung zuweisen. Bei der Testplanung ist zu beachten, dass die Beladung der Testumgebungen mit der richtigen Software und den richtigen Daten pro Version eine gewisse Zeit benötigt, in der die Testumgebung dann evtl. nicht zur Verfügung steht. Bei der Planung der Testdurchführung sind solche für den Tester unproduktiven Zeiten zu berücksichtigen. Die einzelnen Prozessschritte bei der Testdurchführung sind exakt zu planen, und Verzögerungen durch die Bereitstellung der Testumgebung wirken sich auf den gesamten Testprozess negativ aus.

Mit dem vorgestellten Schema, das pro Projekt abgewandelt werden sollte, werden Testumgebungen handhabbar, beschreibbar und vergleichbar. Auch neu auftauchende Anfragen lassen sich fundiert und systematisch beantworten. Darüber hinaus ergibt sich ein einheitliches Verständnis und Vokabular bei allen Beteiligten. Unterschiedliche und diffuse Vorstellungen von Testumgebungen werden dadurch gemeinsam und verständlich dokumentiert [BRAND2015].

Literatur

[BRAND2015] http://pi.informatik.uni-siegen.de/stt/31_1/03_Technische_Beitraege/Brandes_Testumgebungen.pdf

Ein Testkonzept beschreibt für ein spezifisches Projektvorhaben die vorgegebenen und zu erfüllenden Mindestanforderungen für die geplante Testvorgehensweise, die Zuständigkeiten, die Verantwortlichkeiten und Abhängigkeiten. Des Weiteren werden die Rahmenbedingungen (z. B. Testumgebung, Konfiguration des Testsystems, Testvoraussetzungen), Ziele des Testvorgehens, die zu verwendenden Testmethoden und Testverfahren sowie der Umfang der durchzuführenden Testaktivitäten festgelegt [HETT2014].

14.1 Grundlagen

Testfälle müssen immer so beschrieben sein, dass sie ein anderer Mitarbeiter versteht und ohne Rückfragen testen kann. Also je detaillierter, je exakter, desto besser. Für Mitarbeiter, die schon jahrelang in einem Unternehmen als Softwaretester arbeiten, mag sich das manchmal übertrieben kompliziert anhören. Aber man sollte daran denken, dass durch betriebliche Umstrukturierungsmaßnahmen, Krankheit, Fluktuation, Ruhestand etc. es immer wieder vorkommen kann, dass ein anderer Tester auf die Testdurchführung angesetzt wird, und dass man selber nach einigen Jahren so betriebsblind ist, dass man viele Testvoraussetzungen oder viele Befehle bei der Testdurchführung unbewusst verinnerlicht hat und voraussetzt.

Ein grundlegendes Systemverständnis muss man für einen Tester voraussetzen, aber keineswegs eine Kenntnis des Unternehmens und der Testumgebung. Nicht jeder neue Mitarbeiter bringt langjährige Erfahrung im Softwaretest mit, ein höherer Aufwand bei der Dokumentation, weil man auf das Weglassen angeblich bekannter Voraussetzungen verzichtet, zahlt sich in aller Regel schon nach wenigen Testzyklen aus.

Das Testkonzept bestimmt Abgrenzung, Vorgehensweise, Mittel und Ablaufplan der Testaktivitäten. Es bestimmt die Elemente und Produktfunktionen, die zu testen sind, die

Testaufgaben, die durchgeführt werden müssen und das verantwortliche Personal für jede Aufgabe und das Risiko, das mit dem Konzept verbunden ist.

Das Testkonzept beschreibt Testobjekte, Testmethoden, Testinfrastruktur sowie die Testorganisation. Aufgrund dieser Testkonzeption kann eine Abschätzung erstellt werden, wie lange der Test voraussichtlich dauern wird: Es umfasst ebenfalls die Testplanung und die Testfallbeschreibungen. Für jeden Testfall wird eine detaillierte Testfallbeschreibung erstellt. Diese stellt die Testspezifikation dar. Die Testplanung legt den logischen und zeitlichen Ablauf der Tests fest.

Das Testkonzept bildet die Grundlage, auf der die Testorganisation und die Testinfrastruktur bereitgestellt und die Tests durchgeführt werden. Es wird bei neuen Erkenntnissen geändert und nachgeführt. Deshalb macht man sich im Testkonzept zunächst Gedanken über die Geschäftsprozesse, die der zu testenden Anwendung zugrunde liegen [HERM2018].

Um welche Anwendung handelt es sich, welche Abläufe müssen abgebildet werden, in welchem organisatorischen Umfeld, welche User sind involviert? Das alles hat Einflüsse auf Umfang, die Ausprägung und die Details der Durchführung.

Dann ist zu prüfen, wer alles am Testprojekt beteiligt ist. Wie sind die Abhängigkeiten, wo ist es organisatorisch aufgehängt, wie sind die Eskalationsprozeduren? Wer muss wann über was informiert werden? Welche Fakten sind Details, welche entscheidend für den Projektfortschritt, welche Probleme müssen bis zu welcher Stelle berichtet werden? Hierzu gehört, einschätzen zu können, welche Abhängigkeiten bestehen und welche Auswirkungen Verzögerungen, mangelnde Funktionalitäten oder unzureichende Testabdeckung haben kann. Dazu braucht es ein gewisses Systemverständnis, um eine realistische Risikoabschätzung von vornherein treffen zu können und Fallgruben im Projekt zu entdecken bevor man hineinfällt.

Bevor man anfängt, Tests zu beschreiben, muss man das Ziel kennen, wo man mit dem Projekt hin will. Was muss letztlich funktionieren, was bedeutet das für den Test, ist das Projekt richtig gequotet oder von Anfang an viel zu optimistisch eingeschätzt? Wenn das Projekt nur ein Budget von 1 Mio. Euro hat aber 2 Mio. Euro realistisch wären, hat auch der Tester von Anfang an verloren. Entweder die Zeit reicht nicht aus, oder die Kosten werden gesprengt oder die Qualität leidet, oder alle diese Problemkreise kommen zusammen. Wenn man diese Bedingungen vorgesetzt bekommt, kann man nur noch das Beste daraus machen und eben mit einer geringeren Abdeckung testen als es eigentlich erforderlich wäre. In der Regel sind aber derart viele Parameter gleichzeitig zu berücksichtigen und es gibt widersprüchliche Ziele, so dass es schwer fällt alles zu optimieren.

Besonders schwierig wird es auch, wenn man als Testverantwortlicher in ein schon länger bestehendes Projekt gerufen wird, das schon aus dem Ruder gelaufen ist. Dann bleibt nur noch Zeit für Schadensbegrenzung.

Leider ist manchmal gar nicht bewusst wie lange etwas dauert oder was im Einzelnen nötig ist. Aber auch da liegt es teilweise am Tester, die Konsequenzen und zu erwartenden Probleme frühzeitig aufzuzeigen und zu warnen, bevor ein Projekt scheitert. Wenn es von

Anfang an klar ist, dass ein Termin nicht zu halten ist, dann gilt es, dies ebenfalls zu kommunizieren. Manchmal sollen gesetzliche Regelungen derart kurzfristig gelten und haben eine derart knapp bemessene Übergangsfrist, dass man sich mit manuellen Ergänzungen auf dem Beleg oder standardisierten Hinweisen erst mal behelfen muss. In großen wie in kleinen Projekten gilt dabei: Immer die Sache in den Vordergrund stellen, und sich am Endprodukt orientieren.

Oft habe ich schon beobachtet, dass bestimmte Termine von Experten abgeschätzt wurden, aber ein Projektmanager, Entwicklungsleiter oder Controller meinte das müsste viel schneller gehen und die Termine wurden nach vorne verlegt. Es gab danach – was zu erwarten war – Zeitverzug, man musste nachbessern und war vor dem ursprünglich geschätzten Termin nicht fertig, sondern meistens noch später. Die Schätzung der Experten war meist realistischer, wobei auch diese Art der Schätzung oft noch zu optimistisch ist weil viele Mitarbeiter nur ihren eigenen Teil sehen und die Komplexität des Gesamtsystems und die Unternehmensprozesse mit zahlreichen Beteiligten nicht hinreichend berücksichtigt werden.

Wenn dann während des Projekts ein Termin oder ein Meilenstein nicht eingehalten können, ist es erfahrungsgemäß besser, den Zeitplan einmalig gleich um einen längeren Zeitraum zu verschieben. Schlechter ist es, wenn ein Terminverzug unvermeidbar ist, eine zu kurze Verschiebung vorzunehmen und dann diese zu kurze Verschiebung erneut zu verlängern.

Wenn man immer wieder Termine verschiebt, macht man sich unglaublich. Das bedeutet, man kann Zeiten nicht richtig einschätzen. Und wie soll man einem Tester abnehmen, dass er die Qualität des Testobjekts richtig einschätzt, wenn er schon an der Schätzung der Aufgabenpakete scheitert?

Es empfiehlt sich, eine Vorlage für die eigenen Testkonzepte zu erstellen. Im Internet kann man passende Grundlagen finden. Dabei sollte auch definiert sein, für welche Zwecke die Vorlage dient. Wenn einzelne Kapitel nicht erforderlich sind, sollten die Kapitel trotzdem erstellt werden und als „not applicable“ gekennzeichnet sein oder einen kurzen Verweis auf definierte Prozesse enthalten. Das ist besser als Kapitel einfach wegzulassen – so dokumentiert man, dass durchaus an das Thema gedacht wurde, es aber aus bestimmten Gründen für das spezielle Projekt nicht für erforderlich halten oder meinen darauf verzichten zu können. Das Testkonzept sollte grundsätzlich am Anfang des Projekts erstellt werden. Erfahrungsgemäß werden aber viele Details im Laufe des Projekts (aus unterschiedlichen Gründen) anders umgesetzt als es im Testkonzept ursprünglich vorgesehen war. Dann ist es wichtig, das Testkonzept an die geänderten Bedingungen anzupassen. Wenn ein Haus anders gebaut wird, als es der Bauplan vorgesehen hat, ist auch der Bauplan nachzuziehen. Analog verhält es sich mit dem Testkonzept. Wenn man den Aufwand für die Erstellung des Testkonzepts ausschließlich zu Projektbeginn vorsieht, läuft man Gefahr, dass man bei einem späteren Audit oder einer Zertifizierung Probleme bekommen wird, wenn man anders getestet hat als es ursprünglich vorgesehen war.

14.2 Aufbau des Testkonzepts

Das Testkonzept enthält folgende Inhalte:

- **Testziel:** Das Testziel beschreibt, warum getestet wird. Es beleuchtet die Erwartungshaltung, die an den Test gestellt wird und warum zum Nachweis der Qualität gerade der gewählte Testumfang erforderlich ist.
- **Testobjekte:** Die Testobjekte sind für die Festlegung des Testinhalts erforderlich, sie dokumentieren was getestet wird. In den Testobjekten ist das zu testende System beschrieben. Dabei muss auf eine genaue Abgrenzung geachtet werden. Objekte, die beim Test nicht mitgetestet werden, sondern als gegeben betrachtet werden, sind hier ebenfalls anzugeben.
- **Testrahmen mit Testvoraussetzungen/Fehlerklassen/Start- und Abbruchbedingungen:** In diesem Kapitel werden die Randbedingungen des Tests beschrieben. Grundlegende Festlegungen, die für das gesamte Projekt getroffen werden, sind hier zu treffen. Hierzu gehören die Ressourcen, also Mitarbeiter, Testsysteme und die Beschreibung des Testprozesses (die einzelnen Teststufen, ihr Aufbau und ihre Abgrenzung voneinander) und der Projektorganisation.

Es muss in diesem Zusammenhang auch ersichtlich sein, wie die Übergabe der Software an die Tester geschieht. Was erwartet der Tester neben der Software für Angaben von der Entwicklung, also z. B. eine Release Note, eine Featureliste, eine Dokumentation über Einschränkungen?

- **Testumgebung:** Die Testumgebung beschreibt die Konfiguration des Testsystems und legt die Bedingungen des Testumfelds fest. Die Testinfrastruktur dokumentiert das Testsystem, die Testdaten und verwendete Testhilfsmittel.
- **Testfallbeschreibungen:** Die einzelnen Testprozeduren sind Bestandteil der Testkonzeption. In der Testkonzeption wird definiert, welche Testfälle vorgesehen sind. Die Prozeduren werden nach einer festgelegten Struktur erstellt.
- **Testplan:** Der Testplan berücksichtigt die terminlichen Restriktionen für den Softwaretest.

An dieser Stelle muss definiert werden, welche Tester wann und in welchem Umfang zur Verfügung stehen. Was passiert, wenn es Verschiebungen im Plan gibt, wie dynamisch können zusätzliche Ressourcen bereitgestellt werden? Wann kann man mit geringerer Testabdeckung reagieren, inwiefern ist das möglich ohne das Gesamtergebnis zu gefährden? Gibt es Restriktionen bezüglich der Arbeitszeit, Arbeitsdauer, Wochenenden? Besonders brisant wird dieses Thema wiederum bei mehreren Standorten. Da kann es um so triviale Dinge gehen wie unterschiedliche Feiertagsregelungen, vor allem bei ausländischen Standorten, die man in der Planung leicht übersieht. Ist allen Beteiligten am Softwareprojekt wirklich klar, was eine Terminverschiebung bedeutet?

Man sollte auch daran denken, dass der Test im Softwareprojekt eher am Ende der Prozesskette steht. Wenn ein Projekt nicht termingerecht fertig wird, aber die Entwicklung

gerade noch vor dem Abgabetermin fertig wurde, nur eben für den Test nicht mehr ausreichend Zeit war, muss das der Testmanager sehr gut begründen können. Schnell ist man bei der Suche nach dem Schuldigen beim Softwaretest anbelangt.

Meist wird ja, wenn es terminlich knapp wird, gerade die für den Softwaretest vorgesehene Zeit zusammengestrichen und am vorgesehenen Zeitfenster geknappt. In diesem Fall muss eindeutig darauf verwiesen werden, dass die Testabdeckung geringer ist als ursprünglich vorgesehen bzw. dass eine Verstärkung der Ressourcen genau aus diesem Grund in der Phase des Systemtests aufgestockt werden mussten oder dass es sich um einen entwicklungsbegleitenden Test handelte.

Wichtig ist auch die Abgrenzung des Untersuchungsgegenstands: Von Anfang an sollte der Tester darauf hinweisen, was ganz bewusst nicht getestet wird. Wenn der Testmanager das nicht darstellt, erzeugt er einen Erwartungsdruck der dem Systemtest gar nicht angemessen ist. Selbst wenn dann die Tests, die gar nicht durchgeführt werden, auch folgerichtig nicht beschrieben sind, kann es immer danach aussehen, dass sie schlichtweg vergessen wurden. Daher empfiehlt sich der Einsatz von Templates, in denen möglichst viele der Punkte von vornherein genannt sind. Wenn man sie nicht benötigt, kann man das explizit erwähnen. Das ist in jedem Fall besser als nicht zu testende Funktionen einfach unter den Tisch fallen zu lassen.

Im Rahmen der Testvorbereitung ist es wichtig zu prüfen, ob die Testobjekte überhaupt testbar sind. Das ist in vielen Unternehmen immer noch nicht durchgängig erledigt.

Wenn ein Objekt nicht testbar ist, stellt sich als nächstes sofort die Frage, wie denn dann die Funktionsfähigkeit nachgewiesen wird. Teilweise betrifft die Testbarkeit bestimmte Teststufen: So kann z. B. eine bestimmte Funktionalität oder ein bestimmtes Systemverhalten zwar sinnvoll im Komponententest, aber nicht unbedingt im Systemtest oder Abnahmetest nachgewiesen werden. Die Abb. 14.1 zeigt, welche Aktivitäten zur Testvorbereitung durchgeführt werden.

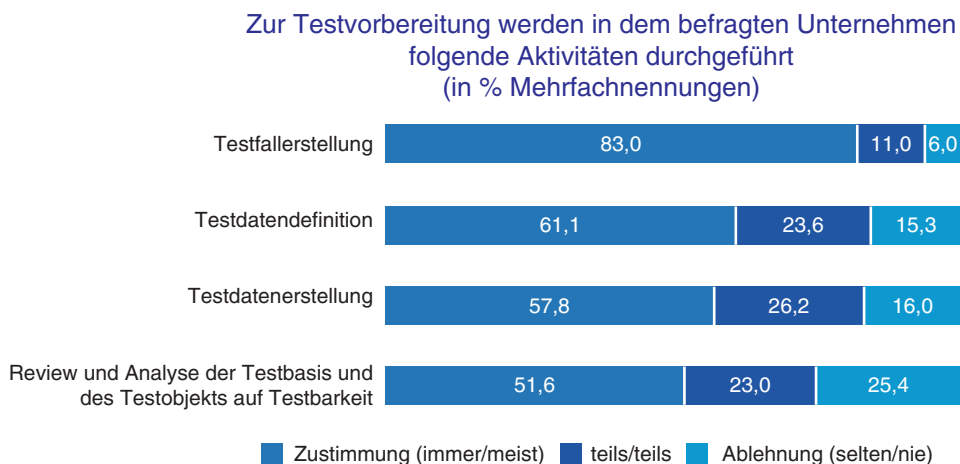


Abb. 14.1 Aktivitäten zur Testvorbereitung

14.3 Testkonzept nach IEEE 829

Das „Institute of Electrical and Electronics Engineers“ ist ein Verband, der Normen zur Standardisierung von Standards publiziert. Die Norm IEEE 829 als „Standard for Software and System Test Documentation“ beschreibt einige Standard-Dokumente, die in einzelnen Phasen des Softwaretests verwendet werden sollen.

Dieser Standard listet für ein Testkonzept folgende Inhalte auf:

Testkonzeptbeschreibung

Kennung/Beschreibung des Testkonzepts, die sicherstellen muss, dass das Dokument in anderen Projektdokumenten eindeutig und klar referenziert werden kann. Ein entsprechendes Benennungsschema wird oft durch das Projektarchiv oder ein zentrales Dokumentenmanagement vorgegeben. Je nach Größe der Organisation, in der das Testprojekt abgewickelt wird, kann diese Dokumentenkennung beliebig kompliziert strukturiert sein. Sie sollte mindestens den Dokumentnamen, die Dokumentenversion und den Freigabestatus enthalten.

Einführung

Die Einführung soll eine kurze Darstellung des Projekthintergrunds bieten, die allen Stakeholdern (Kunde, Management, Entwickler, Tester ...) hilft, den Inhalt des Testkonzepts besser verstehen und einordnen zu können.

Testobjekte

Dieses Kapitel enthält eine kurze Darstellung, aus welchen Teilen bzw. Komponenten die zu testende Software besteht. Auflistung derjenigen Komponenten, die Gegenstand des Tests sind (inkl. Angabe von Versionsnummern, Übergabemedium, Spezifikationen usw.).

Die Systemteile, die nicht getestet werden, sondern als Inhalt der Testumgebung vorausgesetzt werden, sollen ebenfalls aufgelistet werden.

Zu testende Leistungsmerkmale

Dieser Abschnitt beschreibt, welche Funktionen oder Eigenschaften des Systems getestet werden müssen. Die Identifikation der zu testenden Leistungsmerkmale referenziert auf zugehörige Testspezifikationen und ordnet ggf. Testobjekte und Teststufen zu.

Leistungsmerkmale die nicht getestet werden

Um den Testgegenstand sinnvoll abzugrenzen, Missverständnisse zu vermeiden und unberechtigten Erwartungen vorzubeugen muss auch klargestellt werden, welche Aspekte nicht getestet werden sollen oder können, z. B. aus Ressourcenmangel, Termin- und Kostenrestriktionen oder technischen Gründen.

Teststrategie

Die Teststrategie ist die Definition der Testziele, möglichst basierend auf einer Risikoanalyse. Die Teststrategie soll das Risiko aufzeigen, dass durch mangelnde Tests gewisse Fehler nicht gefunden werden, also Risiken, die im Testobjekt bzw. dessen Betrieb liegen.

Daraus folgt eine Priorisierung der Testziele, und es wird sichergestellt, dass die wichtigsten Testfälle vordringlich geprüft werden. Die Teststrategie enthält weiterhin die Auswahl und Darstellung der Testmethoden, die anzuwenden sind. Es muss klar erkennbar sein, ob und warum die ausgewählten Methoden geeignet sind, die Testziele zu erreichen und zwar unter Berücksichtigung vorhandener Ressourcen und identifizierter Risiken.

Abnahmekriterien

Die Testresultate müssen nach Abschluss aller für das Testobjekt vorgesehenen Tests dahingehend überprüft werden, ob eine Freigabe für das jeweilige Testobjekt erfolgen kann und eine Lieferung durchgeführt werden kann. Dazu werden die Abnahme- und Testendekriterien festgelegt.

In der Regel wird eine Kombination der Kriterien aus „Anzahl Testfälle“, „Anzahl der Fehler“ und „Schweregrad der Fehler“ gewählt.

Abnahmekriterien werden individuell pro Testobjekt gewählt. Die Schärfe der Abnahmekriterien orientiert sich an der Risikobetrachtung, d. h. für unkritische Testfälle werden weichere Abnahmebedingungen formuliert als für sicherheitskritische Abläufe. Der Testaufwand wird dadurch auf die wesentlichsten Teile des Testobjekts fokussiert.

Kriterien für Testabbruch und Testfortsetzung

Neben Abnahme- und Testendekriterien müssen auch Kriterien definiert werden, die eine Unterbrechung bzw. den Abbruch von Tests anzeigen. Es kann sein, dass ein Testobjekt einen derart geringen Qualitätsstand aufweist, dass auch nach zahlreichen Tests die Abnahmefähigkeit nicht erreicht wird. Daher werden zusätzliche Kriterien benötigt, die rechtzeitig einen Testabbruch auslösen und weitere unnötige Testaktivitäten unterbinden. Das Testobjekt wird dann, ohne dass alle geplanten Tests bis zum Ende durchgeführt werden, zur Nachbesserung an die Entwickler zurückgegeben. Weiterhin werden Kriterien zur Wiederaufnahme bzw. Fortsetzung dieser Tests benötigt. Hierzu führt man einen Eingangstest mit einer Gruppe ausgewählter Testfälle durch. Nur wenn die Tests mit dieser Auswahl an Tests fehlerfrei bestanden wurden, wird die eigentliche Testdurchführung begonnen.

Testdokumentation

In diesem Abschnitt wird vereinbart, welche Berichte und Ergebnisse die Testaktivitäten liefern und wann, in welcher Form und an welchen Verteiler diese Ergebnisse kommuniziert werden. Dabei geht es nicht nur um die Testergebnisse im engeren Sinn (z. B. Fehlermeldungen, Testprotokolle), sondern auch alle Planungsdokumente (z. B. Testkonzept, Testspezifikationen mit einzelnen Testprozeduren, Zeitplan), Begleitdokumente zu Testobjektübergaben (Testobjektlieferschein), zusammenfassende Teststatusberichte und Statistiken.

Testaufgaben

Die Testaufgaben sind eine Auflistung der Aufgaben, die zur Planung und Durchführung der einzelnen Testfälle erforderlich sind, mit Zuordnung der Verantwortlichkeiten. Der Status der Testaufgaben (offen, in Arbeit, zurückgestellt, erledigt) wird im laufend zu führenden Teststatusbericht verfolgt, auf den in diesem Kapitel verwiesen wird.

Testinfrastruktur

Dieser Punkt enthält eine Auflistung der Bestandteile der Umgebung, die notwendig ist, um die geplanten Tests durchführen zu können: Testplattformen, Arbeitsplätze für Tester, technische Ausstattung der Arbeitsplätze, Testtools, Entwicklungsumgebung, weitere Werkzeuge (z. B. Internetanschluss, Mailzugang, Office-Programme).

Verantwortlichkeiten und Zuständigkeiten

Hier wird die organisatorische Einbindung der Tester in das Projekt beschrieben. Disziplinarische und fachliche Weisungsbefugnis, Aufgabenpakete, Organisation und Aufteilung in Testgruppen und/oder Teststufen, Zuordnung von Personen und Aufgaben, Vertretungsregelungen.

Personal, Einarbeitung, Ausbildung

Darstellung der zur Planumsetzung erforderlichen Personalausstattung (Rollen, Qualifikationen, Personalkapazität, Einsatz- und Urlaubsplanung) für Testpersonal und weitere am Projekt Beteiligte (z. B. Personal zur Administration der Testinfrastruktur), einzubeziehende Entwickler und Kunden, nötige Ausbildungen und Schulungen der Tester.

Zeitplan/Arbeitsplan

Grober Zeitplan der Testaktivitäten und Zuordnung zu den Meilensteinen des Projekts. Diese Planung muss in den Projektplan des Projektmanagements Eingang finden und dort gepflegt werden. Eine regelmäßige Abstimmung zwischen Projektmanager und Testmanager ist nötig: Der Testmanager muss über Lieferprobleme der Entwicklung informiert sein und auf Terminverschiebungen im Tagesgeschäft und in der Detailplanung seiner Testaktivitäten reagieren. Der Projektmanager muss auf Testergebnisse reagieren und ggf. Meilensteine oder Einführungstermine verschieben, Funktionalitäten reduzieren oder zusätzliche Releases für Korrekturzyklen vorsehen.

Planungsrisiken und Unvorhergesehenes

In diesem Abschnitt geht es um Risiken, die sich bei der Umsetzung des Testkonzepts ergeben oder Risiken, die sich darauf ergeben, dass Maßnahmen nicht eingeplant wurden, weil schon zum Zeitpunkt der Planerstellung klar war, dass sie im konkreten Projekt nicht umgesetzt werden können.

Die Liste dieser Risiken sollte während der laufenden Teststatusberichte regelmäßig neu bewertet werden. Manchmal können Risiken nach einigen Wochen als nicht mehr relevant betrachtet werden, während manche Risiken, die zu Erstellung des Testkonzepts nicht betrachtet oder unterschätzt wurden, vielleicht im Zeitverlauf die Erreichung der Projektziele gefährden. Der Nobelpreisträger Friedrich von Hayek bemerkte einst, dass Ökonomie darin bestehe, dem Menschen vor Augen zu führen, wie wenig er wirklich über das weiß, was er planen zu können glaubt. Das gilt für Softwareprojekte in besonderer Weise. Gerade für die Risikobewertung empfiehlt sich ein Workshop mit allen am Projekt Beteiligten, um ein realistisches Bild der Situation von Anfang an zu erhalten.

Genehmigung/Freigaben

In diesem Kapitel sind die betroffenen Personen bzw. Organisationseinheiten aufzuführen, die das Testkonzept freigeben oder zumindest über das Testkonzept informiert werden. Betroffene Personengruppen sind z. B. Entwickler, Projektmanagement, Projektleitungs-ausschuss, Betreiber und Anwender des Softwaresystems, Auftraggeber bzw. Kunde und Tester. Im Testkonzept wird im Ideal die Zustimmung zur Bewilligung und Finanzierung der umfangreichen personellen und technischen Kapazitäten erteilt, die ein angemessener Test des Systems erfordert.

In der Realität bezieht man sich aber im Testkonzept meist auf den Minimalumfang der Testaktivitäten und holt die Zustimmung zu den Maßnahmen ein, um die Qualität für das System nachzuweisen. Dann akzeptiert aber der Projektleiter immerhin, dass auf der Grundlage der Resultate dieser Testaktivitäten Entscheidungen über Systemfreigaben und Auslieferungen getroffen werden, die mit einem gewissen Restrisiko behaftet sind. Da es immer wieder vorkommt, dass gravierende Fehler trotz guter Planung und trotz positivem Projektverlauf erst im Feldeinsatz auftreten, weil einfach vorher niemand das Risiko erkannt hat oder die Tragweite eines Problems unterschätzt hat, ist dieses Risiko hinreichend zu berücksichtigen.

14.4 Testarten, Testinhalte und Testphasen

Im Testkonzept ist festzulegen, welche Inhalte in welcher Testphase getestet werden. In der Praxis hat sich die in Tab. 14.1 dargestellte Aufteilung bewährt.

Tab. 14.1 Testarten, Prüfgegenstände und Testphasen

Testart	Prüfgegenstand	Testphase
Funktionalität	Vollständige Erfüllung der Anforderungen	Modultest, Systemtest
Datenvolumen	Massentest mit großen Datenmengen	Performancetest, Lasttest
Antwortzeit	Messung der Antwortzeiten unter verschiedenen Belastungszuständen und mit unterschiedlicher Hardware	Performancetest, Lasttest
Zuverlässigkeit	Systemverhalten bei dauerhafter Belastung über einen längeren Zeitraum	Performancetest, Lasttest
Fehlertoleranz	Systemverhalten bei Überlast, z. B. zahlreiche gleichzeitige Zugriffe, viele parallele Datenbankabfragen	Performancetest, Lasttest
Sicherheit	Wirksamkeit der Sicherheitsmechanismen	Modultest, Systemtest
Interoperabilität	Kompatibilität mit anderen Systemen, Schnittstellen, Auswirkungen auf andere Systeme und Effekte aus anderen Systemen	Integrationstest, Systemtest
Dokumentation	Vollständigkeit, Konsistenz und Verständlichkeit	Codereview, Dokumentationsreview

14.5 Testkonzept und betriebliche Realität

Einige ausgewählte Szenarien verdeutlichen die Problematiken, die im Laufe des Softwaretests auftreten können:

A) Die Software wird zu spät fertig.

1. Die Planung für ein Release ist wie folgt:

- Montag, 01.02. – Freitag, 12.02. Entwicklung und Software-Integration
- Montag, 15.02. – Freitag, 19.02. Systemtest
- Montag, 22.02. Auslieferung des Systems

Der Auslieferungstermin ist in jedem Fall zu halten, weil sonst eine Vertragsstrafe droht.

2. Die Testfälle und die Testaktivitäten werden derart geplant, dass das fertige System am Montag, 15.2. getestet werden kann. Es wird davon ausgegangen, dass die Fehler so gering sind, dass sie in einem Folgerelease behoben werden können.
3. Die Software-Entwicklung und die Systemintegration sind zum 12.2. noch nicht fertig. Der Systemtest bekommt das Release erst 2 Tage später zum Testen. Der Auslieferungstermin muss aber unbedingt gehalten werden. In diesem Moment ist es bereits erforderlich, dass entweder mehr Testressourcen bereitgestellt werden (um statt 5 Tage nun das gesamte Testprogramm in 3 Tagen zu testen) oder aber die Testabdeckung verkleinern und Features herausstreichen. Das sollte aber bereits am Montag, 15.2. dem Projektmanagement klar sein. Dann liegt es an der Entscheidung des Projektmanagements – ob der Systemtest zusätzliche Ressourcen bekommt (wenn das überhaupt möglich ist – auch hier kann es Restriktionen geben: Personal, Hardware ...), ob es akzeptabel ist die Testabdeckung zu verringern oder ob man den Termin doch verschieben kann/muss. Oft wird genau dieser wesentliche Schritt unterlassen – der Tester hofft eben den Test trotz ungünstiger Rahmenbedingungen gerade noch hinzukriegen und den Aufwand in den engeren Zeitrahmen zu quetschen. Dabei entsteht mehr Druck für die Testabteilung, die Qualität sinkt, man testet weniger Funktionen oder kürzt Tests ab. Das kann bedeuten, dass sich die Software-Entwicklung auch beim nächsten Release Zeit lassen wird oder die Terminsituation nicht ausreichend bemerkt, oder der Testverantwortliche wird mit seinen Prognosen unglaublich – der Testmanager hat vorher fünf Tage im Plan angesetzt und schafft jetzt dasselbe Programm in drei Tagen, er hat also ursprünglich so geplant, dass die Abteilung nicht ausgelastet ist und sich zwei Tage Puffer als Ressourcen vorab gesichert. Dann wird er auch Probleme haben, einen neuen Server, eine neue Lizenz, einen neuen Mitarbeiter zu bekommen, denn mit dem eingeschränkten Budget geht es ja offensichtlich auch. Oder aber der Tester hatte von Anfang an korrekt geplant, die drei Tage sind wirklich zu kurz und das Produkt wird am Ende mit mangelnder Testabdeckung ausgeliefert, in der Folge kommt es zu einem schwerwiegenden Fehler. Dann wird die erste Frage sicher sein, warum der Fehler niemandem vor Auslieferung aufgefallen ist. Die Testabteilung soll mit der

Entwicklungsabteilung ja produktiv zusammenarbeiten, um ein qualitativ hochwertiges Produkt zu liefern. Hier hat die Testabteilung also die Entwicklungsabteilung geschont, weil sie den Verzug akzeptiert hat, aber auch nicht vor Auslieferung des Produkts auf Fehler hingewiesen hat. Das führt zu einer Schwächung der Testabteilung. Dann kann es dazu kommen, dass man sich fragt, wozu man sich überhaupt eine Testabteilung leistet, wenn sie einen schwerwiegenden Fehler nicht bemerkt. Dass die Entwicklung dann zu spät und mangelhaft geliefert hat, ist im Nachhinein schnell vergessen.

B) Die Software-Entwicklung und die Software-Integration liefern eine fehlerhafte Version, die nicht ausgeliefert werden kann.

1. Die Planung für ein Release ist wie folgt:

- Montag, 1.2.–Freitag, 12.2. Entwicklung und Software-Integration Version 1.0,
- Montag, 15.2.–Freitag, 19.2. Systemtest Version 1.0,
- Montag, 22.2. Auslieferung des Systems.

Der Auslieferungstermin ist in jedem Fall zu halten, weil sonst eine Vertragsstrafe droht.

2. Die Testfälle und die Testaktivitäten werden derart geplant, dass das fertige System am Montag, 15.2. getestet werden kann. Es wird davon ausgegangen, dass die Fehler so gering sind, dass sie in einem Folgerelease behoben werden können.

3. Die Testabteilung bekommt das Release Version 1.0 rechtzeitig am 12.2. am späten Nachmittag und startet am Montag, 15.2. mit Ihrem Systemtest. Am späten Nachmittag des 15.2. bemerkt ein Tester einen massiven Softwarefehler. Dieser Softwarefehler stellt weitere Tests in Frage und wirkt sich derart aus, dass entscheidende Funktionen des Systems nicht richtig arbeiten und die Anwendung nach Meinung des Testmanagers mit diesen Qualitätsmängeln nicht ausgeliefert werden kann. Der Testmanager ruft kurz vor 17 Uhr den Projektmanager an und schildert das Problem. Der Projektmanager teilt die Einschätzung des Testmanagers und meint, der Fehler müsse unbedingt noch behoben werden. Der herbeigerufene Leiter der Software-Entwicklung meint, die Fehlerbehebung inkl. Software-Integration werde den gesamten Dienstag beanspruchen, so dass für den Systemtest erst Mittwoch früh eine verbesserte Version zum Test zur Verfügung stehen könne. Diese Version soll dann auch ausgeliefert werden.

Der Systemtest hat also auch in diesem Beispiel nur noch drei Tage Zeit für den eigentlichen Systemtest der korrekten Version. Es steht aber am Dienstag das Testteam wie geplant zur Verfügung. Der Projektmanager meint daher, der Systemtest solle „schon mal den Rest“ weiter testen, denn die Fehler würden sich ja nur auf eine bestimmte Funktionalität beziehen.

In diesem Fall macht es durchaus Sinn, weiter zu testen, und nicht den Dienstag einfach nur abzuwarten oder die Tester für andere Aufgaben zur Verfügung zu stellen.

Eventuell fallen im Systemtest ja weitere Fehler auf, die dann auch in der verbesserten Version noch vorhanden sind. Alle mit dieser Version durchgeführten Tests sind aber unter

Vorbehalt zu betrachten – denn man kann nicht wissen, ob sich die verbesserte Funktion positiv oder negativ auf die Software auswirkt. Angenommen, die Software besteht aus Funktion A, B, C und D. Am Montag ist im Systemtest aufgefallen, dass Funktion B nicht arbeitet. Am Dienstag wird Funktion C getestet. Es kann sein, dass Funktion C Fehler aufweist, die nach der Fehlerbehebung von Funktion B ebenfalls behoben sind. Es kann auch sein, dass Funktion C beim Test am Dienstag einwandfrei funktioniert, aber nach der Fehlerbehebung von Funktion B nicht mehr. Es kann auch sein, dass sich bezüglich Funktion C in der nachgebesserten Version nichts geändert hat. Das alles ist zu berücksichtigen.

Mit etwas Erfahrung und Verständnis über Zusammenhänge der einzelnen Systeme gelangt der Testmanager zu einer Risikoabschätzung für die notwendigen Aktivitäten am Dienstag. Man muss dann aber immer gewahr sein, dass im beschriebenen Beispiel die Testaktivitäten an der Version 1.0 risikobehaftet sind. Es sei denn, man bringt durch einen zusätzlichen Kraftakt doch noch eine komplette Testabdeckung in den verbleibenden drei Tagen hin.

Literatur

- [HERM2018] http://www.hermes.admin.ch/onlinepublikation/content/ergebnis_testkonzept.html
[HETT2014] <http://www.hettwer-beratung.de/konzepte/testkonzept>

Betriebliche Entscheidungen hängen in den meisten Fällen von Zahlen ab. Zahlen begründen Entscheidungen oder regen sie an. Insofern ist es nachvollziehbar, dass Entscheidungsträger in Unternehmen mit einer ständig steigenden Zahlenmenge und einer permanenten Informationsflut konfrontiert werden. Moderne Informationssysteme (Big Data) ermöglichen zahlreiche Auswertungen und Zahlen. Entscheidungen, die aus Zahlen abgeleitet werden, müssen nicht immer richtig sein, wenn Zahlen nur einen Teil der Wirklichkeit abbilden. Auch Zahlen können in die Irre führen. Zahlen verleiten stärker als Worte dazu, ihnen zu vertrauen. Das ist ein Risiko beim Arbeiten mit Zahlen.

Der wachsenden Zahlenmenge durch immer komplexere DV-Anlagen und gestiegene Verarbeitungskapazitäten steht eine konstante Verarbeitungskapazität der Entscheidungsträger gegenüber. Neue Verfahren mögen die Datenmenge steigern, aber nicht in gleichem Tempo immer mehr Daten in Entscheidungen einbinden. Es gibt zwar immer mehr Metriken, aber oft werden gerade aufgrund der Informationsflut die Tragweite und die Bedeutung von Zahlen nicht richtig erkannt.

Es wird daher immer wichtiger, Methoden zu finden, mit deren Hilfe die Informationsflut auf die wenigen wesentlichen Sachverhalte reduziert wird, die wirklich entscheidungsrelevant sind. Außerdem werden Verfahren benötigt, die wichtigsten Sachverhalte und wesentlichen Veränderungen zu erkennen. An diesem Problem setzen Kennzahlen an. Mit dem Informationszeitalter erlangen sie steigende Bedeutung. Sie helfen, Daten zu verdichten und so aufzubereiten, dass die richtigen Entscheidungen gefällt werden können, ohne auf relevante Informationen zu verdichten. Sie machen Projekte miteinander vergleichbar.

Jede Behandlung von Originaldaten birgt eine gewisse Gefahr des Informationsverlustes und der Manipulation. Diese kann durch den Einsatz eines schlüssigen Kennzahlensystems verringert werden. Dabei ist auf eine einseitige Ausrichtung der Kennzahlen sowie auf Abhängigkeiten untereinander zu achten.

In der Folge werden einige Kennzahlen zum Softwaretest erläutert. Das ist selbstverständlich keine erschöpfende Aufzählung aller denkbaren Kennzahlen, sondern kann nur ein kleiner Eindruck sein was mit Kennzahlen alles dargestellt werden kann. Welche Kennzahlen im Einzelnen in welchem Projekt zu beachten sind und in die Entscheidungsprozesse einfließen sollen, ist bei Start des Testprojekts zu klären und abzustimmen.

15.1 Testprozessreife

In einem komplexen Projekt ist es nötig, dem Auftraggeber die Sicherheit zu geben, dass man auch den gestellten Herausforderungen gewachsen ist. Dafür wird die Prozessreife bewertet.

Gerade in Zeiten, in denen die Verringerung der Fertigungstiefe und die Bedeutung des Outsourcings von Geschäftsprozessen wachsen, ist es wichtig, neutrale Kriterien für die Bewertung der Testprozessreife zu definieren. Dadurch soll festgestellt werden, ob ein Anbieter oder auch ein Testteam hinreichend qualifiziert ist, den Systemtest zu stemmen, so dass die Erfolgswahrscheinlichkeit steigt.

Dafür werden folgende Key Indicators herangezogen:

- Reife der Teststrategie,
- Reife des Lebenszyklusmodells,
- Zeitpunkt, ab dem die Tester am Projekt beteiligt sind,
- Reife des Verfahrens zur Aufwandsabschätzung,
- Reife der Testspezifikationstechnik,
- Nutzung statischer Testmethoden,
- Einsatz von Testmetriken,
- Verwendung professioneller Testtools,
- Stabilität der Testumgebung,
- Qualität der Office-Umgebung,
- Motivation und Commitment,
- Qualifikation und Schulung des Testpersonals,
- Reife der verwendeten Testmethoden,
- Status der Kommunikationswege,
- Reife des Testberichtswesens,
- Reife des Fehlerbehandlungswesens,
- Management der Testartefakte,
- Reife des Testprozessmanagements,
- Reife der Systembewertungsverfahren,
- Qualität der Unittests.

Die einzelnen Kriterien sind dabei sinnvoll zu gewichten. Diese Gewichtung kann von Projekt zu Projekt unterschiedlich sein. Man vergibt dabei entsprechende Punkte für die

einzelnen Kriterien und kommt in der Summe zu einer Gesamtpunktzahl aus der die (absolute und relative) Testprozessreife mathematisch bestimmt werden kann.

Durch die Erhebung und Bewertung solcher Kennzahlen lassen sich Projekte besser miteinander vergleichen, und man bekommt genauere Hinweise, in welchem Bereich ein besonderer Handlungsbedarf besteht. Es empfiehlt sich dabei, vor allem die neutralistischen Punkte konzentriert zu optimieren, z. B. die 3 wichtigsten Probleme zuerst anzugehen.

15.2 Ermittlung der Testproduktivität

Die Testproduktivität richtet sich nach dem Grad der Testautomatisierung und der Erfahrung und Motivation der Tester. Wenn man die Testproduktivität einschätzen kann, kann man leichter den zu erwartenden Aufwand eines Testprojekts ermitteln.

Dafür kann man folgende Formel verwenden:

$$\text{Testproduktivität} = \text{TestPoints} / \text{Testertage} \cdot \text{Einflussfaktor}$$

Die Formel von Boehm für die Testaufwandschätzung berücksichtigt die Werte

- Projektbedingungen,
- Produktqualität,
- Systemtyp,
- Testproduktivität,

um daraus den Testaufwand zu ermitteln.

Die fünf Bedingungen, die Boehm vorschlägt und auf ein Testprojekt gut anwendbar sind, sind:

- Grad der Testwiederholbarkeit,
- Stabilität der Testumgebung,
- Kenntnis der Anwendung,
- Zusammengehörigkeit des Testteams,
- Reife des Testprozesses.

Dabei werden diese Projektbedingungen mit einem Exponenten auf einer Skala von 0,91 bis 1,23 bewertet:

- sehr niedrig = 1,23
- niedrig = 1,1
- durchschnittlich = 1,0
- hoch = 0,96
- sehr hoch = 0,91

Der Skalierungsexponent ist der arithmetische Mittelwert dieser fünf Bedingungen.

Wenn also

- die Testwiederholbarkeit durchschnittlich ist,
- die Stabilität der Testumgebung hoch ist,
- die Kenntnisse der Anwendung niedrig sind,
- die Zusammengehörigkeit des Testteams niedrig ist sowie
- die Reife des Testprozesses durchschnittlich ist,

ist im Ergebnis die Skalierungskomponente

$$(1,0 + 0,96 + 1,1 + 1,1 + 1)/5 = 1,03.$$

Das bedeutet, dass man in diesem Fall leicht unterdurchschnittliche Projektbedingungen vorfindet. Mit Hilfe dieser Formel kann man die Risiken besser bewerten und zusätzlich zu erwartenden Aufwand oder Terminverschiebungen besser einschätzen. Für die Kalkulation des Testaufwands muss weiterhin die Qualität und die Komplexität mit berücksichtigt werden.

Die Softwarequalität ist aus Sicht des Tests deren Testbarkeit, das bedeutet, wie leicht bzw. wie schwer es ist, dieses System zu testen. Diese Größe wird anhand des Aufwands zu messen, der erforderlich ist, um ein definiertes Testziel zu erreichen. Je höher dieser Aufwand ist, desto niedriger ist die Testbarkeit. Breite Schnittstellen mit zahlreichen Parametern und überladene graphische Oberflächen mit vielen Widgets deuten auf geringe Testbarkeit hin.

Die Softwarekomplexität bemisst sich aus der Anzahl der Beziehungen zwischen Entitäten. Dabei kommen mindestens folgende vier Komplexitätsmaße in Frage:

- Komplexität der Benutzeroberflächen,
- Komplexität der Systemschnittstellen,
- Komplexität der Datenbanken,
- Komplexität der Anwendungsfälle.

Alle vier Komplexitätsmaße sind rationale Maße auf der Skala von 0,0 bis 1,0.

Komplexität der Benutzeroberflächen = Steuerungseingaben / Gesamteingaben

Die Komplexität der Benutzeroberfläche bezeichnet das Verhältnis der Anzahl der Steuerungseingaben zur Summe der Oberflächeneingaben insgesamt. Je mehr Steuerungseingaben (Radio-Buttons, Menüeinträge, Check-Boxen usw.), desto komplexer ist der Test einer Benutzeroberfläche, wohingegen einfache Textfelder, die den weiteren Ablauf nicht beeinflussen, die Komplexität nicht erhöhen.

Komplexität der Systemschnittstellen = Datentypen / Datenelemente

Mit der Komplexität der Systemschnittstellen wird das Verhältnis verschiedener Datentypen zur Anzahl der Datenelemente pro Schnittstelle berechnet. Je mehr verschiedene Datentypen vorhanden sind, desto aufwändiger wird der Test einer Systemschnittstelle.

$$\text{Komplexität der Datenbanken} = (\text{Schlüsselattribute} + \text{Indizes}) / \text{Attribute}$$

Die Komplexität der Datenbanken bedeutet das Verhältnis der Schlüssel (Primär- und Fremdschlüssel) und der Indizes zur Gesamtanzahl aller Attribute. Je mehr Attribute einer Datenbank Teil von Primär- und Fremdschlüsseln sind und je mehr Attribute indiziert sind, umso schwieriger ist die Testdatenaufbereitung.

$$\text{Komplexität der Anwendungsfälle} = \text{Bedingungen} / \text{Aktionen}$$

Unter der Komplexität der Anwendungsfälle versteht man das Verhältnis der Bedingungen zu den Aktionen. Je mehr Bedingungen einen Vorgang steuern, desto größer ist der Aufwand, den Vorgang bzw. den Anwendungsfall zu testen.

Je größer ein System ist, desto größer ist auch der Aufwand es zu testen und desto schwieriger ist es, das System zu testen. Ein System ist also testbarer je modularer es ist, je weniger Datenbanken bzw. Tabellen es hat, je weniger Schnittstellen es gibt, je schmäler diese sind und je weniger Pfade und steuernde Parameter es gibt. Die Vernetztheit eines Systems deutet auf seine Testbarkeit hin.

Die Testbarkeit kann nicht ohne weiteres als Durchschnitt von Vernetztheit, Datenbankdichte, Schnittstellendichte und Überdeckungskomplexität berechnet werden, da diese Teilmetriken in Abhängigkeit der Art des Testobjekts und der Testsituation unterschiedlich gewichtet werden müssen. Häufig sind in der Phase der Testplanung in der Praxis auch nicht genügend detaillierte Systeminformationen vorhanden, um diese Werte zu ermitteln. In diesem Fall kann der Testplaner die Komplexität mit Hilfe einer nominalen Skala schätzen:

- sehr hoch = 0,8
- hoch = 0,6
- durchschnittlich = 0,5
- niedrig = 0,4
- sehr niedrig = 0,2

$$\text{Testbarkeitsfaktor} = \text{gemessene Komplexität} / \text{mittlere Komplexität}$$

Die Systemtestkomplexität ist der arithmetische Mittelwert der einzelnen Testkomplexitäten. Der Testbarkeitsfaktor ist die gemessene oder geschätzte Systemtestkomplexität, dividiert durch die mittlere Komplexität = 0,5.

15.3 Kennzahlen für die Testeffektivität

Je größer und komplexer der Systemtest ist, desto mehr Tester werden benötigt. In einem Projekt Wiener Banken kamen 96 Tester auf 67 Entwickler, in einem Projekt der US-Regierung gab es über 160 Tester für 100 Entwickler. Man kann davon ausgehen, dass der Test einer service-orientierten Architektur doppelt so viel kostet wie die Entwicklung der einzelnen Web-Services. Vor diesem Hintergrund ist das Management daran interessiert, Zahlen über den Return On Investment (ROI) zu bekommen.

Dafür gibt es im Wesentlichen zwei Kennzahlen:

$$\text{Fehlerentdeckungsrate} = \text{gewichtete Fehler (Tester)} / \text{gewichtete Fehler (gesamt)}$$

Die Fehlerfindungsrate ist der von den eigenen Testern gemeldete Anteil aller gemeldeten Fehler. Je höher der von den Testern selber gemeldete Fehleranteil ist, umso höher ist die Fehlerentdeckungsrate. Dies ist eine klare Aussage über die Effektivität des Testens

$$\text{Vertrauensgrad} = \text{gemeldete Fehler (gewichtet)} / \text{ausgeführte Testfälle} \cdot \text{Testüberdeckung}$$

Als Vertrauensgrad wird das Verhältnis der gefundenen Fehler zur Anzahl der getesteten Testfälle multipliziert mit der Testüberdeckungsrate bezeichnet. Je weniger Fehler pro Testfall aufgedeckt werden und je höher die Testüberdeckung ist, desto größer ist der Vertrauensgrad. Wenn eine ausreichend hohe Testüberdeckung erreicht ist und nur noch wenige niedrig gewichtete Fehler bei vielen Testfällen auftauchen, ist das ein Zeichen für die Systemstabilität und das Vertrauen in den Testbetrieb steigt. Wenn aber entweder viele Fehler gefunden werden oder die Testabdeckung gering ist, steigt das Misstrauen.

15.4 Die COCOMO-II-Gleichung

COCOMO II repräsentiert die Änderungen in der modernen Softwareentwicklung, mit Anpassungen an neue Softwareentwicklungs-Prozessmodelle sowie neuen Entwicklungsmethodiken (z. B. Objektorientiertes Programmieren). Es werden verschiedene Schätzarten unterschieden, die sich auf den Entwicklungsstand des Projekts beziehen.

Der Systemtyp hat einen großen Einfluss auf den Testaufwand. Dafür werden Systeme in vier Kategorien eingeteilt:

- Standalone-Systeme, Single-User-Systeme (Gewichtung = 1),
- verteilte Multi-User-Systeme (Gewichtung = 2),
- integrierte, verteilte Systeme mit mehreren Benutzern (Gewichtung = 3),
- embedded Systeme (Gewichtung = 4).

Bei der Ermittlung des Testaufwands sollten möglichst immer verschiedene Systemtypen miteinander verglichen werden. Der Systemtyp darf nur bei der Berechnung

berücksichtigt werden, wenn das geplante System von einem anderen Typ ist als das Testen, von dem die Produktivität abgeleitet wird. Mit Hilfe dieser Parameter lässt sich der voraussichtliche Testaufwand mit folgender Gleichung schätzen:

$$\text{Testaufwand} = \text{Systemtyp} \cdot (\text{Test Points} / \text{Testproduktivität})^{\text{Skalierungskomponente} \cdot \text{Testbarkeitsfaktor}}.$$

Angenommen, es werden 1600 Test-Points anhand der Konzeptanalyse ermittelt. Im letzten ähnlichen Projekt wurden 1000 Test-Points in 150 Mitarbeitertagen getestet. Das ergibt eine Testproduktivität von $1000/150 = 6,6$ Test-Points pro Mitarbeitertag.

Die Skalierungsexponente wird für dieses Projekt wie folgt eingeschätzt:

- Testwiederholbarkeit = mittel = 1,0
- Stabilität der Testumgebung = hoch = 0,96
- Kenntnis der Anwendung = niedrig = 1,10
- Zusammengehörigkeit des Testteams = hoch = 0,96
- Testprozessreife = hoch = 0,9

Daraus folgt als arithmetischer Mittelwert 0,99.

Die Analyse der Software deutet auf eine überdurchschnittliche Komplexität dieses Systems von 0,60 hin. Durch die Komplexität des Vergleichssystems mit 0,5 ergibt dies einen Testbarkeitsfaktor von 1,2. Es handelt sich, wie im Vergleichssystem, um ein verteiltes Client-Server-System, also Typ 1. Daraus kann man den geschätzten Testaufwand ermitteln:

$$\text{Testaufwand} = 1 \cdot (1600 : 6,6)^{0,99 \cdot 1,2} = 275 \text{ Mitarbeitertage}.$$

Die geringfügig besseren Projektbedingungen, die durch den Exponent 0,99 definiert sind, werden durch die 20 % überdurchschnittliche Komplexität des Systems ausgeglichen, so dass am Ende die Testproduktivität bei 5,8 Testpunkten pro Testertag um 12 % niedriger ausfällt. Dieser Aufwand umfasst genau jene Testaktivitäten, die die alte Produktivitätsmessung umfasst. Wenn also dort der Aufwand für die Testplanung nicht erfasst wurde, wird er auch hier nicht berücksichtigt. Darum ist es unerlässlich, bei der Produktivitätsmessung konsistent zu bleiben und immer die gleichen Aufwände zu erfassen. Nur so lässt sich eine annähernd genaue Hochrechnung für künftige Projekte sichern, ansonsten vergleicht man Äpfel mit Birnen.

15.5 Berechnung der Testdauer und des Testaufwands

Im COCOMO-II-Modell ist die Gleichung für den Testaufwand:

$$\text{TDEV} = \left(C \cdot (P)^F \right) \cdot (1 - \text{SCED} \% / 100),$$

wobei

$$F = (D + 0,2 \cdot (E - B)).$$

In dieser Gleichung sind die einzelnen Parameter wie folgt definiert:

B = Untergrenze der Skalierungskomponente = 0,91

C = Multiplikator = 3,67 für Neuentwicklung

D = Zeitbasiskoeffizient = 0,28 für Neuentwicklung

E = Skalierungsexponent der Entwicklung = 0,91 : 1,23

F = Skalierungsexponent für die Projektdauer

PM = geschätzter Aufwand in Personenmonaten

SCED % = Kompression des Projekts

TDEV = Zeit für die Entwicklung

Demnach ist die gesamte Dauer des Projekts wie folgt zu berechnen:

$$F = (0,28 + 0,2) \cdot (0,96 - 0,91) = 0,29,$$

$$TDEV = 3,37 \cdot (14)^{0,29} = 7,9 \text{ Monate.}$$

Bei dieser Testdauer werden, wie im früheren Projekt, zwei Tester eingesetzt. Nun kommt aber der Schedule-Compression-Faktor hinzu, und der ist beim Testen hoch. Die Testdauer kann durch die Verteilung des Testaufwands auf mehrere Tester um mindestens 60 % verkürzt werden.

$$\text{Das bedeutet: } TDEV = 7,9 \cdot (1 - 60/100) = 3,2 \text{ Monate.}$$

In diesem Beispiel können fünf Tester eingesetzt werden. Anforderungsanalyse und Testplanung werden von zwei Testern durchgeführt. Danach kommen drei weitere Tester zur Erstellung der Testspezifikationen, Erzeugung von Testdaten, Konfiguration der Testumgebung, Durchführung der Testfälle und Auswertung und Interpretation der Ergebnisse hinzu. Das gesamte Testprojekt könnte dadurch innerhalb ca. der halben Zeit abgeschlossen werden.

In der Praxis hat sich aber der Ansatz im Nachhinein in mehreren Projekten bereits als etwas zu optimistisch herausgestellt: Eine wesentliche Voraussetzung für dieses Modell ist nämlich eine vergleichbare Erfahrung, Qualifikation und Motivation dieser Tester. Die Produktivität einzelner Tester hatte dadurch erhebliche Schwankungsbreiten und war miteinander nur schwer vergleichbar. Außerdem stellen fünf Tester durch den vermehrten Kommunikationsbedarf erheblich höhere Anforderungen an die Administration und Projektadministration. Dadurch relativieren sich die errechneten Einsparungen rasch wieder. Man kann durch die Wahl des geeigneten Schedule-Compression-Faktors ein realistischeres Szenario ansetzen.

Trotz aller theoretischen Ansätze gibt es immer noch genügend mögliche Störungen im Testprozess, sei es durch unklar definierte oder zusätzliche Anforderungen, plötzliche Fluktuation im Testteam oder technischen Restriktionen und Abhängigkeiten, die zu Beginn der Testaufwandsschätzung noch unbekannt sind. Daher sollte immer noch ein gewisser Risikoaufschlag von ca. 10 % in die Kalkulation der Testaufwände einfließen.

15.6 Planungsrisiken bei der Aufwandsermittlung

Teilweise passt auch die Planung dann nicht mehr, wenn es Verzögerungen im Entwicklungsprozess gibt: Die Tester sind in ihrer Arbeit ausgebremsst, sie warten auf die zu testende Software und die Spezifikationen sind planmäßig erstellt. Oder aber die Testwerkzeuge funktionieren nicht wie erwartet und man kommt mit der Testautomatisierung nicht weiter. Manchmal tüfelt man an Detailproblemen herum und es fehlt das Know-how an einer ganz bestimmten Stelle, so dass wertvolle Tage verloren gehen. Wenn dann aber diese Hindernisse ausgeräumt sind, startet der Testprozess verspätet und die verlorene Zeit kann nicht mehr aufgeholt werden.

Auf viele Risiken, wie z. B. die pünktliche Lieferung der zu testenden Anwendung oder die Menge der aufgetretenen Fehler, hat ein Tester keinen Einfluss. Zur Ermittlung des Testaufwands empfiehlt es sich daher, Risikoaufschläge von vornherein mit einzukalkulieren. Als Faustregel gilt: Wenn Entwicklungsprojekte mit einer Rendite von über 20 % angesetzt werden, ist die Aufwandsschätzung in der Regel unrealistisch. Ich habe schon mehrfach erfahren, dass bei der Aufwandschätzung ein viel zu niedriger Ansatz gewählt wurde oder die von den Experten ermittelte Schätzung als zu hoch angesehen wurde und für das Management nachträglich nach unten korrigiert wurde. Die Folge waren einige Monate später zahlreiche Krisenmeetings wegen Terminverzug und Overrun der Kosten. Oder man versuchte, über überteuert abgeschätzte Change Requests oder sogar gerichtliche Auseinandersetzungen am Ende zusätzliche Gelder für das Projekt herauszuholen, um die überzogenen Renditeerwartungen zumindest noch teilweise zu erfüllen.

Wenn einzelne Tester oder Entwickler für mehrere Projekte arbeiten, wird die Aufwandschätzung umso komplexer. Dann kann man im Projektverlauf, weil bei einem anderen Projekt gerade Not am Mann ist, nicht auf die Ressource zugreifen, die sowieso einen Engpass darstellt. Wenn das eigene Projekt dann noch halbwegs im terminlichen Rahmen ist, das Projekt das den benötigten Entwickler oder Tester dringend braucht aber schon gewaltig in Schieflage geraten ist und kurz vor der Penaltyzahlung steht, hat man keine Chance mehr das eigene Projekt positiv zu beeinflussen und läuft dadurch ebenfalls zwangsläufig in einen Kosten- und Terminverzug hinein. Wenn man im eigenen Projekt nicht über exklusiv zur Verfügung stehende Ressourcen bestimmen kann, birgt das immer ein gewisses Risiko, wenn irgendwo anders im Unternehmen gerade die Hütte brennt.

Aufwandsberechnungen sind zu einem gewissen Teil politisch zu sehen. Das ist auch eine der Ursachen dafür, dass viele Projekte scheitern. In Wirklichkeit sind sie in diesem Moment aber gar nicht gescheitert, man hat nur in der Anfangsphase an den Aufwand, die Kosten und die Zeit grandios unterschätzt oder unterschätzen wollen. Ein anderes Testteam hätte dann in der Regel auch nicht mehr herausholen können. Das sollte man sich immer vergegenwärtigen, wenn ein Projekt wirklich gescheitert ist, bei dem man maßgeblich mitgewirkt hat.

Die Kosten für die unterschiedlichen städtischen IT-Projekte der Stadt Hamburg laufen immer weiter aus dem Ruder. Außerdem werden die vereinbarten Fertigstellungszeiten nur selten eingehalten. Das ergab sich aus der Antwort des Senats auf eine Kleine

Anfrage des FDP-Abgeordneten Robert Bläsing und aus weiteren Drucksachen zu Softwaregroßprojekten der Stadt. Nach Auswertung des Liberalen-Politikers waren die Kosten von insgesamt 77 Projekten im April 2014 um mehr als 76 Millionen Euro gestiegen, im Spitzenfall liegt die Zunahme bei deutlich mehr als 300 %. Drei Viertel aller Vorhaben wurden oder werden außerdem später fertig als angegeben – in einem Fall liegt die Verzögerung demnach bei 15 Jahren, in sechs weiteren Fällen bei mehr als zehn Jahren.

Die größten Mehrkosten in absoluten Zahlen gab es demnach bei der neuen Software Jus-IT, mit der Jugendhilfe, Sozialhilfe und Wohngeld neu organisiert werden. Sie wurde 21,5 Millionen Euro teurer. An zweiter Stelle steht die städtische Personalsoftware ePers/KoPers, die bereits mit 17,5 Millionen Euro Mehrkosten zu Buche schlägt. Die Einführung des neuen Haushaltswesens (NHH) und die dafür nötige IT-Umstellung wurden neun Millionen Euro teurer als geplant.

Prozentual war die Kostensteigerung beim Projekt WTS Internet Infrastrukturaufbau am höchsten, das um 316 % teurer wurde. Mehr als 200-prozentige Kostensteigerungen gab es auch bei der Softwareeinführung für den elektronischen Petitionsprozess, beim Zentralen Adressmanagement und dem nationalen Waffenregister in Hamburg. Den Rekord bei den Verzögerungen hält das Projekt Archiv Datawarehouse mit 15 Jahren, gefolgt von den Projekten Hauptfürsorge OASIS (13 Jahre) und KITA 2000 (11,5 Jahre). Im Durchschnitt werden die Hamburger IT-Projekte etwa dreieinhalb Jahre später fertig als geplant und etwas mehr als 50 % teurer als veranschlagt. In einer Studie über diese Mehrkosten zeigte sich, dass die Verspätungen und Kostensteigerungen bei Projekten mit so genanntem Lastenheft geringer ausfallen als bei Vorhaben, die ohne Lastenheft umgesetzt wurden [ABEN2014].

Diese Nachricht deutet daraufhin, dass es in der Tat immer noch Projekte gibt, die ohne Lastenheft umgesetzt werden. Das jedoch sollte bei einer großen Behörde (die Stadt Hamburg hat immerhin ca. 70.000 Beschäftigte) nun wirklich nicht mehr passieren.

15.7 Risikozuschläge in Testprojekten

Für die Schätzung zusätzlicher Pufferzeiten, die in Testprojekten benötigt werden, können Risikozuschläge berechnet werden. Dafür muss jedes einzelne Risiko mit der Risikoeintrittswahrscheinlichkeit RW und der Risikoaussetzung RA multipliziert werden. Dabei bezeichnet die Risikowahrscheinlichkeit die prozentuale Wahrscheinlichkeit für den Eintritt des Risikos. Die Risikoaussetzung stellt die Zunahme der Projektdauer in Prozent bei Eintritt des Risikos dar. Die Summe der Produkte dieser beiden Faktoren für alle ermittelten Testrisiken ergibt den Gesamtrisikofaktor. Dieser wird mit 1 addiert und mit der geplanten Testdauer (in Wochen oder Monaten) multipliziert, um die potenzielle Gesamttestdauer unter der Berücksichtigung der Risiken zu ermitteln:

$$\text{Gesamttestdauer} = (1 + \text{SUMME Risiko}(RA \cdot RW)) \cdot \text{geplante Testdauer}.$$

Beispiel

Die offensichtlichen Risiken eines Testprojekts sind

- Verschiebung der Lieferungen mit einer Wahrscheinlichkeit um 40 %,
- hohe Fehlerhaftigkeit der gelieferten Software mit einer Wahrscheinlichkeit von 30 %,
- technische Probleme mit der Testumgebung mit einer Wahrscheinlichkeit von 25 %,
- Probleme mit den Testwerkzeugen mit einer Wahrscheinlichkeit um 20 %,
- Ausfall wichtiger Projektmitarbeiter (wegen Krankheit oder aufgrund Ressourcenengpass in anderen Projekten im Unternehmen) von 15 %.

Potenzielle Auswirkungen dieser Risiken:

- Die Auswirkung der Terminverschiebung auf die Testdauer könnte zwischen 2 und 12 Wochen liegen.
- Die Fehlerhaftigkeit der Software könnte 1 bis 6 Wochen Verzug bedeuten.
- Technische Probleme mit der Testumgebung könnten 1 bis 4 Wochen kosten.
- Probleme mit den Testwerkzeugen werden mit 1 bis 4 Wochen angesetzt.
- Der Ausfall von Personal kann das Projekt nicht mehr als 2 Wochen zurückwerfen, da genügend Reserven vorhanden sind und sich Tester mit relativ geringem Aufwand ersetzen lassen.

Eine Analyse dieser Risiken führt zu folgendem Risikozuschlag:

Terminverschiebung = $[\text{Max} - \text{Min Zeitverlust}] / 2 = 5 \text{ Wochen} \cdot 0,4 = 2 \text{ Wochen}$,

Fehlerhaftigkeit = $[\text{Max} - \text{Min Zeitverlust}] / 2 = 2,5 \text{ Wochen} \cdot 0,3 = 0,75 \text{ Wochen}$,

Testumgebung = $[\text{Max} - \text{Min Zeitverlust}] / 2 = 1,5 \text{ Wochen} \cdot 0,25 = 0,38 \text{ Wochen}$,

Werkzeugprobleme = $[\text{Max} - \text{Min Zeitverlust}] / 2 = 1,5 \text{ Wochen} \cdot 0,2 = 0,3 \text{ Wochen}$,

Mitarbeiterausfall = $[\text{Max} - \text{Min Zeitverlust}] / 2 = 1 \text{ Woche} \cdot 0,15 = 0,15 \text{ Wochen}$.

Der maximale Zeitverlust ist in diesem Fall die Summe der Höchstgrenzen = 28 Wochen bzw. 7 Monate. Der wahrscheinliche Zeitverlust aus der Gewichtung nach der Risikobewertung ist die Summe der einzelnen Risikofaktoren = 3,58 Wochen.

15.8 Vom Entwicklungsaufwand zum Testaufwand

Eine sehr grobe Methode besteht darin, vom Entwicklungsaufwand zwischen 20 und 40 % für den Testaufwand anzusetzen. Zur Ermittlung ist auch der Qualitätsanspruch heranzuziehen, also ob es sich um ein interessantes Gimmick oder ein sicherheitskritisches System handelt, hat direkte Auswirkung auf den Testaufwand.

Dieser Ansatz empfiehlt sich nur als Kontrollgröße. Denn wenn sich der Entwicklungschef verschätzt hat, schreibt man den Fehler bei der Ermittlung des Testaufwands fort.

Eine zu pessimistische Schätzung ist dabei besser als eine zu optimistische. Denn: Verzögert sich die Entwicklung und der Termin zur Fertigstellung der Software rückt immer näher, d. h. der Zyklus der Testdurchführung verkürzt sich, ist der Effekt der optimistischen Schätzung des Testaufwands besonders hoch. Alle Ermittlungen dieser Art sind mit hohen Ungenauigkeiten behaftet und nur geeignet näherungsweise ein Ergebnis zu erhalten.

15.9 Vom Gesamtaufwand zur Detailbetrachtung

Es wäre nun zu einfach, die Anzahl Mitarbeiter vom ersten bis zum letzten Tag durchgängig zu planen. Dafür gibt es zu viele Unwägbarkeiten während des Projektes, die am Anfang der Zeitperiode noch unbekannt sind oder nicht genau vorhergesagt werden können.

Testdurchführung und Testautomatisierung sind davon abhängig, dass der Testmanager wissen muss, was getestet werden kann und überhaupt eine Software zur Verfügung steht.

Bedeutet, wenn Mitarbeiter A, B und C zur Verfügung stehen und C nur die Testautomatisierung erledigen soll, die mit drei Mannmonaten angesetzt ist und das Projekt in Summe 9 Mannmonate dauern darf, dann kann es vorkommen, dass Mitarbeiter A und B zwar ausgelastet sind, aber mit ihren Tätigkeiten erst nach neun Monaten fertig werden damit überhaupt die Testautomatisierung starten kann. Der Testautomatisierer stand zwar zur Verfügung, hat aber die ersten Monate gar nichts zu tun gehabt und das Projekt kommt in Verzug.

Man kann also nicht den Gesamtaufwand gleichmäßig auf den gesamten Zeitraum verteilen, sondern muss die einzelnen Schritte planen und dann pro Periode die benötigten Kapazitäten ermitteln.

15.10 Berücksichtigung vorhandener Ressourcen

Bei der Testplanung muss man überlegen, wer zur Verfügung steht und welcher Tester in welchem Gebiet seine fachlichen Schwerpunkte hat. Die persönlichen und fachlichen Kompetenzen sollten möglichst optimal ausgenutzt werden.

Manche Aufgaben dauern sonst länger als geplant, wenn der einzelne Mitarbeiter sich erst einarbeiten muss oder seine Fähigkeiten nicht optimal ausgenutzt werden. Nicht jeder Mitarbeiter muss alle Aufgaben im Projekt erfüllen können.

Eine verbesserte Planung sähe dann wie in Tab. 15.1 dargestellt aus.

Bei großen Projekten kann man auch das Projekt inhaltlich aufteilen. Sinnvoll ist dabei immer, die Rollen und die zugeordneten Arbeitspakete eindeutig zu beschreiben. Am Ende ist es erforderlich, Tagespläne zu erarbeiten, wobei für A, B und C hervorgeht, welche Aufgaben pro Tag für den einzelnen Mitarbeiter anstehen.

Diese Planung ist ständig an die Gegebenheiten anzupassen und zu reviewen. Immer wenn man bemerkt, dass zu optimistisch oder zu pessimistisch geplant wurde, ist unbedingt zu empfehlen, sofort die Planung zu revidieren, um Fehler nicht in die Zukunft fortzuschreiben.

Tab. 15.1 Testplanung

	Januar	Februar	März
Testplanung	A 50 Stunden		
Testkonzept	A 50 Stunden	A 40 Stunden	
Erstellung der Testbeschreibungen	A 20 Stunden	A 60 Stunden B 130 Stunden	A 20 Stunden B 60 Stunden
Vorbereitung der Testumgebung		A 10 Stunden B 20 Stunden	B 40 Stunden C 10 Stunden
Testdurchführung		A 10 Stunden	B 50 Stunden
Testautomatisierung			A 10 Stunden B 20 Stunden C 120 Stunden
Testreporting		A 20 Stunden	A 20 Stunden B 10 Stunden

Meistens sind die Planungen am Anfang zu optimistisch, weil bestimmte Rahmenbedingungen, fehlendes Know-how, unfertige Umgebungen oder Probleme mit der Infrastruktur das Testprojekt oft wesentlich mehr ausbremsen als man sich das anfangs vorstellt.

15.11 Genügend Fehlerpuffer miteinbeziehen

Ein häufiger Fehler sind zu optimistische Planungen von Anfang an. Der Aufwand für Korrekturen und deswegen notwendige Regressionstests der Anwendung werden häufig unterschätzt. Manchmal wird auch übersehen, dass eine Änderung an einem Teil der Software durchaus in einer anderen Komponente negative Einflüsse haben kann, so dass man zum einen die Gesamtsoftware erneut testen muss, aber auch die gemäß ursprünglicher Planung gar nicht betroffenen Komponenten ändern und erneut testen muss.

Erfahrungsgemäß ist es dann, wenn man bemerkt hat, dass man sich verplant hat, besser einmal beim Management einen größeren Mehraufwand anzufordern als immer wieder per „Salamitaktik“ den Aufwand zu erhöhen.

Manchmal wurde sogar richtig geplant, aber nachträglich bei den Budgetverhandlungen der Aufwand nach unten korrigiert. Im Endeffekt lag dann der geplante Aufwand doch nahe bei der ersten Schätzung.

15.12 Erfahrungen aus früheren Projekten nutzen

Es ist wichtig, Erfahrungen früherer Projekte für die Testplanung mit einzubeziehen. Daher empfehle ich, die einzelnen Tätigkeiten auf Stundenbasis zu erfassen, auch wenn die Erfassung dann etwas länger dauert und etwas mehr administrativen Aufwand erfordert. Aber nur so erkennt man genau, wo der Aufwand genau anfällt und kann dann zumindest beim nächsten Projekt gegensteuern.

Die größten Kostenblöcke sollten dabei mit der Wichtigkeit für das Projekt verglichen werden. Es empfiehlt sich, den Aufwand pro Phase zu ermitteln, also z. B. Testaufwand 30 % der Gesamtprojekts, davon 20 % (also 6 % in Summe) für Testkonzeption, 20 % (also weitere 6 % in Summe) für Testdurchführung, und diese Werte dann mit dem tatsächlich aufgelaufenen Aufwänden zu vergleichen.

Dabei ist wiederum zu beachten, dass Ersparnisse in früheren Phasen negative Auswirkungen auf spätere Phasen haben können: Wenn für das Requirements Engineering im Projekt ein sehr geringer Anteil bereitgestellt wurde und für den Softwaretest ein zu hoher, sieht es zunächst so aus, dass das Requirements Engineering erfolgreicher gewirtschaftet hat. Es kann aber auch heißen, dass durch die Vernachlässigung bestimmter Tätigkeiten am Ende mehr Aufwand veranschlagt werden musste.

Literatur

- [ABEN2014] <http://www.abendblatt.de/hamburg/kommunales/article127889185/IT-Kosten-der-Stadt-laufen-aus-dem-Ruder.html>

Bevor man mit der Testdurchführung loslegen kann, muss man genaue Überlegungen anstellen, was alles zu den Testvoraussetzungen gehört. Welche Hardware, also welcher Prozessor, welches Betriebssystem, welche technische Ausstattung sind zu verwenden? Welche Umgebungsvariablen sind entscheidend – das Wetter, die Tageszeit. Gibt es saisonale Schwankungen?

Am besten ist es, in einem Brainstorming die Testvoraussetzungen zu sammeln und dann zu klassifizieren. Danach muss die Konfiguration des Testsystems exakt definiert werden.

Alle Testergebnisse beziehen sich später auf bestimmte Laborbedingungen und können bei anderen Umgebungsvariablen vollkommen anders sein. Es empfiehlt sich, immer wieder während der Testbeschreibung und der Testdurchführung daran zu denken, das Kap. „Testvoraussetzungen“ permanent zu prüfen, zu hinterfragen und ggf. zu überarbeiten. Manchmal wird das nämlich erst während der Testdurchführung bewusst.

Das kann zum einen die Hardware betreffen: Hardware die im Lager, auf einem Fahrzeug oder auf einer Bohrinself eingesetzt wird, ist ganz anderen Anforderungen ausgesetzt als in einem wohltemperierten Büro. Hardware die im Auto eingesetzt wird, wird daher oft in Klimakammern getestet, bei denen von $-50\text{ }^{\circ}\text{C}$ am Nordpol bis zu Dubai in der Mittagshitze die Temperaturen sehr realitätsgenau nachgebildet werden können.

Der Test einer Applikation ist immer von der entsprechenden Plattform abhängig: Für ein Smartphone oder einem mobilen Scanner muss die Navigation einer Website angepasst werden, die Ergonomie muss also bei der Systementwicklung berücksichtigt werden, und man muss die Anwendung auf unterschiedlichen Zielsystemen testen. Auch der Programmspeicher oder die Taktzahl des Prozessors sind manchmal Voraussetzungen für die Lauffähigkeit bzw. akzeptable Performance eines Programms.

Umgebungsbedingungen sind aber auch für die Software entscheidend: In einem Projekt für ein Video-Überwachungssystem lieferte der Entwickler eines Subcontractors

Komponenten für ein System zu. Beim Integrationstest kam es auf dem Zielsystem zu Fehlern, die lange nicht nachvollzogen werden konnten. Bis jemand von beiden Entwicklern bemerkte, dass der Test auf Windows 2000 durchgeführt wurde, die Entwicklung aber Windows XP verwendete.

Das zeigt, dass manchmal sehr triviale Umgebungsvariablen nicht genügend berücksichtigt werden, denn dasselbe Betriebssystem hatte man stillschweigend vorausgesetzt und auf diesen Unterschied kam man in der Fehleranalyse zunächst gar nicht. Bei sehr komplexen Testumgebungen, die teilweise aus heterogenen Systemen bestehen, muss man die Beladung der Testumgebung mit der richtigen Software und den korrekten Daten gut planen. Ansonsten bekommt man leicht Verzögerungen im Testprozess, weil man mit den Testaktivitäten gar nicht starten kann bzw. von vornherein in der Testabdeckung eingeschränkt ist.

Zu den Testvoraussetzungen zählen u. a.:

- Definition und Implementierung des Testteams,
- Einarbeitung und Schulung des Testpersonals,
- Überarbeiten und Freigeben aller nötigen Testspezifikationen, Anleitungen und Protokollvorlagen,
- Prüfung und Freigabe aller Testwerkzeuge und Testumgebungen,
- Abstimmung der Entwicklung, der Testspezifikationen und Testfälle mit der Design- und Implementierungsphase,
- Konfiguration des Testobjekts (Baseline),
- Einrichtung der Testumgebung vor dem Test,
- Reservierung kritischer Ressourcen (z. B. Messgeräte, Laborzeiten, Testeinrichtungen, Lizenzen für die Testsoftware),
- Schulungen, Einweisungen und Einarbeitungszeiten für die beteiligten Tester.

Systematische Softwaretests im Rahmen der Qualitätssicherung erfordern eine strukturierte Testdokumentation. Diese Dokumentation spezifiziert die korrekte Planung, das Testvorgehen und die Umgebungsvariablen beim Softwaretest. Standardisierte Templates helfen dem Tester bei Erstellung der Spezifikation und Beschreibung der Testprozeduren.

17.1 Struktur der Testbeschreibung

Nach Definition des Testkonzepts kann man mit der Umsetzung der Testbeschreibung mit dem ausgewählten Tool beginnen. Teilweise kann dieser Schritt parallelisiert erfolgen: Das Testkonzept muss noch nicht komplett fertig sein, wenn mit der Beschreibung der Testspezifikationen und Testprozeduren gestartet wird. Die Testbeschreibung muss jedoch zumindest im Entwurf vorhanden sein, bevor mit dem ersten Softwaretest begonnen wird. Man kann und muss die Testbeschreibung während der Testdurchführung sicherlich anpassen und an manchen Stellen verfeinern. Der grundsätzliche Aufbau einer Testspezifikation und der Rahmen muss aber stehen bevor man den ersten Test durchführt.

Dabei ist besonders auf eine geeignete Struktur der Testfälle zu achten:

- funktionale Tests,
- Performancetests,
- Stress- und Lasttests,
- Ergonomie und Usability,
- ...

Die funktionalen Tests nehmen im Allgemeinen den meisten Platz ein. Hier gilt es, eine geeignete Struktur zu finden die sich optimalerweise an der zu testenden Applikation orientiert, also z. B. an den Features, die sie beinhaltet.

Besonders wenn mehrere Autoren eine Testspezifikation erstellen, geht es darum, ein einheitliches Schema zu verwenden und ein gemeinsames Verständnis im Vorfeld zu erarbeiten. Um nicht Testfälle mehrfach zu beschreiben oder Testfälle wegzulassen, ist die Erstellung der Testspezifikation im Vorfeld und regelmäßig während der Erstellung der Testbeschreibung abzustimmen. In diesem Fall ist es besonders wichtig, dass das Tool zur Erstellung der Testspezifikationen Mehrbenutzerfähigkeit erlaubt, weil sonst ein Wildwuchs an Dokumenten (evtl. noch mit lokalen Ablagen) droht.

Ein Tool, mit dem die Testspezifikationen von mehreren Usern parallel erstellt werden können, verhilft auch zu einer enormen zeitlichen Beschleunigung der Spezifikationsphase. Es ist erstrebenswert, die Erfüllung der an die Applikation gestellten Anforderungen über die Testspezifikation bis hin zum Test verfolgen zu können.

Man kann z. B. ein Anforderungsmanagementwerkzeug und ein Testwerkzeug koppeln, so dass eine Verbindung zwischen Anforderungen auf der einen Seite und den Tests und Testergebnissen auf der anderen Seite entsteht. In größeren Unternehmen wird dieser Workflow zwischen den einzelnen Tools am sinnvollsten zentral für mehrere Entwicklungsabteilungen und Projekte definiert, so dass unterschiedliche Anwendungen und Abteilungen davon profitieren können.

Dadurch lässt sich feststellen, ob alle Requirements durch Testfälle nachgewiesen wurden, welche Anforderungen mit welchen und mit wie vielen Tests verifiziert und ob die Funktionen hinreichend validiert wurden. Dieser Nachweis wird auch als „Traceability Matrix“ bezeichnet. Mit deren Hilfe kann man bei einer nachträglichen Änderung einer Anforderung darüber hinaus erkennen, welche Testfälle angepasst werden müssen. Diese Fragen werden vom Auftraggeber, aber auch von der Qualitätssicherung oder im Rahmen von Zertifizierungen regelmäßig gestellt.

Da in vielen Fällen unterschiedliche Tools für Anforderungsmanagement und Testspezifikation eingesetzt werden, ist diese Verknüpfung nicht trivial. Hier zeigt sich wiederum, wie wichtig ein strategisches Testmanagement für den Unternehmenserfolg ist.

17.2 Testspezifikation und Testimplementierung

In der Phase der Testimplementierung wird die Durchführung der Tests vorbereitet. Anhand der entwickelten Testfälle wird die Testumgebung aufgesetzt und die benötigten Testdaten erfasst. Teilweise müssen auch Hilfsmittel (z. B. Simulatoren) konfiguriert und automatisierte Testprogramme erstellt oder vorhandene Skripte angepasst werden, sowie die benötigten Testdaten vorbereitet werden. Manchmal müssen Automatisierungen programmiert, aufgesetzt oder angepasst werden. Die Testumgebung muss vor der Inbetriebnahme sorgfältig überprüft werden.

In der Testimplementierung soll sich die Struktur der Testspezifikation wiederfinden. Parameter, die während der Testimplementierung genutzt werden, müssen bereits in der Testspezifikation beschrieben werden.

Es sollte vermieden werden, dass bei der Testimplementierung Informationen erfasst werden müssen, die bereits in der Testspezifikation stehen. Mehrfache Datenerfassung führt zwangsläufig immer zu einem gewissen Prozentsatz an Fehlern und in jedem Fall immer zu manuellem Mehraufwand. Auch hier zeigt es sich wieder, wie wichtig es ist, Daten im gesamten Prozess immer nur einmal und nur am Entstehungsort erfasst werden. Diese Tatsache wird in vielen Unternehmen noch nicht konsequent genug umgesetzt.

Wenn Änderungen nur an einer Stelle nachgezogen werden, also etwa wenn man die Testimplementierung anpasst aber die Testspezifikation nicht nachzieht, kommt es an dieser Stelle zu Inkonsistenzen. Dann mag sich zwar das Softwareprodukt richtig verhalten und das erwartete Ergebnis beim Testen erzielen, aber es passt nicht mehr zur Testspezifikation, bei einer Weiterentwicklung oder erforderlichen Regressionstests sind dadurch Fehler vorprogrammiert. Eine Synchronisierung der Werkzeuge für Testspezifikation und Testimplementierung ist entscheidend, um Inkonsistenzen und zusätzlichen Aufwand im Softwaretest zu vermeiden.

17.3 Beschreibung der Geschäftsprozesse

Bei der Testbeschreibung empfiehlt es sich, den Standard-Geschäftsprozess ohne Störungen zuerst zu beschreiben. Dann kann man in der Testbeschreibung den Prozess variieren, aber immer nur ein Detail.

Als Beispiel sei der Test eines Handscanner angeführt. Der Handscanner wird von der Post oder einem Kurierdienst verwendet und bei der Auslieferung von Frachtsendungen oder Paketen eingesetzt. Der Standardfall ist, dass das Paket geliefert wird, der Empfänger ist zu Hause, und er nimmt die Ware in Empfang. Störungen sind z. B.: die Ware ist beschädigt, es ist ein Paket zu viel, der Empfänger verweigert die Annahme der ganzen Sendung, es ist ein Paket zu wenig, aber der Empfänger nimmt die Ware trotzdem an, der Empfänger ist nicht zu Hause usw. Die einzelnen Störungen im Geschäftsprozess sind dabei zu klassifizieren und zu priorisieren. Welche Abweichung vom Standardprozess ist also höchstwahrscheinlich die häufigste, welche tritt eher selten auf?

Normalerweise sollte es zu den Geschäftsvorfällen Flussdiagramme oder XML-Daten geben. Wenn nicht, dann muss der Systemtester eine sinnvolle Struktur der Testfälle anhand der beschriebenen Prozesse definieren.

Danach kann man die Testfälle um weitere Komplexitäten ergänzen. In dem obigen Beispiel wäre das z. B., dass der Empfänger in Summe drei Lieferungen erhält, davon wird eine angenommen, bei einer weiteren ist eines der Packstücke beschädigt und wird deswegen annahmeverweigert, die dritte Lieferung ist zwar unvollständig wird aber trotzdem angenommen und der Mangel vermerkt. Anhand von „User-Stories“ lassen sich zu diesem Business Case weitere Abwandlungen finden.

17.4 Aufbau des einzelnen Testfalls

Welche Aspekte im Einzelnen gewählt werden und wie ausführlich die Testspezifikationen zu erstellen sind, ist vom Testobjekt und von den Requirements an das Endprodukt abhängig. Es empfiehlt sich der Einsatz von Templates wenn nicht ohnehin ein Testtool verwendet wird.

In Tab. 17.1 wird ein Beispiel zur Beschreibung eines Testfalls gegeben.

Der Test sollte immer so beschrieben werden, dass ihn auch jemand durchführen kann, der über keine Detailkenntnisse des zu testenden Systems verfügt. Dass ist manchmal schwierig, wenn man das zu Grunde liegende System schon jahrelang kennt.

Idealerweise sollte man jemanden von der Straße holen können, der ein grundlegendes Verständnis von Software hat, ihm die Testanweisung in die Hand drücken und er sollte dann in der Lage sein, ohne Rückfragen den Test eigenständig durchzuführen. Ganz so einfach wird sich ein Test zwar nie realisieren lassen, aber vom Trend her sollte es das Ziel sein.

Tab. 17.1 Beschreibung eines Testfalls

Testfall	Eindeutige Identifikationsnummer
Testgegenstand/ Testobjekt:	Beschreibung der zu testenden Anwendung
Testkonfiguration/ Testvoraussetzung/ Testdaten:	Welche Systemumgebung und welche Umgebungsvoraussetzungen sind Voraussetzung für den Test? Verweis auf die zum Test notwendigen Testdaten und mögliche Randbedingungen
Testbeschreibung:	Genaue Beschreibung des Testfalls und der Schritte die bei der Ausführung zu beachten sind (Testprozeduren)
Bezug:	Bezug zum Pflichtenheft (z. B. Use Case „Datei Öffnen“ S. X). Möglichst viele Punkte aus dem Pflichtenheft durch Testfälle abdecken. Außerdem soll hier der Bezug zu anderen Tests vermerkt werden, insbesondere wenn es sich um eine Wiederholung eines Testfalles handelt.
Priorität:	Priorität des Testfalls (unbedingt erforderlich oder eher nachgeordnet?)
Details:	Evtl. weitere benötigte Details zum besseren Verständnis oder Ergänzungen. Evtl. auch angedeutete Lösungsvorschläge.
Soll-Ergebnis/ Test-Orakel/erwartetes Ergebnis:	Welches Ergebnis wird erwartet bzw. soll laut Spezifikation erzielt werden?
Ist-Ergebnis/ tatsächliches Ergebnis/ Testausgang:	Was ist das tatsächliche Ergebnis?
Bestanden:	War der Test erfolgreich?
Aus welcher Phase stammt der Fehler:	Falls ein Fehler vorliegt, soll hier aufgeführt werden in welcher Phase der Fehler eingebaut wurde (Grobentwurf, Feinentwurf/Design, Implementierung, Test).
Kommentar:	Kommentare und Anmerkungen zu den Ergebnissen.
Tester:	Wer hat den Test durchgeführt?
Datum/Uhrzeit:	Wann wurde der Test durchgeführt?

Vor allem bei Mitarbeitern, die neu in einem Softwareprojekt einsteigen, kommen manchmal Fragen, anhand derer man feststellt, wie sehr einem die gesamte Anwendung schon in Fleisch und Blut übergegangen ist und wie viel Grundlagen man selber nach einiger Zeit unbewusst voraussetzt.

Daher ist es notwendig, besser den Testfall etwas ausführlicher zu beschreiben als sich darauf zu verlassen, dass der Tester es schon verstehen wird. Erfahrenen Testern rate ich manchmal, ihre eigenen Testfälle nach einigen Jahren wieder einmal selber zu lesen. Die meisten wundern sich dann, was sie damals geschrieben haben und fragen sich verwundert was sie eigentlich mit der Beschreibung gemeint hatten.

Bei der Erstellung einer Testanweisung muss man immer auf die genaue Reihenfolge der Anweisungen achten und Fehlermeldungen oder Befehle exakt zitieren. Screenshots sollten soweit es möglich und sinnvoll ist ebenfalls in der Spezifikation enthalten sein. Besonders bei Überarbeitungen einer Testspezifikation ist darauf zu achten, dass die Oberfläche des Systems genau wiedergegeben wird. Ansonsten erzeugt man unnötige Verwirrung beim Tester.

Ein guter Testfall ist dadurch gekennzeichnet, dass er mit hoher Wahrscheinlichkeit einen bisher unbekannten Fehler zu entdecken imstande ist. Ein Testfall ist dann erfolgreich, wenn er einen bisher unbekannten Fehler entdeckt. Allgemein gilt bei der Dokumentation, dass besser etwas ausführlicher dokumentiert werden sollte, um die Verständlichkeit zu fördern. Der Aufwand für eine genauere Beschreibung zahlt sich aus, wenn man dadurch Missverständnisse und Fehler im weiteren Testprozess vermeidet.

17.5 Strukturierung der Testfälle

Beim Test unterscheidet man in der Regel funktionale und nichtfunktionale Tests. Nichtfunktionale Tests betreffen z. B. die Ergonomie, die Antwortzeit, das Lastverhalten (also was passiert bei konkurrierenden Datenbankzugriffen, das Verhalten bei hoher Serverlast).

Es kann sinnvoll sein unterschiedliche Testtools zu verwenden – für die Ergonomie in einer Web-Anwendung eignen sich User Tests im Labor, für das Lastverhalten eine Simulation durch ein Skript und für sehr spezielle Testverfahren mit hoher User-Interaktion oder starkem Automatisierungsaufwand kann es die manuelle Ausführung sein. In jedem Fall sollten die einzelnen Testfälle mit den entsprechenden Anforderungen verbunden bzw. verlinkt sein.

Viele Tools geben dazu entsprechende Möglichkeiten, diese gilt es entsprechend zu nutzen. Denn irgendwann kommt die Frage „Bin ich sicher, dass ich alles was gefordert ist, auch getestet habe?“, die Frage nach der Testabdeckung. Oftmals verlangt bereits der Auftraggeber eine Traceability-Matrix, also eine Beschreibung, mit welchem Test welches Requirement vollständig nachgewiesen wurde.

Der Tester muss immer in der Lage sein, zu begründen, wie die Qualität einer einzelnen Anforderung, einer einzelnen Komponente, einer einzelnen Funktion sichergestellt wurde.

Sollten sich dabei Probleme ergeben, weil der Test sehr aufwendig wird (z. B. wegen der Testumgebung), dann muss man ggf. auf bestimmte Tests verzichten. Das ist aber dann ein Testrisiko, das im Vorfeld zu benennen ist und explizit beschrieben sein muss.

17.6 Anzahl Testfälle

Eine hohe Anzahl von Testfällen erzeugt erfahrungsgemäß immer eine gewisse Anerkennung und wirkt in der Außendarstellung imposant. Um also im Meeting zu punkten, empfehlen sich viele kleine Testfälle eher als komplexe zusammenhängende Prozeduren, die aufeinander aufbauen.

Problematisch wird es auch beim Testdurchlauf, wenn man wenige Testfälle hat, die aber voneinander abhängig sind oder Testprozeduren mit zahlreichen Schritten. Man muss nämlich dann, wenn nur einzelne Punkte daraus nicht funktionieren, den gesamten Test als „nicht bestanden“ bezeichnen. In der Konsequenz kann es dazu kommen, dass man nie fertig wird und ganze Testketten immer wieder von vorne beginnen muss. Soweit es sinnvoll möglich ist, sollten daher besser viele kürzere einzelne Testfälle als wenig lange verknüpfte Testprozeduren konzipiert werden. Dennoch kann man natürlich nicht jeden Testfall gleich schwer bewerten. Es kann sein, dass nur ein Button gedrückt werden muss, es kann aber auch sein, dass eine Abfolge unterschiedlicher Prozesse stattfinden muss.

Eine hohe Anzahl von Testfällen allein sagt noch nichts über die Produktivität oder den Aufwand aus. Manche Testfälle können in wenigen Sekunden erledigt sein, andere benötigen Stunden zur Durchführung. Es empfiehlt sich daher, die Testfälle zu bewerten und damit die Tests zu planen und zu bewerten (Tab. 17.2).

Mit der Priorität der einzelnen Testfälle wird die Testplanung gesteuert. So ist zunächst Testfall 1 und 3 in jedem Fall durchzuführen, anschließend Testfall 2 und dann Testfall 4. Wenn es nun zeitliche Engpässe gibt, kann man im o. g. Beispiel auch eher Testfall 6 anstatt Testfall 5 testen, weil er schneller zu testen ist als Testfall 6, auch wenn die Priorität von Testfall 6 eigentlich niedrig und die von Testfall 5 immerhin mittel ist. Aber bevor man auf bestimmte Tests ganz verzichtet, ist dieses Vorgehen für eine engpassorientierte Strategie durchaus legitim. Das alles muss man jeweils abwägen, wenn man die Testdurchführung plant.

Außerdem sollte noch die Risikobewertung in die Testplanung einfließen (siehe Tab. 17.3): Der Test ist vielleicht aufwendig, aber das Risiko, das man bei einem Weglassen des Testfalls eingeht, so hoch, dass es nicht akzeptabel erscheint auf die Testdurchführung zu verzichten.

17.7 Abhängigkeiten zwischen Testfällen

Besonders schwierig wird es, wenn Testfälle eine bestimmte Voraussetzung zu Testbeginn erfordern, und dann diese Basis benötigt wird, um gewisse Daten zu verändern und dann ggf. in mehreren Stufen zu vergleichen.

Tab. 17.2 Bewertung von Testfällen zur Planung der Testdurchführung

Testfall	Voraussichtliche Dauer	Priorität
1	10 Minuten	Hoch
2	5 Minuten	Mittel
3	30 Minuten	Hoch
4	20 Minuten	Niedrig
5	60 Minuten	Mittel
6	10 Minuten	Niedrig

Tab. 17.3 Bewertung von Testfällen zur Planung der Tests mit Risikobewertung

Testfall	Voraussichtliche Dauer	Priorität	Risiko
1	10 Minuten	Hoch	Hoch
2	5 Minuten	Mittel	Mittel
3	30 Minuten	Hoch	Mittel
4	20 Minuten	Niedrig	Niedrig
5	60 Minuten	Mittel	Hoch
6	10 Minuten	Niedrig	Mittel

Teilweise verändern sich während des Tests auch Testdaten. Das bedeutet, dass die Testvoraussetzungen miteinander verbunden sind, und ein Testfall auf einem anderen Testfall aufbauen muss. Solche Konstellationen machen das gesamte Testvorgehen sehr komplex und fehleranfällig.

Angenommen Testfall 1 verändert die Voraussetzungen an den Testdaten, so dass Testfall 2 nach erfolgreichem Testfall 1 getestet werden muss und Testfall 3 nach erfolgreich durchgeführtem Testfall 2.

Testfall 1 $\Rightarrow$ OK $\Rightarrow$ Testfall 2 $\Rightarrow$ OK $\Rightarrow$ Testfall 3 $\Rightarrow$ OK

Wenn Testfall 1 schon nicht funktioniert, kommt der Tester gar nicht zu Testfall 2. Testfall 3 kann man dann auch nicht durchführen, weil ein korrektes Ergebnis aus Testfall 2 dafür die Voraussetzung ist.

Wenn nun die Software geändert wird, muss der Test mit Testfall 1 erneut starten. Beim nächsten Test ist Testfall 1 OK, Testfall 2 aber nicht. Wieder muss die Software geändert werden, und der Test mit Testfall 1 erneut gestartet werden. Beim nächsten Test ist Testfall 1 OK, Testfall 2 OK und Testfall 3 scheitert.

Man hat also in diesem Fall schon wertvolle Zeit verloren, denn von Anfang kann man nicht die Testabdeckung über alle Tests gewährleisten und es gibt mehrere Entwicklungszyklen. Und dann besteht noch das Risiko, dass sich nach den Software-Änderungen, um Testfall 3 erfolgreich zu testen, die Software dahingehend verändert, dass Testfall 1 wieder nicht funktioniert. Solche Prozessketten sind nur dann sinnvoll, wenn alle Einzeltests mit definierten Schnittstellen schon durchgeführt wurden.

Im obigen Beispiel wäre es besser, die Testvoraussetzungen für alle drei Tests so zu verändern, dass bei allen Tests entsprechend manipulierte Testdaten schon bei Testbeginn vorhanden sind.

17.8 Priorisierung von Testfällen

Oft sind Testprojekte ja nicht so ideal, wie sie im Modell beschrieben sind. Entweder man hat zu spät angefangen, oder den Aufwand unterschätzt, oder die Requirements nicht richtig verstanden und nicht richtig umgesetzt, oder technische Rahmenbedingungen unterschätzt, oder es sind mehrere Probleme oder alles kommt zusammen usw. usw.

Daher ist es immer entscheidend, auf diese Rahmenbedingungen neutral hinzuweisen, aber trotzdem das Beste daraus zu machen. Auf ideale Bedingungen wird man nicht treffen, also lieber mal anfangen und Mut zur Lücke haben anstatt erst mit der Testdurchführung zu beginnen, wenn alle Voraussetzungen geklärt sind, also die Anforderungen erfüllt sind, die Testbeschreibung geschrieben und die Software einen gewissen Reifegrad hat. Immer wieder kommt es leider vor, dass der Test zu spät beginnt und nicht mehr genügend Zeit vorhanden ist, das Systemverhalten im gewünschten Umfang zu bewerten.

Wichtig ist, die notwendigen Soll-Voraussetzungen zu beschreiben und den Ist-Zustand zu beschreiben, und dann mit der Testdurchführung soweit irgend möglich zu starten, aber nicht auf die Behebung der Fehler oder die Fertigstellung gewisser Dokumente zu warten, wenn die Zeit zu sehr drängt.

Manchmal macht es Sinn, den Test schon mit gewissen bekannten Einschränkungen durchzuführen, weil man zu diesem Zeitpunkt ja bereits bemerkt, welche Fehler in jedem Fall verbessert werden müssen, auch wenn noch nicht alle Rahmenbedingungen und Voraussetzungen komplett vorhanden sind. Die Fehler müssen ohnehin irgendwann ausgemerzt werden, und je eher sie im Testprozess auffallen, desto besser. Ein möglichst frühzeitiger Erkenntnisgewinn ist erfahrungsgemäß besser als das Warten auf ideale Bedingungen.

Das alles richtig zu bewerten ist immer Ermessenssache im Einzelfall. Dazu braucht es eine gewisse Portion Erfahrung, fundierte Kenntnisse über Entwicklungs- und Testprozesse und ein fundiertes Wissen über das zu testende System.

17.9 Funktionale und nichtfunktionale Testfälle

Funktionale Testfälle sind die Testfälle, bei denen die korrekte Arbeitsweise der Features und die Richtigkeit der angewendeten Prozeduren nachgewiesen werden. Beim Funktionstest wird geprüft, ob die Antworten des Systems korrekt sind, d. h. ob die auf dem Ausgabemedium erscheinenden Daten den erwarteten Ergebnissen entsprechen. Der Tester muss für jeden Transaktionstyp stellvertretende Variationen mit entsprechenden Testdaten bereitstellen. Dabei genügt es nicht, nur die Antworten am Bildschirm zu kontrollieren, sondern es gilt auch die Ergebnisse in Datenbanken und Berichten oder Datenzustände im Hintergrund zu überprüfen.

Last- und Performancetests

Zur Überprüfung, wie sich das System im Echtbetrieb mit zahlreichen Benutzern verhält, sind Lasttests notwendig. Diese Lasttests benötigen oft spezielle Tools und eine exakte

Planung. Performancetests sollen ein korrektes Systemverhalten bei bestimmten Speicher- und CPU-Anforderungen sicherstellen. Oft wird noch so verfahren, dass zunächst die Funktionalität der Software getestet wird und erst gegen Ende des Projekts mit Belastungstests gestartet wurde. Fehler, die dann gefunden werden, sind aber oft mit einer suboptimalen Software-Architektur begründet. Wenn man diese Fehler spät im Projektverlauf erst aufdeckt, sind umfangreiche Nacharbeiten und nachträgliche Tests nötig, da man dann an die Grundfesten der Software Hand anlegen muss.

Für Belastungstests sind Skripte zu erstellen, da es entscheidend ist, innerhalb kurzer Zeit mehrere Tausend Transaktionen hintereinander und nebeneinander auszuführen, d. h. an mehreren Arbeitsplätzen werden mehrere automatisierte Programme in einer Schleife durchgeführt. Es empfiehlt sich, die Dauer dieser Belastungstests sukzessive zu steigern und dann das Antwortzeitverhalten zu messen, z. B. den Arbeitsspeicher und Threads mit dem Taskmanager zu verfolgen und aufzuzeichnen. Meist handelt es sich um hohe Speicher- oder CPU-Anforderungen. Zunächst lässt man diesen Test einige Minuten laufen, dann einige Stunden, schließlich über Nacht und über ein gesamtes Wochenende.

Stresstests sind in diesem Zusammenhang Tests, die das Verhalten eines Systems unter Ausnahmesituationen verfolgen und aufzeichnen. Dazu werden z. B. über eine Nacht per Skript hohe parallele Zugriffe (Multi-User-Tests: viele Anwender greifen auf ein System zu, simuliert oder real) erzeugt und dann bestimmte Komponenten gemessen.

Crashtests versuchen das System zum Absturz zu bringen. Sie sollen nachweisen, wie fehlertolerant das System reagiert und ob ein Systemabsturz von anderen Instanzen abgefangen werden kann.

Beim Belastungstest wird für jede Transaktion die Zeit zwischen Auslösen des Befehls und der Rückgabe gemessen. Am Ende erhält man eine mittlere Antwortzeit für alle Transaktion und eine Gesamtdauer für die Abarbeitung von n Transaktionen. Außerdem kann man dabei erkennen, ob das System einer solchen Belastung überhaupt dauerhaft Stand hält, ohne Systemabsturz oder undefinierten Zuständen. Daher müssen die Inhalte auch inhaltlich kontrolliert werden.

Sicherheitstest

Die Sicherheit einer Software beschreibt die Fähigkeit, nicht autorisierte Zugriffe, die vorsätzlich oder versehentlich auf Programme und Daten erfolgen, zu verhindern. Je nach der Aufgabenstellung, die an die Software gestellt werden, werden unterschiedliche Sicherheitsanforderungen an sie gestellt. Der Sicherheitstest prüft in diesem Zusammenhang Eignung, Korrektheit, Unumgänglichkeit und Wirksamkeit der in einem System eingesetzten Sicherheitsmaßnahmen.

Dazu werden zunächst die fachlichen und technischen Sicherheitsanforderungen an das System definiert und in einem Review überprüft. Dabei wird geprüft, ob die notwendigen Sicherheitsfunktionen und Sicherheitsmechanismen verfügbar und angemessen sind. Anschließend werden mögliche potentielle Schwachstellen des Systems identifiziert, die ermöglichen können, dass die Sicherheitsvorkehrungen umgangen oder außer Kraft gesetzt werden.

Dann wird in einem „Penetrationstest“ die Existenz der zuvor identifizierten Sicherheitsschwachstellen widerlegt, indem gezielte Angriffe auf das Anwendungssystem simuliert werden. Der Tester versucht also das System anzugreifen und dabei die Sicherheitsvorkehrungen zu umgehen. Das kann durch Manipulationen der Anwendung oder gezielte Angriffe auf den Server geschehen.

Dabei versucht man z. B.

- unerwartete Eingabekombinationen zu erzeugen,
- Reihenfolgen von Funktionen zu ändern,
- nicht erlaubte Funktionen auszuführen,
- Funktionen und Funktionsabfolgen zielgerichtet zu unterbrechen,
- an die Session-ID eines Benutzers zu gelangen, um dann mit dessen Rechten auf eine Web-Anwendung zugreifen zu können (Session-Hijacking),
- den Passwortschutz zu umgehen,
- Passwörter durch Ausprobieren aller Möglichkeiten zu ermitteln (Brute Force Angriff).

Angriffe auf den Web-Server können z. B. dadurch getestet werden, dass man mit einer hohen Anzahl von Verbindungsanfragen dem Web-Server die Ressourcen für normale Anfragen blockiert (Denial of Service-Angriff). Eine andere Möglichkeit für Penetrationstests sind SQL-Injektionen, bei dem Parameter einer Datenbankabfrage mit SQL-Steuerzeichen versehen werden, mit denen Datenbankbefehle in das System übermittelt werden können, die dann nicht zugängliche Daten auslesen oder verändern.

Usability- und Ergonomietest

Übersichtliches Screendesign, klare Menüstrukturen und eindeutige Navigationen innerhalb der Anwendung erleichtern dem Nutzer die Anwendung und verhelfen dem System zu einer höheren Akzeptanz. Vor allem für Webanwendungen ist die Usability ein entscheidendes Kriterium für den Erfolg der Website. Dieses Tests werden auch Oberflächen-tests genannt. Sie testen die Benutzerschnittstelle des Systems (z. B. Verständlichkeit, Anordnung von Informationen, Hilfefunktionen), und werden für Dialogprogramme auch GUI-Tests oder UI-Tests genannt.

In einer umfangreichen Untersuchung über das Zugriffsverhalten von Internet-Anwendungen verbrachten erfahrene Kunden durchschnittlich 25 Sekunden auf der Homepage, unerfahrene Kunden 35 Sekunden. In der Zeit müssen sie überschauen können:

- wo sie gelandet sind,
- ob sie von der Site profitieren können,
- wie sie zu dem für sie relevanten Bereich der Site gelangen.

Testpersonen verließen nutzlos erscheinende Websites in weniger als zwei Minuten. Zu einer gebrauchstauglichen Seite tragen in erster Linie bei:

- gute Navigation,
- gute Suchfunktion,
- gute Gestaltung des Contents.

Erfolgreiche Websites zeigen, dass sie sich für ihre Besucher interessieren. Unkonventionelles oder cleveres Design ist „nice to have“, verweist aber in erster Linie zurück auf den künstlerisch ambitionierten Designer.

Website-Besucher sind typische „Walk-up-and-use-Anwender“. Sie unterscheiden sich wesentlich von „Profi-Anwendern“. Anwender von Profi-Software erhalten Schulungen und arbeiten täglich damit an ihrem Arbeitsplatz. Dafür werden sie bezahlt.

Dagegen kommen Website-Besucher freiwillig; sie können es auch sein lassen oder mit einem Klick die Konkurrenz aufsuchen. Besucher einer informativen Website (Portal, Suchmaschine, E-Commerce) wollen:

- eine bestimmte Information finden,
- etwas erledigen/ein Problem lösen,
- etwas kaufen.

Website-User wollen, stärker als Profi-Anwender, ohne Hindernisse, ohne Wartezeiten und ohne Einarbeitung die Anwendung nutzen. Daher ist es empfehlenswert, sich an den anspruchsvolleren Kriterien für Webtests auch für den Test klassischer Anwendungen zu orientieren.

Durch das Internet und die wachsende Anzahl von Anwendungen auf Smartphones und Apps sind computergestützte Anwendungen stärker in die breite Masse und die Öffentlichkeit gerückt, als das bei vielen klassischen Anwendungen der Fall ist. Dieser Aspekt ist auch beim Testmanagement von Web-Anwendungen zu berücksichtigen.

Folgende Kriterien gelten dabei als allgemeingültig:

a) Werden Hindernisse vermieden?

- Die Website ist unter den jetzt gängigen Browsern voll funktionsfähig. Es wird empfohlen, 5–6 Jahre nach Einführung einer neuen Browserversion zu warten, bevor sie beim Design der Website ignoriert wird. Erst dann können Sie sie beim Design Ihrer Website ignorieren.
- Die Website benutzt korrekten Standard-HTML-Code. Die Seiten werden richtig angezeigt; es erscheinen keine Fehlermeldungen, die die Seiten unbrauchbar machen.
- Es gibt keine toten Links; alle Grafiken werden gefunden; alle Seiten sind fertig, also nicht „under construction“ oder eine „Baustelle“.
- Die Website ist ohne Plugin benutzbar.
- Die Verwendung von Frames hindert den Besucher nicht, gezielt eine Seite Ihrer Website als Bookmark aufzunehmen.
- Der Besucher braucht kein Login.

- Der Besucher muss nicht unnötige Daten von sich preisgeben.
 - Alle Überschriften und Links sind Texte oder mit Texten versehen (und nicht etwa Grafiken); so ist eine Sprachausgabe für Blinde und Sehbehinderte möglich und die Website ist auch bei ausgeschalteter Grafik navigierbar.
 - Die Website wird über Suchmaschinen gefunden; sie enthält aussagekräftige Titel und Textanfänge, außerdem Metadaten wie Autor, Titel und Beschreibung.
- b) Werden Wartezeiten vermieden?
- Ein ästhetisches Design ist „nice to have“ und trägt dazu bei, dass eine Website als angenehm und glaubwürdig empfunden wird. So lässt sich Ladezeit verringern:
- Die Dateigröße der Seiten überschreitet nicht 1 MB.
 - Verzicht auf unnötige Grafiken und Multimedia-Effekte (z. B. große Grafiken ohne Funktion, Splash-Screens und unmotivierter Flash-Anwendung)
 - Die optimale Komprimierung notwendiger Grafiken. Alle Grafiken sind kleiner als 50 K; wenn es mehr als 5 Grafiken auf einer Seite gibt, sind sie kleiner als 100 K.
 - Angabe von WIDTH und HEIGHT bei Grafiken und Tabellen
 - Verwendung mehrerer kleiner Tabellen anstelle von einer komplexen Tabelle
 - Verwendung von Style Sheets
 - Hinzufügen von Endslashes (/) bei URLs
 - Alle Seiten werden sich in weniger als 5 Sekunden geladen.
 - Falls sich die Seite langsamer lädt, gibt es eine Meldung, die den Ladestatus anzeigt (z. B. „50 % geladen“) oder ein visuelles Feedback über die Verzögerung.
 - Alle Informationen im oberen Teil des Bildschirms sind schnell sichtbar, auch wenn sich untere Teile (nach denen man evtl. scrollen muss) noch laden.
 - Barrierefreiheit der Website: Das bedeutet eine transparente Struktur, dass Inhalte mit wenigen Klicks erreichbar und der aktuelle Standort innerhalb der Website erkennbar ist. Dazu gehört eine internetgerechte Aufbereitung des Contents, eine leserfreundliche Formulierung und zielgruppengerechte Farbwahl.
- c) Versteht der User sofort worum es geht?
- Die Struktur der Website orientiert sich an den externen Zielgruppen und ihren Aktivitäten – nicht an der firmen- oder institutioneninternen Organisation und Sichtweise.
 - Der Besucher wird nicht genötigt, aus Optionen auszuwählen, die ihm nicht transparent erscheinen.
 - Alle Optionen sind sichtbar; der Besucher muss sich z. B. nicht durch Aufblättern von Menüs oder anhand obskurer Imagemaps darum bemühen, seine nächsten Handlungsmöglichkeiten zu erkennen.
 - Der Besucher muss lange Texte nicht am Bildschirm lesen, sondern kann eine Printversion ausdrucken.
 - Die Homepage macht in weniger als 20 Wörtern klar, inwiefern der Kunde von ihr profitieren kann.

Nach wie vor halten sich die größten acht Usability-Probleme hartnäckig:

1. Links, die ihre Farbe nicht ändern, wenn sie besucht wurden,
2. Vereiteln der Funktion des Zurück-Buttons,
3. Öffnung neuer Browser-Fenster,
4. Pop-up-Fenster,
5. Design-Elemente, die wie Werbung aussehen,
6. Verstöße gegen Web-weite Konventionen,
7. Textblasen und leerer Hype,
8. dichter Content und schwer überfliegbare Text.

Schnittstellentests

Schnittstellentests werden vor allem in Integrationstests durchgeführt. Sie testen die Funktionalität bei der Zusammenarbeit voneinander unabhängiger Komponenten (für jede der beteiligten Komponenten, anhand der Spezifikation, beispielsweise mit Hilfe von Mock-Objekten).

Es ist nicht immer möglich oder erwünscht, ein einzelnes Objekt vollkommen isoliert zu testen. Soll die Interaktion eines Objektes mit seiner Umgebung überprüft werden, muss vor dem eigentlichen Test die Umgebung nachgebildet werden. Das kann umständlich, zeitaufwändig, oder gar nur eingeschränkt oder überhaupt nicht möglich sein.

In diesen Fällen können Mock-Objekte helfen. Mock-Objekte implementieren die Schnittstellen, über die das zu testende Objekt auf seine Umgebung zugreift. Sie stellen sicher, dass die erwarteten Methodenaufrufe vollständig, mit den korrekten Parametern und in der erwarteten Reihenfolge durchgeführt werden. Das Mock-Objekt liefert keine Echtdaten zurück, sondern vorher zum Testfall passend festgelegte Werte. Das Mock-Objekt kann somit dazu verwendet werden, ein bestimmtes Verhalten nachzustellen.

In Integrationstests werden auch Interoperabilitätstests durchgeführt, welche die Funktionalität bei der Zusammenarbeit voneinander unabhängiger Komponenten unter Einsatz mehrerer Komponenten testen.

Für den Systemtest von Internetanwendungen gibt es folgende Testarten:

Fehlertests prüfen, ob die Verarbeitung von Fehlersituationen korrekt, d. h. wie definiert erfolgt.

Datenkonsistenztests weisen die Auswirkung der getesteten Funktion auf die Korrektheit von Datenbeständen nach (Testbezeichnungen: Datenzyklustest, Wertebereichstest, Semantiktest, CRUD-Test).

Wiederinbetriebnahmetests prüfen, ob ein System nach einem Abschalten oder Zusammenbruch (z. B. ausgelöst durch Stresstest) wieder in Betrieb genommen werden kann.

Installationstests überprüfen die Softwareinstallationsroutinen, ggfs. in verschiedenen Systemumgebungen (z. B. mit verschiedener Hardware oder unterschiedlichen Betriebssystemversionen).

Stresstests sind Tests, die das Verhalten eines Systems unter Ausnahmesituationen prüfen und analysieren.

Crashtests sind Stresstests, die versuchen, das System zum Absturz zu bringen.

Lasttests sind Tests, die das Systemverhalten unter besonders hohen Speicher-, CPU-, o. ä. Anforderungen analysieren. Besondere Arten von Lasttests können Multi-User-Tests (viele Anwender greifen auf ein System zu, simuliert oder real) und Stresstests (dabei wird das System an die Grenzen der Leistungsfähigkeit geführt) sein.

Performance Tests sind Tests, die ein korrektes Systemverhalten bei bestimmten Speicher- und CPU-Anforderungen sicherstellen sollen.

Rechnernetz-Tests weisen das Systemverhalten in Rechnernetzen nach (z. B. Verzögerungen der Datenübertragung, Verhalten bei Problemen in der Datenübertragung).

Sicherheitstests überprüfen, ob ein System potentielle Sicherheitslücken aufweist.

Softwaretests werden oft als analytische Maßnahmen, die erst nach Erstellung des Prüfgegenstandes durchgeführt werden können, definiert. Diese Testmethoden werden beim Softwaretest angewendet. Einige ausgewählte Testmethoden werden in diesem Kapitel beschrieben.

18.1 Äquivalenzklassenbildung

Angenommen, die Eingabewerte sind derart definiert, dass die Software mit ganzzahligen Eingabewerten von 0 bis 200 funktionieren soll. Dazu ist es sinnvoll die in Tab. 18.1 angegebenen Werte zu testen.

Die einzelnen Äquivalenzklassen lauten in diesem Fall:

$$\begin{aligned}x &< 0, \\x &= 0, \\0 &< x < 200, \\x &= 200, \\x &> 200.\end{aligned}$$

Die Werte -1 und $-0,5$ sind Repräsentanten der Äquivalenzklasse $x < 0$, der Wert 0 der einzige Repräsentant der Äquivalenzklasse $x = 0$. Es können also einer oder mehrere Repräsentanten pro Äquivalenzklasse zu einem Test herangezogen werden. Dazu kann man die Äquivalenzklassen weiter in Unterklassen aufteilen. Negative Werte, nicht ganzzahlige Werte und die Werte an den Grenzen sind intensiv zu testen, auch ein Wert mitten im Intervall. Manchmal ist es zusätzlich sinnvoll, mehrere Werte innerhalb des Intervalls herauszugreifen, um eine progressive Entwicklung aufzuzeigen. Auch Werte kurz vor oder kurz nach den Intervallgrenzen sind von besonderem Interesse.

Tab. 18.1 Test mit Äquivalenzklassen

Wert	Erwartetes Ergebnis
-1	Unzulässige Eingabe
-0,5	Unzulässige Eingabe
0	Zulässige Eingabe
0,5	Unzulässige Eingabe
1	Zulässige Eingabe
100	Zulässige Eingabe
199	Zulässige Eingabe
200	Zulässige Eingabe
201	Unzulässige Eingabe

Von der Äquivalenzklasse zum Testfall

In aller Regel besitzt ein Testobjekt mehr als einen Eingabeparameter. Die Äquivalenzklassenmethode liefert für jeden dieser Eingabeparameter mindestens zwei Äquivalenzklassen, einen gültigen und einen ungültigen Wert, der zum Test herangezogen werden muss.

Bei der Erstellung eines Testfalls muss jedem der Eingabeparameter ein gültiger Wert zugewiesen werden. Alle möglichen Kombinationen der gültigen Werte der einzelnen Parameter bilden jeweils einen Testfall. Repräsentanten ungültiger Äquivalenzklassen sind nur mit Repräsentanten anderer gültiger Äquivalenzklassen zu kombinieren. Damit entsteht für jede ungültige Äquivalenzklasse ein „Negativ-Testfall“. Ein „Negativ-Testfall“ bedeutet in diesem Fall, dass die Eingabe (am besten mit einer aussagekräftigen Fehlermeldung) abgelehnt wird, das Programm aber nicht zum Absturz gebracht wird. Beim Test ist zu vermeiden, Repräsentanten mehrerer ungültiger Äquivalenzklassen miteinander zu kombinieren. Ein ungültiger Wert löst eine Behandlung der Ausnahme aus, die davon unabhängig ist, welche Werte die übrigen Parameter haben. Dadurch kann es leicht zu einer Fehlermaskierung kommen, und es ist nicht klar, welcher der falschen Parameter den Fehler ausgelöst hat.

Beispiel für einen Negativ-Test

Dann und nur dann, wenn Bedingung A und Bedingung B erfüllt sind, muss Ereignis C eintreten.

Das bedeutet folgenden Positiv-Testfall:

Eingabe: A = wahr, B = wahr, erwartetes Ergebnis: C = wahr

Das bedeutet im Umkehrschluss folgende Negativ-Testfälle:

Eingabe: A = wahr, B = falsch, erwartetes Ergebnis: C = falsch

Eingabe: A = falsch, B = wahr, erwartetes Ergebnis: C = falsch

Eingabe: A = falsch, B = falsch, erwartetes Ergebnis: C = falsch

Mit der Beschränkung auf den Positiv-Testfall wäre lediglich die Bedingung „wenn Bedingung A und Bedingung B erfüllt sind, muss Ereignis C eintreten“ getestet worden.

Die Bedingung „dann und nur dann wenn Bedingung A und Bedingung B erfüllt sind, muss Ereignis C eintreten“ ist nur vollständig getestet, wenn auch die drei Negativ-Testfälle getestet sind.

Ermittlung der Testabdeckung beim Test von Äquivalenzklassen

Man kann nach der Äquivalenzklassenbildung Testendekriterien ermitteln. In der Pairwise Methode geht man von der Annahme aus, dass Fehler in den meisten Fällen bei der Kombination zweier Parameter entstehen. Dadurch können die bei der Äquivalenzklassenbildung und Grenzwertanalyse erstellten Kombinationen (Testfälle) optimiert werden. Dies geschieht, in dem diejenigen Testfälle nicht mehr betrachtet werden, welche aus Wertepaaren bestehen, die bereits durch andere Testfälle getestet werden. Abb. 18.1 zeigt die Ermittlung der Testabdeckung beim Test von Äquivalenzklassen.

In diesem Beispiel kann man also acht Testfälle auf vier Testfälle reduzieren, ohne die Testabdeckung zu verringern.

Spezielle Eingabewerte in der Anwendung

Angenommen, ein Test für unterschiedliche Ländercodes muss getestet werden und der Programmtest ist noch nicht automatisiert. Dann nimmt man einige bekannte Ländercodes, in denen das Programm ausgeliefert wird und achtet dabei noch auf eine unterschiedliche Anzahl von Stellen (z. B. „F“, „USA“). Bei Postleitzahlenbereichen empfehlen sich ausländische Postleitzahlen, in Deutschland die Eingabe von 4 und 6 Stellen, unsinnige Werte wie „00000“ oder alphanumerische Eingaben. Ist definiert, dass nur zulässige Postleitzahlen verwendet werden sollen? Wenn ja, müssen Postleitzahlen eingegeben werden, die nicht vergeben sind. Ähnlich verhält es sich mit Adressen oder Telefonnummern. Bei bestimmten Wertebereichen mit vorgegebener Länge bestehen die Tests der Äquivalenzklassen u. a. darin, die Nummer um eine Stelle zu lang, eine Stelle zu kurz, mit nicht numerischen Werten einzugeben. Beim Test von Datumsformaten sollten Werte wie „31.04.“ und der

KOMPLETTE TESTDATEN			OHNE DOPPELTE PAARE		
Parameter X	Parameter Y	Parameter Z	Parameter X	Parameter Y	Parameter Z
1	A	I	1	A	I
2	A	I	2	B	I
1	B	I	2	A	II
2	B	I	1	B	II
1	A	II			
2	A	II			
1	B	II			
2	B	II			

Abb. 18.1 Testabdeckung beim Test von Äquivalenzklassen

„29.02.“ in einem Jahr als Eingabedaten verwendet werden, das kein Schaltjahr ist oder der „01.13.“, um falsche Eingaben auszuschließen und kontrolliert abzufangen.

Es gibt verfügbare Standardroutinen für die Datumsermittlung, die Postleitzahlen- oder Bankleitzahlenprüfung, die bereits umfassend getestet sind. Die Verwendung standardisierter Komponenten minimiert auch den Testaufwand. Dadurch bleibt mehr Zeit für schwierige Kombinationen und komplexe Testfälle, was wiederum die Testabdeckung und die Qualität der Software erhöht. Der Testmanager sollte auch bei den Entwicklern und beim Systemarchitekten darauf hinwirken, sich hier an bestehende Standards zu halten. Das betrifft auch Feldlängen und Felddefinitionen.

In einem meiner Projekte ging es um Werksnummern eines Unternehmens. Das Unternehmen war dezentral organisiert und jedes Werk hatte eine Nummer – im Einkaufssystem, in der Personalverwaltung, im Lieferstellenstamm, im Fuhrpark. Bei einer Änderung einer Werksnummer mussten im Unternehmen bis zu 12 Tabellen gepflegt werden. Mal war die Werksnummer 3-stellig, mal 2-stellig, mal alphanumerisch, mal numerisch und mal binär definiert. Durch die zahlreichen Tabellen war bei einer Software-Änderung eine der 12 Tabellen übersehen worden und wurde auch beim Test nicht betrachtet. In der Folge wurden nach einer Software-Änderung Produktionsmengen nicht mehr richtig gezählt, weil die Software für den Vertrieb geändert wurde und die Werksnummern nicht mehr kompatibel waren. Eine Verzögerung des gesamten Projekts trat ein, es musste zusätzlich Software geändert werden, und durch manuelle Buchungen und Korrekturen der Produktionsmengen entstand erheblicher Zusatzaufwand für die kaufmännische Verwaltung.

Standardlösungen sind nicht für jede Spezialanwendung ohne weiteres einsetzbar. Umso wichtiger ist es, als Tester darauf hinzuweisen, dass die Systemarchitektur diesbezüglich untersucht und ggf. verbessert wird.

Erlaubnis definierter Werte

Kann man dem System in einer Konfigurationsmaske erlaubte Wertebereiche definieren? Ist die Berechtigungssteuerung sinnvoll? Wenn wegen jeder Änderung des Wertebereichs das Programm geändert werden muss, ist es nicht pflegefreundlich. Erlaubte Werte sollte normalerweise nicht jeder User definieren können. Aber es muss trotzdem relativ einfach und kurzfristig möglich sein und nicht erst über einen komplizierten Datenübertragungsprozess z. B. in der Folgenacht an den Enduser gehen, sondern auch bei dezentralen Systemen sofort verfügbar sein, denn sonst ist eine Eingabe in der Eingabemaske nicht möglich.

Bei der Testdurchführung sollten unterschiedliche Berechtigungsstufen genutzt werden. Man sollte sich also beim Test einmal als Systemadministrator einloggen und ein anderes Mal als User mit den niedrigsten Autorisierungs-Level. Man muss dafür zusätzliche Werte erfassen oder einen Wert löschen und prüfen, wie sich das System danach verhält. Wird die Eingabe dann erlaubt oder abgewiesen, und erhält der User eine aussagekräftige Fehlermeldung, wenn die Eingabe des geänderten Wertes abgewiesen wird?

Dabei sind auch Datumsbegrenzungen und Tests mit Äquivalenzklassen zu bestimmten Stichtagen zu beachten. Wenn ab 01.01.2019 ein bestimmter Eingabewert nicht mehr erlaubt ist, dann muss die Anwendung einmal mit dem Datum 31.12.2018 und einmal mit dem Datum 01.01.2019 getestet werden. In diesem Beispiel der Test normalerweise erheblich vor dem 01.01.2019 erfolgen sollte, muss man evtl. dafür das Systemdatum manipulieren.

Nutzen von Äquivalenzklassen

Äquivalenzklassen helfen, Testfälle zusammenzufassen. Wenn jedes Requirement mit einem Testfall nachzuweisen ist, ist es sinnvoll, Klassen von Anforderungen zu einem Testfall zusammenzufassen, also keine 1:n-Zuordnung, sondern eine n:n-Zuordnung vorzunehmen.

Es gibt Fälle, wo ein einzelnes Requirement mehrere Testfälle benötigt, damit seine Erfüllung vollständig nachgewiesen werden kann. Andersherum können aber auch mehrere Requirements sinnvoll zu einem Testfall zusammengefasst werden. Wenn mit der Erfüllung der Bedingungen aus einem Testfall der Klasse die Korrektheit gewährleistet ist und bestätigt werden kann, dass dieser Nachweis auf die anderen Testfälle dieser Äquivalenzklasse gültig ist, spart das ggf. erheblichen Testaufwand ein.

18.2 Unterschiedliche Testtiefen

Angenommen, es sind zwei Wochen geplant um ein komplettes Softwarepaket zu testen, der Tester bekommt aber jede Woche eine neue Version. Nehmen wir weiterhin an, Testfall 1–50 wären für die erste Woche und Testfall 51–100 für die zweite Woche vorgesehen, alle Testfälle haben in etwa die selbe Testdauer und Testfall 1–50 hätten hohe und mittlere und Testfall 51–100 eher niedrige Priorität.

Dann würde man Woche für Woche wieder mit den wichtigsten Testfällen beginnen, die Testfälle niedriger Priorität aber nie testen.

Wenn man nun bei jeder neuen Version jedes Mal mit dem Test von vorne anfängt, kann man niemals die erforderliche Testtiefe erreichen und kommt immer nur dazu, die Grundfunktionen zu testen.

Daher sollte man in diesem Fall wie folgt vorgehen:

- Für jede neue Version einen „Quick-Test“ durchführen. Wenn diese Software zum Beispiel unter bestimmten Bedingungen gar nicht erst startet, muss erst dieser Fehler mit hoher Priorität behoben werden. Dann ist es sinnvoll, gleich eine neue Version zu verwenden.
- Für die Hauptfunktionen (also Tests hoher Priorität bis niedriger Priorität) einen Test der wesentlichen Funktionen durchführen. Das bedeutet auch, dass eine gewisse Zeit benötigt wird, wenn diese Version ausgeliefert werden soll. Wenn es vertraglich zugesicherte Termine gibt, will man manchmal dem Kunden als Zwischenstand eine Software

geben, die den aktuellen Bearbeitungsstand widerspiegelt oder den Projektfortschritt dokumentiert, manchmal sind es auch vertragliche Regelungen die eine Software-Lieferung erfordern obwohl es gar nicht passt. Auch dann empfiehlt sich aber nicht im letzten Moment etwas unüberlegt hineinzuintegrieren.

- Mit einer Software-Version, die sich als halbwegs stabil erwiesen hat, werden Tests in der Tiefe durchgeführt. Das muss nicht die allerneueste Version sein, sondern eine Software, die zu einem bestimmten Zeitpunkt funktioniert hat. Wenn man weiß, dass einzelne Features noch nicht fertig sind, kann man sie bei diesem Test weglassen. Bei diesem Test geht es darum, Qualität ins Produkt zu bekommen, kreativ zu sein.

Bei jedem Reporting sollte immer definiert werden, dass nur ein Bruchteil der notwendigen Testabdeckung getestet wurde und ein „Quick-Test“, also ein eher oberflächlicher und kurzer Test der Anwendung, immer ein gewisses Risiko beinhaltet. Für eine Lieferung einer Software in mehreren Releases mag von Zeit zu Zeit ein Quick-Test ausreichend sein, der Testmanager sollte aber darauf drängen, dass er nicht das standardisierte Testvorgehen ersetzen kann.

Erfahrungsgemäß ist zu wichtigen Meilensteinen und immer am Ende eines Projekts ein Test mit einer möglichst hohen Testabdeckung durchzuführen.

Sollten bei einem Quick-Test Fehler entdeckt werden und eine Nachlieferung erforderlich werden, sollte man nach diesem Test mit verminderter Testabdeckung (z. B. um den vertraglich vereinbarten Ausliefertermin einhalten zu können) in jedem Fall nach der Auslieferung noch einen Gesamttest durchführen.

Es ist natürlich besser, wenn ein Softwarefehler vor Auslieferung der Software entdeckt wird. Aber selbst wenn er nach Auslieferung der Software noch bei einem Test im eigenen Unternehmen gefunden wird, ist das immer noch besser, als dass man vom Kunden darauf aufmerksam gemacht wird. Dann war die eigene Softwarequalitätssicherung zwar zu spät dran, aber sie wird immerhin nicht als wirkungslos empfunden. Schließlich muss man sich bei jedem Fehler, den der Kunde bemerkt, fragen lassen, warum einem der Fehler selber nicht aufgefallen ist und warum man selber nicht so getestet hat wie der Kunde. Das wirft in jedem Fall ein schlechtes Bild auf die Testabdeckung und auf das Qualitätsverständnis des eigenen Unternehmens. Leider ist aber nach Auslieferung der vereinbarten Version in der Praxis in aller Regel das Projekt beendet und dann für qualitätssichernden Maßnahmen kein Budget mehr vorgesehen bzw. verfügbar.

18.3 Überdeckungsmaße

Einfache Überdeckungsmaße

Eine Menge von Testfällen kann folgende Kriterien erfüllen:

- Anweisungsüberdeckung: Jeder Knoten im Kontrollflussgraph muss einmal durchlaufen werden (= jede Anweisung wird wenigstens einmal ausgeführt) [QMET2014].
- Zweigüberdeckung: Jede Kante im Kontrollflussgraph muss einmal durchlaufen werden; schließt Anweisungsüberdeckung mit ein.

- Pfadüberdeckung: Jeder Pfad im Kontrollflussgraphen muss einmal durchlaufen werden.
- Der Überdeckungsgrad gibt an, wie viele Anweisungen/Zweige/Pfade durchlaufen wurden [UNIS2014].

Die Abb. 18.2 (Anweisungsüberdeckung) und die Abb. 18.3 (Zweigüberdeckung) zeigen die einzelnen Zustände im Kontrollfluss auf.

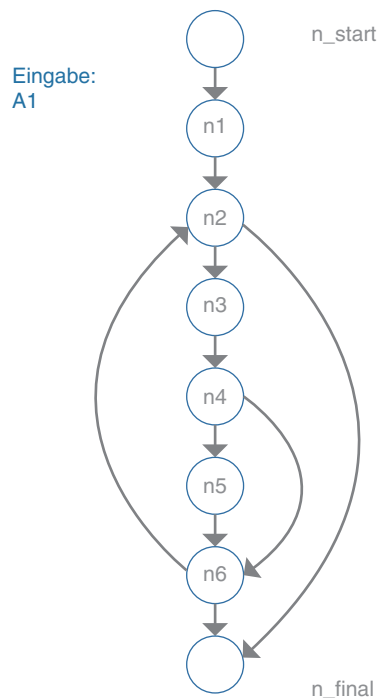
Überdeckung messen mit GCOV

GCOV (GNU COVerge tool) ist ein Werkzeug, um die Anweisungs- und Zweigüberdeckung von C/C++-Programmen zu messen.

```
Dokumentation: man gcov
$ c++ -g -fprofile-arcs -ftest-coverage
-o zaehlezhcn zaehlezhcn.C
$ ./zaehlezhcn
Bitte Zeichen eingeben: KFZ.
Anzahl Vokale: 0
Anzahl Zeichen: 3
$gcov zaehlezhcn
93.33 % of 15 source lines executed in file zaehlezhcn.C
Creating zaehlezhcn.C.gcov.25/37
```

Abb. 18.2 Anweisungsüberdeckung

Anweisungsüberdeckung (C_0)



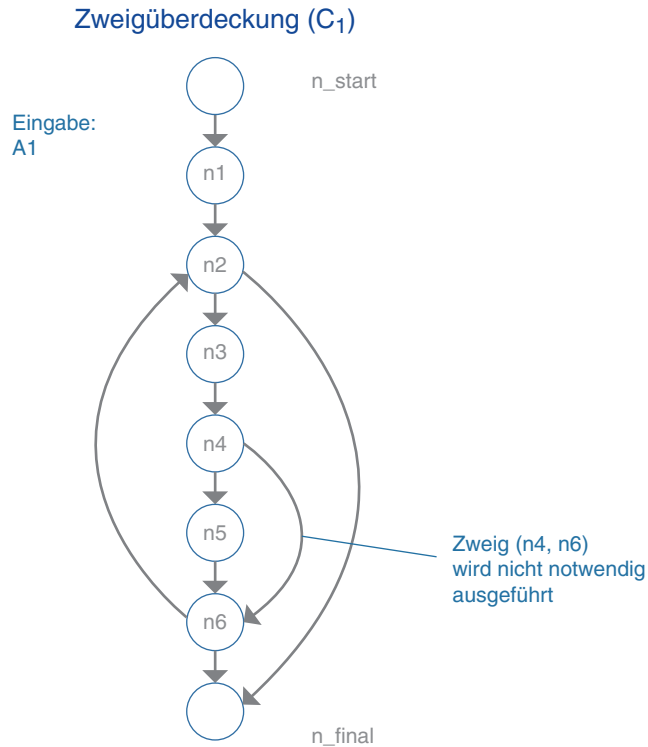


Abb. 18.3 Zweigüberdeckung

Anweisungsüberdeckung messen

```

void ZaehleZchn(int& VokalAnzahl, int& Gesamtzahl)
{
  1 char Zchn;
  1 cin >> Zchn;
  4 while (Zchn >= 'A' && Zchn <= 'Z' &&
Gesamtzahl < INT MAX)
  {
    3 Gesamtzahl = Gesamtzahl + 1;
    3 if (Zchn == 'A' || Zchn == 'E' || Zchn == 'I' || Zchn == 'O' || Zchn == 'U')
    {
      #### VokalAnzahl = VokalAnzahl + 1;
    }
    3 cin >> Zchn;
  }
}
  
```

Zweigüberdeckung messen

```
$c++ -g -fprofile-arcs -ftest-coverage
-o zaehlezhchn zaehlezhchn.C
$ ./zaehlezhchn
Bitte Zeichen eingeben: KFZ.
Anzahl Vokale: 0
Anzahl Zeichen: 3
$gcov zaehlezhchn
93.33 % of 15 source lines executed in file zaehlezhchn.C
Creating zaehlezhchn.C.gcov.
$gcov -b zaehlezhchn
93.33 % of 15 source lines executed in file zaehlezhchn.C
90.91 % of 11 branches executed in file zaehlezhchn.C
36.36 % of 11 branches taken at least once in file zaehlezhchn.C
100.00 % of 10 calls executed in file zaehlezhchn.C
Creating zaehlezhchn.C.gcov.27/37
Gesamtzahl = Gesamtzahl + 1;
if (Zchn == 'A' || Zchn == 'E' || Zchn == 'I' || Zchn == 'O' || Zchn == 'U')
branch 0 taken = 0 %
branch 1 taken = 0 %
branch 2 taken = 0 %
branch 3 taken = 0 %
branch 4 taken = 0 %
branch 5 taken = 100 %
```

Branch 0–4 sind die Einzelbedingungen; Branch 5 ist der „Else“-Fall.

Inkrementelles Messen

```
$ ./zaehlezhchn
Bitte Zeichen eingeben: KFZ.
Anzahl Vokale: 0
Anzahl Zeichen: 3
$gcov -b zaehlezhchn
93.33 % of 15 source lines executed in file zaehlezhchn.C
36.36 % of 11 branches taken at least once in file zaehlezhchn.C
Creating zaehlezhchn.C.gcov.
$ ./zaehlezhchn
Bitte Zeichen eingeben: HUGO.
Anzahl Vokale: 2
Anzahl Zeichen: 4
$gcov -b zaehlezhchn
100.00 % of 15 source lines executed in file zaehlezhchn.C
54.55 % of 11 branches taken at least once in file zaehlezhchn.C
Creating zaehlezhchn.C.gcov.29/37
```

Vergleich von Anweisungs- und Zweigüberdeckung

Tab. 18.2 zeigt Anweisungs- und Zweigüberdeckung im Vergleich [UNIS2014].

Schwächen der Anweisungsüberdeckung

Der folgende Code zeigt, dass das Kriterium der Anweisungsüberdeckung nicht ausreichend ist:

```
x = 1;
if (x >= 1) // statt y >= 1
x = x + 1;
```

Hier wird zwar jede Anweisung einmal ausgeführt; die „Zweigüberdeckung“ fordert aber auch die Suche nach einer „Alternative (was hier schwerfällt ⇒ Defekt).

Pfadüberdeckung

Die Pfadüberdeckung besitzt folgende Kriterien (siehe Abb. 18.4):

- mächtigstes kontrollstrukturorientiertes Testverfahren,
- höchste Fehleridentifizierungsquote,
- keine praktische Bedeutung, da die Durchführbarkeit aufgrund der zahlreichen Testfälle sehr eingeschränkt ist.

Wichtige Variante: strukturierter Pfadtest (jede Schleife wird wenigstens k-mal durchlaufen)

- erlaubt die gezielte Überprüfung von Schleifen,
- überprüft zusätzlich Zweigkombinationen,
- im Gegensatz zum Pfadüberdeckungstest praktikabel,
- Fehleridentifikationsquote: um 65 %.

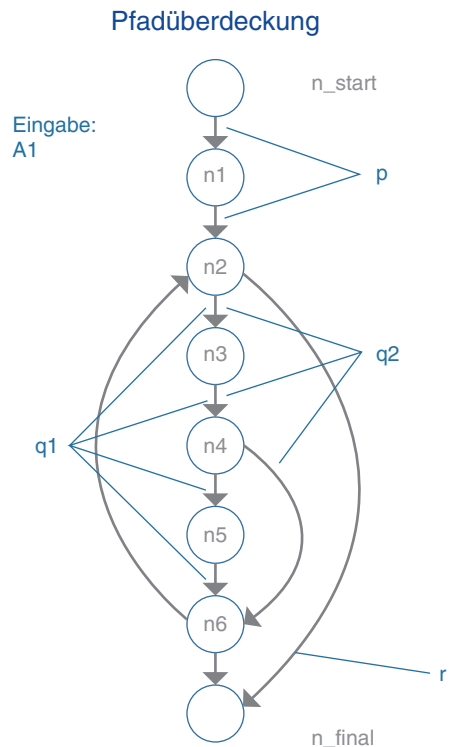
Bedingungsüberdeckung

Beispiel: Bedingungen aus „ZachleZchn“

```
while (Zchn >= 'A' && Zchn <= 'Z' && Gesamtzahl < INT MAX) (A)
if (Zchn == 'A' || Zchn == 'E' || Zchn == 'I' || Zchn == 'O' || Zchn == 'U') (B)
```

Tab. 18.2 Anweisungs- und Zweigüberdeckung

Anweisungsüberdeckung	Zweigüberdeckung
Notwendiges, aber nicht hinreichendes Testkriterium	Gilt als das minimale Testkriterium
Kann Code finden, der nicht ausführbar ist	Kann nicht ausführbare Programmzweige finden
Als eigenständiges Testverfahren nicht geeignet	Kann häufig durchlaufene Programmzweige finden (Optimierung)
Fehleridentifizierungsquote: 18 %	Fehleridentifikationsquote: 34 %

Abb. 18.4 Pfadüberdeckung

Die Struktur der Bedingungen wird vom Zweigüberdeckungstest nicht geeignet beachtet
 ⇒ Bedingungsüberdeckungstests 34/37

Die minimale „Mehrfach-Bedingungsüberdeckung“ ist der wichtigste Vertreter der Bedingungsüberdeckungstests:

- Jede atomare Bedingung muss wenigstens einmal „true“ und einmal „false“ sein.
- Die Gesamt-Bedingung muss wenigstens einmal „true“ und wenigstens einmal „false“ werden,
- schließt Zweigüberdeckung (und somit Anweisungsüberdeckung) mit ein,
- ⇒ realistischer Kompromiss 35/37.

Beispiel

Jede atomare Bedingung und jede Gesamt-Bedingung muss wenigstens einmal „true“ und wenigstens einmal „false“ werden (Tab. 18.3).

Entscheidungsstabellen verhelfen bei der Testdurchführung zu einer schnelleren und exakteren Analyse der Ergebnisse. Außerdem verhindert man, dass bestimmte Konstellationen beim Test übersehen werden. Abb. 18.5 zeigt die unterschiedlichen Überdeckungsmaße.

Tab. 18.3 Entscheidungstabelle

Testfall	1							2	3
Gesamtzahl	0	1	2	3	4	5	6	0	INT MAX
Zchn	,A‘	,E‘	,I‘	,O‘	,U‘	,B‘	,1‘	,a‘	,D‘
Zchn >= ,A‘	T	T	T	T	T	T	F	T	T
Zchn <= ,Z‘	T	T	T	T	T	T	T	F	T
Gesamtzahl < INT_MAX	T	T	T	T	T	T	T	T	F
Bedingung (A)	T	T	T	T	T	T	F	F	F
Zchn = ,A‘	T	F	F	F	F	F	–	–	–
Zchn = ,E‘	F	T	F	F	F	F	–	–	–
Zchn = ,I‘	F	F	T	F	F	F	–	–	–
Zchn = ,O‘	F	F	F	T	F	F	–	–	–
Zchn = ,U‘	F	F	F	F	T	F	–	–	–
Bedingung (B)	T	T	T	T	T	F	–	–	–

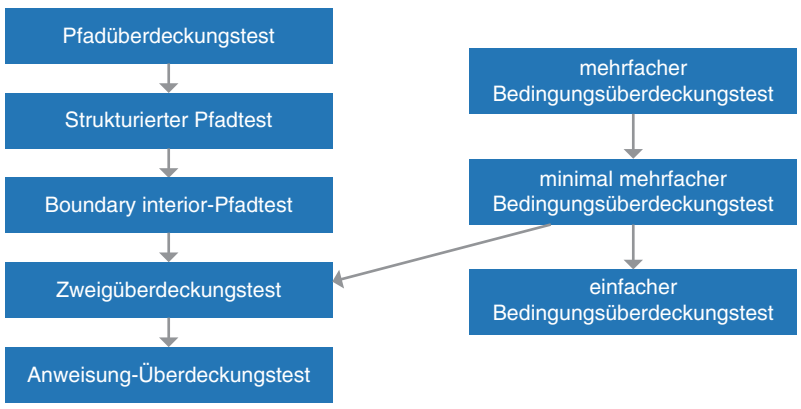


Abb. 18.5 Überdeckungsmaße im Überblick

18.4 Test Maturity Model

Das Test Maturity Model (TMMi) umfasst fünf Ebenen, von denen jede einen Reifegrad des Testprozesses beschreibt.

Auf der untersten Stufe, Level 1, sind diejenigen Testorganisationen angesiedelt, die Software nach dem Prinzip „Trial and Error“ oder „Debugging“ testen. Softwaretests werden vor allem dazu durchgeführt, um Programmier- und Funktionsfehler zu finden und zu beseitigen.

Auf der obersten Stufe, Level 5, haben Unternehmen die Prozessverbesserungen, die sich aus den Ebenen darunter ableiten, in die Testabläufe integriert. Der Test trägt dann nicht nur dazu bei, dass Fehler gefunden, sondern dass sie auch aktiv vermieden werden.

18.5 Test Process Improvement (TPI)

Die Test Process Improvement (TPI) ist eine weitere Methode, um den Reifegrad der Testprozesse zu bestimmen und konkrete Vorschläge zu deren Verbesserung zu erhalten. Das TPI-Modell unterteilt den Testprozess hierfür in 20 verschiedene Kernbereiche, sogenannte Key Areas, denen jeweils Ebenen zugeordnet sind.

Anhand der Ebenen lässt sich der Reifegrad eines Kernbereichs bestimmen – ähnlich wie bei TMMi. Jede höhere Ebene ist in zeitlicher, finanzieller oder qualitativer Hinsicht besser als die vorherige. Damit die Benutzer feststellen können, ob ein Kernbereich die Anforderungen einer Ebene erfüllt, definiert das TPI-Modell für jede Ebene eine Reihe von Kontrollpunkten. Sie dienen als Checkliste, die eine objektive und transparente Bewertung eines Kernbereichs ermöglichen.

Weiterhin berücksichtigt das Modell sinnvolle Kombinationen und Abhängigkeiten der Kernbereichsebenen: So hat es beispielsweise wenig Sinn, eine obere Ebene im Kernbereich „Reporting“ anzustreben, wenn sich das Testmanagement in der Key Area „Dokumentation von Abweichungen“ auf einer niedrigen Ebene befindet. Denn es gibt ja nichts, über das es sich zu berichten lohnt.

Literatur

- [UNIS2014] <https://www.st.cs.uni-saarland.de/edu/einst/12-testen.pdf>
[QMET2014] http://www.qmethods.de/glossar_qualitaetssicherung-test.html

Wenn die Testfälle beschrieben sind soweit es aufgrund der Spezifikationen sinnvoll möglich ist und die Software in testbarem Zustand vorhanden ist, kann mit der Durchführung der Softwaretests gestartet werden. Nun sollte man die Testanweisung exakt durchgehen und so testen, wie es in der Testanweisung beschrieben ist.

Hin und wieder kommt es vor, dass die Testspezifikation noch nicht in dem Zustand ist, wie sie eigentlich vor Testbeginn sein sollte, und die Software trotzdem bereits zur Verfügung steht. Das deutet darauf hin, dass mit den vorbereitenden Testaktivitäten zu spät begonnen wurde oder zu wenige Ressourcen dafür zur Verfügung standen. In diesem Fall sollte mit dem Test gestartet werden, um den Terminverzug nicht noch fortzuführen und nicht andere Projektbeteiligte auszubremsen. Die Entwickler benötigen schließlich ein zeitnahes Feedback über die Testergebnisse. Während der Testdurchführung sind die Testspezifikationen dann nachträglich zu erstellen.

Im Einzelfall ist zu klären, ob besser ein Teil des zur Verfügung stehenden Testpersonals die Testkonzeption erweitert und andere Tester die Testdurchführung starten oder ob alle Tester sobald das Testobjekt vorhanden ist mit der Testdurchführung beginnen und die Testspezifikation während der Testdurchführung überarbeiten. Das hängt von der bereits erreichten Qualität der Testspezifikation, den vorhandenen Testressourcen und der individuellen Projektsituation ab.

Beim ersten Testdurchlauf ist dabei ein Review der Testbeschreibung durchzuführen. Manchmal sind die Eingabemasken oder die Reihenfolge der Feldeingabe bei Erstellung der Spezifikation noch nicht endgültig geklärt, und man sieht das tatsächliche Produkt und den echten Ablauf erst bei der Abarbeitung der Testprozeduren. In jedem Fall muss auch die Testfallbeschreibung geändert werden, wenn der Test anders als ursprünglich beschrieben durchgeführt wird, so dass die Ergebnisse sich immer auf einen konkreten, reproduzierbaren Testfall beziehen, der sich in einer neuen Version erneut so testen lässt.

Dabei ist immer darauf zu achten, dass diese Änderungen nachvollziehbar bleiben. Wenn die Version 1 mit einer anderen Testbeschreibung getestet wurde als die Version 2, vergleicht man Äpfel mit Birnen. Dabei sollte man auch auf den prognostizierten Aufwand

pro Testfall achten: In welchen Fällen wurde die Testdauer verfehlt und warum? Was wurde bei der Erstellung der Testfälle nicht hinreichend berücksichtigt?

Nach dem ersten kompletten Testdurchlauf sollten die geplanten Testdurchführungszeiten in der Testplanung revidiert werden und Anpassungen für die weiteren Versionen so früh wie möglich vorgenommen werden. Bei der Schätzung der Testdauer muss auch die Vorbereitung des Testsystems berücksichtigt werden, also die Installation der Software und die Bereitstellung der Testumgebung. Dieser Aufwand wird oft vernachlässigt.

19.1 Änderungen der Testfallbeschreibung während der Testdurchführung

Angenommen, es wurde ein Testfall formuliert, und mit Version 1 und Version 2 der Software hat man ihn so testen können, wie er ursprünglich beschrieben war und beim Test ein entsprechendes Ergebnis bekommen. Ab Version 3 gibt es eine Änderung der Eingabemaske oder ein zusätzliches Feld, das zu berücksichtigen ist, so dass der Testfall so wie in Version 1 und Version 2 nicht mehr getestet werden kann.

In diesem Fall ist eine nachvollziehbare Versionsverwaltung der Testbeschreibung von Bedeutung: Testergebnisse der Versionen 1 und 2 beziehen sich auf die alte Testfallbeschreibung, das Testergebnis der Version 3 und evtl. weiterer Versionen auf die neue Testfallbeschreibung.

Auf jeden Fall reicht es nicht aus, einfach die Testfallbeschreibung zu ändern und dann die neue Version mit der geänderten Testfallbeschreibung zu testen. Denn dann sind die Testfallbeschreibungen für Version 1 und 2 überschrieben. Wenn man aus irgendwelchen Gründen die Tests zu Version 1 und 2 einmal wieder reproduzieren muss, ist das dann nicht mehr möglich, und die alten Testergebnisse sind verfälscht. Die Testfallbeschreibung wäre in diesem Fall nicht mehr revisionssicher.

Aus Vereinfachungsgründen macht man es in vielen Projekten dennoch häufig so, dass man pragmatisch den Testfall ändert und er eben dann nicht mehr zu den alten Testergebnissen passt, weil die alten Versionen nur Zwischenversionen während des entwicklungsbegleitenden Softwaretests waren, die man nie wieder reproduzieren will/muss.

Dann muss dem Projekt aber trotzdem bewusst sein, dass dieses Vorgehen mit einer umfassenden Qualitätssicherung nicht konform ist. Das ist nämlich ein Aspekt, auf den bei Zertifizierungen Wert gelegt wird.

19.2 Strukturiertes Testen und exploratives Testen

Neben dem Testen der beschriebenen Testfälle, dem „strukturierten Testen“ wie es bisher beschrieben wurde, muss auch exploratives Testen durchgeführt werden. Das ist ein Test, in dem ohne vorheriges Konzept, durch konkrete Anwendungsfälle und freies Ausprobieren, die Testabdeckung erhöht wird und dadurch Fehler gefunden werden können.

Beim explorativen Testen braucht der Tester keine Kenntnisse über die Software und deren Entstehung. Tests werden in Echtzeit geplant, entworfen und durchgeführt.

Wenn man z. B. ein Navigationssystem testet, treten manche Fehler erst auf, wenn man wirklich einmal eine Reihe von Situationen abfährt: Busspuren zwischen der Abbiegespur, Situationen mit zahlreichen Straßen am Flughafen oder am Rastplatz, komplexe Kreuzungen die in unterschiedlichen Richtungen zu durchqueren sind, Straßen, die morgens in die eine und nachmittags in die andere Richtung als Einbahnstraße definiert sind, parallele Straßenverläufe ... das praktische Leben hält so manche Überraschung bereit, die einem im Test im Büro gar nicht aufgefallen wäre.

Exploratives Testen empfiehlt sich bei manchen Fragestellungen, vor allem bei Web-Anwendungen, auch der Einsatz von Mitgliedern der Crowd, oder eine temporäre Ausdehnung der Tester über die Kerntruppe hinaus. Anwender ohne Betriebsblindheit und ohne Hintergrundwissen gehen an das Testobjekt weniger voreingenommen heran als Testexperten, die sich schon monate- oder jahrelang täglich intensiv mit der Applikation beschäftigen. Bei Testern aus der Crowd bemerkt man oft ganz anderer Blickwinkel und stellt Erfahrungswerte fest, die einem bestehenden Testteam das sich jahrelang mit einer bestimmten Anwendung beschäftigt, gar nicht mehr auffällt weil bestimmte Gegebenheiten als selbstverständlich vorausgesetzt und dadurch gar nicht mehr hinterfragt werden.

Exploratives Testen und strukturiertes Testen müssen sich aber sinnvoll ergänzen, denn beide Verfahren haben ihre Stärken und Schwächen. Für den gesamten Test einer produktiven Anwendung einfach nur Leute von der Straße zu holen oder den Softwaretest komplett in die Crowd zu verlagern (die moderne Variante dieses Vorgehens), ist gefährlich.

Cem Karner definierte exploratives Testen als eine Form des Softwaretestens, welche die persönliche Freiheit und Verantwortung des einzelnen Testers hervorhebt, damit die Qualität seiner Arbeit optimiert werden kann, indem Testdesign, Testdurchführung, Testinterpretation und testbezogenes Lernen als sich gegenseitig unterstützende Arbeiten behandelt werden, und gleichzeitig während der Projektlaufzeit fortgeführt werden. Die Betonung liegt darauf, dass der einzelne Tester zwar die persönliche Freiheit bei seiner Arbeitsplanung genießt, aber diese Freiheit auch mit der höheren Verantwortung für seine Arbeit einhergeht. Verantwortung bedeutet in diesem Zusammenhang, dass der Tester auch dafür Sorge trägt, dass seine Arbeitsergebnisse nachgestellt werden können, indem beispielsweise Notizen zu den durchgeführten Testaktivitäten vorliegen [ITAG2014].

Beim explorativen Testen sind also die einzelnen Arbeitsschritte des Testens per Definition miteinander verbunden. James Bach definiert exploratives Testen als „simultanes Lernen, Testdesign und Testdurchführung“. Das bedeutet, dass exploratives Testen die drei Einzelteile der traditionellen Testverfahren nicht vollständig voneinander trennt. Allerdings gibt es im explorativen Testen wesentlich mehr Einzelteile als in einem strukturierten Testansatz. Die Struktur im explorativen Testen besteht darin, die Applikation unter verschiedenen Aspekten zu betrachten. Michael Kelly hat hierfür die FCC CUTS VIDS Touring Heuristik eingeführt. Diese Buchstaben stehen für Features, Complexity, Claims,

Configuration, User, Testability, Scenario, Variability, Interoperability, Data und Structure – als einzelne Perspektiven, mit denen sich der Tester durch die Applikation bewegt. Sie geben der Tätigkeit dadurch eine Struktur.

Eine weitere Struktur für exploratives Testen bietet die „HICCUPS Eselsbrücke“ von James Bach und Michael Bolton. HICCUPS steht für History, Image, Comparable Products, User Expectations, Product, Purpose and Standards & Statuses. Dahinter verbergen sich verschiedene Orakel, die dabei helfen, Inkonsistenzen in einem Produkt festzustellen. Wenn sich beispielsweise von Microsoft 2003 zu Microsoft 2007 die Benutzeroberfläche komplett ändert, dann ist sie inkonsistent zur gewohnten Arbeitsweise der Benutzer, die History in obiger Liste.

Dass die Änderung in Microsoft Office aber mit voller Absicht vorgenommen wurde, verdeutlicht einen weiteren Aspekt von Orakeln: Sie sind Heuristiken und können auch mal daneben liegen. Das bedeutet, dass der Tester mit Hilfe dieser Heuristiken vor allem selber nachdenken sollte.

19.3 Intuitive Testfallermittlung

Um die Erfahrungswerte der Tester zu dokumentieren und auch explorative Tests wiederholbar zu machen, müssen auch für explorative Tests Testfälle aufgeschrieben werden. Die Ermittlung der Testfälle ist zwar intuitiv, aber nicht die Testdurchführung. Es ist wichtig, dass auch andere von den Ideen und der Erfahrung profitieren, mit denen man intuitive Testfälle erst definieren kann. Gerade wenn neue Konstellationen und bisher nicht gegebene Umgebungsvariablen verändert wurden, ist eine exakte Beschreibung und transparente Darstellung für die Reproduktion der Fehler immens wichtig.

Die Testfälle werden ermittelt durch

- Intuition: An welchen Stellen der Software werden Fehler erwartet?
- Erfahrungen: An welchen Stellen sind in früheren Versionen Fehler aufgetreten? Welche Konstellationen führten in der Praxis schon früher zu Problemen?
- Kenntnisse: Wo bestehen bekannte Schwachstellen bei der Entwicklung (Engpässe, Unerfahrenheit, Überforderung) oder bei der Hardware?

Beim „Error Guessing“ handelt es sich um einen fehlerbasierten Ansatz. Testfälle zur Aufdeckung potenzieller Fehler werden beschrieben, gewichtet und mit Risiken bewertet, um dann iterativ nach dieser Fehlerliste getestet zu werden.

Ein besonderes Augenmerk auf Probleme, die schon in früheren Versionen fehlerhaft waren, richtet den Test besser an den Anforderungen der Praxis aus. Andererseits kann es aber auch sein, dass die Probleme jetzt nicht mehr bestehen, weil das Systemdesign und die Anwendung an diesem neuralgischen Punkt erheblich verbessert wurden, dafür andere Komponenten der Software mehr Probleme als früher bereiten. Auch hier gilt es, einzelne Ansätze und Ideen sinnvoll miteinander zu verbinden.

19.4 Durchführung explorativer Tests

Vor allem dann, wenn die Anforderungen die zur Erstellung der Testfälle von mäßiger Qualität sind, kann exploratives Testen helfen dennoch eine passable Testabdeckung zu gewährleisten. Auch bei starkem Termindruck oder bei einmaliger Testdurchführung kann exploratives Testen helfen Zeit zu sparen. Man benötigt dazu aber erfahrene Tester mit intuitiven und flexiblen Ansätzen.

Beim explorativen Test werden die Eigenschaften der einzelnen Funktionen der Software erforscht. Mit der Testdurchführung wird eine unbekannte Anwendung zur Ausführung gebracht und das unbekannte Verhalten des Testobjekts geklärt. Auffälligkeiten, Fehler und Abweichungen dienen zur Erstellung weiterer Testfälle. Dadurch wird deutlich, welche Qualitätsprobleme sich bei der zu testenden Applikation ergeben können.

Beim explorativen Testen werden bestimmte Programmelemente betrachtet und weiter zergliedert. In diesem Zusammenhang wird auch der Begriff „test charter“ für kleinere Programmeinheiten verwendet.

Der Test eines „test charters“ sollte nicht mehr als ein bis zwei Stunden in Anspruch nehmen. Beim Test dieser Einheiten sind folgende Punkte zu überprüfen:

- Warum wird getestet (mit welchem Ziel)?
- Was soll getestet werden?
- Wie soll getestet werden (mit welcher Methode)?
- Welche Probleme sollen nachgewiesen werden?

Folgende Merkmale zeichnen exploratives Testen aus:

- Die Ergebnisse eines Testfalls beeinflussen die Erstellung und Ausführung weiterer Testfälle.
- Die Testdurchführung erzeugt ein Modell des zu testenden Programms, das die Funktionalität und das tatsächliche bzw. erwartete Verhalten des Programms beschreibt.
- Das Modell dient als Testobjekt mit dem Ziel, weitere Aspekte und Verhalten des Testobjekts zu bestimmen, die bisher nicht oder in anderer Form im Modell der zu testenden Anwendung vorhanden sind.

Literatur

[ITAG2014] Exploratives Testen mit Struktur – ein Widerspruch? Markus Gärtner in it-agile.de

Der Testbericht (Testreport) fasst die Ergebnisse der Testdurchführung zusammen. Er zeigt den Handlungsbedarf auf, der zur Nachbesserung erforderlich ist und gibt Aufschluss über die durchgeführten Testmaßnahmen und die Qualität der Software.

20.1 Kriterien für Testberichte

Folgende Kriterien muss ein Testbericht in jedem Fall erfüllen:

- eine Beschreibung der Tests (analog zur exemplarischen Spezifikation),
- eine Bewertung der Testgüte als Abdeckung:
 - abgedeckte Funktionalität im Pflichtenheft (normal: 100 %),
 - abgedeckte Methoden des Programms (Ziel: 100 %),
 - wenn technisch möglich (z. B. mit GCOV): Zweig- und Anweisungsüberdeckung (Ziel: weniger als 100 %),
- eine Beschreibung der Testergebnisse (d. h. Wie viele Tests waren erfolgreich? Wie viele sind fehlgeschlagen?) [UNIS2014].

Ein klares Reporting lässt Probleme frühzeitig erkennen und Gegenmaßnahmen gezielt einleiten. Daher ist es wichtig, die Botschaft klar, strukturiert und inhaltlich vollständig zu adressieren.

Im Testkonzept muss beschrieben werden, was der Testbericht alles enthalten soll, welche Metriken zur Beurteilung des Testergebnisses verwendet werden und warum, welche Aussagen getätigt werden, was der Auftraggeber des Tests für ein Ergebnis erwarten kann. Der Tester soll eine Beurteilung der Software-Qualität abgeben. Im Testkonzept muss spezifiziert werden, wie das mit einem Testbericht in der gewählten Form sichergestellt werden kann.

Ist der Entscheidungsträger mit den Informationen, die hier gegeben werden, wirklich in der Lage das Risiko zu bewerten? Die Konsequenzen für das Endprodukt müssen aus dem Testreport plastisch und deutlich hervorgehen, auch für jemanden, der nicht bis zum tiefsten Detail in der Materie steckt.

Man muss also am Anfang des Projekts abstecken, welche Art von Testreport erwartet wird, wer den Testbericht erhält, welchen Detaillierungsgrad er enthalten soll und welche Aussagen darin unbedingt enthalten sein müssen. Wenn man in einem Project Review Meeting sitzt, den Bericht präsentiert und bestimmte Fragen nicht beantworten kann, sieht das so aus, als ob man das eigene Projekt nicht im Griff hätte.

Ein umfassendes Reporting, anschließend vom Testkoordinator verdichtet, zeigt die Wichtigkeit und die Auslastung der Testabteilung. Die Statusreports sollten mindestens wöchentlich erstellt werden. Ein zusammenfassender Statusbericht vom Testmanager an den Projektleiter kann in der heißen Phase eines Projekts auch täglich erforderlich sein. Dabei ist es aber wichtig, das Testreporting nicht zu stark zu formalisieren, sondern man sollte „auf Knopfdruck“ die wesentlichen Informationen zusammenstellen können. Erfahrungsgemäß liegt in der automatisierten Aufbereitung von Testergebnissen in vielen Organisationen nach wie vor ein großes Optimierungspotenzial.

Am Ende des Reports sollte in einer kurzen Zusammenfassung der Teststatus interpretiert werden. Die „Projekttampel“ (grün/gelb/rot) mag zwar manche Komplexität zu stark reduzieren, dennoch können auch einfache Nachrichten sehr wirkungsvoll sein.

Bei dem Report bestimmter Probleme in Richtung Management sollte man sich auf die fünf wichtigsten Kernpunkte beschränken. Die restlichen Fehler sollten also nicht verschwiegen werden, aber Punkte mit akutem Handlungsbedarf müssen priorisiert werden und jedem Beteiligten transparent sein.

Dabei sollte auf eine Skala in der Bewertung geachtet werden: Wenn es fünf Baustellen gibt und die Probleme überall „superdringend“ und „extrem wichtig“ sind, wird am Ende gar nichts mehr getan.

20.2 Beschreibung der Rahmenbedingungen

Im gesamten Testprozess, von der Definition der Anforderungen bis zum Testbericht, ist es ganz entscheidend, das Testobjekt auch einzugrenzen. Man sollte immer genau beschreiben, was alles getestet wurde, welche Nebenbedingungen vorhanden waren, mit welchen Testdaten geprüft wurde und welche Konfiguration vorhanden war. Dazu gehört auch die Angabe auf welcher Hardware, in welcher Umgebung, welchem Betriebssystem, welchem Prozessor getestet wurde ... was auch immer für diesen Test wichtig ist.

Bei jeder wissenschaftlichen Arbeit geht es darum, den Untersuchungsgegenstand von Anfang an exakt einzugrenzen. Bei einem Vortrag sagt der Redner zu Beginn über welches Thema er referieren möchte, um sich nicht ins Uferlose zu verlieren. Das ist auch für den Softwaretest ganz entscheidend: Man testet ein bestimmtes Produkt in einer bestimmten

Umgebung auf definierte Eigenschaften mit bestimmten Kriterien. Das muss beim Reporting auch immer deutlich gemacht werden.

Gerade wenn der Test auf verschiedene Abteilungen, Standorte oder Funktionsgruppen verteilt ist (funktionaler Test, bestimmte Softwareteile, Integration, Lasttest ...) ist eine genaue Abgrenzung und eine Definition der Grenzen und Schnittstellen entscheidend für den Testerfolg.

Man kann sich darauf einigen nicht jede vorhandene Konfiguration zu testen, wenn man das Risiko für vertretbar hält, aber es muss dann aus dem Testreport einwandfrei hervorgehen, dass man sich für eine bestimmte Konfiguration und eine bestimmte Umgebung entschieden hat.

Wenn danach Fehler auftreten, weil bestimmte Konstellationen nicht getestet wurden, weiß man immerhin woran es liegt und dass man die Testabdeckung besser von Anfang an erhöht hätte. Aber das muss von Anfang an klar sein – denn sonst wird der Tester für das Desaster verantwortlich gemacht.

20.3 Bericht der Testergebnisse

Der Tester sollte persönliche oder emotionale Äußerungen wie „jetzt funktioniert das schon wieder nicht“ oder Sätze wie „katastrophal, vergiss es, so wird das nie fertig“, vermeiden. Zudem sollte der Tester sollte nach außen hin keine destruktive Einstellung zeigen wie „das System bring ich zum Absturz“, auch wenn es ja das Ziel von Stresstests oder Crashtests ist das System herauszufordern und das System wirklich an seine Grenzen bringen. In früheren Zeiten wurde der Überbringer schlechter Nachricht geköpft. Diese Grundhaltung findet sich abgeschwächt immer noch, weil man negative Nachrichten nun mal nicht so gerne hört wie Erfolgsmeldungen.

Ein Tester ist in erster Linie Wissenschaftler, und ein Wissenschaftler analysiert, bewertet, misst und gibt danach ein Statement über die Qualität ab. Es ist Sache der Projektleitung zu entscheiden, ob die Software eingesetzt wird oder nicht. Der Tester kann warnen, er kann Risiken aufzeigen, auch Entscheidungen beeinflussen und eine Richtung weisen, aber er hat in der Rolle des Softwaretesters oder Testmanagers nicht die Aufgabe zu entscheiden, ob die Software in diesem Zustand tatsächlich ausgeliefert wird oder nicht. Der Tester sollte sich als Analyst und Wissenschaftler mit emotionalen Äußerungen oder Schuldzuweisungen zurückhalten, denn das ist der Sache nicht dienlich.

Manchmal bestehen vertragliche Regelungen, nach denen bestimmte Features zu definierten Zeitpunkten fertig sein müssen damit die nächste Zahlung fließt. Manchmal muss man auch einfach (politisch) zeigen, dass das Projekt weitergeht und dass sich irgendwas tut. Von solchen Abhängigkeiten kann der Tester teilweise gar nichts wissen. Der Tester muss aber auch gar nicht all diese Hintergründe kennen – es geht darum das Softwareprodukt in einem bestimmten Zustand auf Herz und Nieren zu prüfen, die Qualität der Software durch diese Testaktivitäten zu stabilisieren und zu sichern. Aber: Der Tester entscheidet

nicht über die Auslieferung des Produkts. In manchen Projekten wird ein Meeting durchgeführt als „Freigabe zur Lieferung“, wobei der Tester seinen Testbericht präsentiert und die Ergebnisse aufzeigt. Dabei soll der Tester seine Einschätzung aus seiner Sicht abgeben und die Risiken benennen und quantifizieren. Aber er wird nicht allein darüber entscheiden, ob die Software ausgeliefert wird oder nicht.

Entscheidend ist immer, dass der Tester klar kommuniziert:

- Was wurde getestet und was wurde nicht getestet?
- Wie schätzt der Tester das Risiko bezüglich der Testabdeckung ein?
- Wie viele Testfälle wurden durchgeführt, wie viele waren davon funktionierten wie erwartet und wie viele nicht?
- Wie schwerwiegend sind die gefundenen Fehler?
- Gab es Fehler, die gar nicht gefunden werden konnten, weil die Basis fehlte, weil also bestimmte Testfälle nicht funktionierten, die Voraussetzung für weitere Tests waren?
- Wie ist die eigene Einschätzung der Qualität?
- Wie wird das Risiko für eine Auslieferung bewertet?

Je nach Managementebene muss man dabei die einzelnen Informationen unterschiedlich verdichten. Die Dokumentation mit Charts und Graphiken soll man dabei nicht unterschätzen. Bekannte Unternehmensberatungen im Topmanagement-Sektor, wie z. B. McKinsey, benötigen nach eigenen Angaben ungefähr die Hälfte ihrer Projektzeit, um Daten zu analysieren, zu interpretieren, daraus Schlüsse zu ziehen und Strategien aufzuzeigen. Die zweite Hälfte der Projektzeit brauchen sie dafür, ihre Analysen und die daraus erkannten Botschaften treffend zu formulieren, die Ergebnisse aufzubereiten und bildhaft zu präsentieren. Softwaretester stecken meist zu tief in technischen Details und vernachlässigen diesen kommunikativen Teil des Reportings.

Man kann das mit einem Hausbau vergleichen: Der Bauherr erscheint während das Haus gerade gemauert wird auf der Baustelle. Das Haus ist noch lange nicht fertig, aber der Auftraggeber will trotzdem sehen, dass der Bau begonnen hat und im Zeitplan ist. Einige Zeit später kommt er wieder vorbei und bemerkt, dass der Elektriker oder der Gipser beschäftigt ist. Der Bauherr sieht, dass es nach wie vor mehrere Monate dauert bis das Haus bezugsfertig ist, hat aber trotzdem ein gutes Gefühl, weil das Haus schon anders aussieht als beim letzten Mal und es auf der Baustelle weitergeht. So ist es mit der Softwareentwicklung auch – der Kunde will wissen, dass es Fortschritte gibt – auch wenn ihm bewusst ist, dass die Entwicklung noch Monate dauern wird.

Der Vergleich mit dem Hausbau und der Softwareentwicklung hinkt etwas, denn manchmal wird bei der Softwareentwicklung der Keller wieder eingerissen, wenn man schon im zweiten Obergeschoss angekommen ist. Das macht aber manchmal auch Sinn, wenn ein Objekt das entscheidend für die grundlegende System- und Software-Architektur ist eben erst später fertig wird (zum Beispiel wegen Abhängigkeiten verschiedener Projekte oder Plattformen). Solche Maßnahmen kosten aber auch Zeit und werden in vielen Fällen unterschätzt.

Selbst wenn der Test zeigt, dass das System in dieser Form nicht eingesetzt werden kann, war der Test an sich dennoch erfolgreich. Auch die Verifizierung, dass ein gewisser Weg nicht gangbar ist oder zu viele Risiken oder Kosten beinhaltet, ist eine Aussage. Technischer Fortschritt besteht auch dort, wo man bemerkt, dass man einen anderen Weg einschlagen muss, und das mit einem definierten Testverfahren nachweisen kann. Edison brauchte zur Erfindung der Glühbirne auch 900 Fehlversuche.

Bei der Entwicklung von neuen Medikamenten dauert es oft Jahre bis zur Freigabe, und viele klinische Tests sind durchzuführen. Dabei kann auch das Resultat stehen, dass das Medikament nicht weiterentwickelt werden kann, dass der Einsatz in der Heilkunde nicht möglich ist. Das ist zunächst frustrierend für das Team, dass lange Zeit mit den Testreihen und umfangreichen Messungen befasst war. Aber die Erkenntnisse, die dabei gewonnen werden, sind trotzdem wesentlich. Auch ein Test, der nachweist, dass es Nebenwirkungen gibt, die einen Einsatz verhindern, ist ein notwendiger Test. Genauso verhält es sich mit dem Test von Software, wenn man merkt, dass man einen anderen Ansatz wählen muss, z. B. eine andere Hardware, eine andere Programmiersprache, eine andere Datenbank, auch wenn das den Terminplan und die Entwicklungskosten erst einmal aus dem Ruder laufen lässt. Deswegen ist es so wichtig, dass genügend Forschungsgelder eingeplant werden, um über „Trial and Error“ Innovationen voranzutreiben, und nicht nur die Umsatzrendite kurzfristig maximiert. Manche Projekte werden hoffnungsfroh entwickelt und dann gestoppt oder nicht ausgeliefert – obwohl die Entwicklung fertig ist und der Test einen guten Job gemacht hat. Es gibt genügend Gründe, die der Softwaretest gar nicht beeinflussen kann.

Literatur

[UNIS2014] <https://www.st.cs.uni-saarland.de/edu/einst/12-testen.pdf>

Neueinführungen oder Wartungsprojekte können in unterschiedlichen Methoden produktiv eingeführt werden. Das hängt einerseits vom betrieblichen Risiko der betroffenen IT-Systeme und Prozesse ab, andererseits von der technischen und fachlichen Realisierbarkeit. Ob die Produktivnahme der Anwendung sukzessive, also etwa im Rahmen von Wartungsaufträgen, oder als gebündelte Gesamteinführung in Form eines Release stattfindet, ist von der Art der Einführung abhängig, von den damit verbundenen Risiken und den zur Verfügung stehenden Ressourcen. Ein Release-Management kann die Einführung oder die Bündelung von Wartungsaufträgen zentral priorisieren und steuern.

Die in Tab. 21.1 dargestellten Möglichkeiten der Produktivnahme stehen zur Verfügung.

Die Bedeutung von Wartungsmaßnahmen wird einem bewusst, wenn man den Umfang von COBOL-Programmen, die vor allem in Banken und Versicherungen zur Verwaltung großer Dateibestände nach wie vor im Einsatz sind, betrachtet: Man geht von ca. 240 Milliarden Zeilen Programmcode aus, ein Upgrade auf eine neuere Programmiersprache würde ca. 960 Milliarden Euro kosten, ein Betrag den selbst die Finanzwirtschaft nicht ohne weiteres aufbringen kann. Die schätzungsweise 10 Milliarden Euro pro Jahr, die deutschlandweit für Pflege und Maintenance von COBOL-Programmen erforderlich sind, nehmen sich demgegenüber ziemlich bescheiden aus.

Das ist auch der Grund dafür, dass nach wie vor COBOL-Programme im Einsatz sind und COBOL-Programmierer händierend gesucht werden. Zum Jahrtausendwechsel, als noch erheblich mehr dieser Altprogramme produktiv waren als heute, mussten zahlreiche Rentner als Entwickler und Tester reaktiviert werden um das Risiko wegen des Datums „01.01.2000“ zu minimieren. Die Ursache für die zu erwartenden Probleme lag damals im Wesentlichen darin, dass Jahreszahlen früher nur zweistellig definiert worden waren um Speicherplatz zu sparen, und man nicht abschätzen konnte, was der Wechsel von „99“ auf „00“ für Auswirkungen haben würde. Im Nachhinein stelle sich zwar

Tab. 21.1 Produktivnahme von Software

Produktivnahme von Software	Durch einzelne Wartungsaufträge, Eilaufträge	Ad hoc, je nach Bedarf (hohe Flexibilität, aber auch hoher Aufwand)
	Als Gesamteinführung (als Release)	– Mit Pilotphase – Mit Übergangsphase (Parallelbetrieb, Teilproduktivstellung) – Als „Big Bang“

heraus, dass das große IT-Fiasko zum Jahrtausendwechsel ausgeblieben war. Das lag aber auch erheblich daran, dass im Vorfeld auf breiter Front ein breites Risikomanagement aufgesetzt worden war und zur Verifikation der Jahrtausendfähigkeit präventiv viel Geld investiert wurde.

21.1 Pilotbetriebe

Bei einer Pilotierung steht die neue Software einer begrenzten Anzahl von Nutzern zur Verfügung. Mit der Pilotierung sollen Aussagen abgeleitet werden, welche die flächendeckende Einführung erleichtern sollen. Vor allem bei strategischen Neuausrichtungen ist die Einbeziehung eines Piloten von Anfang an zu empfehlen, weil man damit die verbundenen Risiken abschätzen und die Akzeptanz des neuen Systems von Anfang an erleichtern kann.

Man setzt sich dadurch auch weniger dem Vorwurf aus, die Entwicklung im Wasserkopf der Zentrale zu vollführen und dann den Praktikern ein unausgeglichenes System aufzudrücken. Gerade Meinungsführer, die schon lange im Unternehmen sind, gegenüber der neuen Software skeptisch sind und dann vielleicht „schlechte Stimmung“ gegen die neu entwickelte Applikation auf breiter Front machen, sollte man daher von Anfang an ins Boot holen und schon die Entwicklungs- und Testphase begleiten lassen.

So wird z. B. ein Warenwirtschaftssystem, das bei 500 Außenstellen eingesetzt werden soll, zunächst bei wenigen ausgewählten Stellen unter genauer Beobachtung getestet. Dafür ist es wichtig, den Pilotpartner repräsentativ auszuwählen und intensiv zu betreuen. In einem meiner früheren Projekte war für den Piloteinsatz eines derartigen Systems ausgerechnet eine Lieferstelle ausgewählt, die im Gegensatz zu anderen Außenstellen fast nur bestimmte Geschäftsprozesse abwickelte: Es wurden nur Lieferscheine und selten Barverkäufe ausgestellt. Dadurch konnte ein erheblicher Fehler bei den Barverkäufen im Pilotbetrieb nicht gefunden werden, weil er schlichtweg an dieser Lieferstelle nicht vorkam. Die Konsequenz war, dass das neue System während des Rollouts mit mehr Barverkäufen massive Fehler in den Backend-Systemen verursachte (in diesem Fall war die gesamte Produktionsmengenerfassung falsch, was man aber wiederum erst nach dem Monatsabschluss überhaupt bemerken konnte) und nachträglich umfangreiche Korrekturmaßnahmen erforderlich wurden.

Es gibt auch Tests, bei denen die gesamte Infrastruktur erforderlich ist, um den kompletten Workflow zu testen und zu validieren. Ein System zur Verarbeitung von Logistikdaten im speditionellen Umfeld brauchte die Sortiermaschine, um den Algorithmus zu evaluieren und noch zu verbessern. Es wäre aber zu teuer und aufwändig gewesen, die gesamte Produktivumgebung im Labor nachzustellen. Daher wurde von ca. 30 Außenstellen zunächst nur eine mit der entsprechenden Software ausgestattet. Ein Produktiveinsatz ist aber nur dann vertretbar, wenn die Software im Testbetrieb schon ausreichend verifiziert wurde und mit Simulatoren bereits im Labor eine entsprechende Last erzeugt wurde, nur eben noch nicht die Anbindung an alle Systeme. In diesem Fall sind Teststufen und Testinhalte aufeinander abzustimmen und im Vorfeld zu definieren und abzugrenzen.

Beim Einbringen neuer Software in den Produktivbetrieb ist unbedingt darauf zu achten, Risiken zu minimieren, auch wenn man „nur“ eine Stelle lahm legt. Ein exaktes Konfigurationsmanagement und eine definierte Integrationsstrategie sind hier unabdingbare Voraussetzungen für den Projekterfolg.

21.2 Produktivnahme mit Übergangsphase

Auch bei Wartungen eines Systems als Ganzes wird manchmal eine Übergangsphase vorgesehen. Die Dauer einer Übergangsphase und die Bestimmung, welche Funktionalitäten während welcher Zeit nicht oder nur teilweise zur Verfügung stehen, muss bei der Planung der Einführung abgestimmt werden. Wenn in der Übergangsphase sowohl das neue als auch das Altsystem zur Verfügung stehen, muss man höhere Wartungskosten einkalkulieren. Daher empfiehlt es sich, diese Phase so kurz wie möglich zu halten. Auch die Verarbeitung in den angeschlossenen Systemen kann erhebliche Mehraufwände und Kosten für die Integration verursachen.

Problematik des „Big Bang“

Große Gesamtproduktivnahmen werden als „Big Bang“ bezeichnet. Damit ist zwar ein großes Vorhaben in kurzer Zeit abgeschlossen, man muss aber das Scheitern, Korrekturmaßnahmen und Nachbesserungsaufwand einkalkulieren. Manchmal ist aber der Big Bang – auch wenn er die Nerven der betroffenen Abteilungen strapaziert – unter dem Strich kostengünstiger und effizienter als eine langwierige Parallelphase von Altsystem und neuer Anwendung. Bei einem Big Bang sollten die Testaktivitäten so umfassend wie möglich und die Testabdeckung möglichst hoch sein, um den Produktivbetrieb nicht nachhaltig zu gefährden. In diesem Fall muss auch über ein Rückfallszenario nachgedacht werden, das die Auswirkungen vorher bedacht und das Risiko realistisch bewertet hat.

Vor allem organisatorische Maßnahmen sind im Rahmen des Big Bangs zu beachten, also die Notwendigkeit von Wochenendeinsätzen oder Überstunden, die mit einem ausreichenden zeitlichen Vorlauf zu klären sind. In den letzten Jahren wird der „Big Bang“ in der Praxis nur noch selten eingesetzt, weil das Risiko des Scheiterns einfach zu groß geworden ist und die meisten Systeme viel zu komplex dafür sind.

Vorteile und Nachteile von Delta-Tests

An irgendeiner Stelle der Software wird eine kleine Funktion geändert. Man ist der Meinung, dass es sich ja nur um eine unwesentliche Änderung handelt, die Funktion wird getestet, kein Fehler bemerkt und freigegeben.

Daraufhin wird die geänderte Software in Betrieb genommen. Plötzlich bemerkt man einen heftigen Seiteneffekt: Eine ganz andere, viel wichtigere und zentralere Funktion, arbeitet plötzlich nicht mehr oder falsch. Durch diese Änderung ist dann ein Problem an einem sehr speziellen Geschäftsvorfall behoben aber ein wesentlicher Geschäftsvorfall, der permanent vorkommt, kann nicht mehr fehlerfrei abgebildet werden. Um dieses Risiko gänzlich zu vermeiden, müsste die gesamte Software neu getestet werden. Da man aber meistens nicht die Zeit hat, wegen einer minimalen Änderung wieder den gesamten Softwaretestzyklus durchzuführen, andererseits mit der Änderung auch nicht länger warten kann oder will, empfiehlt es sich die Tests im Vorfeld zu priorisieren: Die Standardprozesse mit Priorität 1 (hoch) werden dadurch bei jeder, auch minimalsten Änderung, neu getestet, während die exotischen Besonderheiten mit Priorität 3 (niedrig) nur bei einem umfassenden Releasetest getestet werden. Tab. 21.2 zeigt exemplarisch verschiedene Testaktivitäten mit Dauer und Priorität. Der Begriff „Delta-Test“ drückt in diesem Zusammenhang aus, das nur die geänderten Funktionalitäten einer Anwendung einem Test unterzogen werden und nicht die gesamte Testspezifikation erneut getestet wird.

Wenn für die Durchführung maximal zwei Arbeitstage (16 Stunden) zur Verfügung stehen und nur 1 Tester verfügbar ist bedeutet das in diesem Beispiel: Alle Tests mit Priorität hoch (in Summe acht Stunden) können durchgeführt werden, die Tests der Priorität mittel nur teilweise, die Tests mit Priorität niedrig gar nicht.

21.3 Hilfsmodelle

Ich habe schon öfter die Aussage gehört „die Software hat sich ja eigentlich gar nicht geändert“, etwa wenn nur ein einzelner Parameter geändert wurde oder ein kleiner Fehler in einer Ausnahmeroutine verbessert wurde, ansonsten aber am Quellcode definitiv nichts geändert wurde. Dennoch kommt man auch hier grundsätzlich nicht umhin, die gesamte

Tab. 21.2 Testaktivitäten
nach Dauer und Priorität

Test	Dauer	Priorität
A	4 Stunden	Hoch
B	4 Stunden	Mittel
C	3 Stunden	Mittel
D	4 Stunden	Niedrig
E	2 Stunden	Hoch
F	2 Stunden	Niedrig
G	2 Stunden	Mittel
H	4 Stunden	Hoch
I	2 Stunden	Mittel

Software neu zu testen, wenn man sicher gehen will, dass nicht nur der Fehler behoben ist, sondern auch noch weiterhin die Funktionen richtig arbeiten, die in der Vorversion auch funktioniert haben.

Der Aufwand kleiner Programmänderungen nach Fertigstellung des Release wird geheimhin unterschätzt. Hin und wieder bedeutet eine kleine Programmänderung einen gewaltigen Aufwand in der Folge: Der Systemtest, die Validierung, ggf. sogar eine externe Begutachtung der Software müssen dann neu durchgeführt werden, weil sich das Programmobjekt marginal geändert hat.

Moderne Tools zur Konsistenzsicherung der Software erlauben einen besseren Überblick über den Änderungsbedarf, die Architektur und die möglichen Querschlager. Eine Verknüpfung von Anforderungsdokumenten mit dem Quellcode (über mehrere Ebenen) erlaubt es, das Risiko der Softwaretests zu minimieren.

Nachverfolgbarkeit und Nachvollziehbarkeit durch Traceability

Traceability bezieht sich auf die Erfassung aller an die Software gestellten Anforderungen und deren Verknüpfung mit allen Testaktivitäten. Dadurch lassen sich alle Testaktivitäten bis zu den Anforderungen zurückverfolgen. In diesem Fall werden also Kundenanforderungen (Pflichtenhefte, Requirements) über Systemdesign und Architekturdokumente (Lastenhefte), über Entwicklungsdokumente und Programmbeschreibungen bis hin zu Testspezifikationen und Abnahmedokumente miteinander verknüpft, um die logische Umsetzung der Anforderungen nachverfolgbar zu dokumentieren und damit die Qualität für das Endprodukt zu sichern [BOCH2010].

Dadurch lässt sich für alle Abweichungen, die bei der Testdurchführung entdeckt wurden, feststellen, welche Anforderungen nicht oder nicht vollständig erfüllt sind:

Traceability kontrolliert die Testabdeckung und unterstützt dadurch die Effektivität und die Effizienz der Testdurchführung. Jede Anforderung muss mit mindestens einem Testfall verbunden sein und jede Abweichung muss in Zusammenhang mit einem Testfall gemeldet werden. Mit Hilfe der Traceability wird sowohl die Akzeptanz des Systems durch den Kunden als auch durch Dritte (z. B. externe Prüfer, Gutachter, Behörden) sichergestellt. Außerdem können dadurch Auswirkungen von Änderungen im Vorfeld untersucht werden, da der Umfang der Regressionstests sofort visualisiert werden kann.

Bei einer Änderung einer Anforderung bemerkt man mit einer durchgehenden Traceability sofort, welche Einflüsse sich auf das Gesamtsystem ergeben, wo die Systemarchitektur beeinflusst wird, welche Programme geändert und welche Testfälle angepasst werden müssen. Die genaue Kenntnis dieser Auswirkungen ist für die Kalkulation eines Angebots und die Planung von Budget und Terminen von erheblicher Bedeutung.

Literatur

- [BOCH2010] Bochynek N, Delgado JD (2010) Testen in der Finanzwelt. Diaz & Hilterscheid, Berlin. isbn:978-3-00-028082-5

Wichtige Entscheidungen sollten nicht allein vorbereitet und kritische Entscheidungen hinterfragt werden, um das Risiko von Fehlern oder Missbrauch im Unternehmen zu reduzieren. Das gilt auch in besonderem Maße für den Testprozess, weil der Softwaretest ja gerade Fehler minimieren und die Qualität des Endprodukts steigern soll. Deshalb ist ein Vier-Augen-Prinzip zu empfehlen, das sich beim Testprozess durch Reviews umsetzen lässt.

22.1 Allgemeines über Reviews

Ein Review bezeichnet ein bestimmtes Vorgehen zur Prüfung von Dokumenten als auch ein Oberbegriff für statische Prüfverfahren, die von den Beteiligten im Projekt durchgeführt werden. Die Stakeholder sollten daher regelmäßig zu Reviews der Testdokumente eingeladen werden. Der Testplan, das Anforderungsdokument und das Testkonzept sollen in jedem Fall geprüft werden.

22.2 Vorteile von Reviews

Reviews haben für das Projekt und das Unternehmen folgende Vorteile:

- Unstimmigkeiten und Fehler können aufgedeckt werden, bevor der Test in eine falsche Richtung läuft. Die Verbesserung der Dokumente wirkt sich positiv auf den gesamten Entwicklungsprozess aus.
- Fehler werden früh erkannt und erzeugen keine Folgekosten, sondern steigern die Produktivität des Entwicklungs- und Testteams, so dass sich die Entwicklungsphasen verkürzen können.

- Ungenauigkeiten in den Produktspezifikationen können aufgedeckt werden, Widersprüche werden mit dem Auftraggeber geklärt, so dass auch eine Kostenreduktion während der Lebenszeit des Produkts zu erwarten ist, denn nachträgliche Änderungen nach Produkteinführung können im Vorfeld minimiert werden.
- Der Know-how Transfer im Team führt zu verbesserten Arbeitsmethoden und zwingt zu einer klaren Darstellung der Sachverhalte.
- Das gesamte Team wird für die Qualität des untersuchten Prüfobjekts in Verantwortung genommen.

Als Autor des Dokuments sollte man strikt auf die Verbesserung der Qualität des Dokuments achten und sich nicht persönlich angegriffen fühlen, wenn das eigene Testdokument nicht so gut ankommt. Die Reviewer sollten ebenfalls psychologische Aspekte beachten und ihre Kritik konstruktiv vorbringen. Wichtig ist es, jedes Review mit einer definierten Zielvorstellung durchzuführen. Es sind immer alle notwendigen Personen zu beteiligen, aber auch nicht mehr als unbedingt erforderlich. Die Anzahl der nötigen Teilnehmer ist dabei nicht einfach zu ermitteln, vor allem wenn man die Rollen gar nicht genau kennt.

Das Management sollte Reviewprozesse unterstützen und sie aktiv einfordern. Ziel von Reviews sind permanente Optimierungen der internen Prozesse durch eine lernende Organisation.

Der Einsatz von Reviews führt zu einer deutlichen Reduktion von Fehlern: Laut Carpers Jones Studien von ca. 12.000 Projekten führen Requirements Reviews zu einer Reduktion von 20 bis 50 % der zu erwartenden Fehler, Top-Level Design Reviews zwischen 30 und 60 %, detaillierte funktionelle Design Reviews zwischen 30 und 65 % und detaillierte logische Design Reviews zwischen 35 und 75 %.

Wenn man die zusätzlich zu veranschlagende Zeit für Reviews mit 10 bis maximal 20 % ansetzt, zeigt das eindrucklich den durchschnittlichen Nutzen dieser Maßnahmen. Dokumenten-Review müssen sich in alle Projektphasen finden und sich über den gesamten Lebenszyklus des Systems erstrecken, um ihren Nutzen wirklich entfalten zu können.

Das entspricht in etwa der Effektivität von Systemtests (25 bis 65 % aller Fehler). Fehler, die im Review auffallen, können häufig bedeutend kostengünstiger behoben werden, als wenn diese erst während der Testdurchführung gefunden werden.

Dabei laufen verschiedene Qualitätsprozesse ab:

- Der Programmierer bzw. Tester entdeckt selbst eine Verbesserungsmöglichkeit.
- Der Rezensent stellt Verständnisfragen und der Programmierer kann den Code dahingehend verändern (beispielsweise durch geeignete Namensgebung oder Dokumentation), dass diese Fragen beantwortet sind und so die Verständlichkeit verbessert wurde.
- Der Rezensent entdeckt Verbesserungsmöglichkeiten und empfiehlt diese dem Entwickler.

Typische Schwächen, die mit Reviews entdeckt werden können, sind

- Abweichungen von Standards, z. B. Verletzung von Namenskonventionen,
- Verständnis über gemeinsame Begriffe (Glossar),
- Fehler gegenüber (oder auch in) den Anforderungen oder widersprüchliche Anforderungen,
- Fehler im Software-Design oder der Systemarchitektur,
- unzureichende Wartbarkeit der Programme,
- falsche Interpretation und fehlerhafte Umsetzung von Anforderungen,
- Unvollständigkeit von Anforderungen,
- falsche oder fehlende Schnittstellenspezifikationen.

22.3 Reviewprozess

Ein typisches Review besteht aus folgenden Phasen:

- **Planung**
 - Auswahl der beteiligten Personen und Besetzung der Rollen,
 - Festlegung der Vor- und Nachbedingungen.
- **Kick-Off**
 - Verteilung der Dokumente,
 - Erläuterung der Ziele und des Prozesses,
 - Prüfung der Vorbedingungen.
- **Individuelle Vorbereitung**
 - Dokumentation von potentiellen Fehlern, Fragen und Kommentaren.
- **Reviewsitzung**
 - Diskussion und Protokollierung der Ergebnisse,
 - Empfehlungen geben oder Entscheidungen über Fehler treffen.
- **Überarbeitung (rework)**
 - Beheben der gefundenen Fehler, typischerweise durch den Autor.
- **Nachbearbeitung (follow up)**
 - Überprüfung der Überarbeitung,
 - Prüfung von Testendekriterien.

Manchmal wird auf die Reviewsitzung verzichtet und das Dokument dann nur per Mail an die Reviewer verschickt. In der Praxis spart man dadurch nach meiner Erfahrung nichts ein, denn es führt eher zu endlosen Reviewprozessen und manche Reviewer müssen mehrfach daran erinnert werden, dass der Endtermin des Reviews längst überschritten ist. Wenn man alle Beteiligten an einem Tisch sitzen hat, kann man Missverständnisse viel eher ausräumen und Formulierungen direkt anpassen. Dabei kann man auch „Reviewsitzung“ und „Rework“ gleich zusammenfassen. Für die am Review Beteiligten steigt der Aufwand

zwar vordergründig betrachtet leicht an, aber das Review wird schneller fertig und das Dokument ist dann wirklich von allen Beteiligten auch verabschiedet, was mittelfristig die Effizienz steigert.

Wichtig ist auch, dass die richtigen Reviewer für die zu überprüfenden Objekte benannt sind und die Reviewer mit der richtigen Perspektive das Testobjekt betrachten. Für eine sorgfältige Vorbereitung von Reviews ist daher die Beachtung dieser Punkte unbedingt zu empfehlen.

Tools für das Testmanagement bieten Mechanismen zur komfortablen Erfassung, Organisation und Steuerung der Prioritäten von Testfällen. Dadurch kann der Status der Testaktivitäten überwacht, dokumentiert und statistisch ausgewertet werden. Es wird ersichtlich, welche Testfälle durchgeführt wurden und welches Ergebnis bei welchem Testfall erwartet wurde und tatsächlich auftrat. Das Testmanagement ist auf die Unterstützung von Testtools angewiesen, um den Testfortschritt jederzeit zu messen, eine stringente Testplanung und ein klares Testreporting durchführen zu können.

Testmanagementwerkzeuge sollen anforderungsbasiertes Testen unterstützen und daher Systemanforderungen (Requirements) erfassen oder aus Requirements-Management-Tools mit denjenigen Tests verknüpfen können. Dadurch wird ersichtlich, ob die entsprechende Anforderung erfüllt ist.

23.1 Evaluation von Testtools

Ein entscheidender Erfolgsfaktor für die Sicherstellung einer hohen Softwarequalität ist der Einsatz eines zentralen Tools für das Testmanagement, damit alle Team-Mitglieder jederzeit Zugriff auf wichtige Informationen und einen vollständigen Überblick über alle Testaktivitäten erhalten. Wenn eine solche Lösung die Zusammenarbeit von (auch geografisch verteilten) Teams unterstützt und alle Beteiligten mit denselben eindeutigen Informationen arbeiten können, werden beispielsweise auch die Iterationen zur Fehlerbehebung auf ein Minimum beschränkt bleiben.

Das eingesetzte Tool muss den gesamten Testprozess unterstützen. Beim Einsatz eines Testtools sollte man sich zunächst genau überlegen, welche Anforderungen man an das Tool hat.

Das Testtool sollen alle durchgeführten Teststufen unterstützen, also etwa nach dem V-Modell den Entwicklertest, den Integrationstest und den Systemtest. Diese Tests können unterschiedlich komplex sein. Eine Integration der Tests in ein gemeinsames Tool und die Möglichkeit, zwischen den einzelnen Teststufen Tests auszutauschen, sollte in jedem Fall gegeben sein. Der Blickwinkel mag anders sein, beim Entwicklertest geht es nur um den Programmcode aus der Perspektive des Programmierers, beim Systemtest oder User Acceptance Test um das Gesamtsystem aus der Perspektive des Endanwenders.

In Großunternehmen, wo manchmal schon zahlreiche Testtools nebeneinander existieren, empfiehlt es sich, statt das x-te Testtool einzuführen – welches für die eigene Anforderung exakt zu passen scheint – zu evaluieren was vorhanden ist und sich strategisch auf eine oder wenige (dabei aber auch die Abhängigkeit von einem Zulieferer beachten!) Tools zu spezialisieren. Nicht für jede Anforderung ist jedes Tool gleich gut geeignet, hier gilt es einen Kompromiss zu finden.

23.2 Suche geeigneter Werkzeuge

Die Flut an Excel-Tabellen, die über uns alle in den letzten Jahrzehnten hereingebrochen ist, lässt es leider nicht vermeiden, dass man Tests vorfindet, die in einer Excel-Tabelle definiert sind, in der dann die einzelnen Zellen rot und grün markiert werden. Zur Darstellung komplexer Zusammenhänge ist dieses Verfahren völlig ungeeignet. Es fehlt die Verbindung zum Anforderungstool, die Versionsverwaltung ist nicht ausreichend, die Methode ist schlichtweg unprofessionell.

Es ist zu bedenken, dass Projekte wachsen und dass man ohne eine wirkliche Versionsverwaltung und mit verteilten Daten mit dem Werkzeug „Excel“ nicht glücklich werden wird. Nach einiger Zeit hat man ein gewaltiges Chaos im Projekt, das dann kaum noch verwaltet werden kann. Sofern es sich nicht um ein einmaliges, in wenigen Tagen abgeschlossenes Kleinstprojekt handelt, ist eine gezielte Analyse erforderlich, um das richtige Test-Tool auszuwählen.

Excel und Word sind in der Regel aus folgenden Gründen ungeeignet für die Verwaltung von Testfällen:

- unübersichtlich bei Planung der Tests,
- unhandliche Zuordnung von Testfall zu Tester,
- mangelnde Flexibilität bei zusätzlichen Releases oder geänderten Anforderungen,
- keine Historie der Testfälle,
- Dokumentation nur begrenzt möglich,
- keine Möglichkeit, Anhänge oder Screenshots hinzuzufügen,
- manuelle Strukturierung der einzelnen Dokumente,
- Gesamtüberblick über mehrere Testpläne nur über Verdichtung von Informationen möglich, Suche nach bestimmten Begriffen schwierig,
- keine Mehrbenutzerfähigkeit, bei Zugriff mehrerer Anwender völlig ungeeignet,

- keine Möglichkeit des Datenaustauschs mit anderen Anwendungen bzw. nur sehr eingeschränkt über selbsterstellte Skript-Programme,
- keine zeitliche Entwicklung der Testdurchführung,
- keine direkte Verknüpfung mit Fehlern für notwendige Nachtests,
- Protokollierung der Testergebnisse nicht detailliert genug,
- keine Versionsverwaltung,
- Dateibasierte Speicherung von Daten,
- lokale Ablagen (absichtlich oder unabsichtlich) möglich, dadurch unterschiedliche Versionen, beim Zusammenführen verschiedener Informationen aus unterschiedlichen Dateien entsteht hoher manueller Aufwand und die Fehlerquote steigt,
- Ermittlung des Teststatus nicht ersichtlich,
- aktueller Teststatus ist bei mehreren Versionen nicht aus den Requirements ablesbar.

Excel und Word haben für manche Einsatzzwecke ihre Berechtigung, zumal das Know-how allgemein vorhanden ist und kaum Einarbeitungsaufwand erforderlich ist, aber die Möglichkeiten zur Testfallsteuerung besitzen sie nur sehr eingeschränkt.

Allgemein gesagt, kann eine geeignete Software den Testprozess in folgenden Bereichen unterstützen:

- Planung der Testprozesse,
- Einsatzplanung der Testressourcen,
- Dokumentation eindeutiger Testszenarien,
- Gewährleistung reproduzierbare Testabläufe,
- Überwachung des Testverfahrens,
- Dokumentation der Testergebnisse,
- Definition, Priorisierung und Verfolgung der Aktivitäten zur Fehlerbehebung,
- Dokumentation der Abnahme [[SCHL2009](#)].

Es empfiehlt sich, in einem Brainstorming unternehmens- und projektspezifische Anforderungen, die eine Testsoftware erfüllen soll, zu sammeln. Dazu sollte man die einzelnen Kriterien gewichten und die spezifischen Anforderungen an die Testsoftware in ein Lastenheft übernehmen. Dieses Lastenheft kann man von einem externen Entwickler bewerten lassen, den man nach Referenzen fragen kann.

Des Weiteren sollte eine Teststellung für das Testtool vereinbart werden und das Tool in einem ausgewählten Projekt eingesetzt werden, um täglich damit zu arbeiten und den gesamten Funktionsumfang auszuprobieren. Wichtig ist auch, dass ein Entwickler des Anbieters für Ihr Unternehmen jederzeit zur Verfügung steht.

Eine Lösung von der Stange, die evtl. in einigen Punkten an Ihr Unternehmen individuell angepasst wird, ist fast immer besser als eine Eigenentwicklung. Wenn das eigene Unternehmen nicht selbst als Anbieter entsprechender Software auf dem Markt agiert, sollte man besser die eigene Kernkompetenz weiter ausbauen und sich nicht in hobbymäßiger Programmierung versuchen. Diese Aussage gilt analog für Tools für das Fehlermanagement.

Von Schnellschüssen in der Form, dass ein Praktikant mit Hilfe einer Access-Tabelle ein Testtool zusammengestrickt hat, ist dringend abzuraten. Es ist bei so manchem Unternehmen schon vorgekommen, dass dann bei einer neuen Version Testfälle verschwunden sind. Bei der Auswahl des geeigneten Testtools ist auch auf Skalierbarkeit und hinreichende Flexibilisierung zu achten.

23.3 Anforderungen an geeignete Test-Tools

Ein gutes Test-Tool sollte in jedem Fall folgende Möglichkeiten beinhalten:

- Verwaltung mehrerer Versionen der Software (Major-/Minor-Releases),
- Übersicht über den aktuellen Status der einzelnen Testergebnisse,
- standardisierte Dokumentation der Testergebnisse,
- leichte Auswertbarkeit der Testergebnisse,
- Zuordnung von Testressourcen zu Tests,
- Definition der notwendigen Kriterien pro Testfall (eindeutige ID, Beschreibung, Priorität, Testdauer, Voraussetzungen, Aktion, erwartetes Ergebnis ...),
- Erweiterungsmöglichkeit um zusätzliche Optionen,
- Status der Testabdeckung je Version,
- definierte Zugriffsebenen und Rechte,
- übersichtliche Struktur der Verzeichnisse,
- Verknüpfung zu Anforderungen,
- Verknüpfung zum eingesetzten Fehlerverfolgungssystem,
- übergreifende Sicht auf die einzelnen Fehler,
- keine redundante Datenpflege (Testfälle, Requirements),
- Generierung geeigneter Metriken zum Testreporting (Standardauswertungen und Möglichkeit, selber Reports zu generieren),
- einfaches Berechtigungssystem,
- Web-basiert und Pflege über Browser,
- Verwaltung mehrerer Produkte und Projekte möglich, leichte Skalierbarkeit,
- Mandantenfähigkeit [[VIVI2011](#)].

Weitere individuelle Forderungen sind im Unternehmen bzw. im Projekt zu definieren. Ein testweiser Einsatz oder eine Pilotierung eines Tools in einem überschaubaren Projekt ist in jedem Fall dringend zu empfehlen.

Die Entscheidung für ein Testtool ist eine strategische Entscheidung und sollte auch für das gesamte Unternehmen oder zumindest alle betroffenen Abteilungen und Bereiche, die jemals an einem Projekt arbeiten, so gelten und durchgesetzt werden.

Wildwuchs an dieser Stelle oder „Synergie-Effekte“ die sich dann doch nicht einstellen, können einem Testprojekt das Genick brechen. Gerade bei Mergern oder bestehenden Projekten die zusammengelegt werden, besteht die Gefahr, dass man sich über „low

hanging fruits“ freut. Viele Synergieeffekte gerade in komplexen Projekten lassen sich aber meist nur mittelfristig realisieren und mit sehr genauer Analyse der zugrundeliegenden Prozesse umsetzen. Dazu fehlt dann die notwendige Geduld oder das notwendige Budget.

Schließlich muss aber festgehalten werden, dass auch das beste Testtool kein Allheilmittel ist, sondern lediglich eine Hilfestellung sein kann.

Die Definition der Prozesse im Unternehmen, ein fundiertes Testkonzept, nachvollziehbare, detaillierte und vollständige Testszenarien, durchdachtes Fehlermanagement und professionelles Reporting sind wesentliche Bausteine für ein erfolgreiches Testmanagement. Ein Testtool kann unterstützen, Abläufe zu optimieren und den Verwaltungsaufwand zu reduzieren, aber es kann nie das fachliche und technische Knowhow erfahrener Softwaretester ersetzen.

Literatur

- [VIVI2011] http://www.vivit-germany.org/pdf/H2011/Vivit11_18_Schmidt_parcIT.pdf
[SCHL2009] <http://www.bernhardschloss.de/images/Testmanagment%20in%20IT-Projekten.pdf>

Als Testabdeckung bezeichnet man das Verhältnis an tatsächlich getroffenen Aussagen eines Tests gegenüber den theoretisch möglich treffbaren Aussagen bzw. der Menge der gewünschten treffbaren Aussagen. Die Testabdeckung dient als Gradmesser für die Qualitätssicherung und wird häufig in Metriken verwendet.

Die Testabdeckung lässt sich durch eine Erhöhung der Messungen und Testfälle optimieren, sie wird in der Praxis aber durch die Kosten, die für die Testaktivitäten verbunden sind, begrenzt.

Das folgende Beispiel zeigt, dass ein komplettes „Austesten“ eines Programmes so gut wie unmöglich ist:

- Es gibt genau x Möglichkeiten ein Programm auszuführen und wenn alle x Möglichkeiten im Test getestet werden, ist das Programm komplett ausgetestet.
- Angenommen, eine Prozedur erwartet 2 Integer-Werte als Eingabedaten.
- Bei 32 Bit Länge ergeben sich bereits $2^{32} \cdot 2^{32}$ Kombinationen von Eingabewerten (ungefähr $1,8 \cdot 10^{19}$).
- Selbst wenn ein automatischer Testtreiber in 1 ms einen Testfall ausführen kann, dauert der vollständige Test aller Eingabewerte ca. 570 Millionen Jahre!

Wenn man sämtliche Kombinationen der Software testen würde, würde man also zumindest Hunderte und Tausende von Testfällen behandeln müssen. Mit einer Testautomatisierung kann man dann zwar manuelle wiederholte Tätigkeiten vermeiden, aber es bleibt der Aufwand der Automatisierung, der Pflege, der Aktualisierung und der einmaligen Erstellung der Skripte. Auch automatisierte Testfälle müssen zumindest einmal manuell ausgeführt worden sein, und bei einer Vielzahl von Testfällen muss man mit einer gewissen Fehlerquote rechnen. Außerdem müssen die Ergebnisse der automatischen Tests ausgewertet, archiviert und interpretiert werden. Wenn man andererseits nur sehr oberflächlich

einige wenige Funktionen der Software testet, sinkt die Zeit der Testdurchführung und man minimiert die Kosten. Dann läuft man aber Gefahr, für wesentliche Funktionen nicht die notwendige Qualität sicherzustellen.

Dazu kommt das Reporting automatisierter Testfälle: Selbst wenn auch hier die Ergebnisdateien maschinell erzeugt werden, so bleibt immer noch die Interpretation der Ergebnisse und die Untersuchung der Fehlerursachen bei fehlgeschlagenen Testskripten.

Also empfiehlt es sich, ein gesundes Maß zu testen, und dabei das Risiko und die Kosten gegeneinander abzuwägen.

In einer Kurve kann man Kosten und Testumfang zur Abschätzung visualisieren (Abb. 24.1).

Um dieses Optimum zu ermitteln, kann man sich mit Formeln, Metriken und Schätzungen nähern, die in der Fachliteratur und teilweise auch in diesem Buch beschrieben sind. Meist sind nicht alle Daten vorhanden um die Ermittlung rechnerisch exakt genug durchzuführen. Wenn man einzelne Faktoren zu gering oder zu hoch ansetzt, multiplizieren sich die einzelnen Abweichungen. Deswegen sollte zu einer theoretischen Betrachtung eine pragmatische und individuelle Betrachtung der Gegebenheiten im Projekt kommen. Langjährige Erfahrungswerte und eine Portion Kreativität sind erforderlich um die ideale Testabdeckung zu ermitteln.

Das Optimum ist bei jeder Software, je nach Sicherheitsrelevanz, Zielgruppe, Einsatzbereich anders.

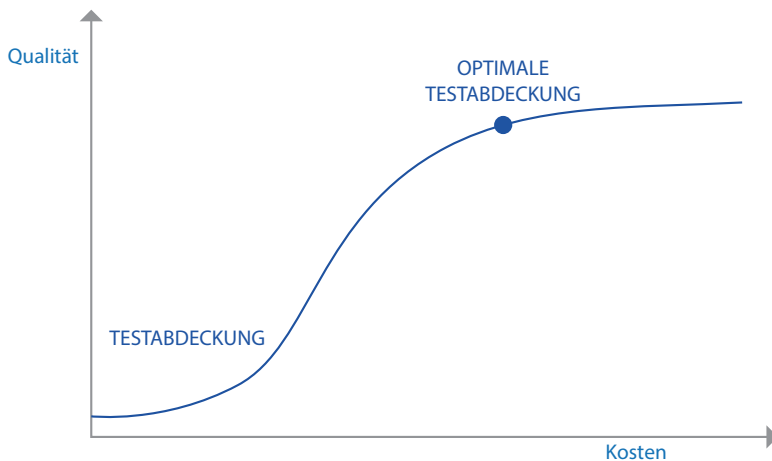


Abb. 24.1 Die optimale Testabdeckung

Man muss in jedem Projekt immer genügend Zeit dazu vorsehen, um das Wesentliche übersichtlich und vergleichbar in Reports darzustellen. Das betrifft auch die Präsentation der Ergebnisse der Testaktivitäten.

Für ein klares Testreporting empfiehlt sich der Einsatz von Testmetriken. Beim Reporting sind alle Mitarbeiter nach ihren Ergebnissen zu befragen. Metriken helfen dabei, Vergleichbarkeit herzustellen:

- Wer ist für welchen Test verantwortlich?
- Was wurde getestet?
- Welche Aufgaben wurden erledigt?
- Wo liegen die größten Probleme?
- Stockt der Softwaretest gerade und warum?

Metriken sind gängige Instrumente, die mittels Kennzahlen versuchen, den Softwareprozess kontrollierter ablaufen zu lassen. Sie können für die Testüberwachung eingesetzt werden.

Mit Metriken kann man z. B. erstellen:

- Prozentsatz bereits beschriebener Anforderungen,
- Prozentsatz bereits getesteter Testfälle,
- Anzahl gefundener Fehler.

Es ist dabei wichtig, keine der Metriken absolut zu behandeln, sondern im Zusammenhang zu sehen.

25.1 Arten von Testmetriken

Nützliche Maße für die Verfolgung des Testprozesses sind beispielsweise

- testfallorientierte Metriken, z. B. Anzahl der Testläufe, Anzahl der erfolgreichen Tests, Anzahl der Testwiederholungen,
- fehlerbasierte Metriken, z. B. Anzahl der Fehlermeldungen,
- testobjektbasierte Metriken, z. B. Codeüberdeckung.

Gründe, die für den Einsatz von Metriken sprechen, sind:

- Identifikation von risikoreichen Bereichen, die zusätzlichen Testaufwand erfordern,
- Identifikation von Schulungsbedarf,
- Identifikation von Prozessverbesserungsmöglichkeiten,
- Statusverfolgung und Statuskontrolle (Control/Tracking Status),
- Grundlage für Schätzungen,
- Rechtfertigung für Budget, Tools und Schulungen,
- Indikator, wann welche Aktionen gesetzt werden müssen/können [[HAHS2014](#)].

Diese Maßzahlen können während des Testprozesses Indikationen dafür sein, ob sich die Aufwände erhöhen, bis das Testendekriterium erreicht werden kann.

Folgende Metriken sind erfahrungsgemäß in vielen Fällen sinnvoll.

- Testfälle/Funktionen,
- Anzahl Testfälle,
- Anzahl gefundene Fehler in Summe,
- Fehler pro Testfall (fehlgeschlagene Testfälle/Anzahl Testfälle),
- geleistete Testabdeckung in Prozent,
- aktueller und geplanter Status der Testaktivitäten.

Es sollten dabei Indexwerte und geeignete Key Performance Indicators gebildet werden, welche die Softwarequalität der eigenen Anwendung am besten treffen. Dabei ist zu prüfen, welche Aussagen im Einzelfall wem berichtet werden sollten. Eine Metrik „Anzahl der Testfälle“ z. B. vermengt umfangreiche, komplexe und langwierige mit einfachen und schnell durchzuführenden Testfällen. Wenn nun am Anfang des Projekts die einfachen, schnell durchzuführenden Testfälle durchgeführt werden, steigt die Kurve der Anzahl Testfälle schnell nach oben und verleitet dazu, sich zurücklehnen zu können. Aber wenn es zu den schwierigeren Testfällen kommt, lässt sich dieses Tempo nicht mehr durchhalten, und die Kurve nach oben wird flacher, obwohl sich an der Effizienz des Testprozesses und am Einsatz des Testteams gar nichts geändert hat. Dann kann es zu kritischen Rückfragen nach dem Testaufwand und der Kostenentwicklung kommen.

Der Tester muss beim Reporting im Vorfeld planen, welche Kriterien er bewerten will und wie entscheidend ein gewisses Kriterium ist. Meist empfehlen sich mehrere Metriken, um das Bild möglichst abgerundet erscheinen zu lassen. Es ist aber darauf zu achten, dass es nicht zum Information Overflow kommt.

Es muss auch allgemein geklärt sein, was der entsprechende Indexwert aussagt, und was er nicht aussagt. Das ist ein nicht zu unterschätzender Mehraufwand am Beginn des Projekts, nützt aber gerade in hektischen Testphasen und hilft dabei, Abweichungen frühzeitig zu erkennen und gezielt gegensteuern zu können.

Soll der Testreport zeigen, dass die Funktionalität gestiegen ist? Dann ist die Anzahl der implementierten Funktionen entscheidend. Ist eine hohe Testabdeckung von zentraler Bedeutung, um die Produktivität des Testteams nachzuweisen, weil das Testteam aufgestockt werden muss, weil Risiken aufgewiesen werden sollen? Dann ist die Anzahl durchgeführter Testfälle ein zentraler Punkt im Testreporting. Soll auf die zu erwartende Zuverlässigkeit des Produkts hingewiesen werden? Dann ist das Verhältnis der fehlgeschlagenen zu den durchgeführten Testfällen zu messen. Soll gezeigt werden, dass die Testabdeckung gestiegen ist? Dann empfiehlt es sich den Testfortschritt durch automatisierte Skripts in einer Metrik zu demonstrieren.

Dabei können unterschiedliche Verdichtungsstufen gewählt werden – einen groben Testüberblick für das Topmanagement und detaillierte Angaben auf den Ebenen darunter.

Allgemein sind die Gradmesser für den Projektfortschritt früh zu erstellen und die Erwartungen an den Projektfortschritt allen Beteiligten im Projekt am Anfang zu kommunizieren. Dann schafft man es eher, korrigierend einzugreifen als wenn die Termine schon hoffnungslos überschritten sind und die Kosten bereits davongelaufen sind.

25.2 Bewertung der Komplexität von Testfällen

Die Komplexität von Testfällen drückt sich in vier Komplexitätsmaßen aus:

- Testdatendichte
- Testdatenkomplexität
- Testdatenvolumen
- Testfallintensität

Im Folgenden werden die entsprechenden Formeln näher beschrieben.

Testdatendichte

$$\text{Testdatendichte} = \text{Testdatentypen} / \text{Testdaten}$$

Die **Halstead-Metrik** ist eine 1977 von Maurice Howard Halstead vorgestellte Softwaremetrik. Sie gehört zu den statischen, analysierenden Verfahren der Komplexitätsmessung von Software.

Hierbei wird die Systemkomponente nicht aktiv ausgeführt (wie bei den dynamischen Verfahren), sondern es werden gezielt Informationen über den Prüfling mit analytischen Mitteln gesammelt. Die Halstead-Metrik legt die Annahme zugrunde, dass ausführbare Programmteile aus Operatoren und Operanden aufgebaut sind. Die Definition, was die zu betrachtenden Operatoren und Operanden sind, ist dabei eine der Aufgaben vor dem Einsatz einer Halstead-Metrik. Es werden dabei für jedes Programm folgende Basismaße gebildet:

- Anzahl der verwendeten unterschiedlichen Operatoren n_1 und Operanden n_2 , zusammen die Vokabulargröße n .
- Anzahl der insgesamt verwendeten Operatoren N_1 und Operanden N_2 , zusammen die Implementierungslänge N .

Aus diesen Basisgrößen kann man verschiedene Kennzahlen berechnen.

- Schwierigkeit ein Programm zu schreiben bzw. zu verstehen, z. B. im Rahmen eines Code-Reviews:

$$D = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$$

$$D = (n_1 / 2) \times (N_2 / n_2)$$

- Aufwand:

$$E = D \times V$$

- Implementierungszeit:

$$T = (E / 18) \text{ Sekunden}$$

Die Testdatendichte ist an die Halstead-Metrik angelehnt und bezeichnet die Anzahl der unterschiedlichen Testdatentypen relativ zur Anzahl der Testdatenausprägungen. Je mehr Datentypen es gibt, umso schwieriger ist es, diese Datentypen zu erzeugen, ganz gleich, ob dies manuell oder maschinell geschieht. Sie müssen unterschiedlich behandelt werden. Andererseits ist es einfacher, aus einem Datentyp viele Instanzen zu generieren [GOLL2011].

Testdatenkomplexität

$$\text{Testdatenkomplexität} = \text{Steuernde Testdaten (Prädikate)} / \text{alle Testdaten}$$

Die **Chapin-Q-Metrik** betrachtet nicht nur Input und Output eines Moduls, sondern verwendet einen Faktor zur Gewichtung des Dateninputs im Modul und misst dadurch den Anteil der Bedingungs- und Ergebnisdaten von allen verwendeten Daten.

Dabei gibt es

- „P“ Daten: Daten zur Steuerung erforderliche Eingaben,
- „M“ Daten: Daten, die im Laufe bei der Programmdurchführung vom Modul verändert werden,
- „C“ Daten: Eingaben, die Entscheidungen oder Selektionen steuern,
- „T“ Daten: Daten, die unverändert im Modul verwendet werden.

Jedem dieser Datentypen wird dabei ein Gewichtungsfaktor zugeordnet, um die Komplexität der Kontrollstruktur zum Aufruf des Moduls anzuzeigen.

In diesem Zusammenhang wird die Testdatenkomplexität in Bezug auf das Verhältnis der steuernden Testdaten zur Summe aller Testdaten gemessen. Steuernde Testdaten bestimmen den Verlauf des Testfalls. Nur wenn die Variablen definierte Werte annehmen, schlägt das System den beabsichtigten Pfad ein. Deswegen müssen die Parameter, die in der Software Entscheidungen auslösen, bei der Testdurchführung gezielt dahingehend manipuliert werden, dass das erwartete Ergebnis eintritt. Je mehr Steuerdaten vorliegen, desto komplexer wird der Test.

Testdatenvolumen

$$\text{Testdatenvolumen} = 1 - \left(\text{Testfälle} / (\text{Argumente} + \text{Ergebnisse}) \right)$$

Die **Henrys-Interface-Metrik** misst die Anzahl der Zugriffe von fremden Funktionen/Methoden in ein Modul (englisch *fan-in*) beziehungsweise Anzahl der Aufrufe fremder Funktionen/Methoden aus einem Modul (englisch *fan-out*) relativ zu allen Funktionen/Methoden des Moduls. Das Testdatenvolumen ist in Anlehnung daran ein Maß für das Verhältnis der Ein- und Ausgabedaten zu den Testfällen. Jeder Testfall verlangt, dass ein oder mehrere Eingabedaten gesetzt werden. Diese Argumente bilden eine Input-Domain. Andererseits erwartet jeder Testfall ein oder mehrere Ausgabedaten bzw. Ergebnisse. Je mehr Argumente ein Testfall enthält, umso aufwändiger ist die Testdatengenerierung. Je mehr Ergebnisse ein Testfall besitzt, umso komplizierter gestaltet sich wiederum die Validierung der Testergebnisse. Die Komplexität eines Testfalls steigt mit der Anzahl seiner Ein- und Ausgaben.

Testfalldichte

$$\text{Testfalldichte} = 1 - \left(\text{Anwendungsfälle} / \text{Testfälle} \right)$$

Jeder Anwendungsfall muss mit mindestens zwei Testfällen nachgewiesen werden, mit einem für den positiven und einem für den negativen Ausgang. Je nachdem, wie komplex der Anwendungsfall ist, kann er beliebig viele Testfälle haben, einen für jeden Zustand und einen für jeden möglichen Ausgang. Je mehr Testfälle pro Anwendungsfall vorhanden sind, desto größer die Testfalldichte.

In diesem Zusammenhang können weitere Indexwerte ermittelt werden:

Architekturüberdeckung

Architekturüberdeckung = getestete Architekturelemente: Summe der Architekturelemente zur Messung der Übereinstimmung der fachlichen mit den technischen Problemen

Datenüberdeckungsmaß

Datenüberdeckungsmaß = Anzahl geprüfte Datenelemente: Summe aller Datenelemente zur Kontrolle von Ausgabemasken, Reports und Datenbanktabellen

Funktionsüberdeckung

Funktionsüberdeckung = getestete Anwendungsfallvarianten: Summe Anwendungsfallvarianten zur Analyse der Vollständigkeit der getesteten Funktionen

Anforderungsüberdeckung

Anforderungsüberdeckung = getestete Anforderungen : alle Anforderungen zur Messung der Vollständigkeit der Requirements durch Testfälle

Komplexitätsfaktor

Komplexitätsfaktor = Aggregierte Testfallkomplexität / Mittlere Testfallkomplexität

Die aggregierte Testfallkomplexität ist der arithmetische Mittelwert der einzelnen Komplexitätsmaße. Sie hat einen negativen Einfluss auf den Testaufwand. Der Komplexitätsfaktor zur Justierung der Testquantität ist die aggregierte Testfallkomplexität, dividiert durch die mittlere Testfallkomplexität = 0,5.

Testfallintensität

Je mehr Funktionen ein Test durchläuft, desto größer ist die Testfallintensität. Dabei gilt die Formel

$$\text{Testfallintensität} = 1 - (\text{Testfälle} / \text{Zielfunktionen})$$

Testfallqualität

Die Testfallqualität ist das gewichtete Mittel aus Testfallwiederholbarkeit, Testfallüberdeckung und Testfallkonformität.

Ein hoher Anteil an automatisierten Testfällen erhöht dabei die Wiederholbarkeit eines Tests.

Daher gilt

$$\text{Testfallwiederholbarkeit} = \text{automatisierte Testfälle} / (\text{automatisierte} + \text{manuelle Testfälle})$$

Eine hohe Testfallwiederholbarkeit bedeutet weniger nicht reproduzierbare Fehler. Fehler, die nicht reproduzierbar sind, sind deswegen besonders gefährlich, weil man die genauen Nebenbedingungen nicht kennt und sie deswegen einerseits nach einem einmaligen Auftreten im Systemtest oft nach Auslieferung erneut auftreten, andererseits einen hohen Analyseaufwand nach sich ziehen. Daher ist eine möglichst frühzeitige und möglichst umfassende Automatisierung der Testfälle auch so wichtig für den Projekterfolg.

Testfallüberdeckung

Die Testfallüberdeckung lässt sich durch das Verhältnis der spezifizierten Fachfunktionen zu den Zielfunktionen, also den Funktionen die explizit von den Testfällen durchlaufen werden, bestimmen.

$$\text{Testfallüberdeckung} = \text{Zielkonzeptfunktionen} / \text{spezifizierte Funktionen}$$

Diese Metrik kann speziell für den Unittest angewendet werden, um eine möglichst hohe Testbarkeit der einzelnen Module schon zu einer frühen Testphase zu prüfen:

$$\text{Testfallüberdeckung Unittest} = \text{Zielcodefunktionen} / \text{codierte Funktionen}$$

Testqualität

Die Testqualität sagt aus, wie viele Mängel der Test aufgedeckt hat. Die Qualität kann dabei in Unittests und Systemtest-Qualität unterteilt werden. Die Qualität der Tests wird demnach mit folgenden Formeln berechnet.

$$\text{Alle Mängel} = \text{Unittestmängel} + \text{Systemtestmängel} + \text{Produktionsmängel}$$

$$\text{Qualität des Unittests} = \text{Unittestmängel} / \text{alle Mängel}$$

$$\text{Qualität des Systemtests} = \text{Systemtestmängel} / \text{alle Mängel}$$

$$\text{Qualität des Tests insgesamt} = (\text{Unittestmängel} + \text{Systemtestmängel}) / \text{alle Mängel}$$

Die Unittestqualität sollte bei 0,5, die Systemtestqualität bei 0,4 und die Gesamttestqualität über 0,9 liegen. Das heißt es sollten maximal 10 % der bekannten Mängel Produktionsmängel sein, also vom Anwender bzw. im Rahmen der Abnahme entdeckt worden sein.

25.3 Problematik von Metriken

Man erwartet in der Regel bei einer Metrik immer einen Fortschritt im Testprozess. Anhand der Anzahl durchgeführter Testfälle lässt sich zum Beispiel die Effizienz der Testprozesse oder die Intensität der Arbeit bewerten.

Allerdings kann es sein, dass die Metrik keinen Fortschritt zeigt, und trotzdem wesentliche Erkenntnisse gewonnen wurden:

- Musste außerplanmäßig eine neue Version eingespielt werden?
- Musste der Test zwar abgebrochen werden, konnten aber behobene Fehler erfolgreich nachgetestet werden?
- Wurde der Testumfang erweitert?
- Ergaben sich aus dem Test wesentliche Erkenntnisse über ungeklärte Anforderungen, die in das Requirements Engineering einfließen können und die Systemdefinition erweitern und verbessern?
- Hat man durch den Test bemerkt, dass der Entwicklungsansatz falsch war oder die Systemarchitektur verändert werden muss?
- Hat man durch erneute Automatisierungen zwar Aufwand investieren müssen, aber dafür die Quantität und Qualität der Testfälle für die Zukunft erheblich verbessert, so dass man künftig eine höhere Testabdeckung (z. B. durch eine Kombination von weiteren Parametern), eine Vermeidung manuellen Aufwands bei der Testdurchführung oder ein exakteres automatisiertes Testreporting erhält?

25.4 Verwässerung von Metriken

Ich hatte in einem früheren Projekt, wo es um die Zertifizierung nach CMMI ging, mehrere Metriken für mehrere Teilprojekte erstellt, um den Fortschritt zu dokumentieren. Bei einer Messgröße bis 80 %, war das Projekt rot markiert, zwischen 80 und 95 %, gelb und bei einem Wert über 95 % grün.

Dann ging es darum, dass kein Indexwert rot werden durfte (denn dann hätte man das Management aufgeschreckt) und auch nicht grün (denn dann wäre ja alles OK, man könnte aufhören zu arbeiten, wollte aber natürlich nicht auf das schon genehmigte Budget verzichten und keinesfalls Stunden zurückgeben), also immer gelb mit einer leicht in Richtung grün ansteigenden Tendenz. Das verleitet natürlich dazu, solche Metriken zu fälschen: Man hält eigentlich schon positiv getestete Testfälle zurück, weil man sie ja zum Reporting später nochmals benötigt. Wenn bei Testdokumenten der Fortschritt nicht ausreichend ist, legt man ein weiteres Dokument an und schreibt nur die Überschrift in das Dokument – Hauptsache man kann irgendwie einen Fortschritt begründen. Meist wird sowieso nicht so genau nachgefragt.

Ganz extrem wird es, wenn erfolgreiche Arbeit mit der Anpassung von Werten in der Darstellung verschlechtert wird: Man erwartet bei einer gewissen Anzahl Testfälle einen gewissen Prozentsatz an Fehlern. Wenn nun die Programmierung sehr gut arbeitet, findet man nur wenige Fehler, was bei negativer Betrachtungsweise bedeuten kann, dass die Entwicklung zu viel Aufwand betrieben hat oder dass der Tester zu wenig geprüft hat oder zu unqualifiziert für seinen Job ist ... also kann man Personal bei der Entwicklung oder bei den Testern abbauen – man sollte sich immer in die Rolle des „Advocatus Diaboli“ hineinversetzen, bevor man Zahlen in einem Bericht weitergibt.

In vielen Unternehmen existiert eine projektbezogene Stundenschreibung der Mitarbeiter. Wenn man 100 einfache Testfälle durchführt, die nur je fünf Minuten dauern, ist man natürlich schneller fertig als wenn man nur zehn komplexe Testfälle durchführt, die aber

je zwei Stunden dauern. Im Reporting schnellst aber die Erfolgsquote bei den 100 kurzen Testfällen zu stark nach oben und bei den zehn komplizierten Testfällen zu langsam. Die Messlatte wird dann bei den kurzen Testfällen angelegt, es empfiehlt sich also aus diesem Aspekt kurze und länger dauernde Testfälle in einer bestimmten Periode zu kombinieren, was allerdings vom Aspekt der zu testenden Funktionalität völlig sinnlos sein kann.

Bei einer Metrik, die keine stetige Kurve nach oben zeigt, oder wo die Kurve gegenüber früheren Perioden abflacht oder wo die geplanten Ziele nicht erreicht werden, wird das Management leicht nervös. Der Berichterstatter wird also dazu verleitet, die Ergebnisse positiv darzustellen, also möglichst viele einfache Tests zuerst durchzuführen oder dann wenn es mal besonders gut läuft die positiv getesteten Testfälle zurückzuhalten. Dazu kommt, dass beim Reporting nach oben immer weniger von den Problemen ankommt, denn jede Reportingebene will sich selber gut verkaufen und ist dazu geneigt, etwas besser zu berichten als der wirkliche Zustand des Softwareprodukts tatsächlich ist. Am Ende wundert man sich dann, dass doch immer alle Parameter in die richtige Richtung gezeigt haben, das Projekt aber dennoch gescheitert ist. Auch die letzten paar Prozent des Projekts werden meist unterschätzt: man denkt man sei schon zu 95 % fertig, übersieht aber, dass die Besonderheiten weit mehr Zeit erfordern, dass die Dokumente zeitaufwändigen Reviews unterzogen werden müssen und vernachlässigt den Korrekturaufwand.

Dazu kommt, dass man die Metriken auch richtig interpretieren muss. Erst vor kurzem ist es mir selber so ergangen, dass ich in einer Fehlermetrik zwar immer die Anzahl der noch offenen Fehler berichtet hatte, aber die Tragweite für die Fehlerbehebung unterschätzt hatte. Als dann am Ende des Projekts noch zahlreiche Fehler gefunden wurden, ergab sich plötzlich ein Terminverzug von mehreren Wochen.

Mit Metriken kann man schließlich auch Aussagen so verfälschen, dass sie einem politischen Ziel dienen. Der Satz „Traue keiner Statistik, die du nicht selbst gefälscht hast“ bewahrheitet sich hier immer wieder.

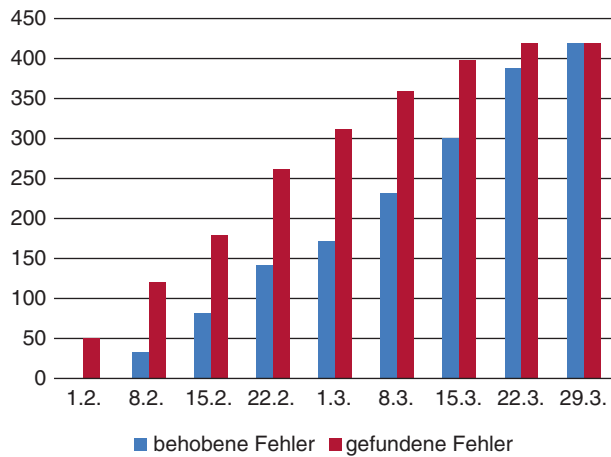
Abb. 25.1 und Tab. 25.1 zeigen ein Beispiel für eine Fehlermetrik. Es verdeutlicht die Entwicklung der beim Systemtest entdeckten und behobenen Fehler in einem bestimmten Zeitraum.

25.5 Metriken, bei denen man (fast) nur verlieren kann

Es gibt Metriken, die problematisch sind, weil man die Ergebnisse fast immer dem Tester negativ auslegen kann. Ein Beispiel dafür sind Metriken, die Fehler prognostizieren („Error-Forecast-Metriken“). Man kann aufgrund der Anzahl an Codezeilen Vorhersagen treffen, wie viele Fehler man bis zu welchem Zeitpunkt erwartet. Dieses Verfahren birgt aber mehrere Probleme:

- Wenn die Anzahl der Fehler höher ist als erwartet, bedeutet das entweder, dass die Qualität der Entwicklung zu schlecht ist oder intensiver getestet wurde als vorgesehen – dass also dem Softwaretest zu viele Ressourcen zugestanden wurden. Beides ist für die Stakeholder demotivierend.

Abb. 25.1 Fehlerstatistik



Tab. 25.1 Fehlerstatistik

Fehlerstatistik	1.2.	8.2.	15.2.	22.2.	1.3.	8.3.	15.3.	22.3.	29.3.
Behobene Fehler	0	30	80	140	170	230	300	390	420
Gefundene Fehler	50	120	180	260	310	360	400	420	420

- Wenn die Anzahl der Fehler niedriger ist als erwartet, bedeutet das entweder, dass die Qualität der Entwicklung zu gut ist (also die Entwicklung zu teuer) oder nicht genügend getestet wurde – dass also der Softwaretest nicht exakt genug gearbeitet hat und Funktionen nicht ausreichend getestet hat oder gewisse Punkte übersehen hat.

Das Dilemma ist hierbei, dass der Entwickler möglichst fehlerfreie Software ausliefern will, der Tester aber möglichst viel Fehler finden will. In dieser Hinsicht ist der Softwaretest vordergründig eine eher destruktive Tätigkeit, die aber für den Projekterfolg und die Softwarequalität unbedingt erforderlich ist. Es ist aber problematisch, wenn in einer Kenngröße unterschiedliche Interessenlagen vermengt werden.

Dass der Forecast genau getroffen wird, ist sowieso unwahrscheinlich. Für diesen positiven Fall der nur in vielleicht 0,1 % aller Fälle auftritt hätte man die Metrik im Vorfeld gar nicht benötigt. Das soll aber im Umkehrschluss nicht bedeuten, dass Metriken generell sinnlos sind – im Gegenteil, wenn Tatsachen messbar gemacht werden, gewinnt man eine umfassende fundierte Aussage über den tatsächlichen Zustand des Testobjekts, das nicht aufgrund von Stimmungen und Emotionen beeinflusst ist und kann besser fundierte Entscheidungen auf der Grundlage von Zahlen treffen.

Man kann daher einen solchen Indexwert trotzdem als Gradmesser für Entwicklungs- und Testqualität heranziehen, dann muss aber jedem im Projekt und im Management bewusst sein, welche Auswirkungen mit den Aussagen verbunden sind. Es zeigt sich, dass Metriken helfen, den Projekterfolg zu bewerten, dass dabei aber immer mehrere Metriken und die sinnvollsten Indexwerte richtig miteinander in Beziehung gesetzt werden müssen, um ein realistisches Bild der Lage zu erhalten und weder Entwickler noch Tester als unfähig erscheinen zu lassen.

25.6 Aussagen unterschiedlicher „Key Performance Indicators“

Die Testabdeckung beschreibt, wie sicher Sie sind, eine möglichst umfangreiche Menge von Tests durchgeführt zu haben. Die Fehler pro Testfall beschreiben die Effizienz des Softwareprojekts.

Für Fehler kann es unterschiedliche Ursachen geben:

- zeitliche Probleme,
- unklare Requirements,
- Aufgabenstellung nicht verstanden,
- Entwickler nicht genügend geschult,
- Entwicklungsmethode neuartig,
- hohe Komplexität,
- Abhängigkeiten,
- Verschiebung von Prioritäten.

Am Anfang des Entwicklungsprozesses hat man noch wenig getestet, also auch wenig Fehler gefunden. Wenn das Projekt schon ein paar Monate läuft und die Tests umfassend durchgeführt sind, sind die Fehler wesentlich höher als am Anfang. Trotzdem ist die Software ja nicht schlechter geworden, sondern hat sich sogar weiterentwickelt. Daher muss man immer die Testabdeckung mit einbeziehen.

Siehe Beispiel in Tab. 25.2: Am Anfang in Version 1 konnten nur wenige Testfälle getestet werden, weil die Software nur wenige Features beinhaltete.

Dass beim Test der Version 4, die bereits einen kompletten Funktionsumfang hatte, dann 200 Fehler gefunden wurden, sagt nicht aus, dass die Version 4 um das 20-fache schlechter war als Version 1.

Nicht jeden Fehler gleich bewerten

Angenommen, Version 3 hatte 15 Fehler. 10 Fehler davon bezogen sich auf Rechtschreibfehler, ein fehlendes Komma oder Formatierungen im Text. Bei Version 4 kann die Software nach der Installation nicht gestartet werden. Da man also gar nicht mit dem Test starten kann, hat folgerichtig die Software nur einen Fehler ... Ist deswegen ein bedeutender Entwicklungsschritt vollzogen worden? Nein, es ist in jedem Fall zunächst ein gewaltiger Rückschritt. Das bedeutet, es ist neben der Anzahl immer auch die Schwere der Fehler zu bewerten.

Es empfiehlt sich (vor allem bei großen Projekten) auch für die Fehlerintensität Metriken einführen, denn ob in einem Hilfetext ein Komma fehlt, ist weit weniger

Tab. 25.2 Testabdeckung und gefundene Fehler

Version	1	2	3	4	5
Getestete Testfälle	50	100	200	500	500
Gefundene Fehler	10	30	100	200	50
Umfang der Software	30 %	50 %	80 %	100 %	100 %

wichtig, als ob die Software gar nicht startet. Das fehlende Komma im Hilfetext hätte in diesem Beispiel Faktor 1 und der Fehler der die Software gar nicht erst starten lässt den Faktor 1000. So bekommt man ein realistischeres Bild von der vorhandenen Softwarequalität.

Dann bewertet man, welche Faktoren ausschlaggebend waren und konzentriert sich auf die Behebung der Fehler. Wichtig ist, dass man hierzu ein gemeinsames Verständnis entwickelt und nicht einseitig Randprobleme optimiert, obwohl die Hauptfehler woanders liegen.

Den Testaufwand mit geeigneten Parametern bewerten

Allgemein gilt folgende Formel für die Berechnung des Testaufwands :

$$\text{Testaufwand} = (\text{Testfälle} \cdot \text{Testfallkomplexität}) / (\text{Testproduktivität} \cdot \text{Testfallqualität}).$$

Die Testproduktivität ist dabei die Anzahl der durchgeführten Testfälle pro Personentag.

Diese Größe kann individuell stark abweichen und hängt u. a. vom individuellen Kenntnisstand der Tester, von der Komplexität der Software, dem Auslastungsgrad, der Schnittstellen für die Anwendung und der internen Abläufe und der Motivation und Produktivität der Tester ab.

Voraussetzung für valide Daten ist dabei, dass diese Parameter für sich ermittelt wurden und genügend belastbare Daten vorhanden sind, um sie eindeutig zu ermitteln.

Wenn wenige Fehler gefunden wurden, kann das auch heißen, dass das Testobjekt (noch) nicht hinreichend getestet wurde. Die Verfahrensanweisungen zum Test (Testprozeduren) sind meist zu Anfang eines Projekts noch nicht fertig.

Beispiel

Die fertige Testdokumentation umfasst 1000 Testfälle. Die Testdokumentation wird vom 15.1. bis 28.2. erstellt. Es gibt 3 Software-Versionen, zum 10.2., zum 10.3. und zum 10.4.

Am 10.2. wird bereits die erste Softwareversion dem Test zur Verfügung gestellt. Zu diesem Zeitpunkt sind aber erst 500 Testfälle komplett beschrieben. Wenn nun die Anwendung mit diesen 500 Testfällen komplett getestet wird, erreicht man trotzdem nur eine Testabdeckung von 50 %. Nehmen wir weiter an, beim Test 500 Testfälle im ersten Release im Zeitraum vom 10.2. bis 20.2. werden 100 Fehler gefunden.

Am 10.3. wird termingerecht die zweite Software-Version zum Test bereitgestellt. Zu diesem Zeitpunkt sind alle 1000 Testfälle komplett beschrieben und werden auch komplett getestet. Beim Test der 1000 Testfälle im zweiten Release werden weitere 100 Fehler gefunden (der Einfachheit halber sind in dem Beispiel die 100 Fehler der ersten Version noch nicht verbessert). Das bedeutet dann natürlich nicht, dass die Software schlechter geworden ist, sondern einfach nur, dass wegen einer höheren Testabdeckung und einer Weiterentwicklung der Funktionalitäten ein umfangreicherer und detaillierterer Softwaretest möglich war.

Ähnlich verhält es sich, wenn bestimmte Features noch gar nicht vorhanden sind. Wenn im Release zum 10.3. erst 5 von 10 Funktionen vorhanden sind, die auch alle wie spezifiziert komplett getestet wurden, zum nächstfolgenden Release am 10.4. aber alle 10

Funktionen die dann ebenfalls komplett getestet werden, dann hat sich die Testabdeckung bezogen auf die vorhandene Software nicht geändert. Die Testabdeckung die sich auf den gesamten Funktionsumfang bezieht hat sich aber verdoppelt.

Das alles ist in die Interpretation des Testergebnisses einzubeziehen. Es geht also darum, die geeigneten Metriken in Beziehung zu setzen und daraus korrekte Schlussfolgerungen zu ziehen. Entscheidend ist daher, die einzelnen Key Performance Indicators entsprechend zu gewichten und zu prüfen, welcher für das Testreporting zu welchem Zeitpunkt geeignet ist. Ansonsten kommt man schnell zu einer verzerrten Wahrnehmung der Wirklichkeit und zieht daraus falsche Schlüsse für das weitere Vorgehen.

Das „magische Dreieck“: Kosten – Qualität – Termine

Bei gegebener Funktionalität muss das Projektmanagement immer zwischen Kosten, Qualität und Zeit bzw. Terminen abwägen. Das sich daraus ergebende Dreieck (siehe Abb. 25.2) wird als „magisches Dreieck“ bezeichnet, da die Optimierung eines Kriteriums zu Lasten der anderen geht [PROJ2018].

Softwaretests sind dafür da, die Qualität zu verbessern, wirken sich aber tendenziell eher negativ auf Kosten bzw. Termine aus.

Dem Testmanager obliegt hier die Aufgabe, darüber zu wachen, dass die Qualität so gut wie möglich eingehalten wird und zu warnen, wenn die Qualität nicht mehr hinreichend gewährleistet werden kann. Dennoch bleibt es immer Aufgabe des Projektmanagements darüber zu entscheiden, ob Testressourcen ausgebaut und Testaktivitäten erweitert werden. Ein Schlüssel dabei ist aber auch, die Methoden effizient einzusetzen und die geeigneten Maßnahmen zu definieren. Man kann Software so testen, dass sie höchste Qualitätsansprüche erfüllt und trotzdem die Kosten eben nicht aus dem Ruder laufen, indem man die Testverfahren richtig auswählt, die Prozesse in sich schlüssig definiert und die richtigen Testfälle automatisiert testet.

Das Testreporting soll so angelegt sein, dass zur erforderlichen Testabdeckung eine Empfehlung zur Auslieferung der Software ausgesprochen werden kann. Die letzte Entscheidung, ob die Software tatsächlich ausgeliefert wird oder nicht, obliegt aber immer dem Management.

Abb. 25.2 Das „magische Dreieck“ Zeit – Kosten – Qualität



25.7 Beispiel für eine Fehlermetrik

Tab. 25.3 und Abb. 25.3 zeigen ein Beispiel für eine Fehlermetrik.

Tab. 25.3 Fehlermetrik

	1505	1506	1507	1508	1509	1510	1511	1512
Testfälle in Summe	250	250	250	250	250	250	250	250
Ursprüngliche Planung	40	70	110	150	190	220	240	250
Aktuelle Planung	40	70	105	130	180	220	240	250
Fehlgeschlagene Testfälle	0	0	10					
Erfolgreiche Testfälle	38	65	95					
Fehler mit Priorität „Hoch“	0	2	2					
Fehler mit Priorität „Mittel“	0	2	3					
Fehler mit Priorität „Niedrig“	0	2	5					

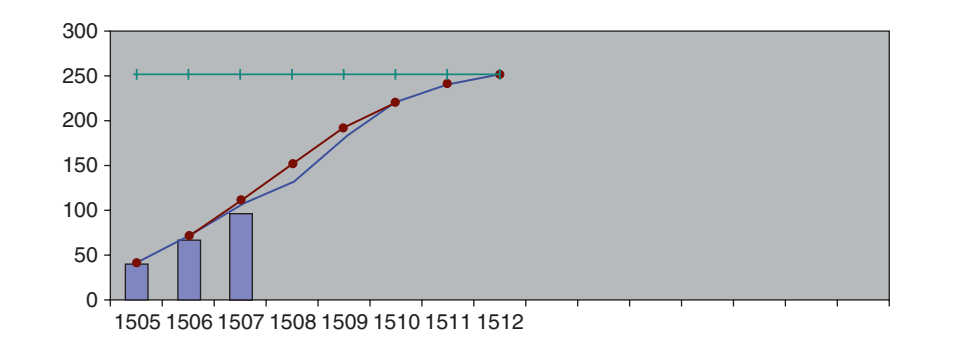


Abb. 25.3 Fehlermetrik

Literatur

[PROJ2018]

www.projektmanagementbau.de

[GOLL2011]

Goll J (2011) Methoden und Architekturen der Softwaretechnik. Vieweg & Teubner, Wiesbaden

[HAHS2014]

<http://michael.hahsler.net/stud/done/nest/Diplomarbeit.pdf>



Tests, die wiederholt ablaufen, können automatisiert werden. Das verhilft zu höherer Testabdeckung, einer verlässlichen Reproduzierbarkeit, spart Zeit und Kosten. Wenn man 1000 Kombinationen von Testfällen manuell testen will, ist man damit ein paar Wochen beschäftigt. Ein Roboter der dafür ein Skript abarbeitet, schafft das in wenigen Stunden.

Sich wiederholende, stupide Testtätigkeiten lässt man sicherer und günstiger mit einem automatisierten Skript ablaufen und setzt den Faktor Mensch gezielt dort ein, wo er einen wesentlichen Beitrag liefern kann und jedem Testroboter überlegen ist: beim kreativen Design auf Basis bereits ausgeführter Testfälle, beim explorativen Testen, bei der Ermittlung zusätzlicher Testabdeckung.

Die Automatisierung betrifft dabei in der Regel die Testdurchführung. In Einzelfällen können auch die Erstellung der Testfälle und das Testreporting automatisiert ablaufen, eine abschließende Interpretation und Bewertung der Ergebnisse wird aber immer der Testexperte durchführen müssen.

Der Aufwand für Qualitätssicherung steigt mit der Komplexität der Anwendung. Manchmal ist es rein zeitlich gar nicht anders möglich, um zahlreiche Kombinationen abzutesten als mit einer effizienten Automatisierung, den nötigen Qualitätsbedarf zu erreichen.

Wenn man prüfen möchte, ob alle geplanten Änderungen fachlich und technisch korrekt umgesetzt sind, steht man vor dem Dilemma, entweder einen hohen manuellen Aufwand zu erzeugen oder auf Qualität zu verzichten. Für den manuellen Aufwand fehlen oft die Softwaretester, und selbst wenn genügend Personal vorhanden ist, rechnet sich ihr Einsatz ab einer bestimmten Zahl von Testfällen nicht mehr.

Dazu kommen enge Zeitfenster zur Durchführung der notwendigen Testaktivitäten. Es ist ja nicht nur die reine Anzahl an Mitarbeitern, die oft einen Engpass bedeutet, sondern ihre zeitliche Verfügbarkeit. Gerade wenn eine bestimmte Testmannschaft mehrere Projekte zu testen hat, besteht manchmal das Problem, dass zu einigen Phasen die verfügbaren Tester zu 300 % und zu anderen nur zu 40 % ausgelastet sind. Neben der Verfügbarkeit der einzelnen Mitarbeiter kommt die Verfügbarkeit der Testanlagen hinzu.

Eine besondere Herausforderung besteht dann, wenn die Tester in verschiedenen Projekten mitwirken und sich in einzelnen Projekten die Termine verschieben. Diese Terminverschiebungen können dann zu Domino-Effekten in anderen Projekten führen und schließlich alle Projekte gefährden. Gerade bei mehreren parallelen Projekten die auf dieselben Ressourcen (Personal, aber auch Hardware, zugreifen) ist daher eine fundierte Projektplanung unbedingt erforderlich. Gerade wenn man bei der Hardware knapp ist, geschieht es schon mal, dass eine Testanlage „zerpflückt“ wird und dann zur Fortführung der Testaktivitäten zu einem späteren Zeitpunkt die ursprünglich eingesetzte Hardware nicht mehr zur Verfügung steht. Dann ist natürlich auch keine Vergleichbarkeit der Testergebnisse mehr gegeben.

26.1 Chancen und Risiken der Testautomatisierung

Die Durchlaufzeiten der Testzyklen lassen sich mit geeigneten Programmen zur Testautomatisierung erheblich verkürzen. Testdaten und Testfälle sind wieder verwendbar und können auch von verschiedenen Mitarbeitern, auch im Outsourcing-Betrieb, durchgeführt werden.

Selbst umfassende Portfolios von Testfällen lassen sich mit überschaubarem Aufwand einfach umsetzen und durchführen. Gerade bei zahlreichen Releases mit immer wiederkehrenden Tests bestimmter Basis-Features rechnen sich Maßnahmen zur Testautomatisierung schon nach wenigen Zyklen. Testautomatisierung verhilft also in erster Linie zu höherer Qualität und besseren Ergebnissen.

Wenn man z. B. eine Tabelle mit Ländercodes hat, die man in einem Webportal eingeben kann, macht es wenig Sinn, dass der Tester alle nacheinander manuell eingibt und einzeln die Ergebnisse abhakt. Das kann ein programmiertes Skript wesentlich besser, zuverlässiger und schneller erledigen.

Bevor man automatisierte Testverfahren einführt, müssen aber standardisierte Abläufe vorhanden sein. Daran scheitert die Automatisierung in vielen Fällen. Man benötigt rückverfolgbare und wiederholbare Testverfahren, die vom individuellen Profil des Softwaretesters weitgehend unabhängig sind. Automatisierte Testverfahren eignen sich besonders, wenn die Testdurchführung zeitlich getaktet ist und die Testfälle ständig bzw. häufig durchgeführt werden [BUCS2015].

Automatisierung rechnet sich nicht, wenn die Tests nur einmalig oder seltene Male durchzuführen sind bzw. die Änderungen zwischen den Versionen starke Auswirkungen auf die Automatisierung haben. Vor allem gilt dies bei Änderungen in der graphischen Benutzeroberfläche. Schon in vielen Projekten ist es zu geringfügigen nachträglichen Änderungen der Benutzeroberfläche gekommen, die auf einen Schlag wochenlange Testautomatisierungsarbeiten zunichte gemacht und sogar Projekttermine verschoben haben.

Automatisierung bedeutet auch nicht, dass danach der gesamte Testprozess auf Knopfdruck funktioniert. Man sollte also zunächst nicht die Hoffnung zu haben, man mache damit den Tester überflüssig. Auch in einer industriellen Fertigung gibt es neben

Tab. 26.1 Manuelle und automatisierte Testdurchführung

Kriterium	Manuelle Testdurchführung	Automatisierte Testdurchführung
Testdurchführung	Langsam und kostenintensiv	Schnell und billig
Aufwand bei Re-Test für Fehlerbehebungen	Hoch	Niedrig
Möglichkeit, Testergebnisse während der Durchführung zu interpretieren	Ja	Nein
Exakt reproduzierbar	Ja	Nein
Flexibilität der Tester	Abnehmend	Konstant
Motivation der Tester	Abnehmend	Konstant
Anwendbarkeit für Scrum (kontinuierliche Integration)	Nein	Ja
Definition der Testfälle	Beschreibung	Beschreibung und Skripterstellung
Zeit für Testvorbereitung	Niedrig	Hoch

automatisierten Abläufen immer wieder manuelle Tätigkeiten. Wenn eine Fertigungsstraße geändert oder neu eingerichtet werden muss, sind oft umfangreiche und zeitaufwändige Vorarbeiten erforderlich bis der Roboter für die Massenproduktion loslegt.

Das Starten der einzelnen Testläufe, die richtige Konfiguration der Testruns und die nachträgliche Dokumentation bzw. Ablage der Testergebnisse und die Interpretation der Testergebnisse verbleiben in jedem Fall beim Tester. Diesen Aufwand darf man nicht unterschätzen. In diesem Zusammenhang ist es auch wichtig, wie gut der Scheduler für die Skriptprogramme ausgereift ist.

Die Tab. 26.1 zeigt in der Übersicht die Unterschiede manueller Testausführung und automatisierter Softwaretests.

26.2 Vorgehen bei der Testautomatisierung

Sinnvollerweise sucht man sich zunächst die Testfälle, die leicht zu automatisieren sind. Dazu ermittelt man Testfälle, die eine hohe Priorität haben und die sich bei der Testdurchführung als sehr aufwendig erweisen (leider sind oft die Testfälle, die sehr aufwendig in der Durchführung sind, auch schwer zu automatisieren).

Die Testfälle fasst man in einer Tabelle zusammen (siehe Tab. 26.2). Dann entscheidet man anhand dieser Übersicht über die Priorität und das Vorgehen bei der Automatisierung der Testfälle. Testfälle, deren Aufwand zur manuellen Testdurchführung hoch, deren Aufwand zur Testautomatisierung gering und deren Priorität hoch ist, haben dabei den höchsten Nutzen und sollten vordringlich mit automatisierten Skripts umgesetzt werden.

Bei der Testautomatisierung kommt ein „bestanden“ oder „nicht bestanden“ heraus. Dabei sind jeweils die Prüfkriterien festzulegen: Zum Beispiel Füllen einer Tabelle, zusätzliche

Tab. 26.2 Priorisierung der Testautomatisierung

Testfall (ID)	Aufwand zur manuellen Testdurchführung	Aufwand zur Testautomatisierung	Nutzen automatisierter Tests	Priorität	Fazit
1					
2					
3					

Angaben wie Zeitstempel, ID usw. Man muss dabei auch daran denken, was passiert wenn das Skript abbricht: Fehleingaben verhindern z. B. die Abarbeitung des Programmteils, es gibt einen undefinierten Abbruch und das Programm steht. Zum Beispiel waren 1000 Durchläufe für den Nachlauf geplant doch nach dem vierten Programmdurchlauf stieß das Programm auf einen Fehler und der Test brach ab.

Auch ein Testprogramm braucht eine Testphase, man muss die Ergebnisse vergleichen und den Algorithmus anpassen. Manchmal sind mehrere Automatisierungsschritte nacheinander erforderlich und es empfiehlt sich, zunächst nur einen Teilablauf zu automatisieren und danach eine größere und komplexere Prozesskette.

Am besten ist es, immer wieder neu zu beginnen, also den definierten Ausgangszustand vor dem Test herzustellen und das evtl. zeit- oder ereignisgesteuert zu machen.

26.3 Planung der Testautomatisierung

Manchmal soll es die Testautomatisierung im letzten Augenblick richten. Da werden dann Skripts programmiert, ohne den Test jemals inhaltlich und gedanklich in letzter Konsequenz durchdrungen zu haben und ohne alle Schritte manuell durchgeführt zu haben. Wenn man aber einfach nur drauf los codiert, erreicht man nicht unbedingt den gewünschten Erfolg.

Das V-Modell gilt in seinen Grundzügen auch für das Automatisieren von Tests. Man muss die genauen Abfolgen nacheinander abarbeiten, jeden Schritt bedenken und dokumentieren. Nur so kann das Skript später gepflegt, angepasst, weiter optimiert werden.

Für das Design der Testautomatisierung muss ebenfalls Know-how vorhanden sein. Notwendig ist weiterhin, dass andere Projektbeteiligte (Management, Entwickler, Tester, Fachbereich) von der Testautomatisierung überzeugt sind und die Prioritäten entsprechend gesteuert werden und die notwendigen Ressourcen zur Verfügung gestellt werden. Die Analyse der Spezifikationen für das automatisierte System fehlt erfahrungsgemäß in den meisten Fällen, so dass Ziel und Umfang des Automatisierungssystems missverstanden werden und dadurch die Entwicklung des Automatisierungssystems an den Erfordernissen des Softwaretests vorbegeht.

Da man im Life Cycle der Software den Automatisierungsgrad vergrößern will, ist von Anfang an eine geeignete Systemarchitektur für die Testautomatisierung erforderlich. Erfahrungsgemäß sollte an dieser Stelle ausreichend Know-how aufgebaut werden.

26.4 Voraussetzungen für automatische Testdurchführung

Wie bereits beschrieben gibt es drei wesentliche Anschlusspunkte der automatischen Testdurchführung:

- Zugriff des Tools auf Schnittstellen der Applikation, die getestet werden soll,
- Schemata im Ablauf der Testfälle,
- ein gewisser Reifegrad der zu testenden Anwendung und ihrer Umgebung.

Diese Punkte sehen auf den ersten Blick recht unempfindlich aus: Schnittstelle ist die Benutzeroberfläche, von der erwartet wird, dass sie einerseits bedienbar und andererseits auch logisch und konsistent ist. Schemata im Ablauf der Testfälle sind eine weitere, die Usability unterstützende Eigenschaft, die von vielen Applikationen ohnehin erfüllt werden soll.

Allerdings ist hier leider nicht alles Gold, was glänzt: Gerade in diesen zwei Bereichen sind einerseits Tools sehr empfindlich, andererseits sind sie der Fokus von Weiterentwicklungen von Software.

Graphische Neugestaltungen von Applikationen etwa, die an der eigentlichen Funktionalität nichts ändern, verursachen Wartungsaufwand für Automatisierer oder Erweiterungen um Schnittstellen für die Automatisierung durch die Entwickler. Selbiges gilt für Umgestaltungen der Abläufe in der zu testenden Applikation. Soll die Automatisierungssuite nachhaltig und über einen längeren Zeitraum eingesetzt werden, kann hier erheblicher Wartungsaufwand entstehen. Verletzungen des dritten Punktes treten zumeist zusammen mit Verletzungen der ersten zwei Punkte auf, allerdings kann auch eine Applikation, deren Gestaltung und Ablaufdesign als fertig betrachtet wird, an mangelndem Reifegrad leiden. Daraus resultiert, dass die Protokolle von automatischen Testdurchläufen wenig aussagekräftig sind, da die Fehler die Ausführung vieler Testfälle verhindern. Eine weitere Konsequenz besteht darin, dass das Auftreten der aufgezeigten Fehlerwirkungen zu häufig für eine sinnvolle Auswertung oder nicht reproduzierbar ist.

26.5 Codenahe und GUI-Testautomatisierung

Die codenahe Testautomatisierung sollte in einer frühen Projektphase angesetzt werden. Dann lassen sich Fehler in den einzelnen Komponenten relativ schnell finden, und die Fehler pflanzen sich nicht über Integrationsschritte auf die Systemebene fort. Testautomatisierung für die graphische Benutzeroberfläche sollte eher spät im Projektverlauf durchgeführt werden, denn oft gibt es an den Masken nach den ersten GUI-Tests noch Änderungen.

Meist wird der Fehler gemacht, die GUI frühzeitig und die codenahe Automatisierung spät im Projektzyklus zu starten. Das ist einer der wesentlichen Gründe, warum es mit der Testautomatisierung dann nicht wirklich vorangeht und der Automatisierungsgrad auf einem niedrigen Level verharret [COMP2012].

Testautomatisierung setzt auch eine konsequente Testdatenverwaltung voraus. Die Testautomatisierung verlangt eine gegebene Konfiguration zu Beginn der Testaktivitäten.

26.6 Untersuchungen vor der Automatisierung

Testautomatisierung zahlt sich nicht immer sofort im ersten Projekt aus. Langfristig gesehen bringt sie aber in jedem Fall eine höhere Qualität und eine Kosteneinsparung. Ob eine Testautomatisierung im speziellen Fall überhaupt sinnvoll ist, liegt an der Komplexität der Umsetzung, an der Häufigkeit der Durchführung und der Lebensdauer des Projekts.

Bei der Analyse zur Automatisierung von Testfällen müssen beide Größen berechnet und bewertet werden: Aufwand und Kosten für manuellen Test gegenüber Aufwand und Kosten der Testautomatisierung. Dabei ist zu bedenken, dass sich der Erfolg von Automatisierungsmaßnahmen oft erst mittelfristig auszahlt und dass auch bei Testautomatisierung nach wie vor manueller Aufwand übrigbleibt, nur eben in geringerem Maße.

Testautomatisierung lohnt sich dann am meisten, wenn sich der Code nur selten ändert. Eine schnelle Abfolge in den Releasezyklen, eine Verkürzung des Time-to-Market und eine Reduzierung von Testbudgets betreffen immer auch die Testautomatisierung. Die Umstellung von klassischen auf agile Vorgehensweisen, die gerade in den letzten Jahren in vielen Unternehmen und Projekten durchgeführt wird, erfordert eine vorhandene Testautomatisierung und regelmäßige Regressionstests. Es muss genügend Geld und Zeit vorhanden sein, Änderungen aus der Entwicklung in den Testskripten nachzubilden. Bei der Planung von Erweiterungen und Anpassungen der Software ist unbedingt die Testautomatisierung mit einzubeziehen. Problematisch wird es bei Großprojekten, wenn die Testautomatisierung ein Eigenleben entwickelt und sich nicht genügend mit dem Entwicklungsprozess abstimmt. Das kriegt man dann manchmal erst mit, wenn ein Skript was monatelang problemlos lief plötzlich auf einen Fehler läuft.

26.7 Ablauf der Testautomatisierung

Im Allgemeinen wird eine spezialisierte Entwicklungsumgebung für eine Skriptsprache benutzt. Die Tools greifen auf Schnittstellen der zu testenden Anwendung zu. Manchmal sind dies Komponenten der Benutzeroberfläche (Buttons, Textfelder, Tabellen usw.). Ein bestimmter Prozess wird dann aufgezeichnet und in Skripte umgesetzt, die mittels Spreadsheets oder Datenbanken angesteuert werden können. Auch wenn viele Toolhersteller damit werben, dass keine Programmierkenntnisse Tests automatisiert werden können, sollte man doch bedenken, dass die optischen Darstellungen mit Hilfe einer graphischen Oberfläche letztlich Programmcode sind. Wenn während der Testautomatisierung ein Fehler auftritt, muss man immer untersuchen, ob der Fehler an der zu testenden Applikation, oder am Skript liegt.

Ein Tester, der die Rolle des Testautomatisierers einnimmt, muss Prozesse und Methoden im Testbetrieb kennen und über Erfahrung in der Entwicklung von Softwaresystemen verfügen. Es soll als Ergebnis schließlich ein stabiler, wartungsfreundlicher und effizienter Code rauskommen.

Oft fehlt es an Unterstützung durch andere Projektbeteiligte. Management, Entwickler, Tester und Fachbereich müssen von der Sinnhaftigkeit der Automatisierung überzeugt sein. Der Automatisierer muss genügend Zeit bekommen, um sich mit der zu testenden Applikation fachlich und technisch mit ausreichendem Tiefgang auseinanderzusetzen.

Analysen der Anforderungen oder Spezifikationen für das automatisierte System fehlen oft vollständig, wenn man von den Testfällen absieht, deren Automatisierung geplant ist. Auch dieser gut abzuschätzende Aufwand darf nicht außer Acht gelassen werden, sonst besteht die Gefahr, dass Zweck und Umfang des Automatisierungssystems missverstanden werden, und somit die Entwicklung des Automatisierungssystems an den eigentlichen Bedürfnissen des Tests vorbeigeht.

Ein weiteres Problem, das aus der Softwareentwicklung bekannt ist, kann hier auftreten. Erfahrungsgemäß wird die Komplexität des Vorhabens selbst unterschätzt. Was als einfaches System mit ein oder zwei Proof-of-Concept Testfällen beginnt, wächst oft sehr schnell und Teile des Systems finden Wiederverwendung in anderen Bereichen. Testautomatisierungssysteme fordern ihre Erweiterung ein: Gefundene Fehler im automatischen oder manuellen Test oder gar im produktiven Einsatz fordern dann neue Testfälle, die ebenfalls automatisiert werden sollen. Um das System ausreichend skalieren und erweitern zu können, ist eine wohlüberlegte Architektur notwendig. Gerade im Bereich der Testautomatisierung gibt es hierfür einige recht flexible Ansätze, deren Anwendung in Betracht gezogen werden sollten.

26.8 Testergebnisse bei automatisierten Tests

Da die Dokumentation der Testergebnisse und die Auswertung automatisierter Testläufe ebenfalls aufwändig ist, sollte von Anfang an das Reporting ebenfalls automatisiert ablaufen, z. B. indem die Ergebnisse automatisch in eine Tabelle übertragen werden, dort sortiert und nach vorhandenen definierten Algorithmen standardisiert interpretiert werden können. Eine gleichartige Struktur der Ergebnisdateien hilft bei der Auswertung der Testergebnisse enorm weiter, sonst hat man zwar erfolgreich Tests mit Skripten automatisiert ist aber tagelang mit der Auswertung beschäftigt.

Wenn die erwarteten Ausgaben vordefiniert sind, kann man die Ergebnisprüfung der automatisierten Testdaten ebenfalls automatisiert ablaufen lassen. Man muss in diesem Fall aber genau auf die richtigen Formate achten. Es empfiehlt sich hierbei besonders, den automatisierten Testlauf zunächst eine begrenzte Zeit, etwa eine Stunde lang, laufen zu lassen und dann die automatisiert erzeugten Ergebnisse manuell zu vergleichen. Danach kann man das Skript auch über eine größere Datenmenge oder einen längeren Zeitraum

laufen lassen, z. B. in einem ersten Anlauf eine ganze Nacht und in einem erweiterten Szenario ein ganzes Wochenende lang.

Bei Abweichungen der erwarteten Ausgaben lassen sich automatische Fehlermeldungen generieren, die an das Entwicklerteam weitergeleitet werden. Dabei ist aber darauf zu achten, dass nicht Tausende von Fehlermeldungen erzeugt werden, obwohl es sich letztlich nur um einen einzigen Fehler handelt, der immer wieder auf die selbe Ursache zurückzuführen ist und nur in unterschiedlichen Ausprägungen auftritt (Beispiel: Rundungsfehler im Ergebnis).

Für die Testausgaben bei automatisierten Tests muss auf eine eindeutige Zuordnung der Testprotokolle und Logfiles zu den Testfällen geachtet werden. Es empfiehlt sich eine Verlinkung in den Testprotokollen. In einem meiner Projekte lagen die Testprotokolle der automatisierten Skripts dann mit einem kryptischen Namen in einem lokalen Verzeichnis und eine Zuordnung der Logfiles zum Testreport war im Nachhinein nur mit erheblichem manuellen Aufwand möglich. Auch hier zeigte sich wieder, dass man sich Gedanken über den gesamten Testprozess machen muss und organisatorische Festlegungen nicht unterschätzen sollte. Daher empfiehlt es sich immer wieder, einen stringenten Ablauf von A bis Z zu konzipieren, selbst wenn man dann nicht alle Punkte sofort erledigen kann, anstatt nur punktuell Optimierungen herbeiführen zu wollen.

26.9 Qualifikation der Softwaretester bei automatisierten Tests

Für automatisierte Tests müssen die Tester über eine höhere Qualifikation verfügen als manuelle Tester.

Der Tester in der Rolle des Testautomatisierers (der technische Tester) muss wissen, welche Optionen mit dem Tool möglich sind. Er muss in der Lage sein, sie zu konfigurieren und zu warten. Er muss programmieren können, einzelne Skripts integrieren und die Prozesse verstehen. Im Falle von Abbrüchen und Fehlern muss er korrigierend eingreifen können und die Fehlerursachen verstehen. Der Tester in der Rolle des Testplaners (der fachliche Tester) muss die Anforderungen und die Testdatenmuster exakt definieren und Zusammenhänge zwischen Ein- und Ausgaben verstehen und beschreiben. Der fachliche Tester in einer automatisierten Testumgebung muss das erwartete Testergebnis in einer automatisch verarbeitbaren Form spezifizieren können, also deskriptiv programmieren können.

Dadurch steigen die notwendigen Qualifikationen an die Tester in einer automatisierten Testumgebung. Der automatisierte Test kann bei gleichbleibenden Qualitätsanforderungen und gleicher Testabdeckung mittel- bis langfristig zum Abbau von Testressourcen führen, allerdings steigen die Anforderungen an die Testexperten welche die automatisierten Abläufe überwachen. Ein kurzfristiger Erfolg ist mit Testautomatisierung nicht möglich, weil sich automatisierte Skripts meist erst nach mehreren Durchläufen zu rechnen beginnen.

In der Regel wird aber ohnehin eher zu wenig als zu viel getestet. Daher sollte Testautomatisierung in erster Linie zur Steigerung der Testabdeckung, Senkung der Fehlerquote (auch Tester machen Fehler bei der manuellen Testdurchführung!) und zur umfangreicheren Qualitätssicherung angewendet werden, bevor man übereilt Personal abbaut unter der Annahme kurzfristige Renditeziele zu erreichen.

Testautomatisierung ist ein Teilbereich des Softwaretestens, der sich spezialisierter Tools bedient. Für eine erfolgreiche Automatisierung ist Mehraufwand zu Beginn notwendig, der allerdings bei guter Planung und kompetenter Umsetzung zu wesentlicher Verbesserung und Effizienzsteigerung des Tests führt.

Professionelle Testautomatisierer, die mindestens über Grundkenntnisse im Bereich der Softwareentwicklung und über Vorgehensweisen und Prozesse im Test verfügen, sind für die Umsetzung nötig. Im Idealfall bringen sie umfangreiche Erfahrung im Bereich der Testautomatisierung mit.

26.10 Automatische Testskripts und Änderungen in der Software

In einem meiner früheren Testprojekt wurde ein automatischer Programmdurchlauf entwickelt bei denen in einem Menü bestimmte Optionen nacheinander ausgewählt und die Programmteile dann automatisiert durchgeführt wurden. Die Programmierung des Skriptes hatte ein paar Monate gedauert, das Skript lief nun endlich über Nacht und in einem ersten Massentest über ein ganzes Wochenende.

Die graphische Oberfläche änderte sich während des Projekts einige Releases später, das Skript war damit also unbrauchbar und musste geändert werden, was erneut einige Wochen dauerte. Bei einem „Design for Test“ Ansatz soll das geschilderte Szenario von vornherein vermieden werden.

Manchmal ist der Programmierer für das Testskript schneller als der Entwickler des Programms. Ein bestimmter Programmcode ist noch nicht fertig, aber die Programmierung eines Skripts, um einzelne Optionen abzuarbeiten kann schon definiert werden. Daher ist es wichtig, dass der Testautomatisierer über dieselben Informationen verfügt wie der Programmierer der Funktion. Auch bei Änderungen im Programmcode muss man immer das Testprogramm berücksichtigen. Hier zeigt sich wieder, wie wichtig ein durchgängiges und klares Anforderungsmanagement und eine Verknüpfung der Anforderungen zur Programmierung und zum Testing sind. Eine stetige Rückkopplung zwischen Entwicklungs- und Testabteilung ist dazu erforderlich.

Gerade an dieser Stelle ist die Kommunikation der Beteiligten im Projekt für einen Projekterfolg entscheidend. Hier zeigt sich wieder einmal eindrucklich, dass es meist nicht die komplexen ingenieurtechnischen mathematischen Herausforderungen und Algorithmusoptimierungen sind, die Projekte scheitern lassen. Die gewaltigsten Probleme werden immer wieder durch relativ lapidare Details ausgelöst:

- Der Tester wusste nicht, dass die Version auf einem anderen Rechner getestet werden sollte.
- Es wurde eine Tabelle doppelt gepflegt und die Daten waren widersprüchlich.
- Das Kopieren der Festplatten zur Datensicherung dauerte zu lange, dadurch konnten bestimmte Testläufe nicht mehr termingerecht gestartet werden.
- Zur angekündigten Samstagsarbeit war der Server nicht verfügbar und das automatisierte Testtool lief auf einen Fehler.

Das sind alles Punkte, die eigentlich kein großes Nachdenken erfordern, aber vielleicht gerade aufgrund ihrer Trivialität häufig nicht bedacht werden.

Wichtig ist es, in Prozessen zu denken, also das Testprojekt von A bis Z als Ablauf unterschiedlicher Beteiligten zu sehen und die Schnittstellen exakt zu definieren. Oft werden die zentrale Bedeutung dieser organisatorischen Abläufe und die notwendig zu beachtenden betrieblichen Details gerade in Testumgebungen unterschätzt.

26.11 Automatische Generierung von Testdaten

Bei großen Projekten mit zahlreichen zu testenden Variablen kann es sinnvoll sein, Testdaten automatisch zu erzeugen. Voraussetzung dafür ist, dass bereits Test- oder Produktionsdaten vorliegen oder alle Wertebereiche der Daten spezifiziert sind. Bei vorliegenden Test- oder Produktionsdaten werden diese übernommen, nach definierten Regeln von einem Programm geändert und danach als neue Daten ausgegeben. Wenn die Wertebereiche der Daten spezifiziert sind, sind Verzeichnisse aller verwendeten Datenattribute zu erstellen, in denen die Wertebereiche der einzelnen Attribute definiert sind. Aus solchen Wertebereichen, verbunden mit der Zuweisung der Attribute zu Datenobjekten, lassen sich automatisch Kombinationen von Datenwerten (Objektzuständen) generieren.

26.12 Fehlerfreie Ergebnisse ergeben vollständige Reproduzierbarkeit

Menschen machen Fehler, das gilt auch für Softwaretester. Nach einer gewissen Phase lässt die Konzentration nach und die Fehlerquote steigt, gerade bei sich wiederholenden Tätigkeiten. Ein Testautomat arbeitet stetig und gleichmäßig, die Wahrscheinlichkeit einen Fehler zu finden ist nach 100 Stunden Durchlauf so hoch wie in der ersten. Menschliche Leistungen hingegen schwanken erheblich. Dazu kommt das Problem, dass Tester organisatorisch eingebunden werden müssen, dass die Anforderungen an die Kommunikation steigen, dass es Testkoordinatoren und Gruppenleiter geben muss, die die Tester und die Testaufgaben koordinieren, aber nicht selber Tests durchführen. Die Auswahl der geeigneten Tester mit der geeigneten Qualifikation und der erforderlichen Motivation ist ebenfalls nicht trivial. Daher ist gerade dann, wenn umfangreiche Tests mit massivem

Personaleinsatz erforderlich wären und die Testfälle relativ einfach umzusetzen sind, immer der Automatisierung der Vorzug zu geben.

Es sollte darauf geachtet werden, Testautomatisierung in allen Teststufen sinnvoll einzusetzen, ohne Skripte mehrfach zu programmieren. Eine definierte Programmbibliothek und eine Ablage der automatisierten Skripte sind ebenfalls unbedingt zu empfehlen.

Testautomatisierung sollte nach meinen bisherigen Erfahrungen zunächst auch dem Offshoring von Tests vorgezogen werden, so wie es in der Produktion und der klassische Industrie ja auch schon seit langem geschieht. Nur wenn die Automatisierungspotenziale weitestgehend erschöpft sind und die verbliebenen Tests zu komplex sind sollte man über sinnvolle Offshore-Modelle nachdenken.

26.13 Fazit

Testautomatisierung ist ein Teilbereich des Softwaretesten, der sich spezialisierter Tools bedient. Für eine erfolgreiche Automatisierung ist Mehraufwand zu Beginn notwendig, der allerdings bei guter Planung und kompetenter Umsetzung zu wesentlicher Verbesserung und Effizienzsteigerung des Tests führt. Professionelle Testautomatisierer, die mindestens über Grundkenntnisse im Bereich der Softwareentwicklung und über Vorgehensweisen und Prozesse im Test verfügen, sind für die Umsetzung nötig. Im Idealfall bringen sie umfangreiche Erfahrung im Bereich der Testautomatisierung mit. Voraussetzung für eine gute Testautomatisierung ist ein Reifegrad der Entwicklung, der dem Rahmen der Testautomatisierung entspricht. Unterstützung durch Management, Entwicklung und Testteam sind ebenfalls wichtige Punkte, deren Bedeutung nicht unterschätzt werden darf.

Literatur

- [COMP2012] <http://www.computerwoche.de/a/die-zehn-groessten-fehler-im-software-test-management,1877489>
- [BUCS2015] Bucsis T, Baumgartner M, Seidl R, Gwihs S (2015) Basiswissen Testautomatisierung. dpunkt, Heidelberg

Der Begriff Offshoring bezeichnet eine Form der Verlagerung unternehmerischer Funktionen und Prozesse ins Ausland. Auslöser für eine Offshoring-Entscheidung sind in der Regel die im Ausland günstigeren Rahmenbedingungen. Da die IT-Industrie bereits in den 90er-Jahren Entwicklungsdienstleistungen ins Ausland verlagerte, folgte dem auch bald der Softwaretest. Das betraf anfangs vor allem Indien und osteuropäische Länder (Nearshoring).

Beim Offshoring stand oft der Gedanke der Kosteneinsparung im Vordergrund: Wenn die Personalkosten in Indien oder Osteuropa geringer sind, dann kann man Kosten einsparen, wird schneller fertig oder kann die Testabdeckung erhöhen, gerade bei hohem manuellen Aufwand. Soweit die Theorie.

Man muss aber eine Reihe von Faktoren bedenken wenn man Offshoring-Projekte angeht. Ganz unabhängig davon, ob es sich nur um den Softwaretest handelt oder um ein komplettes Entwicklungsprojekt:

- Vertragliche Risiken: Wer ist das Partnerunternehmen, ist es das eigene Unternehmen oder ein Dienstleister?
- Wie ist die Rechtssicherheit im Zielland, lassen sich vertragliche Ansprüche durchsetzen oder gibt es ein Risiko bezüglich Korruption?
- Erfahrung des Offshore-Partners: Nur als langfristige Perspektive ist Offshoring von Testaktivitäten sinnvoll. Zu differenzieren ist hierbei, ob Software-Entwicklung und Test offshore erfolgen sollen, oder nur Software-Entwicklung oder Test. Wenn man unterschiedliche Offshore-Partner für Entwicklung und Test auswählt, werden die Kommunikation und die Planung extrem schwierig.
- Eine gegenseitige permanente Abstimmung und Wissenstransfer zwischen dem Offshore-Partner und dem Auftraggeber muss gewährleistet sein.

- Genaue Spezifikationen, methodische Planung und definierte Entwicklungsprozesse sind bei Offshore Projekten noch wesentlich wichtiger als bei Testaktivitäten im eigenen Hause. Ausgereifte und standardisierte Abläufe, definierte Schnittstellen und gemeinsame Infrastruktur (idealerweise web-basiert) sind unabdingbare Voraussetzungen dafür, dass Offshore-Partner und Cloud Services einen Mehrwert bringen können. Die Industrialisierung der Software-Entwicklung befindet sich in den meisten Organisationen noch nicht auf der Reifestufe, wo Offshoring bereits sinnvoll angewendet werden kann.
- Gemeinsames Verständnis über Softwaretest – nicht mehrere Ansätze für Inhouse Testing einerseits und billige manuelle Test-Offshore-Partner andererseits: Die Testphilosophie muss unternehmensweit einheitlich sein, sonst werden die Kostenvorteile konterkariert.
- Dynamik in den Lohnkosten: Gerade in typischen Offshore-Regionen wie Indien oder Osteuropa entwickeln sich die Lohnkosten sehr dynamisch, so dass sich die Kostenvorteile bei den Personalkosten mehr und mehr relativieren. Man muss zusätzlich bedenken, dass Personalkosten nur einen Teil der Entwicklungskosten ausmachen. Auch die Fluktuation ist in vielen typischen Offshore-Ländern wesentlich höher als in Europa.
- Kulturelle Probleme: Dazu gehören mangelnde Flexibilität, mangelnde Erfahrung und unterschwellige Vorurteile, Termintreue. Ein erfahrener Tester weiß, dass er oft während der Testdurchführung noch Anpassungen vornehmen muss, Dinge ergänzen muss, dann doch nicht ganz so wie es beschrieben ist den Test durchführen muss und dann die Testbeschreibung noch einmal ändern und ständig hinterfragen muss. Das fehlt bisweilen beim Test im Offshoring – da wird dann buchstabengenaue das getestet, was in der Spezifikation steht, man muss die Testbeschreibung also besser und genauer erstellen als wenn man es selber testet. Das lohnt sich dann nur für sich wiederholende Aufgaben. Das sind aber genau die Tests, die man auch automatisieren kann. Problematisch sind aber auch spezielle Rahmenbedingungen in bestimmten Ländern: In Indien muss man das Kastenwesen strikt beachten – wenn der Projektleiter nicht einer höheren Kaste als der Tester angehört, ist das Projekt von Anfang an zum Scheitern verurteilt. Kulturelle Unterschiede wirken sich auf die Kommunikation aus, und Projekte scheitern oft an mangelnder oder falscher Kommunikation. Inzwischen hat sich aber das Offshore-Geschäft im Testing schon weitestgehend professionalisiert, und typische Offshore-Länder wie Indien oder China haben dabei teilweise die Lernkurve in schnellerer Zeit bewerkstelligt als die klassischen Industrieländer.
- Nicht zu unterschätzen ist auch, dass bestimmte Begriffe aus dem täglichen Leben einem Mitarbeiter im Offshore-Land nicht bekannt sind. Es ist schwierig, einem Indier eine „Fehlbelegungsabgabe“ zu erklären, die genaue Kenntnis dieses Begriffs kann aber für den Test einer Anwendung für eine Hausverwaltung entscheidend sein.
- Große Unterschiede in der Arbeitsweise einzelner Tester: Während einige Tester sehr gewissenhaft und fleißig testen und sich dabei auch Gedanken über exploratives Testen machen, kratzen andere nur an der Oberfläche oder sind nicht wirklich bei der Sache. Dieses Problem hat man auch bei Testern in der eigenen Organisation. Wenn man die

Tester aber bei ihrer täglichen Arbeit sieht, bekommt man viel eher einen persönlichen Eindruck davon, wer zu den Leistungsträgern und wer zu den „low performern“ gehört. Wenn die Tester nur ein paar Kilometer entfernt sitzen, ist das schon wesentlich schwieriger, und in einem anderen Land hat man darauf kaum Einfluss. Dazu kommt, dass die Qualifikation der Tester noch nicht überall so standardisiert ist wie im eigenen Land bzw. die Standards nicht ohne weiteres vergleichbar sind.

- **Termine:** Wir Europäer sind Termintreue gewöhnt – schon von Kindesbeinen an leben wir nach der Uhr. Es gibt pünktliche Zeiten fürs Mittagessen und der Bus fährt immer pünktlich. In anderen Ländern ist das Leben weit weniger von der Uhr bestimmt. Das muss man einfach wissen, ohne es zu werten. Die deutsche Art zu arbeiten fördert eine gewisse Unflexibilität und unterdrückt so manche Kreativität und Spontaneität. Andererseits müssen aber Termine beachtet werden wenn ein Projekt zeitgerecht fertig werden muss.
- **Vorurteile:** Unterschiede zwischen Kulturen und Ländern können eine Bereicherung sein, amerikanisches Marketing mit deutscher Gründlichkeit und asiatischem Fleiß zusammen verhilft einem Produkt oder einer Dienstleistung zu einem positiven Ergebnis. Das muss aber auch von allen Beteiligten gelebt werden. Wenn man abschätzig auf andere Länder herunterblickt und sich für die höher entwickelten hält, Informationen zurückhält um die anderen auszubremsten und nicht in der Lage ist, in der Verschiedenheit Chancen zu sehen dann ist ein Testprojekt im Offshoring zum Scheitern verurteilt.
- **Sprachliche Hürden:** Auch wenn Englisch weit verbreitet ist – nicht alle formulieren so, dass es der andere exakt versteht. Gerade komplexe technische Sachverhalte lassen sich in einer gemeinsamen Sprache leichter vermitteln.
- **Technische Hürden:** Durch neue Kommunikationsmedien und Internet sind technische Hürden geringer geworden und die Welt wird mehr und mehr zum globalen Dorf. Trotzdem muss man wissen, dass in Indien an vielen Orten 1 Stunde Stromausfall pro Tag normal ist. Oder aber die ausländischen Dienstleister arbeiten nicht auf derselben Hardware, können nicht auf den gemeinsamen Server zugreifen oder die Datenübertragung dauert einfach viel zu lange. Die Kommunikation mit Offshore Projekten lief in einem meiner Projekte über einen schwer verständlichen Mobilfunk-Anschluss im Zielland, in der regelmäßigen Telefonkonferenz hörte man vor allem Nebengeräusche. Wenn man Offshoring derart unvorbereitet und unprofessionell umsetzt, muss man sich über mangelnde Akzeptanz, Vorurteile und schlechte Resultate nicht wundern. Wenn das Projekt komplett im Web abgewickelt werden kann, sind technische Hürden geringer, aber auch da steckt manchmal der Teufel im Detail. Bisweilen sind es Zugriffsbeschränkungen, andere Server, andere Hardware und technische Hürden, die das Projekt scheitern lassen. Das alles sind Randbedingungen, die nicht zu unterschätzen sind und die in den Wirtschaftlichkeitsberechnungen im Management meist nicht entsprechend gewürdigt werden. Vielleicht liegt es auch daran, dass schon viel zu viel in Meetings am grünen Tisch entschieden wird als sich die Situation vor Ort einmal näher anzusehen.
- **Zeitumstellung:** Die Idee, dass man ein Projekt dreimal schneller erledigen kann, indem man die Zeitzonen ausnutzt ist bestechend: 8 Stunden arbeitet A in den USA; 8

Stunden B in Deutschland, 8 Stunden C in Ostasien – man nutzt die Zeitverschiebung aus, wird zum globalen Unternehmen und beschleunigt dadurch die Entwicklung. In der Praxis ist es dann aber oft so, dass B nicht anfangen kann, weil er auf C warten muss und die Arbeit dann am Ende doch eher dreimal länger dauert statt dreimal so schnell fertig ist und dass die erhoffte Beschleunigung ganz und gar nicht eingetreten ist. Bei der Testplanung müssen auch die lokalen Gegebenheiten wie freie Tage oder Feiertage anbelangt bedacht werden. An die Steuerung des Testprojekts und an das Projektmanagement werden in jedem Fall höhere Anforderungen gestellt, wenn Teile des Projekts offshore vergeben wurden.

Oft wird Test-Offshoring aus Kostengründen begonnen, ohne die beschriebenen Randbedingungen zu beachten. Und viel zu oft – auch in namhaften Konzernen – gleicht die Kosten-Nutzen-Rechnung von Offshore-Aktivitäten einer „Milchmädchenrechnung“.

Eine Verzögerung des Projekts wegen mangelnder Abstimmung verursacht oft weit mehr Aufwand als eine Einsparung in Stundensätzen der Offshore-Entwickler. Es ist in meiner beruflichen Erfahrung schon mehrfach vorgekommen, dass mehrere hochbezahlte deutsche Experten unproduktiv auf eine neue Software warten mussten, weil ein Programmierer mit einem bestimmten Entwicklungs- oder Testschritt in einem „Billiglohnland“ nicht rechtzeitig mit einer Komponente fertig wurde. In einem anderen Fall war die Anforderungsspezifikation unklar, es kamen aber von der Entwicklung aus dem Offshore-Land keine Rückfragen, sondern es wurden nur genau die Funktionen buchstabengetreu implementiert, die vorher lückenhaft spezifiziert waren. Die Folge war eine Verzögerung des gesamten Projekts über mehrere Wochen.

Test-Offshoring kann dann sinnvoll sein, wenn man große Projekte, eine hervorragende Organisation und die Themen Requirements-Analyse, Entwicklung und Testkonzept bereits sehr gut umgesetzt hat. Wenn man Test-Offshoring einsetzt, muss man auch eine definierte Schnittstelle im Offshore-Land haben, jemanden der genau weiß, worum es im Projekt geht, der den gesamten Prozess und die Auswirkungen im Blick hat und der in der Lage ist die Rahmenbedingungen und die Verhältnisse vor Ort genau zu beachten. Nur so kann man verhindern, dass das Offshore Team eine unerwünschte Eigendynamik entwickelt oder an den tatsächlichen Projektzielen vorbei testet. Man muss also eine langfristige Strategie verfolgen und die Abläufe im Vorfeld der Verlagerung genau prüfen, bevor man ein Offshore-Projekt, sei es in der Entwicklung oder im Softwaretest, startet.

Nach einer Studie des „Instituts für Zukunftsstudien und Technologiebewertung“ aus dem Jahre 2013 findet beim Offshoring großer IT-Unternehmen in Länder mit deutlich geringeren Lohnkosten die eigentliche Forschung und Entwicklung (inkl. Anforderungsspezifikation und Dokumentation) nach wie vor in Deutschland statt, während in Indien, seltener auch in China, dann nur die reine programmtechnische Ausführung oder die Testdurchführung stattfindet. Die Schwierigkeiten liegen schon bei dieser eingeschränkten Art des F+E-Outsourcing in der Kommunikation und Abstimmung. Die Vorgaben müssten deshalb recht aufwändig extrem genau spezifiziert werden. Die Kostenersparnis in einschlägigen indischen Regionen liegt heute allerdings durch Qualifikations- und

Lohnkostensteigerungen bei höchstens noch ca. 5 bis 10 % gegenüber Deutschland. Zunehmend attraktiver ist das Nearshoring, z. B. in Polen oder Rumänien.

Offshoring nach Indien oder China wird deshalb inzwischen immer weniger aus reinen Kostengründen genutzt, sondern zunehmend, um die asiatischen Märkte besser zu erreichen [IZTW2007]. Während in den deutschsprachigen Unternehmen Kostensenkung lange Zeit das Hauptmotiv für Offshoring gewesen ist, war in den USA von Anfang an der Gewinn an externem Know-how als wichtiger angesehen worden. Nach einem Offshoring Hype Anfang der 2000er-Jahre wird das Thema inzwischen bereits sehr viel differenzierter betrachtet.

Ein weiteres zukünftiges Hauptmotiv für Offshoring in der IKT-Branche ist in Deutschland zunehmend der Mangel an qualifizierten Fachkräften, da bereits heute zahlreiche Stellen für Informatiker nicht besetzt werden können und sich dieses Problem angesichts des demographischen Wandels in den nächsten 15 Jahren noch erheblich verschärfen wird. Dadurch werden Unternehmen daran gehindert, entsprechend den Marktanforderungen zu wachsen. Große Konzerne, die für einzelne Entwicklungsprojekte oft mehrere Hundert Spezialisten auf einmal benötigten, haben bereits heute gar keine andere Wahl mehr, als solche Vorhaben nach Indien oder China zu verlagern.

Als Fazit ist festzustellen, dass Offshore-Testaktivitäten langfristig an Bedeutung gewinnen und in der globalen Wirtschaft auch vermehrt eingesetzt werden. Das einzelne Testprojekt wird aber durch Offshore-Aktivitäten komplexer. Man muss daher eine Reihe von Voraussetzungen bedenken und einige Herausforderungen bestehen, damit Testen im Offshore- oder Nearshore-Betrieb wirklich die gewünschten Vorteile bringt und nicht zum Scheitern des Projekts führt.

Literatur

[IZTW2007] https://www.izt.de/fileadmin/downloads/pdf/IZT_WB88.pdf

Für den Test von Internet-Anwendungen gilt grundsätzlich dasselbe wie für den Test von Software-Applikationen. Der wesentliche Unterschied ist, dass im Internet fehlerhafte Qualität viel schneller einer erheblich größeren Gruppe von Nutzern auffällt.

Damit eine Web-Applikation erfolgreich ist, sollte sie spätestens in drei bis sechs Monaten online gehen können. Das bedeutet, dass das Testprojekt hier wesentlich dynamischer arbeiten muss als bei einem typischen Software-Entwicklungsprojekt. Beim Test von komplexen Anlagen in Maschinenbau oder bei Komponenten für die Automobilindustrie sind durchaus Testprojekte in der Dauer zwischen einem und zwei Jahren die Regel.

Agile Ansätze empfehlen sich daher besonders für den Test von Internet-Applikationen. Das Arbeiten nach dem V-Modell ist für einen typischen Webtest zu starr und zu komplex.

28.1 Struktur bei Web-Tests

Zur Strukturierung von Webtests empfiehlt sich folgende Aufteilung:

- technische Tests
 - Darstellung der Anwendung in unterschiedlichen Browsern und unterschiedlichen Betriebssystemen,
 - Betrachtung des gesamten Anwendungssystems: Browser, Netzinfrastruktur, Webserver, Anwendungsserver/Middleware, Datenbank,
 - Test des Lastverhaltens und der Performance.
- inhaltliche und funktionale Tests
 - Funktionsfähigkeit
 - Benutzerakzeptanz (verständliche und bequeme Oberfläche)
 - Leistungsfähigkeit

- Skalierbarkeit
- Sicherheit
- Zuverlässigkeit
- Verfügbarkeit

Bei Online-Shop-Anwendungen kommt es besonders darauf an, den gesamten Workflow des Nutzers zu testen: Von der Produktsuche geht es zur Auswahl des Produkts, von dort zur Detailbeschreibung des Artikels, dann zum Warenkorb, danach zur Bestätigung des Kaufvorgangs und schließlich zur Bezahlung und zur Bestätigung der getätigten Transaktion.

28.2 Fragen an die Web-Applikation

Jeder einzelne Schritt ist zu testen und entsprechende Varianten sind zu definieren.

Fragen dazu sind u. a.:

- Mit welchen Suchbegriffen könnte mein Kunde das Produkt suchen?
- Wird das Produkt dann mit diesen Suchbegriffen auch gefunden, und an welcher Stelle in Google steht das Suchergebnis?
- Sind die Produkte richtig strukturiert?
- Ist die Beschreibung des Produkts aussagekräftig? Werden Bilder angezeigt, wird das Produkt optimal präsentiert? Kann der Nutzer der Produktbeschreibung alles entnehmen, was zur Kaufentscheidung wichtig ist?
- Werden Upselling- und Cross-Selling -Potenziale ausgenutzt? Beispiel: Eine Seite für Damenmode bietet auch Schuhe oder Schmuck an. Dadurch können höhere Umsätze pro Transaktion erzielt werden.
- Können Produkte in den Warenkorb hineingelegt und auch wieder gelöscht werden? Behalte ich jederzeit die Übersicht über den Kaufvorgang?
- Werden unterschiedliche Zahlungsoptionen angeboten?
- Wird der Kaufvorgang dem User bestätigt?

Bei einem Online-Shop sollte auch bedacht werden, dass er an das eigene Warenwirtschaftssystem angebunden ist. Hieraus ergeben sich weitere Tests.

Bei den funktionalen Tests ist u. a. zu prüfen:

- Gibt es Rechtschreib- oder Grammatikfehler?
- Wie wirkt der Inhalt visuell auf den Nutzer? Werden Design-Standards eingehalten?
- Ist das Logo immer an derselben Stelle, ist die Navigation klar und durchgängig?
- Wie verhält sich die Anwendung in unterschiedlichen Auflösungen, unterschiedlichen Browsern und unterschiedlichen Betriebssystemen?
- Wie wird die Darstellung auf Mobile Devices (iPad, Smartphone usw.) angezeigt?

- Gibt es Links, die ins Leere führen? Sind alle Downloads durchführbar?
- Was passiert bei abgeschalteten Plug-Ins (JavaScript, Flash usw.) oder deaktivierten Cookies?
- Sind die Zugriffsmechanismen sicher? Wie ist die Verschlüsselung (z. B. von Zahlungsinformationen) gewährleistet?
- Ist die Anwendung sicher vor Störungen?

Beim Performancetest wird geprüft:

- Ist die Antwortzeit für den User ausreichend? Antwortzeit User = Bearbeitungszeit des Browsers + Datenübertragungsdauer des Netzwerks + Antwortzeit Server.
- Ist die Bandbreite des Web-Servers ausreichend?
- Ist die Performance auch bei maximaler Benutzerzahl akzeptabel?
- Was passiert, wenn die maximale vorgesehene Benutzerzahl überschritten wird? Eine hohe Anzahl von Nutzern kann man mit Hilfe von Tools simulieren.
- Wie hängt die Performance von den Datenübertragungsraten ab? (Das sollte zwar im Jahr 2019 kein Problem mehr darstellen, aber immer noch fehlen Datenautobahnen, vor allem in ländlichen Regionen.)
- Wie verhält sich die Performance im Vergleich zu Wettbewerbsseiten?
- Inwiefern wirken sich Verschlüsselungsmechanismen negativ auf die Performance aus? [[FRAN2007](#)]

Das Internetbusiness ist etwa 20 bis 25 Jahre jünger als das Softwaregewerbe. Dementsprechend ist auch der Reifegrad für Testaktivitäten für Webtests noch nicht so weit fortgeschritten.

Fehlende Strategien, veraltete Technik und Qualitätsmängel schaden dem Image des eigenen Unternehmens. Links, die ins Leere gehen, diffuse Fehlermeldungen oder mangelnde Aktualität schrecken den User ab. In vielen Fällen macht man sich mehr Gedanken über das Design als über den Inhalt. Anstatt den eigenen USP herauszustellen und dem User nutzwolle Informationen zu liefern, werden aufwändige Graphiken und Animationen entwickelt, die lange Ladezeiten verursachen, aber kaum Zusatzinformation für den Endanwender bieten.

Es gibt inzwischen genügend prominente Beispiele gescheiterter Internet-Anwendungen. Bei so mancher Website kam es schon kurz nach dem Launch der Anwendung zu einem Serverabsturz oder langen Wartezeiten.

Der Test von Web-Anwendungen ist wegen einiger Randbedingungen manches Mal problematischer als der Test von Software-Anwendungen:

- Der Zeitdruck ist oft höher, die Projekte kurzfristiger geplant, formale Entwicklungsmodelle werden seltener angewendet.
- Das Benutzerprofil der Zielgruppe ist nicht ausreichend bekannt.
- Die Testumgebung muss die Produktivumgebung abbilden.

- Die Lokalisierung des Fehlers wird in komplexen Infrastrukturen schwierig.
- Die Abschätzung der Zugriffszahlen ist problematisch. Ein detailliertes Webcontrolling mit Messungen der Zugriffszahlen über einen längeren Zeitraum unter der Berücksichtigung von Spitzenzeiten und Größe der Downloads ist dafür eine wesentliche Voraussetzung [QUAL2004].

In Großunternehmen mit einer Vielzahl unterschiedlicher Websites und zielgruppen- bzw. produktionsspezifischen Websites empfiehlt sich die Durchführung von übergreifenden Testmaßnahmen. Dazu gehören standardisierte Testverfahren zur Überprüfung der Einhaltung des Corporate Designs, der Styleguides oder der Markenkommunikation. Als Ergebnis steht ein Bericht über Abweichungen und Fehler von den Grundlagen und Empfehlung von Maßnahmen zur Korrektur dieser Fehler.

Beispiele: Suche nach der Website

Als Anbieter ist man meist betriebsblind und Experte. Man definiert das eigene Produkt exakt, verwendet Fachtermini, aber keine umgangssprachlichen Begriffe. Das führt dazu, dass man oft zu eingeschränkte Suchbegriffe definiert. Dann testet man zwar, ob die Website mit diesen Suchbegriffen gefunden wird, das kann dann auch positiv nachgewiesen werden, aber mit diesen Suchbegriffen wird viel zu selten gesucht.

Hier empfiehlt es sich, User zu befragen und sich die Suchtrends unter Google anzusehen. Google erstellt auch selber Vorschläge, wenn eine Adwords-Kampagne gestartet wurde, welche Suchbegriffe noch relevant sein könnten. Da es sich um eine automatisierte Anwendung handelt, muss man aber sehr genau nachvollziehen, welche Begriffe wirklich sinnvoll sind.

28.3 Phasen beim Web-Test

Grundsätzlich ist der Webtest in allen Phasen des Produktionszyklus einer Website einzusetzen: Im Vorfeld eines Relaunches oder einer Neuerstellung, während der Implementierung und nach dem erfolgten Relaunch. Nachhaltigen Erfolg kann nur durch eine Vorgehensweise sichergestellt werden, bei der die Vorteile des produkt- als auch die des prozessorientierten Qualitätsmanagements miteinander verbunden werden. Je nach Phase können unterschiedliche Ziele mit den Webtestergebnissen verfolgt werden.

Abb. 28.1 zeigt einen idealtypischen Prozess mit den einzelnen Phasen beim Test einer Web-Applikation.

28.4 Testabdeckung beim Test von Internetanwendungen

Eine Anwendung ist in unterschiedlichen Browsern und mit unterschiedlichen Betriebssystemen zu testen. Es wäre aber äußerst ineffizient, wenn dazu der gesamte Softwaretest wiederholt würde. Es reicht in diesem Fall meist aus, sich auf die Usability zu beschränken



Abb. 28.1 Testphasen beim Webtest

und wenige funktionale Standard-Tests mit den unterschiedlichen Kombinationen aus Browser und Betriebssystem durchzuführen. Ansonsten hat man z. B. schon mit wenigen Kombinationen Windows Vista, XP, Browser Chrome, Internet Explorer, Firefox sechs Testdurchläufe, also fünf zusätzliche Kombinationen. Wenn man den Test für eine Kombination einfach hochrechnet bekommt man einen exorbitant hohen Testaufwand, ohne wahrscheinlich einen wesentlichen Zusatzgewinn zu erzielen.

Hier empfiehlt es sich ohnehin, die Häufigkeit der verwendeten Kombinationen aus der im Unternehmen eingesetzten Webanalyse-Software zu ermitteln und dann die Kombinationen entsprechend zu priorisieren. Das gilt auch für besondere Endgeräte – wenn man die Software für jede mögliche Plattform optimieren will muss man sich erst überlegen, wie oft welcher User überhaupt irgendeine exotische Umgebung benutzen.

Aus der verwendeten Webcontrolling-Software lassen sich auch mittelbar Werte über die erforderliche Last herauslesen. Wenn die Website maximal von 10 Usern gleichzeitig betrachtet wird, muss man keinen Lasttest für den Zugriff von 10.000 Usern durchführen.

Andererseits ist es nicht sinnvoll, eine Web-Applikation nur auf ihre Funktionalität zu testen und das Verhalten unter Last, also wenn zahlreiche Benutzer gleichzeitig auf die Anwendung zugreifen und Abfragen generieren, zu verzichten. So geschah es bei einem neuen Stellenportal der Arbeitsagentur, als auf die Simulation der Last verzichtet wurde und der Server am ersten Tag nach der Einführung in die Knie ging und stundenlang nicht mehr erreichbar war.

Bei Lasttests muss man auch bedenken, plötzliche Sprünge im Lastverhalten zu simulieren. Daher sollte man gerade bei Internetanwendungen die Zugriffsstatistiken auswerten, um saisonale, regionale oder Spitzenlasten im Tagesverlauf vorher zu ermitteln und beim Simulationstest dann auf die höchsten ermittelten Werte noch einen Sicherheitspuffer von ca. 20 % oben drauf zu setzen. Wenn bei einem neuen Portal noch nicht bekannt ist, wie das Lastverhalten sich darstellt, kann man auf Erfahrungswerte in Google Trends setzen.

Beispiele werden in Abb. 28.2 dargestellt.

Modelleisenbahnen verzeichnen in der Weihnachtszeit ein erheblich höheres Interesse als in den Sommermonaten. Das Beispiel „Truthahn“ zeigt die hohen Zugriffsraten rund um „Thanksgiving“ Ende November.

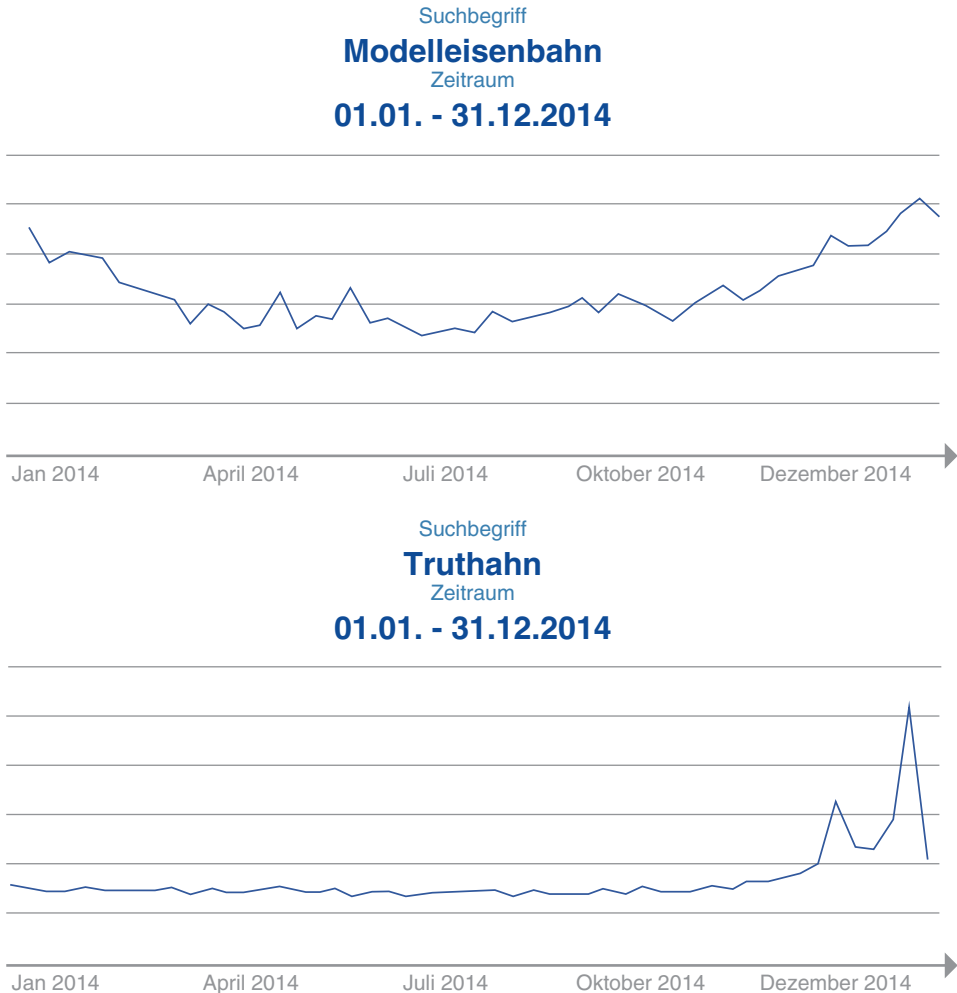


Abb. 28.2 Interesse im zeitlichen Verlauf für bestimmte Suchbegriffe

28.5 Test unterschiedlicher Sprachen

Beim Test unterschiedlicher Sprachen muss vor allem auf die richtige Architektur der Anwendung geachtet werden: Die Texte sollten in Tabellen stehen, die dann dynamisch herangezogen werden. Wenn das zu tief im Quellcode versteckt ist, wird die Übersetzung problematisch. Zu beachten ist beim Rollout, dass dann die Texte auch aktuell übersetzt sind. Ansonsten kann es zu Fehlern bei der Softwarenutzung kommen, weil alter Beschreibung und neue Funktionalität nicht zusammenpassen. Auch hier gilt es, die einzelnen Sprachen bei der Testplanung zu priorisieren, und dabei nach den Userzahlen pro Sprache die Übersetzungen zu steuern.

Texte sollten mittels Textbausteinen über ein Content Management System verwaltet werden. Hier ist zu beachten, dass wegen der unterschiedlich langen Wörter das Layout in der Fremdsprache plötzlich ganz anders aussehen kann. Das ist vor allem zu bedenken, wenn erklärende Bilder und Graphiken auf der Website vorhanden sind, die evtl. auseinandergerissen werden und nicht mehr im Kontext zu den Erklärungen stehen.

28.6 Look and Feel

Beim Test von Websites kommt es im Allgemeinen – stärker als beim Test einer Softwareanwendung – auf das äußere Erscheinungsbild des Testobjekts an. Hier entscheidet oft der erste visuelle Eindruck. Eine Website wendet sich schließlich in aller Regel an die breite Masse und nicht an Anwender, die in ihrem Bereich Experten sind und sich jeden Tag mit der entsprechenden Materie beschäftigen.

Beim Webtest ist daher u. a. zu untersuchen:

- Was sind gute Elemente des Appeals, was sind Störfaktoren?
- Wie gut findet sich der Besucher auf der Website zurecht?
- Ist die Navigationsstruktur klar?
- Wie benutzerfreundlich erscheint die Website?
- Ist die Website verständlich?
- Welche Inhalte sind besonders interessant und sollten im Vordergrund stehen?
- Ist die Website barrierefrei?

28.7 Test nach Zielgruppen

Beim Webtest sollten zunächst die einzelnen Zielgruppen klar definiert werden, die auf das Angebot im Internet zugreifen, z. B. Interessenten, Stammkunden, Teilnehmer an einem Online-Spiel, Servicemitarbeiter, potentielle Mitarbeiter im Rahmen ihrer Bewerbung. Dabei ist zu prüfen, welche Bereiche welche Zielgruppe am ehesten frequentiert. Eine Zielgruppe kann nach Region, Alter, Einkommen, Hobbies, Geschlecht und vielen anderen Kriterien beschrieben sein. Zusammen mit der Marketingabteilung sollte daher zunächst eine Kundensegmentierung stehen. Es geht bei der Website schließlich darum, zielgruppenrelevante Informationen zu präsentieren.

Dabei ist u. a. zu klären:

- Erhält jede Zielgruppe schnell und direkt die Informationen, die sie wünscht?
- Sind die Informationen detailliert genug?
- Erhält jede Zielgruppe einen Mehrwert?
- Sind die fachlichen und technischen Beschreibungen von Experten erstellt bzw. geprüft?

- Sind die Inhalte stets aktuell?
- Lässt sich die Website leicht aktualisieren? Haben alle Beteiligte Zugriff auf ein Redaktionssystem und sind nicht Abteilungen voneinander oder einer Webagentur abhängig?
- Gibt es die Rubriken FAQ, Glossar, Impressum, Neuigkeiten, Informationen zum Unternehmen (z. B. Finanzberichte), einen Newsletter?
- Kann man mit dem Unternehmen in Kontakt treten und ist Interaktivität gewährleistet?
- Ist der Bestellprozess eindeutig oder gibt es Brüche im Bestellprozess?

28.8 A/B-Testing und Multivariate Testing

Beim A/B-Testing und beim Multivariate Testing wird ermittelt, welche Version einer Website den meisten Erfolg verspricht.

Zunächst muss man dabei definieren, was „meister Erfolg“ bedeutet:

- die längste Zeit pro Besucher auf der Website,
- die meisten Verkäufe,
- die höchsten Umsätze,
- Steigerung von Umsätzen für bestimmte Produkte mit besonders hohen Margen.

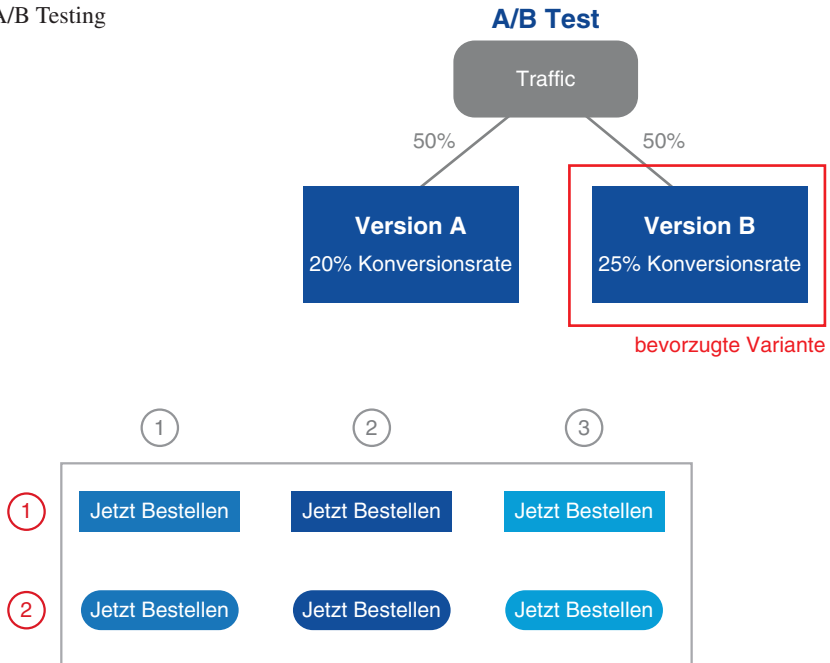
Dann werden diese Werte erfasst und deren Wichtigkeit bewertet. Daraus bildet man geeignete Indizes.

Beim A/B-Testing (siehe Abb. 28.3) werden zwei Alternativen miteinander verglichen. Dieser Test wird auch als Split-Run-Test bezeichnet, denn im Vorgehen wird die Zielgruppe in zwei Untergruppen aufgeteilt: Gruppe A und Gruppe B. Entsprechend der Aufteilung der Zielgruppe wird auch das Werbemittel wie beispielsweise eine Annonce unter Google Adwords zweigeteilt produziert: die Originalvariante und eine veränderte Variante. Anschließend wird bei der Gruppe A das Originalwerbemittel und bei der Gruppe B das veränderte Werbemittel eingesetzt und die Reaktionen verglichen. Mit Reaktion ist hierbei das gewünschte Wirkungsergebnis eines Werbemittels gemeint. Das kann z. B. der erzielte Umsatz, die Anzahl der Verkäufe oder das Interesse das man am Zugriffsverhalten messen kann, sein. Wichtig ist, dass Zielgruppen A und B ähnlich sind [CONV2014].

Man kann z. B. die Annonce mit Ausprägung A am Dienstag von 13 bis 15 Uhr und die Annonce mit Ausprägung B am Dienstag von 15 bis 17 Uhr schalten.

Beim Multivariate Testing werden ganze Bereiche und Prozesse (z. B. Landingpages und Registrierungs- oder Bestellprozesse) miteinander verglichen. Dadurch wird das Zusammenwirken mehrerer unterschiedlicher Website-Elemente geprüft. Abb. 28.4 zeigt ein Beispiel mit unterschiedlichen Buttons zur Bestellung. Man kann zum Beispiel messen, bei welcher Form und Farbe des Buttons der Aufforderung zur Bestellung die Konversionsrate bzw. der erzielte Umsatz am höchsten ist.

Dadurch ergeben sich sehr schnell viele Möglichkeiten. Das führt sowohl zu technischen Probleme als auch zu Schwierigkeiten mit der statistischen Vergleichbarkeit.

Abb. 28.3 A/B Testing**Abb. 28.4** Multivariate Testing

Bei 30 zu testenden Kombinationen muss man schon an die 100.000 Zugriffe pro Monat auf der Website erzielen, damit man valide statistische Aussagen treffen kann. Bei weniger als 1000 Zugriffen pro Kombination ist die Aussagekraft nicht gegeben, weil zusätzliche Einflussfaktoren (Wochentag, Uhrzeit) auch noch berücksichtigt werden müssen.

Wenn eine Alternative zwar Sonntag von 14 bis 16 Uhr wesentlich interessanter war als die andere, aber Dienstag von 10 bis 12 Uhr der anderen Alternative weit unterlegen, müsste untersucht werden, welche User dem Unternehmen denn nun mehr bringen – die am Sonntag von 14 bis 16 Uhr oder die am Dienstag von 10 bis 12 Uhr, denn die User vom Sonntag könnten zum Beispiel eher Privatpersonen und die am Dienstag eher Unternehmen sein. Es handelt sich in diesem Fall also um völlig andere Zielgruppen, anderes Suchverhalten, andere Umsatzmöglichkeiten. Wobei das wiederum nicht für jede Website so gelten muss ...

Beim Multivariate Testing wird die zu testende Seite zunächst nach bestimmten Kriterien analysiert:

- Relevanz: Ist die Website relevant für den User?
- Klarheit: Ist dem User klar, was er auf der Website tun soll?
- Ablenkung: Gibt es Ablenkungsfaktoren auf der Website?
- Unsicherheit: Wird der User beim Besuch der Website durch irgendwelche Einflüsse verunsichert?
- Dringlichkeit: Ist dem User klar, dass er sich schnell entscheiden muss?

Im Multivariate Testing definiert man dann auch bestimmte Userprofile. So kann man den Usability-Test im Labor mit den Ergebnissen aus dem Aufbau eines Multivariate Tests vergleichen:

- Schüler, 15 Jahre, mittelgroße Stadt,
- Studentin, 25 Jahre, Großstadt,
- Arbeiter, männlich, 30 Jahre, Kleinstadt,
- nicht erwerbstätig, weiblich, 35 Jahre, Dorf,
- selbständig, männlich, 45 Jahre, Kleinstadt,
- Beamte, weiblich, 50 Jahre, Großstadt,
- Hausfrau, weiblich, 60 Jahre, Dorf,
- Rentner, männlich, 70 Jahre, mittelgroße Stadt.

Man kann natürlich auch andere Kategorien wählen, etwa nach Einkommen, Parteipräferenz, Religionszugehörigkeit, Anzahl Kinder ... oder auch nach Interessen und Hobbies. Es werden dann Hypothesen zur Optimierung der zu testenden Seiten erstellt. Im Testkonzept für den Multivariate Test muss darauf geachtet werden, dass der Test eindeutig ausgewertet werden kann und die Auswertung das gewünschte Ergebnis beschreibt.

Beim Usability-Test werden Testpersonen gesucht, die die gewünschten Kriterien erfüllen, und lassen sie bewerten, wie ihnen die zu testende Website gefällt. Oft werden diese Testpersonen in ein Labor eingeladen, wo sogar die Augenbewegungen wissenschaftlich gemessen werden. Das Testlabor hat den Vorteil, die Testumgebung definieren und für alle Testpersonen in gleicher Weise gestalten zu können, so dass der User während des Webtests nicht von Umweltfaktoren abgelenkt ist.

Dabei werden beim Usability-Test gezielte Fragen gestellt:

- Wie wirkte der Hintergrund?
- Ist die Navigation klar?
- Sind die Texte verständlich?

Außerdem wird der User nach dem spontanen Eindruck gefragt, den er in den ersten Sekunden seines Besuches hat [FRAN2007].

Beim Multivariate Test handelt es sich um einen Live-Test, das bedeutet man muss aufgrund des Suchverhaltens Annahmen treffen, zu welcher Gruppe der User gehört. Der Besucher A kommt um 10:00:00 auf die Website und erhält Kombination 1, Besucher B kommt um 10:00:01 auf die Website und erhält Kombination 2 usw. Man kann dabei Graphiken, Buttons, Navigations- oder Textelemente auf der Website ändern.

Sinnvoll ist in diesem Zusammenhang auch die Besucher nach Segmenten (z. B. Browser, geographische Herkunft) zu unterteilen und dann den unterschiedlichen Segmenten verschiedene Varianten von Websites vorzuführen. Die Segmentierung der User ist dabei eine eigene Fragestellung: Welche Merkmale eignen sich – das Geschlecht, das Alter, der Wohnort (Region, Stadt/Land), das Monatseinkommen?

Daher gilt es zu prüfen, welche Zielgruppen des jeweiligen Unternehmens welche gemeinsamen Merkmale aufweisen. Meist sind es mehrere Kriterien, die zusammentreffen müssen. Man kann sich dabei auch an der Milieuforschung orientieren.

Beim Multivariate Test wird meist die Gestaltung der Website getestet. Die Bandbreite reicht von Textbausteinen, Überschriften, Bildern und Buttons bis zu kompletten Inhaltsblöcken und ganzen Seiten oder Seitenabfolgen. Dabei können auch unterschiedliche Besuchersegmente zum Element von Tests werden. Je nach Nutzermerkmal, wie beispielsweise verwendeter Browser oder Herkunft (z. B. geografische Herkunft, E-Mail, SEM oder SEO), aber auch Uhrzeit des Besuches, können unterschiedliche Varianten und Kombinationen untersucht werden. Anhand zentraler Performance-Daten, wie Konversionsraten für jede Variante, lassen sich während und bei Abschluss des Tests die erfolgreichste Variante identifizieren.

Anforderungen

Multivariates Testen stellt ein anspruchsvolles Projekt dar, das eine Reihe von Anforderungen und möglichen Fallstricken birgt. Es umfasst in der Regel verschiedene Beteiligte wie die mit dem Webdesign betraute Agentur sowie interne Teams aus dem fachlichen und technischen Bereich, die zielführend in das Projekt eingebunden werden sollten. Daher ist ein professionelles Projektmanagement für die Durchführung von Optimierungen mit Hilfe von Multivariate Testing sehr wichtig.

Unerlässlich sind auch eine vorausgehende Machbarkeitsanalyse und ein sachkundiges Testdesign. Denn für den Erfolg des Projekts sollten einige grundlegende Anforderungen erfüllt sein.

Voraussetzung für valide Ergebnisse aus Multivariaten Tests sind ein begrenzter Testzeitraum sowie ausreichende Website-Besuchszahlen. Wenn die Testdauer zu lang gewählt ist, können externe Rahmenbedingungen wie zum Beispiel saisonale Effekte die Ergebnisse beeinflussen und eine eindeutige Beurteilung einer Darstellungsvariante behindern.

Um aber innerhalb eines begrenzten Zeitraumes eine aussagekräftige Menge an Daten zu gewinnen, muss die Website wiederum ein Mindestbesuchsvolumen aufweisen. Dieses erhöht sich, je mehr Varianten getestet werden. Denn als Richtwert gilt, dass jede Testvariante ca. 200–300 Konversionen aufweisen sollte, um die Leistung der Varianten gesichert vergleichen und beurteilen zu können und keine statistische Verzerrungen wegen nicht ausreichender valider Daten zu erhalten. Daran sieht man, dass Multivariate Tests sich wirklich nur bei Großunternehmen oder Web-Portalen mit zahlreichen Zugriffen rechnen können.

Zudem gilt: Je aktiver die Website, desto größer der Gewinn, der durch die Conversion-Optimierung herbeigeführt wird. Und desto schneller wird der Aufwand für die Tests amortisiert.

Ein Beispiel

In einem Bestellprozess sollen insgesamt vier verschiedene Testelemente mit einer unterschiedlichen Anzahl an Varianten getestet werden. Ziel ist die nachhaltige Erhöhung der Online-Bestellungen und des Online-Umsatzes.

Anzahl der Testvarianten

Testelement 1 (Besucher-Browsertyp):	3 Varianten
Testelement 2 (Headline):	3 Varianten
Testelement 3 (Angebot):	2 Varianten
Testelement 4 (Button-Farbe):	3 Varianten
Zeitraum:	30 Tage

Berechnung der für die Durchführung des Tests notwendigen Conversions:

Notwendige Conversions = (Anzahl an Kombinationsmöglichkeiten · Mindestanzahl Conversions je Kombination)

In diesem Beispiel also

$$(3 \cdot 3 \cdot 2 \cdot 3) \cdot 200 = 10.800$$

Berechnung des für die Conversions notwendigen Besuchsaufkommens:

Conversion Rate ursprüngliche Variante:

- 1,5 %

benötigte Anzahl Visits:

- 720.000

Berechnung der Umsatzsteigerung nach dem Test:

Conversion Rate beste Kombination:

- 1,8 %

Uplift (aus der Conversion-Steigerung resultierender Anstieg der Bestellungen):

- 15 %

Bestellungen pro Monat vor Test:

- 15.000

Zusätzliche Bestellungen pro Monat:

- 2250

Durchschnittlicher Bestellwert:

- € 50

Umsatzplus pro Monat:

- € 112.500

Ein zentraler Erfolgsfaktor für Multivariate Testing ist das Testdesign, für das verschiedene statistische Methoden wie z. B. die **Taguchi-Methode oder iterative Verfahren** zur Verfügung stehen. In iterativen Verfahren wird während der Testphase anhand von Zwischenauswertungen die Anzahl der getesteten Varianten sukzessive angepasst beziehungsweise auf die erfolgreichsten Varianten reduziert.

Die Taguchi-Methode eignet sich für Fälle, in denen die Website im definierten Testzeitraum nicht das für die Variantenanzahl nötige Besuchs- und Conversion-Aufkommen aufbringen kann. Um dennoch alle möglichen Einflussfaktoren auf die Conversion-Rate zu berücksichtigen, werden bei der Taguchi-Methode sogenannte orthogonale Felder benutzt, durch die vor Beginn der Tests die Anzahl der Testvarianten auf die einflussreichsten Faktoren reduziert werden kann. Aus den Ergebnissen der getesteten Varianten wird mithilfe statistischer Modelle der Einfluss der ausgelassenen Varianten errechnet.

Da diese Methode jedoch umstritten ist, empfehlen wir, nur so viele Varianten in einem Test zu berücksichtigen, wie im Rahmen des Besuchsaufkommens der Website tatsächlich getestet werden können.

Testumfang festlegen

Einer der häufigsten Fehler bei Multivariate Tests und A/B-Tests ist, dass einfach darauf los getestet wird.

Beim Multivariate Testing werden einzelne Elemente der Website bewertet: Funktionierte das Bild des roten Apfels oder das der gelben Birne besser? Allerdings unterscheiden sich beide Obstsorten nicht nur in der Form, sondern auch in der Farbe. Wirkte die Frau oder der Mann auf dem Bild besser? Oder war dabei nicht das Geschlecht entscheidend, sondern ihre Kleidung?

Positiv bei Webtests ist, dass man mit einer relativ kurzen Vorbereitungszeit schnell mit dem Test loslegen kann und im Web sofort Ergebnisse erkennbar sind, also z. B. eine höhere Konversionsrate erzielen oder mehr Umsatz generieren kann. Selbst einfache Farbänderungen können sich dabei schon merkbar positiv auswirken. Die Wirkungen der Änderungen sind dabei immer individuell pro Seite zu betrachten.

Während ein sich bewegender Sliderbutton auf der Landingpage gut funktionieren kann, kann er sich auf anderen Seiten konversionshemmend auswirken. Es ist zu beachten, dass für jede Website eine andere Auswahl von Produkten, Seiten, Branche, User (Zielgruppe) und Besuchsabsichten existiert. Das alles ist bei den Testaktivitäten mit einzubeziehen.

Bei einem Webshop wurde z. B. dem User bei Verlassen der Seite während eines Antrags eine Warnbox angezeigt, mit der er darauf aufmerksam gemacht wurde, dass damit möglicherweise alle bisherigen Einträge verloren gingen. Damit wollte man Fehlbedienern eine letzte Chance geben – und siehe da, knapp 20 % der Abbrüche konnten damit tatsächlich verhindert werden. Hier liegt die Schlussfolgerung nahe, dass tatsächlich jeder

fünfte Antragsteller das Formular aus Versehen geschlossen hatte. Man muss dann in der Folge aber die Ursachen für die hohe Fehlbedienungsrate suchen und sollte es nicht nur bei der Warnbox belassen.

Professionelle Webanalyse-Werkzeuge verfügen über „Wärmebilder“ auf der Website, aus denen ermittelt werden kann, welche Bereiche stärker und welche seltener angeklickt werden. Diese visuellen Darstellungen zeigen eindrücklich, wo man die Navigation umstellen sollte oder wo der User verwirrt ist und deswegen den Besuch abbricht. So wurde bei einem Formular einer Website bei fehlenden Eingaben des Users eine Fehlermeldung angezeigt. Sie verschwand aber nicht sofort, wenn die fehlenden Eingaben ergänzt wurden, sondern erst bei erneutem Absenden des Formulars. Hier scheiterten etliche Besucher, die eigentlich den Kaufvorgang abschließen wollten, weil sie der Meinung waren, wegen ihrer individuellen Daten sei etwas falsch und brachen den Kaufvorgang ab. Solche Effekte kann man mit Bildschirm- und Videotracking ermitteln, die reinen Trackingdaten einer Statistiksoftware würden sie nicht anzeigen.

Über Korrelationen, welche Kunden welche Produkte kaufen, kann man ermitteln, welche Werbung wohl sinnvoll angezeigt wird.

Beispielsweise bei Zalando, Otto oder Amazon bekommt der User auf der Website bereits Hinweise auf Cross-Selling-Potenziale: „Kunden, die dieses Produkt haben, haben auch diese Produkte gekauft ...“, das ist vor allem bei Kleidung oder Schuhen sinnvoll.

Dieses Verfahren lässt sich auch auf Produkte anwenden, die zunächst einmal gar nicht im Zusammenhang stehen. Auch in Supermärkten wird mittels Data Mining bereits ermittelt, welche Produkte an welcher Stelle zusammen angeordnet werden sollten. So ist zum Beispiel herausgekommen, dass Windeln neben Bier platziert werden sollten, weil große Windelkartons oft von jungen Vätern gekauft werden. Ähnlich verhält es sich, wenn man Werbung für Outdoor-Artikel bekommt, wenn man sich für eine bestimmte Automarke interessiert hat.

Durch die Verknüpfung und Korrelationen innerhalb großer Datenmengen kann man das Surfverhalten des Einzelnen auswerten und ihm dann zielgenauer als je zuvor individualisierte Werbung zukommen lassen.

Man kann es auch wie folgt ausdrücken: *„Sage mir welche Seite Du im Internet besuchst, und ich sage Dir, wer Du bist“.*

Literatur

- [QUAL2004] http://ois-quality.de/Documents/Quality_Point-Webtest-040226.pdf
- [FRAN2007] Franz K (2007) Handbuch zum Testen von Web-Applikationen. Springer, Berlin/Heidelberg/New York. isbn:978-3-540-24539-1
- [CONV2014] <http://conversionconsult.de/was-wir-machen/ab-und-multivariate-tests-on-site-opti-mierung/>



Nachdem anfangs nur Daten, später auch Anwendungen auf Server in der Cloud verlagert wurden, werden inzwischen mehr und mehr menschliche Arbeiten in die Cloud erledigt [COWU2014]. Genauso wenig, wie man als Unternehmen wissen muss, wo Daten abgespeichert sind, muss man wissen, welche Personen letztlich eine Arbeit erledigen. Entscheidend ist nur, dass sie „irgendwer“ erledigt und das Unternehmen möglichst wenig mit diesen virtuellen Mitarbeitern zu tun hat – solange die vorgegebenen Qualitätsstandards eingehalten und die geforderten Aufgaben erfüllt werden. Diese Verlagerung in die Cloud begann mit Entwicklungsdienstleistungen und hat seit einigen Jahren auch den Softwaretest erreicht.

Die Industrialisierung von Testprozessen bedeutet also nicht unbedingt Testautomatisierung. Bei Last- oder Performancetests, bei denen gewisses Zugriffsverhalten simuliert wird und dadurch ein gewisses Lastverhalten synthetisch erzeugt wird, lässt sich die Cloud zur Testautomatisierung verwenden [ZDNE2013].

Beim Test grafischer Benutzeroberflächen oder komplexer vernetzter Softwaresysteme, deren Test ein hohes Systemverständnis oder fachliche Hintergründe voraussetzt, sind die Einsatzmöglichkeiten für Cloud-Lösungen eher begrenzt. Anders sieht es beim Test mobiler Apps oder intuitiv zu nutzenden Internet-Anwendungen aus.

Die meisten Vorteile ergeben sich aber in der Cloud durch die Bereitstellung der Testinfrastruktur. Hunderte oder Tausende Menschen können gemeinsam eine Applikation testen, und die virtualisierte Testumgebungen lassen sich grafisch konfigurieren, und Rüstzeiten für Testumgebungen werden erheblich verringert. Auch hierbei ist wiederum der Test mobiler Apps besonders geeignet, während bei einer sehr individuellen und komplexen IT-Landschaft mit anspruchsvollen Anwendungen weniger Nutzen zu erwarten ist.

Für eine mobile App, die vielleicht ein paar Tausend Mal heruntergeladen wird und die einen kleinen Umfang hat, lassen sich nur geringe Umsätze realisieren, so dass auch die Testaktivitäten begrenzt sein müssen. Hier muss das Testkonzept und die Testbeschreibung

übersichtlich bleiben und die Testdurchführung in wenigen Stunden oder Tagen erledigt sein. Hier schafft ein einzelner Tester rein zeitlich kaum eine sinnvolle Testabdeckung.

Problematisch ist allerdings in jedem Fall das Zusammenfassen der Ergebnisse von zahlreichen unterschiedlichen Nutzern. Es muss jeweils geprüft werden, ob der Fehler bereits von einem anderen Anwender gemeldet wurde, ob die Fehlerbeschreibung für den Entwickler ausreichend ist und ob die Reproduzierbarkeit der Fehler gegeben ist. Denn meist testet die Cloud ohne ein detailliertes Testkonzept explorativ, und dem Testmanager, der für das Cloud-Testing-Projekt meist ebenfalls nur für wenige Tage oder Wochen abgestellt ist, obliegt es den Überblick zu wahren und die entsprechenden Änderungsaktivitäten zu veranlassen.

Auch für die Bewertung des Fehlverhaltens und die Verwaltung der Fehler müssen in jedem Fall Kapazitäten innerhalb des Unternehmens bzw. des Softwareprojekts vorgehalten werden.

Literatur

[ZDNE2013] <http://www.zdnet.de/88172544/pro-und-contra-software-tests-in-cloud/>

[COWU2014] <http://www.computerwoche.de/a/cloudworker-ein-modell-mit-risiken-und-nebenwirkungen,3063439>



Smartphones und Tablets haben inzwischen viele Lebensbereiche erobert. Mobile IT-Systeme bieten neue Anwendungsbereiche und gestatten von der einfachen App bis zur komplexen Shopping-Lösung neue Möglichkeiten und mehr Flexibilität für den Nutzer. Mobile Anwendungen bedeuten neue Anforderungen für das Testmanagement.

30.1 Besonderheiten beim Mobile Testing

Bei mobilen Anwendungen ist vor allem zu beachten:

- Ist die Anwendung intuitiv bedienbar und die Navigation klar?
- Wie verhält sich die Anwendung auf unterschiedlichen mobilen Endgeräten?
- Wie sind Reaktionszeiten der Applikation?
- Wie sind die Anforderungen an Speicherplatz?
- Wie sieht es mit der Energieeffizienz aus?

Im Testreport ist es besonders wichtig, darauf hinzuweisen, auf welchen mobilen Endgeräten getestet wurde.

Auch die Umgebungsbedingungen sind im Freien wesentlich weiter gefächert als im Büro – sei es Lärm, Sonneneinstrahlung oder Kälte. Auch die Verfügbarkeit von WLAN ist noch lange nicht überall so wie sie sinnvollerweise schon sein sollte – der Ausbau der Datenautobahnen ist zwar ein hehres politisches Ziel, aber in der Praxis noch weit entfernt.

Daher muss man sich beim Test mobiler Applikationen besonders gut in den Enduser hineinversetzen und überlegen, an welchen Orten und in welchen Situationen er häufig auf die mobile Anwendung zugreifen möchte. Das kann in der Folge auch bedeuten, dass sich

auch der Software-Entwickler aus seinem Büro entfernen muss, um die Resultate seiner Änderungen in einem möglichst realitätsnahen Umfeld zu erkennen.

Der Verbreitungsgrad und der Erfahrungsschatz bei mobilen Anwendungen sind deutlich kleiner als bei stationären Programmen. Daher ist ein umfassendes Requirement-Management hier manchmal, so notwendig es auch wäre, nicht zu erwarten, weil die Computeraffinität der Zielgruppe nicht so ausgeprägt ist. Gerade für mobile Applikationen bieten deshalb agile Ansätze viele Optionen iterativ, im Laufe des Projekts, Anforderungen zu erarbeiten und die Kreativität der Anwender zu nutzen.

Bei mobilen Endgeräten ist der Aspekt der Usability und Ergonomie stärker betont als bei der klassischen Softwareentwicklung. In vielen Fällen wird das Testen mobiler Anwendungen noch als Ableger oder Randprodukt betrachtet, was sich auch an den Preisen der Apps zeigt. Aber gerade durch den stärkeren Fokus auf die Gebrauchstauglichkeit können mobile Anwendungen das Testen für stationäre Anwendungen beeinflussen, denn Funktionalität allein reicht für ein Produkt nicht aus, wenn die Software mehr und mehr Lebensbereiche durchdringt [RITZ2008].

30.2 Auswirkungen auf das Testmanagement

Bei klassischen Anwendungen gibt es ein eingespieltes Testteam innerhalb des Unternehmens. Die Tests werden strukturiert durchgeführt, die Aufgabenstellung nach dem V-Modell erarbeitet und umgesetzt. Wenn die Anwendung komplex ist und die Entwicklungszyklen mehrere Monate bis Jahre dauern und eine gewisse Infrastruktur voraussetzen, wird das auch in Zukunft der Fall sein. Man hat Zeit für die Testautomatisierung, zur Beschreibung dedizierter Testprozeduren und kann Offshoring punktuell einsetzen.

Auf der anderen Seite hat man aber mehr und mehr mobile Anwendungen, schnellere Entwicklungszyklen und webbasierte Oberflächen, die ein ganz anderes Vorgehen für den Softwaretest sinnvoll erscheinen lassen. Hier kommen agile Ansätze wie Scrum besser zur Geltung. Man muss immer die Art der Anwendung betrachten und sich dann für ein Verfahren entscheiden, und dabei auch Ansätze mischen.

Es ist nicht möglich alle Vorteile eines theoretischen Modells zu erreichen, wenn man es nicht konsequent umsetzt. Das muss allen Beteiligten beim Testmanagement bewusst sein. Manchmal ist es aber besser, einen Ansatz in bestimmten Punkten abzuwandeln, weil man sonst die Organisation überfordert und es besser ist, einen modellbasierten Ansatz zu 80 % in einem akzeptablen Zeitraum umzusetzen anstatt 99 % nie.

Literatur

[RITZ2008] <http://it-radar.eu/uploads/reports/TranskriptProf.Ritz.pdf>

Unter **agilem Testen** versteht man das Testen von Software im Rahmen eines agilen Entwicklungsprojekts. Daher muss sich der Test in einem agilen Entwicklungsprojekt auf die Unterstützung des Entwicklungsteams konzentrieren. Viele Eigenschaften von agilen Testern helfen dabei auch einem traditionellen Tester.

Ausgangspunkt für agile Softwareentwicklung ist das agile Manifest. Auf dieses Manifest berufen sich die meisten Methoden der agilen Entwicklung wie beispielsweise eXtreme Programming oder Scrum. Das agile Manifest lässt bewusst offen, wie im Einzelfall die Entwicklung durchgeführt wird. Kern des agilen Manifests sind die vier Werte, die eine Gegenüberstellung traditioneller Entwicklungsansätze mit agilen Ansätzen formulieren:

- Individuen und Interaktionen gelten mehr als Prozesse und Tools.
- Funktionierende Programme gelten mehr als ausführliche Dokumentationen.
- Die stetige Zusammenarbeit mit dem Kunden steht über Verträgen.
- Der Mut und die Offenheit für Änderungen stehen über dem Befolgen eines festgelegten Plans.

Auf der Basis dieser Werte wurden verschiedene Praktiken entwickelt, die immer wieder im Zusammenhang mit agiler Entwicklung erwähnt werden. Zu diesen gehören unter anderem:

- Paarprogrammierung
- testgetriebene Entwicklung
- gemeinsamer Codebesitz
- ständige Refaktorisierung
- Entwicklung in Iterationen bzw. Zyklen
- Story Cards

- Feature Driven Development
- ständige Integration (Daily Build)
- ständig lauffähiges Produkt

Die Methoden der agilen Programmierung verwenden jeweils einige der oben beschriebenen Praktiken.

Einige Werte scheinen direkt mit der Kleinheit der Projekte zusammenzuhängen. Welches Milliardenprojekt würde schon ohne Verträge gestartet werden? Resultiert der Mut zur Offenheit ohne Plan nicht direkt im Chaos? Andererseits zeigen Großprojekte, dass sich einige Praktiken wie beispielsweise der Daily Build durchaus in großen Projekten erfolgreich einsetzen lassen [[JACO2016](#)].

31.1 Besonderheiten beim Agile Testing

Agiles Testen folgt den Prinzipien des agilen Manifests und wendet die agilen Prinzipien auf das Testen an.

Entwicklungsmethoden wie Extreme Programming, Scrum, Kanban werden dazu eingesetzt, um die Entwicklungszyklen eines Projekts und die Time-to-Market zu verkürzen, und schneller Rückmeldungen und Ergebnisse über die Software zu erhalten, anstatt erst ein umfangreiches Softwarepaket zu entwickeln und anschließend testen zu lassen [[NAGA2018](#)]. Dadurch enthalten alle am Projekt Beteiligten stets das aktuelle Bild über den laufenden Stand der Entwicklung und der Produktqualität.

In agilen Projekten geht man davon aus, dass Änderungen von Anforderungen während des Projekts normal sind. Es werden also nicht erst alle Requirements formuliert, sondern nur ein Bruchteil, diese Anforderungen werden dann umgesetzt und getestet, und iterativ wird die Software weiterentwickelt.

Die Auslieferung neuer Software geschieht bei agilen Ansätzen in „Sprints“ im Rhythmus von 2 oder (in der Praxis seltener) 4 Wochen. Der Softwaretest muss sich an diese höheren Taktfrequenzen anpassen.

Aus „User Stories“ (Kundenanforderungen) werden zu Beginn des Sprints Tasks (festgelegte Aufgaben) für die Entwickler abgeleitet. Planung, Verfolgung und Steuerung agiler Projekte werden in Task Boards durchgeführt.

Am Anfang der agilen Entwicklung wurde zwar agil entwickelt, der Unittest vielleicht auch noch agil, der Systemtest aber doch wieder nach der klassischen Methode durchgeführt. Erst nach und nach wurden agile Methoden auch verstärkt im Integrationstest und im Systemtest angewendet.

Man spricht in diesem Zusammenhang auch von „Continuous Integration“ und „Test Driven Development“. Man überlegt sich also während der Entwicklung schon, wie das fertige Stück Software optimal in Testprozeduren nachgewiesen werden kann.

Agiles Testen muss derart gestaltet sein, dass es die Ziele agiler Softwareentwicklung optimal unterstützt, nämlich

- hohe Kundenzufriedenheit, die durch die bessere Kooperation zwischen dem Kunden bzw. Kundenvertreter (bei Scrum beispielsweise der „Product Owner“) und dem Entwicklungsteam entsteht,
- hohe Produktqualität, weil die Entwicklungsaktivitäten von Anfang an auf das Vermeiden und Auffinden von Fehlern konzentriert,
- hohe Entwicklungsgeschwindigkeit, die durch die Reduktion von Blindarbeit (unnötige Dokumentation, unnötige Fehlerkorrekturarbeit usw.) bewirkt wird,
- schnelle Reaktion auf Änderungen, die durch kurze Zyklen und Iterationen ermöglicht wird,
- hohe Teammotivation, die durch Selbstmotivation und dauerhaft einhaltbares Arbeitstempo unterstützt wird.

Dabei müssen unter Umständen Werte des Testens im „klassischen“, meist phasenorientierten oder iterativ-inkrementellen Vorgehen Berücksichtigung finden, z. B. in einem regulierten Umfeld:

- gute Planbarkeit, Verfolgbarkeit und Nachweisbarkeit der Testaktivitäten
- hohe Systematik des Testentwurfs, daraus resultierend eine hohe und reproduzierbare Testabdeckung [ISTE2009]

Testgetriebene Entwicklung: Testgetriebene Entwicklung bedeutet, dass zuerst Testfälle spezifiziert (bzw. automatisiert) werden und dann erst der Code geschrieben wird, auf den diese Testfälle angewendet werden sollen. Neben diesem methodischen Aspekt hat der Begriff „testgetrieben“ noch eine zweite Dimension: Der Begriff verschiebt das Machtgefälle innerhalb der Software-Entwicklung hin zum Testen. In den Projekten werden zwar Abnahmekriterien und Qualitätsstandards definiert. Methodisch bleiben diese Ansätze allerdings hinter der testgetriebenen Entwicklung zurück. Betrachtet man die organisatorische Dimension, dann wurde eine ähnliche Machtverschiebung hin zur Integration, zum Netzwerktest, implementiert [JACO2016].

In agilen Projekten wird das System schneller und dadurch auch häufiger geändert als in klassischen Testprojekten. Daher müssen auch dieselben Tests öfter wiederholt ausgeführt werden. Dadurch rechnen sich Maßnahmen der Testautomatisierung in agilen Projekten wesentlich eher als in klassischen Testprojekten. In der Regel ist eine existierende Testautomatisierung eine wesentliche Voraussetzung dafür, dass ein Projekt überhaupt agil durchgeführt werden kann.

Das **Task Board** klassifiziert die Testaufgaben und gliedert sie in Test Tasks. Jede Task hat dabei einen Zustand, also z. B.:

- offen
- in Bearbeitung (Testfall und Testprozedur werden geschrieben)
- Test durchgeführt
- Fehler gemeldet
- abgeschlossen

Im Idealfall haben nach dem jeweiligen Sprint alle Test Tasks den Zustand „abgeschlossen“ erreicht. Wenn im Rahmen ein und desselben Sprints mehrere Testversionen vorhanden sind (Spike), können diese Zustände für jeden getesteten Spike erneut als Regressionstest durchlaufen werden.

31.2 Testmanagement in agilen Testprojekten

Für das Testmanagement in agilen Testprojekten gelten einige Besonderheiten:

- Der Product Owner ist immer im Projekt involviert. Das heißt, die betreffende Person kann und muss jederzeit für Fragen zur Verfügung stehen, etwa: Wie ist diese Anforderung genau gemeint? Wie wichtig ist die Umsetzung dieser Anforderungen im aktuellen Sprint? Sind die konzipierten Testfälle adäquat oder gibt es Missverständnisse zwischen Kunde und Entwicklung?
- Der Fokus liegt beim Testen also eher auf der Validierung als auf einer Verifikation: Das gewünschte Ergebnis soll als Referenz dienen!
- Die Tester übernehmen mehr Verantwortung, Tester und Entwickler sind gleichberechtigte Mitglieder im Scrum Team. Die klassische Sichtweise, dass der Tester nur die Umsetzung dessen überprüft, was ein Entwickler aus einer Anforderung gemacht hat, verschiebt sich zugunsten einer gleichberechtigten Kooperation. Ergebnis: Ein Tester bringt seine Ideen, etwa eine andere Risikosicht, von Anfang an ins Projekt ein.
- Geringere Diversifikation der Rolle des Testers: Die Tester bearbeiten im Rahmen eines Sprints alle Aufgaben, wie beispielsweise das Ableiten von Testfällen aus Anforderungsspezifikationen, die Automatisierung und die Durchführung von Testfällen sowie die Bewertung der Ergebnisse. Dabei können durchaus unterschiedliche Tester verschiedene Aufgaben wahrnehmen, aber anders als in klassischen Entwicklungsprojekten finden die einzelnen Aktivitäten (Testbeschreibung, Testautomatisierung, Testdurchführung) parallel und nicht nacheinander statt.
- Den „Nur-Testdurchführer“ oder den „Nur-Testautomatisierer“ wird es nicht geben; Testen wird als ganzheitliche Aktivität verstanden. Für regelmäßige Regressionstests müssen automatisierte Testfälle vorliegen.
- Tester sind an allen Sprint Meetings beteiligt.
- Tester sehen, welche aktuellen Probleme in der Entwicklung auftauchen und können ggf. durch entsprechende Tests schnell reagieren.
- Die Kommunikationswege sind direkt; Tester können dadurch die Feinplanung des Sprints maßgeblich beeinflussen.

Voraussetzung für agiles Testen ist eine offene Kommunikationskultur in Entwicklung und Test unter ständigem Einbezug des Kunden sowie der gegenseitigen Anerkennung von Entwicklern, Testern und Projektmoderatoren im Projektteam.

31.3 Kombination traditioneller Testmethoden und agilen Methoden

In manchen Teststufen, wie Unittest oder Integrationstest, treten die Vorteile der agilen Methoden deutlicher hervor als beim Systemtest. Vor allem, wenn die Anforderungen erst im Lauf der Entwicklung herausgearbeitet werden, sind agile Methoden den klassischen Methoden überlegen.

Daher gibt es auch Mischformen, bei denen auf den höheren Teststufen nach wie vor nach V-Modell und bei den entwicklungsnahen Teststufen nach agilen Methoden getestet wird. Das bewirkt eine bessere Systematik der Testdurchführung. Man muss sich dabei aber bewusst sein, dass diese Mischformen ein hohes Maß an Kommunikationsaufwand erfordern. Gerade in Transformationsphasen, wenn also etwa der Integrationstest bereits agil, der Systemtest aber noch klassisch durchgeführt wird, ist das Risiko von weggelassenen oder doppelt durchgeführten Testaktivitäten sehr hoch, weil eine permanente Abstimmung in Detailfragen erforderlich ist. Viele Projekte befinden sich derzeit in einer derartigen Phase, was die Effizienz der Testausführung in der Praxis oft verringert und der Qualität eher abträglich ist, weil die Rahmenbedingungen nicht ausreichend geklärt sind.

Literatur

- [NAGA2018] <http://www.anecon.com/angebot/software-test/testprozess/testautomatisierung/segen.html>
- [ISTE2009] http://www.iste.uni-stuttgart.de/fileadmin/user_upload/iste/se/people/schmidberger/downloads/Grundlagen_Testen-110_k.pdf
- [JACO2016] http://pi.informatik.uni-siegen.de/stt/30_3/01_Fachgruppenberichte/TAV/05_Jacobs_AgilittGroprojekte_V1.pdf

In den Augen des Auftraggebers brauchen Tests zu lange und sind zu teuer. Tester kommunizieren gerne im Fachjargon, z. B. über formale Abdeckungskriterien, Begriffe, deren Bedeutung dem Nicht-Tester häufig verschlossen bleibt. In den Augen des Testmanagers hingegen beschränkt sich das Qualitätsbewusstsein seines Auftraggebers auf Lippenbekenntnisse. Dieser gönnt den Entwicklern Verspätungen, Budgetüberschreitung und unvollständige Lieferungen und will dies durch Einsparungen beim Testen kompensieren. Er versteht die Testberichte nur auf einem rudimentären Niveau, z. B. einer Ampel. Der Testmanager kommt sich dann vor wie der einsame Rufer in der Wüste.

Heutige Managementansätze basieren hingegen auf Corporate Governance, einem (meistens nationalen) Richtliniensystem, das die optimale Unterstützung der Geschäftsziele des Unternehmens auf strategischer, funktionaler und operationaler Ebene verlangt. Börsennotierte Unternehmen sind sogar durch Regelungen wie dem Sarbanes-Oxley Act (SOX) oder „Basel II“ dazu verpflichtet. Die IT als wesentlicher Unternehmensbereich ist auch betroffen und hat mit Managementansätzen der IT-Governance darauf reagiert. Diese Anforderungen sind vom Testmanagement zu berücksichtigen.

Das geschäftsbasierte Testmanagement (englisch: Business Driven Test Management, Abkürzung: BDTM) ist der Test Governance-Ansatz der Methode TMAP Next.

Merkmale des **BDTM-Ansatzes** sind:

- Der Gesamtaufwand für das Testen bezieht sich auf Risiken des Systems, das für eine Organisation getestet werden soll. Der Einsatz von Menschen, Ressourcen und Budget konzentriert sich auf die Teile des Systems, die für die Organisation am wichtigsten sind. TMap bietet die Möglichkeit festzustellen wie viele Risiken durch den gewählten Test abgedeckt werden und so das Restrisiko abzuschätzen.

- Der geschätzte Aufwand und die Planung des Testprozesses sind eng mit der festgelegten Teststrategie verbunden. So kann leicht auf Änderungen eingegangen werden, da immer ein Plan zur Verfügung steht wie viel Zeit, Budget und Ressourcen benötigt werden.
- Der Kunde wird in den Testablauf eingebunden, so dass auf die Wünsche des Kunden besser eingegangen werden kann. Ein BDTM-Ansatz kann auch die Konsequenzen zukünftiger und vergangener Entscheidungen sichtbar machen.

Dieser Ansatz geht davon aus, dass der Testprozess die „Test Governance“ berücksichtigt:

- Der Tester weist Eigenschaften des Produktes nach und liefert ein Ergebnis und deckt Fehler auf. Dieses Testergebnis erzeugt Vertrauen.
- Der Tester informiert über Zeit, Kosten und den Testfortschritt.
- Der Tester benennt Risiken.

Test-Governance ist demnach eine Testpolitik, die in Testhandbüchern, Testkonzepten und -planungen umgesetzt werden muss. Folgende praxiserprobte Elemente des Testmanagements können hierzu beitragen:

- Risikobasiertes Testen
- Rückverfolgbarkeit der Testergebnisse auf die Anforderungen
- professionelle Planung und Steuerung des Testprozesses als Teilprojekt
- regelmäßiges Berichten über die Ergebnisse, Kosten und den Zeitverlauf des Testens an den Auftraggeber.

Beim geschäftsbasierten Testmanagement (s. Abb. 32.1) hilft das Testmanagement seinem Auftraggeber dabei, die Teststrategie festzulegen und ermöglicht ihm, den Testprozess unter Berücksichtigung der Governance verantwortlich zu lenken. Dazu werden folgende Schritte durchgeführt:

- 1) Formulierung des Auftrags und Zusammenstellen der Testziele unter der Beachtung von erwarteten Ergebnis (Qualitätsanspruch), Risiko, Zeit und Kosten
- 2) Bestimmung der **Risikoklasse**: Aufgrund der Produktrisikooanalyse werden Merkmale und zu testende Teilobjekte festgelegt. Für jedes Testobjekt wird in Bezug auf die Testziele und die relevanten Merkmale eine Risikoklasse hinterlegt.
- 3) Festlegung der Testintensität: Je höher das Risiko, desto mehr muss das Testobjekt getestet werden.
- 4) Rückmeldung von Ergebnis, Risiko, Zeit und Aufwand an den Auftraggeber. Das Ergebnis ist eine Testplanung, die eine Steuerung des Testprozesses durch den Auftraggeber ermöglicht und die richtige Abwägung zwischen den konkurrierenden Zielen zu treffen.
- 5) Auswahl von Testtechniken: Der Testmanager leitet aus der Testintensität konkrete Maßnahmen und Techniken ab, um das Testziel zu erreichen. Mit Hilfe dieser Techniken werden die bei der Testfallerstellung die Testfälle aus der Testbasis abgeleitet.

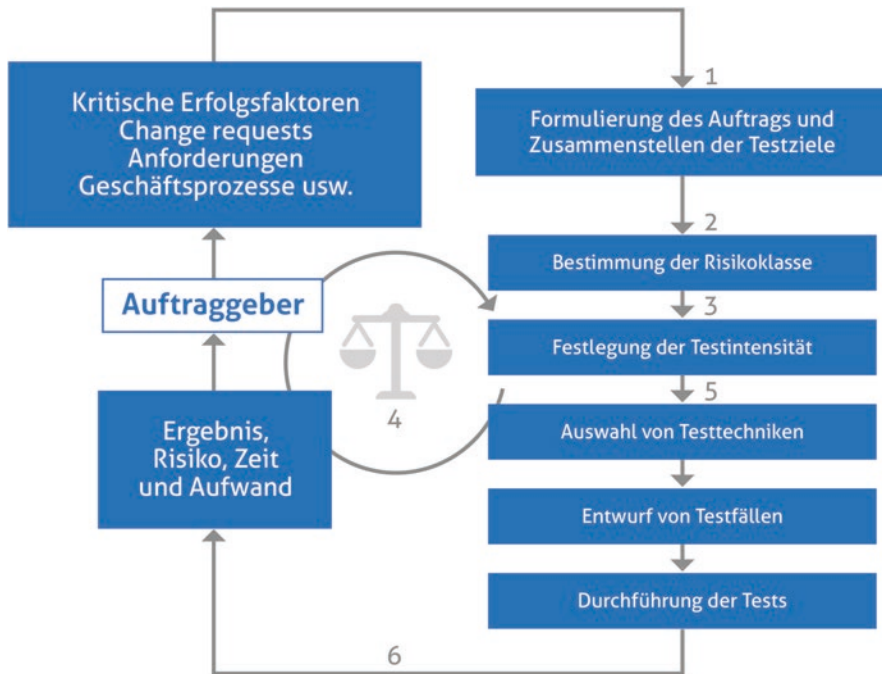


Abb. 32.1 Geschäftsbasiertes Testmanagement (BDTM-Ansatz)

6) Rückmeldung von Ergebnis, Risiko, Zeit und Aufwand an den Auftraggeber: Während des gesamten Testprozesses bietet der Testmanager dem Auftraggeber und anderen Stakeholdern adäquate Einsicht in und Kontrollmöglichkeiten über die Governance Aspekte

- Ergebnis (die erreichten Testziele)
- Risiken des Testobjekts und des Testprozesses
- Kosten
- Zeit/Testfortschritt

Die konsequente Umsetzung des geschäftsbasierten Testmanagements führt zu folgenden Vorteilen:

- Der Testprozess wird für den Auftraggeber und die Stakeholder transparent und ist in allen IT-Governance Aspekten steuerbar.
- Der Testmanager kommuniziert und berichtet in der Sprache seines Auftraggebers und liefert dabei Informationen, die für den Auftraggeber in seinem Kontext verwendbar sind.
- Der Testmanager steht nicht im Zielkonflikt mit seinem Auftraggeber, sondern kooperiert mit ihm als dessen Dienstleister [SIGS2008].

Das Testen wird also darauf ausgerichtet, zum Business Case optimal mit Informationen über Produktrisiken, Projektrisiken, Planung und Kostenschätzung des Testens und Zeitschätzung beizutragen. Das Testmanagement soll für Transparenz sorgen, die Kommunikationslücke zum Kunden schließen – und den Kunden dadurch bei seiner Entscheidung unterstützen. Auch wenn dieses Prinzip einfach erscheint, wurde es bisher in der Praxis kaum konsequent umgesetzt. Das Verfahren **TMap** macht es durch den BDTM-Ansatz praktikabel [AALS2008].

Literatur

- [SIGS2008] https://www.sigs-datacom.de/uploads/tx_dmjournals/hamburg_vonalst_OS_testing_08.pdf
- [AALS2008] http://pi.informatik.uni-siegen.de/stt/28_1/01_Fachgruppenberichte/TAV/TAV-26P9AalstHamburg_neu.pdf

Sogetis Modell der Testprozessverbesserung („TPI“) wurde in der zweiten Hälfte der neunziger Jahre entwickelt, als Unternehmen die negativen Folgen fehlerhafter Systeme zu spüren bekamen und sich darüber bewusst wurden, dass es immer wichtiger wird, ihre Testprozesse zu verbessern. Im Laufe der Zeit wurden mit Anwendung dieses Modells eine Menge Erfahrungen gesammelt und eine Reihe von Best Practices entwickelt. Daraus entstand das TPI Next Modell zur Testprozessoptimierung, mit dem die Optimierungsschritte gezielt an die spezifischen Bedürfnisse eines Unternehmens angepasst werden können.

33.1 Kernbereiche im TPI Next Modell

Das **TPI Next Modell** besteht aus 16 Kernbereichen, die es ermöglichen, den Testprozess detailliert zu bewerten und schrittweise zu optimieren. Der Reifegrad jedes Kernbereichs kann separat gemessen werden. In ihrer Gesamtheit umfassen die Kernbereiche alle Aspekte des Testprozesses.

Die Kernbereiche werden in die Kategorien Stakeholderbeziehungen (SB), Testmanagement (TM) und Testkompetenz (TK) unterteilt. Durch die Klassifizierung der Kernbereiche mit Hilfe einer Testreifematrix werden die relativen Stärken jeder Kategorie nach der Bewertung des Istzustands eines Testprozesses deutlich erkennbar.

Die 16 Kernbereiche sind im Einzelnen (in Klammern die zugeordnete Kategorie):

1. Engagement der Stakeholder (SB)
2. Grad der Beteiligung (SB)
3. Teststrategie (SB)
4. Testorganisation (SB)
5. Kommunikation (SB)

6. Berichterstattung (SB)
7. Testprozessmanagement (TM)
8. Kostenschätzung und Planung (TM)
9. Metriken (TM)
10. Fehlermanagement (TM)
11. Testwaremanagement (TM)
12. Methodisches Vorgehen (TK)
13. Professionalität der Tester (TK)
14. Testfalldesign (TK)
15. Testwerkzeuge (TK)
16. Testumgebung (TK)

Die einzelnen Testprozesse werden dabei in einen bestimmten **Reifegrad** eingestuft. Mit Erreichen eines höheren Reifegrads nimmt die Reife des Testprozesses zu und der geschäftliche Mehrwert steigt. Zur Ermittlung und Einstufung des Testprozesses in den entsprechenden Reifegrad existieren definierte Kriterien.

Die einzelnen Reifegrade lauten dabei „Initial“, „Kontrolliert“, „Effizient“ und „Optimierend“.

Im Automotive TPI sind diese Kernbereiche ebenfalls enthalten und werden durch einen weiteren Kernbereich „Integrationstests“ ergänzt [SCHL2010].

33.2 Der Änderungsprozess beim geschäftsbasierten TPI

Der Änderungsprozess beim geschäftsbasierten TPI besteht aus mehreren Schritten (s. Abb. 33.1):

- Schaffung des Bewusstseins: Es wird den Stakeholdern transparent, warum es erforderlich ist, den Testprozess zu optimieren.
- Festlegung von Zielen, Betrachtungsbereich und Vorgehen: Es wird ein Ziel festgelegt, wo sich der Testprozess kurz- oder langfristig befinden soll. Ansatzpunkte, die bereits in der gegenwärtigen Situation existieren, werden dazu geprüft.
- Bewertung des Istzustands: Die Beurteilung der jetzigen Situation wird vorgenommen. Stärken und Schwächen des Istzustands werden definiert.
- Definition von Verbesserungen: Es wird geprüft, welche Maßnahmen durchgeführt werden müssen, um die Ziele zu erreichen.
- Erstellung eines Maßnahmenplans: Dabei wird ermittelt, wie die gewünschte Situation erreicht werden kann, wer dabei involviert ist und welche Ressourcen benötigt werden.
- Implementierung der Maßnahmen: Diese Phase besteht aus Umsetzung der Verbesserungsmaßnahmen, Anwendung neuer Verfahren und Vorlagen, Sammeln von Praxiserfahrung.
- Bewertung und Neuausrichtung: Es wird geprüft, ob das Ziel erreicht wurde und die Situation den Wünschen entspricht. Das weitere Vorgehen wird geprüft.

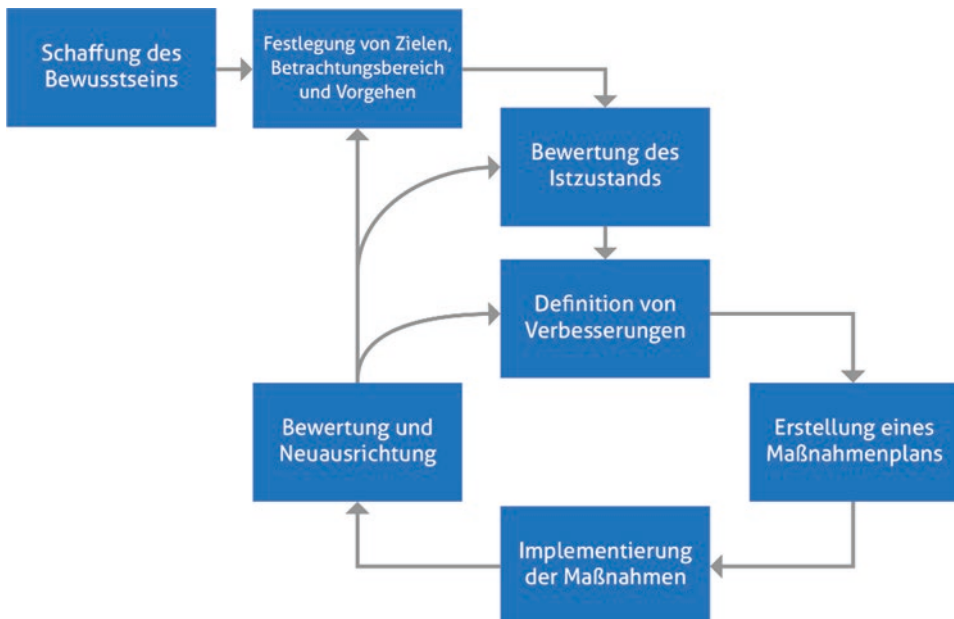


Abb. 33.1 Änderungsprozess und Kernaktivitäten bei TPI Next

Bei jeder Phase gibt es bestimmte Erfolgsfaktoren. Meist ist es besonders schwierig, die von Änderungen Betroffenen zu überzeugen, Widerstände zu überwinden und ihr Engagement zu fördern. Das trifft aber auf jegliche Änderungsprozesse zu.

Für das Testen sind im Laufe dieses Änderungsprozesses jedoch einige Punkte besonders zu beachten.

1) Schaffung des Bewusstseins:

- Notwendigkeit der Verbesserung des bestehenden Testprozesses erkennen
- Anerkennung des BDTPI-Modells zur Umsetzung
- Verständnis, dass mit der Änderung des Testprozesses eine Änderung der Arbeitsweise einhergeht

Gerade in Organisationen, in denen ein Team von Softwaretestern schon viele Jahre nach bestimmten, eingeschliffenen Mustern arbeiten, ist es oft schwierig, Verbesserungspotenziale aufzuzeigen und die Mitarbeiter zu motivieren.

2) Festlegung von Zielen, Betrachtungsbereich und Vorgehen:

Im Rahmen eines BDTPI-Assessments werden die Testaktivitäten festgelegt und beschrieben. Der Zeitplan wird aufgestellt und die notwendigen Ressourcen ermittelt. In einem kontrollierten Testprozess zielt der Testaufwand auf die kritischen Teile eines Systems ab, wobei die wichtigsten Fehler bei geringstem Kostenaufwand gefunden werden. Dabei ist es erforderlich, messbare Kriterien von Anfang an zu definieren, um den Grad der Zielerreichung konkret nachweisen zu können.

Der Betrachtungsbereich kann dabei eine bestimmte Teststufe oder bestimmte Testarten sein.

Wenn Ziele und Betrachtungsbereich festgelegt wurden, wird das weitere Vorgehen definiert. Falls der Betrachtungsbereich sehr weit gewählt ist, empfiehlt es sich, zunächst ein Referenzprojekt zu verwenden. Ein Referenzprojekt muss so gewählt sein, dass es allgemeine Aussagen erlaubt die auf andere Projekte gut übertragbar sind und dass die Informationen gut zugänglich und für eine Analyse geeignet sind.

3) Bewertung des Istzustands:

Zur Bewertung des Istzustands können Interviews und bestehende Dokumentationen herangezogen werden. Die dabei gewonnenen Informationen müssen strukturiert analysiert werden. Bemerkungen und Befunde werden in einer Tabelle notiert.

4) Definition von Verbesserungen

Die Ziele zur Verbesserung des Testprozesses werden als kurz- mittel- oder langfristige Ziele beschrieben. Nach Definition des Sollzustandes wird der Verbesserungspfad gewählt. Es empfiehlt sich, eine Gruppierung der Ziele vorzunehmen. Man kann sich dabei an den Kriterien der Basisgruppierung im TPI Next Modell orientieren:

- Engagement der Stakeholder
- Grad der Beteiligung
- Teststrategie
- Testorganisation
- Kommunikation
- Berichterstattung
- Testprozessmanagement
- Kostenschätzung und Planung
- Metriken
- Fehlermanagement
- Testwaremanagement
- Methodisches Vorgehen
- Professionalität der Tester
- Testfalldesign
- Testfallwerkzeuge
- Testumgebung

In der Regel müssen diese Ziele aber konkretisiert und auf das eigene Unternehmen und die besonderen Herausforderungen im Softwaretest abgestimmt werden, da sie sonst zu allgemein bleiben.

Dann wird geprüft, ob die einzelnen Verbesserungsvorschläge machbar sind und mit welchen geeigneten Maßnahmen ihre Umsetzung erfolgen kann.

Mögliche Verbesserungsmaßnahmen sollten dabei auf die IT- und Geschäftsziele bezogen werden. Mögliche positive Auswirkungen auf folgende Punkte sind dabei besonders zu beachten:

- Flexibilität der Testressourcen
- Qualität des Testprozesses
- Vorbereitung der Testorganisation für zukünftige Änderungen
- Wissensaustausch und -konsolidierung

5) Erstellung eines Maßnahmenplans:

Es ist erforderlich, einen Implementierungsplan mit definierten Phasen und Milestones zu definieren. Das Management muss die Umsetzung der Maßnahmen aktiv verfolgen. Schwierig wird es vor allem dann, wenn laufende Projekte eine höhere Dringlichkeit besitzen als die Änderung von Prozessen

Nach meiner Erfahrung ist es wenig hilfreich, die Verbesserung von Abläufen gegen den Zeitdruck laufender Projekte auszuspielen. Dabei gewinnt immer das Projekt, weil dafür gerade Geld bereitsteht und es darum geht, das Projekt fertigzustellen, mit welchen ineffizienten Prozessen auch immer das am Ende geschieht. Ein Projekt bringt sofort Einnahmen, eine Prozessverbesserung erst mittelfristig, dafür aber langfristig umso mehr. Daher ist es sinnvoll, Verbesserungen innerhalb existierender Projekte von Anfang an einfließen zu lassen, Maßnahmen schon ab Projektstart nach neuen Verfahren zu beginnen und dabei Erfahrungen im Projekt zu sammeln, und dann nach Projektende diese prozessualen Optimierungen auf das Gesamtunternehmen zu übertragen. Das ist auch dafür wichtig, dass die Prozessänderungen nicht als theoretischer Überbau erlebt werden, sondern als effiziente Maßnahmen die bei der täglichen Projektarbeit sofort Nutzen bringen. Problematisch dabei ist lediglich, dass dieses pragmatische Vorgehen eine hohe Flexibilität und einen ausgeprägten Veränderungswillen der Stakeholder erfordert. Das einzelne Projekt kann dadurch zusätzlich temporär belastet werden. Der Projektleiter ist also besonders zu motivieren und auch zu unterstützen um diese notwendigen Änderungen als Pilotanwender umzusetzen.

6) Implementierung der Maßnahmen:

In dieser Phase wird der Plan umgesetzt. Die Auswirkungen des Änderungsprozesses zeigen sich hier bereits in der täglichen Praxis. Auf Basis von BDTPI-Kontrollpunkten werden regelmäßig Soll-Ist-Vergleiche durchgeführt um die Zielerreichung zu messen und bei Problemen gegensteuern zu können.

7) Bewertung und Neuausrichtung:

In dieser Phase werden die Ergebnisse der Optimierungsmaßnahmen mit den ursprünglichen Zielen, Plänen und Anforderungen verglichen. An dieser Stelle kann der Prozess eine neue Richtung einschlagen, wie z. B. die Neubewertung oder eine neue Definition der Verbesserungsmaßnahmen oder sogar ein Abbruch oder eine Unterbrechung des Prozesses.

Bei Implementierungsprozessen, die länger als 6 Monate dauern (erfahrungsgemäß ist das in den meisten Organisationen der Fall), empfiehlt es sich, mithilfe des BDTPI-Modells Zwischenbewertungen des Istzustands bei den im Prozessverlauf erreichten Meilensteinen durchzuführen [SOG2011]. Bei einem ausgewogenen An-

satz werden alle Aspekte des Testprozesses gleich gewichtet und gleich behandelt. Bei der geschäftsbasierten Anwendung dieses Modells kommt es zu einer Gewichtung, bei dem die einzelnen Kernbereiche unterschiedlich große Beiträge zu den relevanten Geschäftstreibern beisteuern. Die wichtigen Kontrollpunkte erhalten dadurch eine höhere Priorität bei der Umsetzung.

Literatur

- [SOG2011] Sogeti (2011) TPI Next – Geschäftsbasierte Verbesserung des Testprozesses. dpunkt, Heidelberg
- [SCHL2010] <https://thequaliteers.de/nachrichten-leser/items/so-koennen-sie-tpi-fuer-sich-anwenden-teil-1.html>



Die Testumgebung soll die tatsächliche Umgebung so realitätsgetreu wie möglich darstellen. Sonst kann es schnell passieren, dass ein Projekt alle Tests bestanden und grünes Licht für die Freigabe erhält, und ausfällt, sobald es live geht.

Für den Aufbau einer Testumgebung müssen alle relevanten Anforderungen und Informationen zusammengestellt werden, die notwendig sind. Nicht jedes Teilsystem benötigt jedoch eine vollständige Testumgebung; so müssen z. B. Testumgebungen in Batch-Bereichen – im Gegensatz zu Online-Bereichen – oft nur aus einer Datenbasis bestehen.

Folgende Anforderungen und Fragen sind beim Aufbau einer Testumgebung zu klären:

- parallele Testaktivitäten innerhalb einer Umgebung
- separate Testumgebungen für unterschiedliche Tester
- Anzahl der logischen Datenbanken
- verwendete Systeme zur Transaktionsbearbeitung
- benutzte Datenbanksysteme
- benötigte Dateitypen
- benötigte Treiber
- Einsatz permanenter Systemdateien
- benötigte Hilfsprogramme (z. B. zur Kontrolle der Datenbanken)
- Einsatz von Capture-/Replay-Tools zur Testautomatisierung
- Schnittstellen innerhalb der Testumgebung

Jedes Teilsystem und jede Teststufe sollte eine eigene Testumgebung zur Verfügung haben, damit der Test ohne Einflüsse von Entwicklern und Testern anderer Teilsysteme/ Teststufen ausgeführt werden kann.

Tab. 34.1 Testumgebungen und Releases

	Januar	Februar	März	April
Testumgebung A	Release 1.2	Release 2.0	Release 2.1	Release 2.2
Testumgebung B	Release 1.1	Release 1.2	Release 2.0	Release 2.1
Testumgebung B	Release 1.0	Release 1.1	Release 1.2	Release 2.0
Produktion	Release 1.0	Release 1.1	Release 1.2	Release 2.0

Zur Entwicklungsumgebung müssen mindestens zwei zusätzliche Testumgebungen eingerichtet werden: Eine für den Integrationstest und eine für den Systemtest, an welchem bei Bedarf mehrere Anwendungssysteme beteiligt sind [HETT2018].

Die Testumgebung für den Systemtest sollte so lange zur Verfügung stehen, wie das getestete Release in Produktion ist, um Fehler, die in der Produktion auftreten, jederzeit auf einer produktionsnahen Umgebung nachtesten zu können. Ich habe selber schon mehrfach erlebt, dass die Testumgebung nach Produktiveinführung des Systems zerpfückt wurde und dann in umständlichen Prozeduren unter hohem Zeitdruck wiederhergestellt werden musste.

Das bedeutet in der Praxis, dass unterschiedliche Releases auf unterschiedlichen Testumgebungen zur Verfügung stehen müssen (Beispiel siehe Tab. 34.1).

In diesem vereinfachenden Beispiel findet jeweils genau zum Monatswechsel ein Rollout in die Produktion statt und in der Produktion ist auch nur ein Release vorhanden. Sollte der Rollout sich über längere Zeit hinziehen, sind in dieser Phase mehrere Releases in der Produktion vorhanden und entsprechend viele Testumgebungen vorzuhalten, bis der letzte Standort auf das neue Release umgestellt ist.

Mittels Virtualisierung – entweder am eigenen Standort oder in einer Cloud – können Systeme oder Services in einem frühen Entwicklungsstadium getestet werden, bevor die entsprechende Infrastruktur existiert [SQSS2018].

Dazu ist ein erfolgreiches Testumgebungsmanagement erforderlich.

Häufig ist die Verfügbarkeit realistischer Testumgebungen durch folgende Probleme erschwert:

- fehlende und instabile Komponenten
- wachsende und sich verändernde Entwicklungsumgebungen
- nicht verfügbare Systeme und Services von Dienstleistern oder externen Partnern
- zu hohe Komplexität für den Betrieb in Testlaboren (z. B. Mainframes oder große CRM und ERP-Systeme)
- interne und externe Ressourcen, die mit anderen Partnern geteilt werden müssen

Aufgrund wachsender Systemanforderungen haben viele Organisationen Hardware- und Betriebssystem-Virtualisierungen eingeführt, um ihren Teams ständigen Zugriff auf realistische Entwicklungs- und Testsysteme zu ermöglichen.

Die Virtualisierung der Kernelemente, wie dem Betriebssystem, der Konfiguration und der Plattform selbst, erhöht die Unabhängigkeit vom Produktivsystem und erhöht die

Verfügbarkeit spezieller Systemtypen. In Kombination mit einer Cloud-Infrastruktur bietet die Virtualisierung sogar eine nahezu unbegrenzte Bandbreite zur Skalierung der voneinander abhängigen Systeme.

Es gibt aber nach wie vor Beschränkungen, weil eine Hardware- oder Betriebssystem-Virtualisierung für große Mainframe-Anwendungen, Third-Party-Anwendungen oder große ERP-Systeme oftmals gar nicht möglich ist.

Eine weitere Herausforderung stellt die Konfiguration der Systeme dar, die sich oft sehr kompliziert und zeitaufwändig darstellt. Inzwischen wird vermehrt dazu übergegangen, nur die Teilsysteme zu virtualisieren, die für das aktuell von den Testern und Entwicklern betrachtete Verhalten notwendig ist.

Testumgebungen bieten oftmals keine ausreichende Leistungsfähigkeit für realistische Performancetests. Die Bereitstellung mehrerer virtualisierter Systeme auf einer Hardware erhöht zwar die Verfügbarkeit der Testsysteme, verringert dabei aber häufig auch die verfügbare Leistung. Virtualisierte Systeme stellen in der Regel kein realistisches Systemverhalten zur Verfügung. Dadurch sind die erzielten Testergebnisse nur bedingt übertragbar.

Eine Virtualisierung des Anwendungsverhaltens hingegen ermöglicht eine Skalierung realistischer Performanz-Eigenschaften unabhängig vom emulierten Live-System. Dafür wird während der Ausführung des Testobjekts das Verhalten der Randsysteme sowie der relevanten Transaktionen, Nachrichten, Services etc. und der zugehörigen Daten in einem sogenannten „Virtual Asset“ erfasst. Wenn ein Virtual Asset, das die realen Performanz-Eigenschaften beschreibt, einmal erzeugt wurde, so kann dieses beliebig konfiguriert werden um realistische Performance-Eigenschaften des Systems zu simulieren. Dazu werden entsprechende Parameter für Timing, Latenz und Verzögerungen eingestellt. Hierdurch können Performance-Tests gegen das Virtual Asset mit realistischen Performance-Eigenschaften durchgeführt werden, anstatt mit Testsystemen, die nur eingeschränkte Performance-Eigenschaften bieten, arbeiten zu müssen [SIGS2011].

Literatur

- [SQSS2018] <https://www.sqs.com/de/services/testumgebungen-environment-management.php>
[SIGS2011] https://www.sigs-datacom.de/uploads/tx_dmjournals/kusterer_OS_testing_11.pdf
[HETT2018] <https://www.hettwer-beratung.de/konzepte/testkonzept/testumgebung/>

35.1 Grundlagen von Schätzverfahren

Testmanager stehen oft vor der Herausforderung, den Aufwand für ein Projekt abzuschätzen.

Die Schätzung der zu erwartenden Testaufwände ist von zahlreichen Parametern abhängig. Es gibt einige Faustformeln, die sich dazu verwenden lassen, einen halbwegs realistischen Wert anzusetzen.

Auf der Basis des „reinen Entwicklungsaufwands = 1“ kommen hinzu:

0,1 für politisches Projektmanagement

0,2 für technisches Projektmanagement

0,1 für Qualitätsmanagement

0,1 für Anpassen von Entwicklungswerkzeug, Integration und Configuration Management

0,5 für Testaktivitäten

und wenigstens 0,1 für Dokumentation

Unter dem reinen Entwicklungsaufwand (= 1) versteht man dabei:

Anforderungs-, Change- und Configuration-Management (= 0,3)

Entwurf und Schnittstellenspezifikation (= 0,4)

Codierung (= 0,3)

Da man, wie oben bemerkt, zum reinen Entwicklungsaufwand 50 % für den Test (durch die Software-Entwickler selbst) hinzurechnen sollte, gilt grob: Wer Software entwickelt (als Auftragnehmer), aber nicht ihr Nutzer sein wird, muss etwa 1/3 des Gesamtaufwandes für Test einkalkulieren, den er selbst durchzuführen hat.

Dabei ist zu beachten, dass man eine regressionsfähige Testsuite aufbauen, die regelmäßige automatisierte Tests erlaubt. Diese Testsuite muss

- alle wesentlichen Funktionen der Anwendung zum Gegenstand haben,
- immer dann, wenn modifizierter Code in Betrieb genommen werden soll, auch komplett neu abgearbeitet werden.

Diese Testsuite so anzulegen, dass sie das System sein ganzes Leben lang begleiten kann, ist eine schwierige, kostspielige Aufgabe. Die Testtreiber und Testauswertungsfunktionen zu pflegen, d. h. auf jeweils neue Releases der Anwendung anzupassen und vor Inbetriebnahme jedes neuen Releases oder jeder Bug-Fix-Auslieferung den kompletten Acceptance-Test neu durchzuführen, kann sehr aufwendig sein. Solchen Aufwand über eine Faustregel zu quantifizieren ist unmöglich – ihn gezielt klein zu halten gelingt nur dann, wenn die Software-Architekten die Anwendung ganz gezielt so entworfen haben, dass sie einfach testbar wird [GREI2018].

Einige Grundregeln sind bei der Durchführung von Aufwandschätzungen zu beachten:

- Die Aufwandsschätzung ist eine Aktivität, die regelmäßig während des Projektes durchgeführt werden muss. In der Anfangsphase stellt eine high level Aufwandsschätzung die Machbarkeit des Projektes fest. Später müssen sehr sorgfältige und verfeinerte Analysen und Aufwandsschätzung durchgeführt werden und während des Projekts regelmäßig nachjustiert werden.
- Die Beteiligten müssen sich über das Testobjekt und den Testumfang einig sein.
- Vor allem in größeren Projekten empfiehlt es sich, unterschiedliche Teams die gleiche Aufgabe abschätzen zu lassen und deren Ergebnisse zu vergleichen. Große Differenzen in den Zahlen zeugen für Unklarheiten im Testvorgehen.
- Die Unbekannten in der Aufwandsschätzung sind zu quantifizieren und durch adäquate Puffer zu kompensieren.
- Alle Teststufen und Projektphasen sind bei der Testaufwandsschätzung zu berücksichtigen.
- Die Aufwandsschätzung sollte in Punkten oder Arbeitsstunden und nicht in Kalendertagen vorgenommen werden. Kein Team ist zu 100 % effektiv! Man kann einen getrennten Konvertierungsfaktor einsetzen, um die Aufwände in Kalendertage umzurechnen.
- Ein Faktor für nicht vorhersagbare Vorkommnisse (etwa mehrfache Testzyklen zur Fehlerbehebung) sind einzuplanen.
- Die Aufwandsschätzung und sowohl das Verfahren für die Kalkulation als auch die Kalkulationsbasis sind zu dokumentieren [THIN2018].

35.2 Modelle zur Testaufwandsermittlung

Mehrere Modelle sind dazu geeignet, Testaufwände zu erheben:

1. **COCOMO II:** mehrere, unterschiedlich detaillierte algorithmische Modelle zur Aufwandsschätzung zu unterschiedlichen Zeitpunkten im Softwarelebenszyklus

2. **FPA = Function Point Analysis:** Quantifizierung der Funktionalität eines Softwareprodukts aus Anwendersicht. Dabei ist ein Function Point die Maßeinheit des funktionalen Umfangs der Anwendungssoftware.
3. **TPA = Test Point Analysis:** Die TPA ist geeignet zur Schätzung des Aufwands für High-level Tests, z. B. Test eines integrierten Softwareprodukts. Dabei werden statische und dynamische Testpoints gezählt, die wiederum auf der Anzahl der Function Points der zu testenden Software basieren. Als Input der Schätzung dienen außerdem ein funktionaler Entwurf einschließlich detaillierter Prozessbeschreibungen und ein logisches Datenmodell. Die Ermittlung eines organisationsspezifischen Fähigkeitsfaktors erfordert außerdem Daten abgeschlossener Projekte der schätzenden Organisation.
4. **Modell von Sneed und Jungmayr:** Eine mathematische Formel auf Basis von CO-COMO II berechnet aus Konzept, Entwurf und Sourcecode die Testbarkeit, die Anzahl erforderlicher Testfälle und den Testüberdeckungsgrad. Die Testerproduktivität (je Zeiteinheit durchgeführte Testfälle) und ein Skalierungsfaktor, der sich aus der Testumgebung, den Testwerkzeugen, der Erfahrung und Zusammengehörigkeit der Testmannschaft und der Reife des Testprozesses zusammensetzt, ergeben den voraussichtlichen Testaufwand (in Zeiteinheiten).

Diese Ermittlung ist aber kompliziert, da einige implizite Annahmen diesem Modell zugrunde liegen:

- Die Testfälle hinsichtlich der einzelnen Bearbeitungszeit miteinander vergleichbar sein.
 - Der Wiederholungsfaktor (Anzahl der benötigten Testzyklen) kann vor Testbeginn ermittelt werden.
 - Die Testerproduktivität ist bekannt. Dafür müssen die Stabilität des Testsystems und die Verfügbarkeit der Testumgebung von vornherein bekannt sein, was oft nicht gegeben ist.
 - Die Testbarkeit muss mithilfe geeigneter Metriken gemessen werden.
5. **Modell von Clazolari, Tonella und Antonio:** Dieses Modell erklärt die Entwicklung des Testaufwands in Analogie zur Systemtheorie der Evolution: Fehler werden als Beute, Tester als Jäger modelliert, die die Population der Fehler reduzieren. Zwei Differenzialgleichungen beschreiben die dynamische Variation der beiden Populationen. Zur Testaufwandsprognose werden die Parameter in beiden Gleichungen anhand von Daten geschätzt, die die Entwicklung des Testaufwands im Projekt wiedergeben. Beide Gleichungen enthalten die Parameter a , die asymptotische Fehlerkorrekturrate, und b , die Fehleranzahl, ab der die Fehlerkorrekturrate halbiert ist. Um die Entwicklung des Testaufwands (also die Population der Jäger) zu berechnen, müssen darüber hinaus die Parameter c , Effizienzfaktor genannt, und m (für Mortalität), d. h. die intrinsische Abnahme des Korrekturaufwands im Zeitverlauf, ermittelt werden.

Die Anwendung dieses Modells zur Testaufwandsschätzung wird dadurch erschwert, dass die Parameterschätzung nicht exakt genug beschrieben ist. Es müssen bereits mehrere Releases des Produkts entwickelt worden sein, das im zu schätzenden Projekt weiterentwickelt wird, und zu jedem Release muss detailliert erfasst worden sein, wie viele Fehler zu welchem Zeitpunkt gefunden wurden. Die Trendlinien, die zur Parameterschätzung benötigt werden, stellen hohe Anforderungen an die Qualität der Datenerfassung.

6. **Use Case Points – Modell von Nageswaran:** Nageswarans Ansatz basiert auf der Annahme, dass sich aus der Anzahl der Use Cases mittels Zählung der Use Case Points die Anzahl der erforderlichen Testfälle bestimmen lässt. In Verbindung mit organisationsspezifischen Erfahrungswerten hinsichtlich der erforderlichen Zeit bzw. des Aufwands zur Durchführung von Testfällen ergibt die Anzahl der Testfälle den Testaufwand. Zur Ermittlung der Anzahl der zu testenden Use Case Points werden die Anzahl der Anwender, die Anzahl und Komplexität der produktinternen und -externen Schnittstellen sowie die Anzahl und Komplexität der Use Cases berücksichtigt. Zur Testaufwandsschätzung wird die ermittelte Anzahl Uses Case Points anhand eines technischen Komplexitätsfaktors angepasst. Zuschläge für die Projektkomplexität und Koordinations- und Managementaufgaben erhöhen den Wert der angepassten Use Case Points. Zur Testaufwandsschätzung wird schließlich ein organisationsspezifischer Umwandlungsfaktor benötigt, der für bestimmte Kombinationen einer Programmiersprache mit einer Technologie den Aufwand für Planung, Testfallerstellung und Testdurchführung zum Test eines Use Case Points angibt.
7. **Modell von Cangussu, DeCarlo und Mathur:** In diesem Modell wird basierend auf Konzepten und Techniken der Regelungstechnik der Testprozess gestaltet und der Testaufwand geschätzt, der zur Verringerung der Fehleranzahl und Einhaltung des vorgegebenen Auslieferungstermins erforderlich ist. Eingabeparameter sind dabei die Anzahl Tester je Zeiteinheit, die Komplexität des zu testenden Softwareprogramms, die Qualität des Testprozesses, der Anzahl bei Testbeginn im Programm enthaltener Fehler, die Konstante der Funktion „dem Testprozess anhaftender Widerstand gegen die Fehlerreduzierung“ und die Funktion der Entwicklung der Fehleraufdeckungsrate im Projekt. Auch in diesem Modell werden Daten aus vergleichbaren Projekten zur Parameterschätzung herangezogen. Das Modell geht davon aus, dass Teststufen nach Bedarf, jedoch nicht zeitlich geplant, stattfinden, und dass die Anzahl der enthaltenen Fehler schon zu Projektbeginn bekannt bzw. ermittelbar ist. Beides trifft in der Praxis nicht zu. Die Annahme, dass zu jedem Zeitpunkt des Projekts die verbleibende Fehleranzahl proportional zum Testaufwand ist, ist ebenfalls nicht plausibel. Der Aufwand der Fehlerbehebung und die Anzahl der Testzyklen pro Fehlerbehebung sind sehr unterschiedlich.
8. Eine Weiterentwicklung der genannten Modelle ist das **allgemeine Modell des Testaufwands**: Demnach müssen zur Aufwandsermittlung mehrere Merkmale betrachtet werden:
 - a) Organisationsspezifisch ausgeprägte Merkmale:
 - Zu den Merkmalen des Entwicklungsprozesses gehören die **Prozessreife** (je höher die Produktqualität dank der Reife des Entwicklungsprozesses ist, desto weniger Fehler enthält das Produkt) und die Einordnung des Testens in den Entwicklungsprozess. Entwicklungsprozesse und damit die Ausprägung ihrer Merkmale verändern sich im Zeitverlauf, wobei dies mehr oder weniger zielgerichtet geschieht. Betrachtet man den Entwicklungsprozess als Abbild der Ablauf- und

Aufbauorganisation in Softwareentwicklungsprojekten einer Organisation, wird klar, dass Änderungen vielfältige Konsequenzen haben (z. B. für die Koordination oder das Machtgefüge innerhalb des Unternehmens oder des Projekts) und die Änderung der Einstellungen und Verhaltensweisen der Betroffenen voraussetzen. Organisationsänderungen führen nach meiner Erfahrung häufig zu Unsicherheit und Verlustängsten und bremsen die Tätigkeiten zunächst aus. Die einzelnen Teammitglieder müssen sich erst neu sortieren und verharren gerade dann wenn die neue Organisation nicht abschließend geklärt ist erst einmal in Wartestellung. Ich hatte bereits mehrfach Projekte erlebt, die hauptsächlich aus genau diesem Grund aus dem Termin- und Kostenrahmen liefen, nicht mehr fertiggestellt wurden oder bei denen der Scope reduziert wurde.

- Zu den Merkmalen der Organisation werden die Organisationskultur, die Arbeitsteilung und die Koordination gezählt. Die **Organisationskultur** bezeichnet die Art zwischenmenschlicher Beziehungen (Konkurrenz oder Kooperation), Standards (z. B. ungeschriebene Verhaltensregeln) und Symbolsysteme (z. B. Umgangsformen), die die Motivation und den Teamgeist der Tester sowie Effektivität und Effizienz des Projektteams wesentlich mitbestimmt. Das sind Einflussfaktoren, die sehr schwer in Zahlen zu bewerten sind und einem Projektmanager oder Testmanager das Leben schwer machen können. In einem Projekt, in dem ich vor kurzem tätig war, war das Testteam aus zwei Abteilungen zusammengewürfelt worden. Diese Abteilungen hatten aus historischen Gründen sich gegenseitig abgelehnt. Schwelende Konflikte führten zu einer großen Unzufriedenheit und gegenseitigen Schuldzuweisungen bei Problemen. Das Projektziel wurde unter großen Anstrengungen erreicht, diese Faktoren hatten es aber allen Beteiligten sehr schwer gemacht. Obwohl das Testobjekt am Ende eingeführt wurde, behielten viele Stakeholder das Projekt in negativer Erinnerung. Leider wurde auf ein „Lessons Learned“ in diesem Fall verzichtet. Die **Arbeitsteilung** gilt als ein Grundproblem von Organisationen und determiniert ebenfalls die Leistungsfähigkeit und Leistungsbereitschaft von Mitarbeitern. Die ausgewogene oder einseitige Belastung von Mitarbeitern führt häufig zu Fluktuation. Unter **Koordination** ist die wechselseitige Abstimmung von Elementen eines Systems zur Optimierung desselben zu verstehen. Je komplexer die zu erfüllenden Aufgaben in einer Organisation sind, desto mehr erfolgt die erforderliche Koordination durch informale Kommunikation. Für den Testaufwand ist es entscheidend, wie viel Aufwand die organisationsspezifische Gestaltung der Koordination mit sich bringt und wie viel Informationen dabei verloren gehen.

b) Projektspezifisch ausgeprägte Merkmale:

Es gibt zahlreiche verschiedene Einflussgrößen, die sich in vielen Fällen gegenseitig beeinflussen und teilweise voneinander abhängig sind. Diese Wechselwirkungen machen eine Aufwandsschätzung sehr schwierig, zumal es für diese Beobachtungen bisher keine Theorie gibt, die dieses Wirkungsgefüge hinreichend beschreibt und erklärt. Empirische Analysen ohne jegliches theoretische Fundament können keine inhaltvollen Aussagen liefern.

- **Merkmale des Produkts:** Zu diesen Merkmalen gehören die **Marktbedeutung** (strategische Bedeutung des Softwareprodukts, Anzahl der Kunden früherer Releases, Anzahl erwarteter Kunden und geplanter Preis des Produkts), die **Neuartigkeit** (Auswirkung auf das Fehlerpotenzial bei der Umsetzung) und die **Dynamik der Anforderungen**. Unklare oder widersprüchliche Anforderungen bewirken das Risiko fehlerhafter Entwicklung und höherer Testaufwände. Dasselbe gilt für Anforderungen, die sich während des Projektverlaufs ändern, da die Anzahl an Entwicklungs- und Testzyklen steigt. Außerdem haben der **Grad der Kundenindividualität** (kann der Kunde das Produkt eigenständig konfigurieren?), die **Kundenerwartungen** (Kritikalität, Existenz von Vergleichsprodukten) und der **Umfang und die Komplexität des Softwareprodukts** (Anzahl der Anforderungen, eingesetzte Entwicklungstechnologien und Anzahl externer und interner Schnittstellen) einen erheblichen Einfluss auf das Produkt und den Testaufwand. Schließlich beeinflussen die angestrebte **Qualität** (Funktionalität, Zuverlässigkeit, Benutzbarkeit und Effizienz) und die **Dokumentation** der Spezifikation und die Qualität der Entwurfsdokumente die Testintensität und dadurch auch den Aufwand.
- **Merkmale des Teams:** Dazu gehören die **Erfahrung der Tester** (auch der Entwickler), die **Motivation** und die **Einstellung zur Qualität**, die **Teamgröße** und die **Zusammenarbeit** (Kontinuität des Teams, Verfolgung eines gemeinsamen Ziels) und die Kommunikation (Sprache, Standorte) und die geeignete **Rollenbesetzung** (übernehmen Tester oder Entwickler die Testdurchführung?).
- **Merkmale der Ressourcen** sind **personelle Restriktionen** (Verfügbarkeit der Teammitglieder, Abhängigkeit von Zulieferern), die **Kundenbeteiligung am Test** (eine enge Zusammenarbeit zwischen Entwicklern, Testern Management und Anwender reduziert den Testaufwand), die Anzahl, Qualität und Wiederverwendbarkeit der vorhandenen **Testfälle und Testdaten**, die Zeit für die Projektdurchführung und die Verfügbarkeit der **Testinfrastruktur** und die Nutzung von Testwerkzeugen.
- **Merkmale der Testorganisation:** Die Auswahl der geeigneten Testfälle und die geeignete Verteilung des Testaufwands auf einzelne Teststufen um sowohl mehrfache Aufwände durch redundante Tests in verschiedenen Teststufen als auch fehlende Tests zu vermeiden wirken sich ebenfalls auf den Testaufwand aus.

In weiteren Modellen zur Testaufwandsschätzung werden u. a. außerdem folgende Einflussgrößen genannt:

- **Testbarkeit** der Software
- Fähigkeiten der Entwickler
- Verwendete **Testmethoden** [DOWI2009]

In der Praxis werden die unterschiedlichen Parameter häufig nicht ausreichend berücksichtigt. Das führt dabei in der Regel zu viel zu optimistischen Schätzwerten zum Testaufwand. Ein weiteres Problem in diesem Zusammenhang ist, dass man oft nur den eigenen Bereich

sieht und nicht den gesamten Prozess betrachtet: Verzögerungen in der Entwicklung ziehen Verzögerungen beim Test nach sich und erhöhen dadurch den Testaufwand. Sofern Erfahrungswerte aus ähnlich gelagerten Projekten vorhanden sind und das bestehende Testteam schon einmal ein ähnliches Softwareprodukt getestet hat, sind die Schätzungen noch am genauesten.

Man sollte jedenfalls bei den einzelnen Schätzungen immer einen genügend großen Sicherheitsaufschlag mit einkalkulieren. Aus meiner Erfahrung liegt dieser Aufschlag zwischen 10 % und 20 % des ermittelten Testaufwands. Damit liegt man am Ende dann meistens einigermaßen richtig.

Leider werden auch viel zu selten Projekte nachkalkuliert, indem man die ursprüngliche Schätzung mit den am Projektende tatsächlich erreichten Stunden vergleicht. Wenn das Projekt bzw. Produkt erst einmal eingeführt ist, ist es nicht mehr von Interesse und aus dem Focus verschwunden. Aber schon aus diesem Vergleich heraus könnte man wertvolle Informationen für die Zukunft gewinnen, um die im Unternehmen relevanten Einflussgrößen richtig einzuschätzen.

Literatur

- [GREI2018] <http://greiterweb.de/spw/Software-Entwicklungsaufwand.htm>
[THIN2018] <http://www.think-pi.de/aufwandschätzung>
[DOWI2009] Dowie U (2009) Testaufwandsschätzung in der Softwareentwicklung. <https://elib.uni-stuttgart.de/handle/11682/5524>

Weiterführende Literatur

SQ-Magazin Ausgabe. 23.06.2012. Agiles Testen Eine Managementsicht. Uwe Hehn Method Park Software AG
<http://www.enzyklopaedie-der-wirtschaftsinformatik.de/> Erläuterung von Fachbegriffen in mehreren Kapiteln

Stichwortverzeichnis

A

Abnahmetest 81
A/B-Testing 256
Acceptance Test Driven Development (ATDD) 47
Änderungskritikalität 53
Änderungsqualität 53
Änderungsquantität 53
Äquivalenzklasse 173
Akzeptanztest 81
Anforderungsanalyse 61
Anforderungsbeschreibung 61
Anforderungsüberdeckung 222
Anforderungsverfolgung 61
Angemessenheit 23
Anpassbarkeit 24
Anweisungüberdeckung 178, 182
Architekturüberdeckung 222
Austauschbarkeit 25
Automatisierung 232

B

Barry-Boehm-Kurve 96
BDTM-Ansatz 273
Bedingungsüberdeckung 182
Behavior Driven Development (BDD) 47
Belastungstest 167
Benutzerdokumentation 66
Benutzeroberfläche, graphische 235
Big Bang 201
Black-Box-Test 85
Branch Coverage 90
Browser 253

C

Chapin-Q-Metrik 220
Cloud 263
 Service 244
COCOMO II 148
Condition Coverage 91
Continuous integration 39
Conversion 260
Crashtest 167
Cross-Selling 250

D

Data Mining 262
Datenkonsistenztest 171
Datenüberdeckungsmaß 222
Decision Coverage 90
Defekt 109
Delta-Test 202
Denial-of-Service-Angriff 168

E

Entscheidungstabelle 184
Entwicklungsaufwand 287
Entwicklung, testgetriebene 269
Error Guessing 190

F

Fehler 94
Fehlerdichte 107, 111
Fehlerfindungsproduktivität 107
Fehlerfindungsrate 52

Fehleridentifikationsquote 182
Fehlerintensität 228
Fehlerkosten 96
Fehlermeldung 103
Fehlermetrik 230
Fehlerstatistik 225
Fehlertest 171
Fehlertoleranz 23
Fehlertracking 102
Fehlerverfolgung 102
Fehlerverwaltung 104
Fehlerzahl 58
FPA (Function Point Analysis) 289
Funktionsüberdeckung 222

G

GNU COVeRage tool 179
Google Trends 253
Grey-Box-Test 86
GUI 235

H

Halstead-Metrik 219
Henrys-Interface-Metrik 221
HIL-Simulation 51
HIL-Test 51
Historie 4

I

IEEE 829 136
Inkrementelles Messen 181
Installationstest 171
Internet-Anwendungen 251
Interoperabilität 23
ISO/IEC
29119-2 25
33063 25

K

Key Performance Indicator 218
Koexistenz 25
Komplexitätsfaktor 222
Konfigurationsmanagement 35
Konfigurationsverwaltung 57
Konformität 23
Konsistenzsicherung 203

Konversionsrate 259
Korrektheit 23
Kundenindividualität 292

L

Line Coverage 89

M

Manifest, agiles 267
Marktbedeutung 292
Mock-Objekt 171
Modell des Testaufwands, allgemeines 290
Modell von
Cangussu, DeCarlo und Mathur 290
Clazolari, Tonella und Antonio 289
Sneed und Jungmayr 289
Modifizierbarkeit 24
Multivariate Testing 256

N

Nearshore 247
Negativ-Test 174

O

Offshoring 243
Ordnungsmäßigkeit 23
Organisationskultur 291

P

Pairwise Methode 175
Path Coverage 92
Penetrationstest 168
Pfadüberdeckung 179, 182
Piloteinsatz 200
Pilotierung 200
Portierbarkeit 81
Positiv-Testfall 174
Produktivbetrieb 201
Produktivnahme 199
Programmierfehler 112
Prozess
agiler 37
phasenorientierter 37
Prozessoptimierung 50
Prozessreife 290

Q

Qualitätsanspruch 56
Quick-Test 54, 177, 178

R

Rechnernetz-Test 172
Reifegrad 278
Release 55
 Management 35
Reporting 225
Requirements Engineering 61
Review 205
Risiko 125
 Aussetzung 152
 Bewertung 153, 164
 Eintrittswahrscheinlichkeit 152
 Klasse 274
Rollenbesetzung 292

S

Schedule-Compression-Faktor 150
Schnittstellentest 171
Shift-Left 44
Sicherheitstest 167
Simulator 160
Skalierungsexponent 146, 149
Skript 160
Softwarekomplexität 146
Softwareproduktreife 8
Softwarequalität 146
Sprint 268
Standardisierung 5
Statusbericht 194
Statusverfolgung 57
Systemarchitektur 73
Systembewertungsverfahren 144
Systemdesign 70
Systemverhalten 167

T

Taguchi-Methode 261
Task Board 269
Testabdeckung 215, 227
Testanweisung 162–163
Testart 139
Testaufwand 29, 148, 228, 287

Testaufwandschätzung 145
Testautomatisierer 30, 239
Testautomatisierung 232
Testbarkeit 147, 292
Testbericht 193
Testberichtswesen 144
Testbeschreibung 159
Test charter 191
Testdaten 121
Testdatendichte 219
Testdatenkomplexität 220
Testdatenmanagement 122
Testdatenvolumen 221
Testdauer 150
Testdesign 3
Testdesigner 29
Testdokumentation 159
Testelement 259
Testen
 exploratives 188
 strukturiertes 188
Testende 58
Testendekriterien 137
Testendekriterium 218
Testfall 162
 funktionaler 166
Testfallbeschreibung 187
Testfalldichte 221
Testfallintensität 222
Testfallqualität 222
Testfallüberdeckung 223
Test, funktionaler 94
Test-Governance 274
Testgüte 193
Testimplementierung 161
Testinfrastruktur 138, 263, 292
Testkonzept 131
Testkoordinator 28
Testkriterium 59
Test Lifecycle 34
Testmanagement 33
Testmanagementwerkzeug 209
Testmanager 27
Test Maturity Model (TMMi) 184
Testmethode 132, 173, 292
Testmetrik 217
Testobjekt 124
Test-Offshoring 246
Testorganisation 59, 132, 292

Testphase 139
Testplanung 119, 154
Testplattform 138
Test Process Improvement (TPI) 185
Testproduktivität 145, 228
Testprozedur 162
Testprozess 2
Testprozessmanagement 144
Testprozessreife 144
Testpyramide 86
Testqualität 223
Testrahmen 134
Testreport 193
Testspezifikation 66, 160
Teststatusbericht 137
Teststrategie 2, 50, 119, 136
Teststufe 101
Testtiefe 177
Testtool 138, 210
Testumgebung 127, 134, 283
Testvariante 260
Testverfahren 232
Testvoraussetzung 157
Testwiederholbarkeit 149
Testziel 127
TMap 276
TPA (Test Point Analysis) 289
TPI Next Modell 277
Traceability 203
 -Matrix 163

U

Überdeckungsgrad 179
Überdeckungsmaß 178
Upselling 250
Usability 24, 168
Use Case Points – Modell von Nageswaran 290
User-Stories 161

V

Validierung 82
Verifikation 82
Versionsverwaltung 57
Vertrauensgrad 52, 148
Virtualisierung 284
V-Modell 82

W

Wärmebild 262
Web-Applikation 253
Web-Server 251
White-Box-Test 85
Wiederinbetriebnahmetest 171

Z

Zehnerregel 96
Zeitverhalten 24
Zielgruppe 255
Zugriffsverhalten 168
Zweigüberdeckung 178



Jetzt im Springer-Shop bestellen:
springer.com/978-3-658-19844-2

